

دستور کار آزمایشگاه سیستم‌عامل

دکتر لیلا شریفی

اکرم سلطانی

تابستان ۱۴۰۵

فهرست آزمایش‌ها

آزمایش ۱ : سیستم‌های لینوکسی ۹

۱-۱- نرم‌افزار آزاد ۱۱

۱-۲- سیستم‌عامل گنو/لینوکس ۱۲

۱-۳-توزیع‌ها ۱۴

۱-۴-کاربردهای لینوکس ۱۶

۱-۵- نصب لینوکس ۱۸

آزمایش ۲ : تنظیمات و راهبری در سیستم‌های لینوکسی ۴۱

۲-۱- قابلیت تغییر و تنظیم ۴۳

۲-۲- خط فرمان ۴۵

۲-۲- ۱- انواع پایانه در لینوکس ۴۷

۲-۳-رفع مشکل در لینوکس ۵۰

۲-۴- انواع کاربران ۵۰

۲-۵- فایل‌سیستم ۵۲

۲-۶- پارتیشن‌بندی ۵۶

۲-۷- سطح اجرایی ۵۹

۲-۸- فرآیند بوت سیستم ۶۱

آزمایش ۳ : خط فرمان ۶۷

۳-۱-دستورات خط فرمان ۶۹

۳-۲- راهبری ۶۹

۳-۳- مجوزها و کاربران ۷۳

۳-۴- جستجو ۸۰

۳-۵- مدیریت فایل و دایرکتوری ۸۳

۳-۶- پردازش متن ۸۶

۳-۷- مدیریت دیسک ۸۸
۳-۸- مشاهده اطلاعات سیستم ۹۵
۳-۹- مشاهده و مدیریت فرآیندها ۹۶
۳-۱۰- راهنما و مستندات ۱۰۲
۳-۱۱-مخازن نرم‌افزاری ۱۰۳
آزمایش ۴ : اسکریپت‌نویسی پوسته ۱۰۷
۴-۱- معرفی ۱۰۹
۴-۲- پیش‌نیازها ۱۱۰
۴-۳- اولین اسکریپت ۱۱۹
۴-۴- متغیرها ۱۲۲
آزمایش ۵ : اسکریپت‌نویسی پیشرفته ۱۲۹
۵-۱- مرور مفاهیم پایه ۱۳۱
۵-۲- ساختار شرطی If ۱۳۱
۵-۳- عملگرهای منطقی ۱۳۴
۵-۴- حلقه تکرار for ۱۳۶
۵-۵- حلقه تکرار while ۱۳۷
۵-۶- متغیرهای محیطی پوسته ۱۳۹
۵-۷- متغیرهای خاص ۱۴۱
آزمایش ۶ : وقفه‌ها ۱۴۷
۶-۱- وقفه چیست؟ ۱۴۹
۶-۱-۱- انواع وقفه ۱۵۰
۶-۲- مشاهده وقفه‌های سیستم ۱۵۳
۶-۳- برنامه‌نویسی به زبان C در لینوکس ۱۵۵
۶-۴- غیرفعال‌سازی وقفه تایمر ۱۵۷
۶-۵- فراخون سیستمی ۱۶۰
۶-۶- سیگنال ۱۶۳

آزمایش ۷ : فرآیندها ۱۷۷
۷-۱- فرآیند چیست؟ ۱۷۹
۷-۲- مشاهده فرآیندها ۱۸۰
۷-۳- ایجاد و مدیریت فرآیندها ۱۸۰
۷-۴- ایجاد فرآیند فرزند ۱۸۱
۷-۵- ارتباط بین فرآیندی ۱۸۳
۷-۵-۱- پایپ ۱۸۴
۷-۵-۲- حافظه مشترک ۱۸۵
۷-۵-۳- سوکت ۱۸۹
آزمایش ۸ : رشته‌ها ۱۹۷
۸-۱- چرا چند رشته‌ای؟ ۱۹۹
۸-۲- ایجاد و از بین بردن رشته‌ها ۲۰۰
۸-۳- ارسال آرگومان به رشته و برگرداندن نتایج از آن ۲۰۵
۸-۴- چند رشته با تابع مشابه ۲۰۷
۸-۵- رشته‌های تودرتو ۲۱۱
آزمایش ۹ : سمافور ۲۱۵
۹-۱- ساختار سمافور ۲۱۷
۹-۲- انواع سمافور ۲۱۸
۹-۲-۱- سمافور باینری ۲۲۰
۹-۲-۲- سمافور شمارشی ۲۲۱
آزمایش ۱۰ : تحلیل کارایی با Perf ۲۲۹
۱۰-۱- معرفی ۲۳۱
۱۰-۲-نصب و تنظیمات ۲۳۳
۱۰-۳- دستورات فرعی ۲۳۵
۱۰-۴- پروفایل کردن پردازنده ۲۳۶
۱۰-۵- پروفایل کردن برنامه‌های سطح کاربر ۲۴۰

۱۰-۶- کاربردهای تکمیلی ۲۴۸
آزمایش ۱۱ : کامپایل هسته لینوکس ۲۵۵
۱۱-۱- هسته لینوکس ۲۵۷
۱۱-۲- مزایای هسته ماژولار ۲۶۰
۱۱-۳- چالش‌های هسته ماژولار ۲۶۱
۱۱-۴- کامپایل و آپدیت دستی هسته ۲۶۲
۱۱-۵- آپدیت خودکار هسته لینوکس ۲۶۸
۱۱-۶-بازگشت به هسته قبلی و حذف هسته‌های اضافه ۲۷۱
آزمایش ۱۲ : ماژول‌های هسته ۲۷۵
۱۲-۱- ماژول هسته چیست؟ ۲۷۷
۱۲-۲- ماژول‌های سیستمی ۲۷۷
۱۲-۳- ماژول‌های کاربر ۲۷۹
۱۲-۳-۲- ایجاد ماژول شخصی ۲۸۰
۱۲-۳-۲- کامپایل ۲۸۲
۱۲-۳-۳- بارگذاری، بررسی اجرا و خروج ۲۸۴
۱۲-۳-۴- دریافت پارامتر ۲۸۵
آزمایش ۱۳ : مجازی‌سازی با KVM ۲۹۱
۱۳-۱- مجازی‌سازی ۲۹۳
۱۳-۲- مجازی‌سازی با KVM در لینوکس ۲۹۷
۱۳-۲-۱- نصب و راه‌اندازی kvm ۲۹۷
۱۳-۲-۲- نصب سیستم‌عامل مجازی در KVM ۳۰۰
۱۳-۲-۳- دسترسی به سیستم‌عامل مجازی ۳۰۴
۱۳-۲-۴- مدیریت ماشین‌های مجازی ۳۰۴
آزمایش ۱۴ : مجازی‌سازی نوع یک با ESXi ۳۰۹
۱۴-۱- معرفی ۳۱۱
۱۴-۲- نصب سرور ۳۱۱

۱۴-۳- مدیریت ماشین‌های مجازی ۳۱۴
۱۴-۴- اتصال کاربران ۳۱۹
آزمایش ۱۵ : حافظه مجازی ۳۲۳
۱۵-۱- شیوه‌های مدیریت حافظه ۳۲۵
۱۵-۲- حافظه مجازی ۳۲۷
۱۵-۳- نظارت بر تخصیص حافظه مجازی ۳۳۰
۱۵-۴- برنامه‌ای برای تخصیص حافظه ۳۳۳
۱۵-۵- دستورات بررسی حافظه سیستم ۳۳۴
آزمایش ۱۶ : فایل‌سیستم ۳۴۱
۱۶-۱- انواع فایل‌سیستم ۳۴۳
۱۶-۱-۱- FAT ۳۴۳
۱۶-۱-۲- NTFS ۳۴۶
۱۶-۱-۳- EXT ۳۴۹
۱۶-۲- ایجاد فایل‌سیستم ۳۵۵
۱۶-۳- بررسی سلامت فایل‌سیستم ۳۶۵
۱۶-۴- اشکال‌زدایی فایل‌سیستم ۳۶۶
پیوست ۱ : راهنمای گزارش کارنویسی ۳۷۵
پیوست ۲ : راهنمای سریع دستورات لینوکس ۳۷۹
کتاب‌شناسی ۳۸۵
فهرست موضوعی ۳۸۹

پیشگفتار

کمبود منابع عملی و کاربردی برای درس سیستم‌عامل در محیط‌های آموزشی و دانشگاهی پیوسته احساس می‌گردد. کتاب‌های نظری بسیارند؛ اما متنی که دانشجو را با پیاده‌سازی مفاهیم درگیر نماید، پراکنده و ناکامل است. اغلب منابع دانشگاهی، فقط به یک یا در نهایت چند موضوع محدود پرداخته‌اند و در طی جستجوهای انجام‌گرفته، مرجع کاملی که همه مباحث و سرفصل‌ها را به‌طور کامل و یک‌جا پوشش دهد، یافت نشد. با مشاهده این خلا تصمیم گرفتیم منبعی گسترده و کامل در این زمینه گردآوری نماییم. این کتاب، نتیجه سال‌ها تدریس، پژوهش و کار عملی در این حوزه می‌باشد.

سرفصل‌های کتاب بر اساس مصوب وزارت علوم طراحی گردیده‌اند و با مراجع دانشگاه‌های معتبر همسو می‌باشند. در هر آزمایش، ابتدا خلاصه‌ای از مفاهیم بیان شده و سپس دستورکارهای عملی، کدهای قابل اجرا و شبیه‌سازی مفاهیم را در بر گرفته است. در پایان هر آزمایش نیز تمرین‌ها و گزارش‌کارهایی برای سنجش میزان درک دانشجو از مطالب و به چالش کشیدن دانسته‌های وی در نظر گرفته شده است. کدهای مورد استفاده در کتاب، در صفحه گیت‌هاب[1] ویژه کتاب قرار داده شده‌اند. به این ترتیب، خواننده می‌تواند به‌صورت مستقیم وارد مرحله اجرا شود.

امیدواریم این کتاب بتواند گامی هر چند کوچک در بهبود آموزش عملی مفاهیم سیستم‌عامل بردارد و مورد توجه اساتید، دانشجویان و علاقه‌مندان این حوزه قرار بگیرد. آماده شنیدن نظرات و پیشنهادات خوانندگان گرامی در راستای بهبود محتوا در چاپ‌های بعد هستیم و از آن استقبال می‌کنیم.

---

[1] https://github.com/SoltaniAkram/OSLab-Codes-UU

آزمایش ۱ : سیستم‌های لینوکسی

اهداف:

- آشنایی با مفاهیم اساسی لینوکس به‌عنوان یک سیستم‌عامل

ابزار و مفاهیم مورد نیاز:

- فایل ISO نصب یک توزیع لینوکسی (ترجیحا آخرین نسخه اوبونتو یا مینت)
- USB Flash/ DVD
- نرم‌افزارهای ایجاد فلش قابل بوت
- مدیر ماشین‌مجازی
- یک پارتیشن خالی در سیستم

محتوا:

- معرفی مفهوم نرم‌افزار آزاد
- اجزای تشکیل دهنده سیستم‌عامل گنو/لینوکس
- توزیع‌ها و کاربردهای لینوکس
- نصب لینوکس
  - Live Distribution
  - نصب مجازی و محلی

۱-۱- نرم‌افزار آزاد

در جامعه لینوکس، عبارتی به این مضمون وجود دارد: «لینوکس یک سیستم‌عامل نیست، بلکه فرهنگ آزادی است.» ریشه این ایده به «جنبش نرم‌افزار آزاد» برمی‌گردد که در سال ۱۹۸۳ توسط آقای ریچارد استالمن[1] راه‌اندازی و در ۱۹۸۵ با تاسیس «بنیاد نرم‌افزار آزاد[2]»، پایه‌های آن مستحکم شد. نماد این بنیاد، گوزن آمریکای شمالی است که گِنو[3] نام دارد.

از دیدگاه بنیاد نرم‌افزار آزاد، برای اینکه یک برنامه آزاد شناخته شود، باید چهار حق اساسی را برای کاربران خود قائل باشد. این چهار مورد، آزادی‌های صفر تا سه نامیده می‌شوند و به شرح زیر هستند:

- آزادی صفر : آزادی اجرای برنامه برای هر هدفی
- آزادی یک: آزادی مطالعه نحوه کار برنامه و تغییر آن (پیش نیاز: دسترسی به کدمنبع[4] برنامه)
- آزادی دو: آزادی تکثیر و توزیع مجدد برنامه
- آزادی سه: آزادی تقویت و بهبود برنامه و نشر مجدد آن برای همگان (پیش نیاز: دسترسی به کدمنبع برنامه)

اعطای قسمت عمده‌ای از این آزادی‌ها، با انتشار برنامه به‌صورت بازمتن محقق می‌شود، اما یک شرط نهایی نیز وجود دارد که رشد جنبش نرم‌افزار آزاد را تضمین می‌نماید: «اگر این آزادی‌ها در اختیار شما قرار گرفته است، دیگران نیز باید از آن بهره‌مند شوند.» به بیان دیگر، اگر یک برنامه آزاد را دریافت کرده و آن را تغییر داده و سپس بخواهید این

---

1 Ricahrd Stallman
2 Free Software Foundation
3 GNU
4 Source Code

نسخه تغییر یافته را با دیگران به اشتراک بگذارید، آن‌گاه موظف خواهید بود که برنامه را همچنان به شکل آزاد منتشر کنید.

در نرم‌افزارهای بسته مانند ویندوز یا فتوشاپ، قانون از تولیدکنندگان حمایت می‌کند. اگر بخواهیم که از آزادی‌های کاربران نیز پشتیبانی بشود، به یک متن حقوقی نیاز داریم. این متن حقوقی، اجازه‌نامه عمومی همگانی[5](GPL) نام دارد و اگر یک نرم‌افزار براساس مجوز GPL منتشر شده باشد، قانون موظف است حقوق پدیدآورنده را در مورد آن تضمین نماید.

۱-۲- سیستم‌عامل گنو/لینوکس

دغدغه اصلی در بنیاد نرم‌افزار آزاد این بود که خود کاربران، کنترل کامل نرم‌افزاری که از آن استفاده می‌کنند را در دست داشته باشند تا بتوانند با بررسی، بهبود و به اشتراک گذاری نرم‌افزارها، در ساختن دنیایی بهتر و آزادتر سهیم شوند. بدیهی است که این مسیر باید از پایین‌ترین لایه‌ی نرم‌افزاری یعنی سیستم‌عامل آغاز شود.

۱-۲-۱-ایجاد یک سیستم‌عامل آزاد

بهترین معماری موجود برای سیستم‌عامل در آن برهه زمانی، UNIX نام داشت که البته به‌صورت تجاری و بسته برای کامپیوترهای بزرگ ارائه می‌شد. اعضای پروژه GNU نیز نگارش سیستم‌عامل آزاد خود را بر پایه همان معماری آغاز نمودند. براساس معماری مذکور، تکمیل چهار مولفه ضروری بود:

- **هسته**[6]: بخش مرکزی سیستم‌عامل که مدیریت منابع سخت‌افزاری و ارتباط بین نرم‌افزار و سخت‌افزار را بر عهده دارد.

---

[5] General Public Licence
[6] Kernel

- **محیط توسعه نرم‌افزار:** مانند کامپایلرها و کتابخانه‌های مختلفی که برای ساخت برنامه‌ها در سیستم‌عامل، ضروری می‌باشند.
- **دستورات عمومی و روزمره:** مانند دستورات لازم برای کپی کردن فایل‌ها، برنامه‌های گرافیکی، پوسته‌ی[۷] متنی برای تفسیر و اجرای فرمان‌های کاربر، ابزارهایی برای قالب‌بندی دیسک و ... .
- **مستندات**: راهنماهای سیستم‌عامل و توضیحات متنی مرتبط.

پروژه گنو با سرعت به پیش رفت و سه مولفه‌ی آخر، با کیفیت خوبی تکمیل گشت. به‌طوری‌که، بسیاری از سیستم‌عامل‌های دیگر بویژه یونیکس، شروع به استفاده از نرم‌افزارهای آزاد تولید شده برای این سه بخش نمودند.

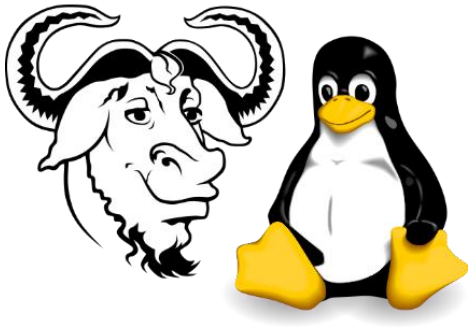
۱-۲-۲- پیدایش گنو/لینوکس

مولفه باقیمانده برای ساخت این سیستم‌عامل آزاد را شخصی به نام «لینوس توروالدز» تکمیل کرد. او دانشجوی کارشناسی علوم کامپیوتر بود و برای سرگرمی شروع به نوشتن یک هسته سیستم‌عامل به نام «لینوکس» کرد. این پروژه در سال ۱۹۹۱ تحت مجوز GPL منتشر شد و واکنشی مثبت و گسترده را برانگیخت به‌طوری‌که از آن موقع تا کنون ایشان و توسعه‌دهندگان بسیاری به تکمیل و نگهداری این هسته مشغول بوده‌اند و مدیریت این پروژه همچنان بخش عمده‌ای از فعالیت حرفه‌ای وی را تشکیل می‌دهد. در نهایت، این هسته با سه مولفه‌ی دیگر همراه شد و «سیستم‌عامل گنو/لینوکس» به وجود آمد.

نکته: هسته‌های دیگری نیز برای تکمیل این پروژه به وجود آمدند که برخی از آن‌ها مانند FreeBSD و Minix3 در حوزه خود موفق نیز بوده‌اند.

---

[7] Shell



تصویر ۱-۱- نماد لینوکس (Tux) در راست و پروژه گنو (GNU) در چپ

۱-۳-توزیع‌ها

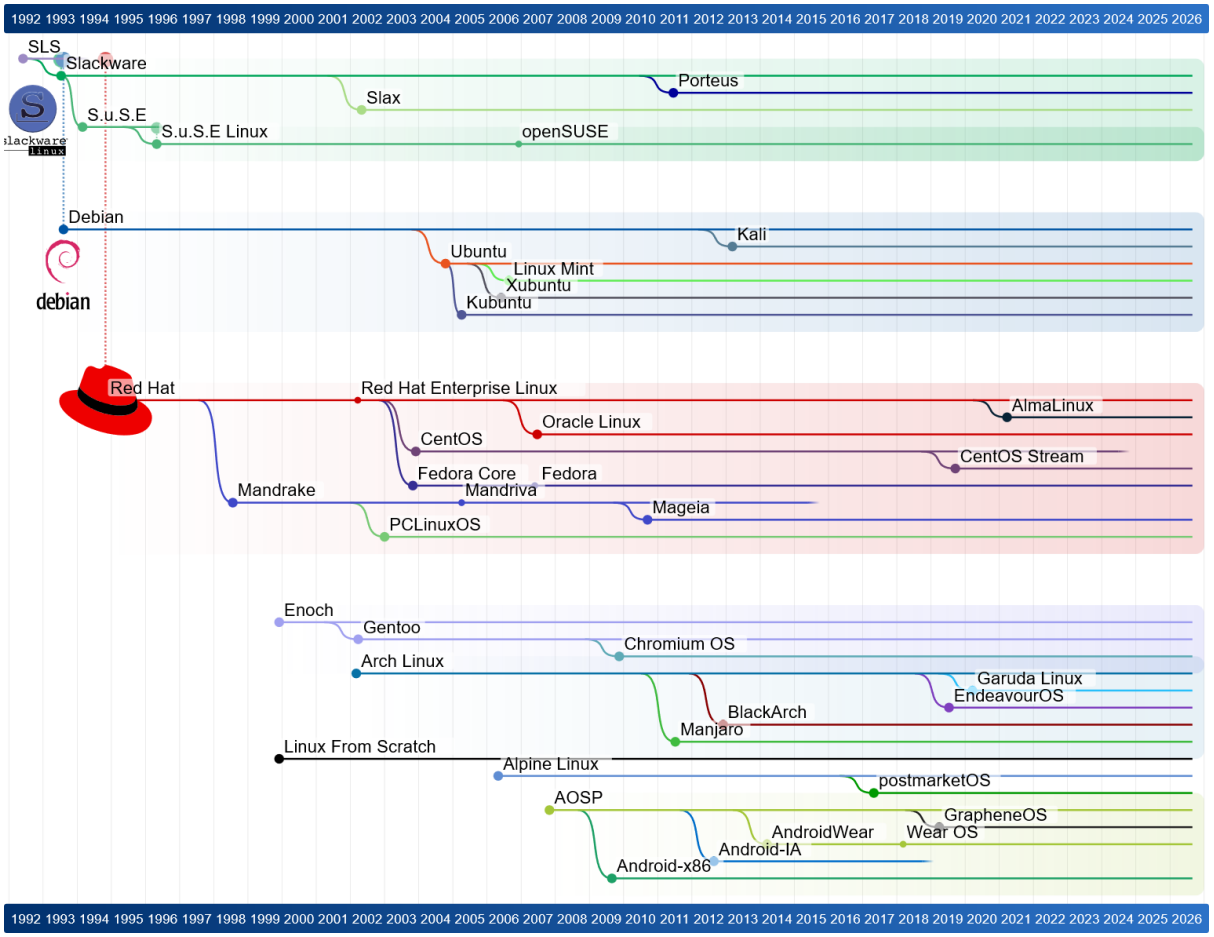
برای بوت کردن یک کامپیوتر با گنو/لینوکس در سال ۱۹۹۲ لازم بود کاربر، هسته را با یکی از ابزارهای گنو به فایل اجرایی تبدیل کرده و آن را در یک موقعیت خاص روی هارد بنویسد. سپس باید فلگ‌هایی را برای بوت شدن تنظیم می‌کرد و هر یک از برنامه‌های کاربردی و کتابخانه‌های نرم‌افزاری مورد نیاز را جداگانه کامپایل کرده و براساس استاندارد یونیکس روی دیسک می‌نوشت. درنهایت اگر همه چیز درست انجام شده بود، سیستم با گنو/لینوکس بالا می‌آمد. این فرآیند حتی برای افراد حرفه‌ای نیز یک پروژه پرچالش به شمار می‌آمد. بنابراین، افرادی انجام این مراحل پیچیده را بر عهده گرفته و محصول نهایی را به شکل یک مجموعه دیسک با قابلیت نصب عرضه کردند. این مجموعه‌ها، توزیع‌های لینوکس[8] نامیده شدند. یکی از قدیمی‌ترین توزیع‌ها که هنوز فعال یا به اصطلاح «زنده» است، Slackware نام دارد.

در این توزیع‌ها علاوه بر هسته لینوکس و ابزارهای گنو، بسیاری از برنامه‌های آزاد دیگر نیز عرضه می‌شدند. به‌عنوان مثال ممکن بود یک محیط‌کار گرافیکی، مجموعه اداری LibreOffice، یک برنامه گفتگوی برخط و مرورگر فایرفاکس، درون یک توزیع قرار بگیرند. در این حالت کاربر پس از نصب لینوکس، به این ابزارها و برنامه‌ها نیز دسترسی داشت و

---

[8] Linux Distros

دیگر نیازی به نصب جداگانه برای آن‌ها نبود. علاوه بر این گاهی توسعه‌دهندگان، برنامه‌های مخصوص به خودشان را هم به توزیع اضافه کرده و یک توزیع ویژه برای یک کاربرد تخصصی ارائه می‌کردند.



تصویر ۱-۲- تایم‌لاین پیدایش و تکامل توزیع‌های لینوکسی

از جمله شناخته‌شده‌ترین توزیع‌ها، ردهت[9] و دبیان[10] می‌باشند که بسیاری از توزیع‌های پرکاربرد دیگر، از آن‌ها منشعب شده‌اند. برای مثال اوبونتو از دبیان نشات گرفته و فدورا[11]، شاخه‌ای از ردهت می‌باشد. در تصویر ۱-۲ برخی از توزیع‌ها و انشعاب‌هایشان را می‌بینید.

۱-۴-کاربردهای لینوکس

سایت دیستروواچ[12] مرجعی برای بررسی تمام توزیع‌های لینوکس است و توزیع‌های فعال لینوکس را زیر نظر دارد. هر یک از این توزیع‌ها با هدفی متفاوت و برای کاربردی خاص ساخته شده‌اند. برخی از این کاربردها عبارتند از:

- **لینوکس سرور**
  - وب سرورها
  - سیستم‌های مخابراتی
  - سازمان امنیت ملی آمریکا، پایگاه‌های پرتاب سفینه‌های فضایی، سیستم‌های کنترل ترافیک و کنترل پرواز
  - سرور شرکت‌های انیمیشن سازی و جلوه‌های ویژه
  - کامپیوترهای بزرگ[13]

---

[9] RedHat
[10] Debian
[11] Fedora
[12] https://www.distrowatch.com/
[13] Mainframes

- **ابررایانه‌ها**

سریعترین ابررایانه[۱۴] در حال حاضر - که در سال ۲۰۲۶ قرار داریم - با نام LineShine شناخته می‌شود. این ابررایانه از سیستم‌عامل Kylin که به‌عنوان یک نسخه سازمانی از سیستم عامل لینوکس توسعه داده شده است، استفاده می‌کند. اغلب ابررایانه‌های جهان نیز از توزیع‌های تجاری لینوکس مانند SUSE یا Red Hat استفاده می‌نمایند.

- **لینوکس دسکتاپ**

توزیع‌های لینوکسی دسکتاپ نیز وجود دارند اما در طی سال‌های گذشته همواره سهم بسیار کوچکی را به خود اختصاص می‌دادند، هر چند این سهم در سال جاری به حدود ۷.۵ درصد رسیده است. اما چون این اطلاعات معمولا از ترافیک وب استخراج می‌شوند و با توجه به اینکه ربات‌ها و ترافیک خودکار مانند خزنده‌های وب و هوش مصنوعی اغلب روی سرورهای لینوکس اجرا می‌شوند؛ باید از بخشی از این سهم چشم‌پوشی نمود.

- **لینوکس توکار**

سیستم‌های توکار[۱۵] سیستم‌هایی هستند که در ظاهر یک کامپیوتر نیستند اما در باطن یک سیستم‌عامل آن‌ها را اداره می‌کند. مانند خودروها و خانه‌های هوشمندی که از هسته لینوکس استفاده می‌کنند و یا شتاب دهنده‌های ذرات. این سیستم‌ها با لینوکس راه‌اندازی می‌شوند زیرا به دلیل آزاد و بازمتن[۱۶] بودن، امکان حذف بخش‌های غیرضروری و کوچک‌سازی آن تا حد ممکن وجود دارد.

---

[14] Supercomputer
[15] Embeded
[16] Open Source

۱-۵- نصب لینوکس

در این قسمت با مراحل نصب لینوکس، هم بر روی ماشین‌مجازی و هم به‌صورت محلی به‌عنوان سیستم‌عامل دوم، آشنا خواهیم شد.

۱-۵-۱- سیستم‌عامل زنده

لینوکس یک سیستم زنده[17] است؛ به این معنی که رسانه نصب لینوکس می‌تواند یک کامپیوتر را به شکل کامل بوت کند تا بتوان از لینوکس به شکل زنده و بدون نصب استفاده کرد. به این ترتیب، کاربر فرصت بررسی یک توزیع خاص را بدون انجام مراحل نصب دارد. بدین‌منظور، کافی است پس از بوت کردن سیستم با رسانه نصب، گزینه Live یا Try را انتخاب کنید و یا از ابتدا نسخه لایو را دانلود و سیستم را با آن بوت نمایید.

سیستم‌عامل‌هایی که قابلیت اجرا به‌صورت «زنده» را دارند می‌توانند بدون انجام فرآیند نصب، از روی دی‌وی‌دی یا فلش‌درایو اجرا شوند. در این حالت، تمام اطلاعات برای بارگذاری سیستم‌عامل، بر روی حافظه اصلی کامپیوتر قرار می‌گیرد و هیچ چیزی روی هارد نوشته نمی‌شود. بنابراین به فایل‌ها یا سیستم‌عاملی که از قبل بر روی هارد نصب شده است، آسیبی نخواهد رسید. پس از اتمام کار با لینوکس زنده، می‌توان سیستم را راه‌اندازی مجدد کرده و فلش‌درایو یا دیسک جانبی را خارج نمود. کامپیوتر دوباره به همان حالت قبلی باز می‌گردد و با همان سیستم‌عامل پیش‌فرض خود بارگذاری خواهد شد. ویندوز نیز در نسخه ۸ Enterprise قابلیت اجرای زنده را اضافه کرده بود اما از ویندوز ۱۰ به بعد آن را حذف کرد.

از جمله مزایای استفاده از یک توزیع زنده می‌توان به موارد زیر اشاره نمود:

- دسترسی به یک سیستم‌عامل همراه که در هر زمان و مکان قابل استفاده است

---

[17] Live Distribution

- فراهم آوردن محیطی امن به‌منظور انجام امور حساس مانند عملیات بانکی بدون نگرانی از بدافزارها و یا برجای گذاشتن ردی روی سیستم
- امکان ارزیابی توزیع‌های مختلف بدون نیاز به نصب
- عیب‌یابی ویندوز و بازیابی فایل‌ها در صورت از کار افتادن سیستم
- امکان احیای سیستم‌هایی که سخت‌افزار قدیمی دارند

۱-۵-۲- نصب اوبونتو بر روی هارد دیسک به‌صورت راه‌اندازی دوگانه

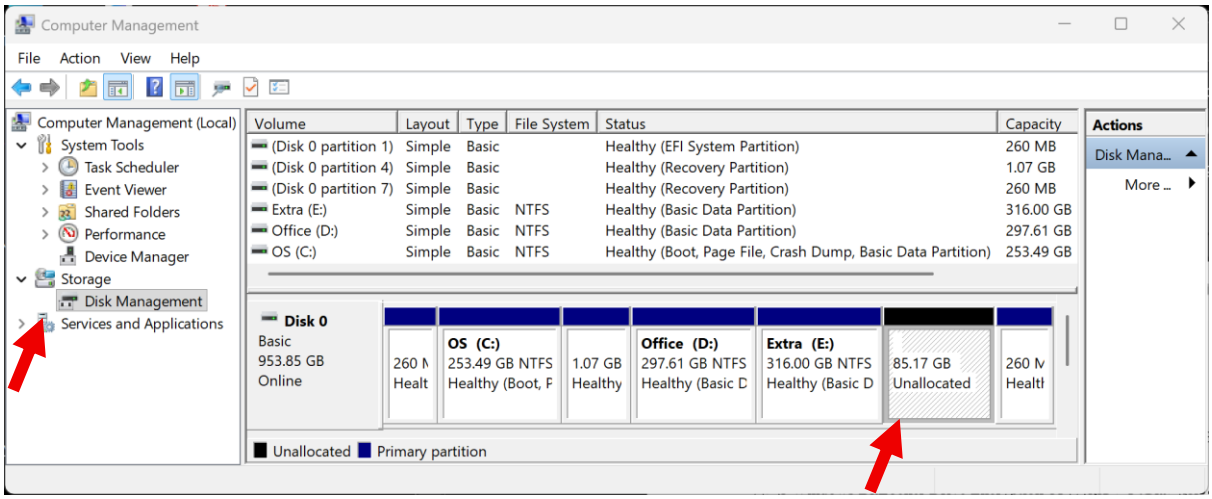
بسیاری از کاربران، روش نصب مستقیم[۱۸] لینوکس بر روی کامپیوتری که دارای ویندوز نیز هست را انتخاب کرده و سیستم را به‌صورت راه‌اندازی دوگانه[۱۹] مورد استفاده قرار می‌دهند. در این روش، برای سوئیچ بین سیستم‌عامل‌ها، باید سیستم را مجددا راه‌اندازی نمایید و در منوی بوت، سیستم دلخواهتان را انتخاب کنید. برای نصب همزمان لینوکس و ویندوز، لازم است هارد دیسک خود را پارتیشن‌بندی کنید تا فضای جداگانه‌ای برای هر یک از سیستم‌های عامل ایجاد شود. در اینجا فرض می‌کنیم ویندوز از قبل روی سیستم شما نصب شده و اکنون می‌خواهید سیستم‌عامل دوم را نصب کنید.

ابتدا وارد ویندوز شده، بر روی آیکون MyComputer راست‌کلیک کرده و گزینه Manage را انتخاب نمایید. از پانل سمت چپ در پنجره‌ی باز شده، وارد بخش Disk Management شده و یکی از درایوهای خود را به دو قسمت تقسیم کنید. یعنی با استفاده از گزینه Shrink پارتیشن مربوطه را کوچک کرده و با فضای آزاد شده، یک پارتیشن جدید بسازید. پارتیشن جدید باید حداقل ۲۵ گیگابایت فضا داشته باشد - البته این مقدار به توزیعی که قصد نصب آن را دارید بستگی دارد. حال با راست‌کلیک روی درایو و انتخاب گزینه Delete، درایو را پاک کنید تا همانند تصویر ۱-۳،

---

[18] Native
[19] Dual Boot

آن بخش با عنوان Unallocated نمایش داده شود. به این ترتیب هنگام نصب، تنها از این درایو استفاده خواهیم کرد و محتوای بقیه درایوها به خطر نمی‌افتند.



تصویر ۱-۳- مجزا کردن یک پارتیشن برای نصب لینوکس

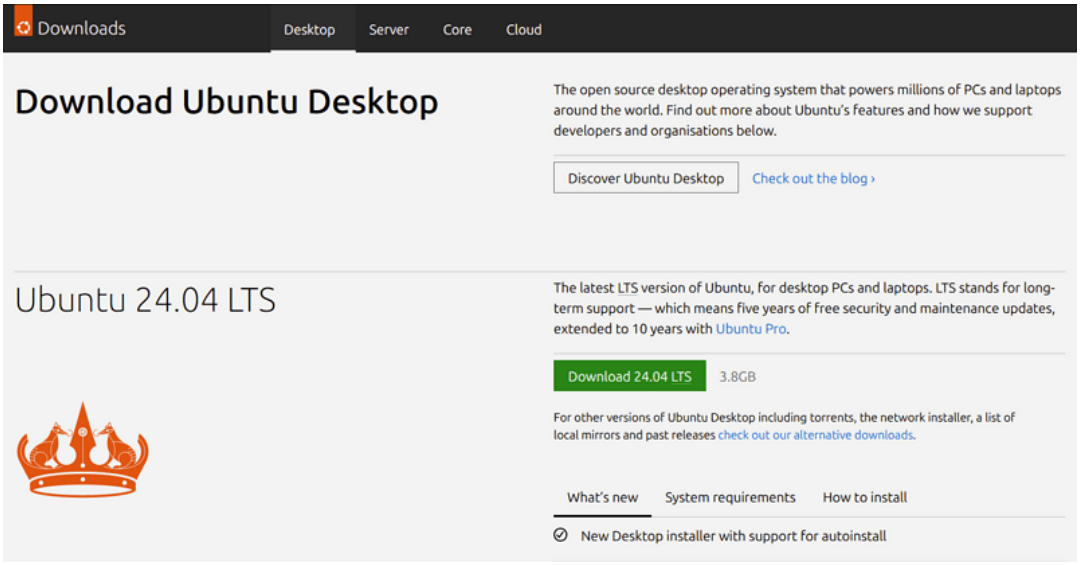
نکته: اگر ویندوز شما از Bitlocker برای رمزگذاری داده‌ها استفاده می‌کند، حتما قبل از نصب محلی لینوکس یا استفاده از توزیع‌های زنده، کلیدهای رمزگذاری BitLocker خود را بازیابی و ذخیره نمایید.

توجه: برای نصب روی ماشین‌مجازی به بخش ۱-۵-۳ مراجعه کنید.

مراحل نصب

۱. دانلود فایل ISO نصب

از وب‌سایت رسمی اوبونتو[۲۰] آخرین نسخه Ubuntu Desktop را دانلود نمایید. معماری مناسب (۳۲ یا ۶۴ بیتی) را بر اساس نوع سیستم خود انتخاب کنید. برای دانلود سایر توزیع‌های محبوب می‌توانید به وب‌سایت لینوکس[۲۱] مراجعه نمایید. همان‌طور که در تصویر ۱-۴ مشاهده می‌کنید، در این راهنما از نسخه اوبونتو 24.04 LTS (منتشر شده در آوریل ۲۰۲۴) استفاده شده است.



تصویر ۱-۴- دانلود اوبونتو

---

[20] https://ubuntu.com/download/desktop

[21] https://linux.org/pages/download

۲. ایجاد فلش درایو قابل بوت

برای ایجاد یک فلش درایو قابل بوت[۲۲]، نرم‌افزارهای مختلفی وجود دارد. از جمله Rufus, Balena Etcher, UNetBootin, UltraIso, Nero و غیره. اینجا از نرم‌افزار Rufus[23] استفاده خواهیم کرد، زیرا علاوه بر سادگی، قدرتمند است و نسخه پرتابل نیز دارد. اما اگر به‌دنبال نرافزاری هستید که روی هر سه سیستم‌عامل لینوکس، ویندوز و مک اجرا شود، می‌توانید از Balena Etcher[24] استفاده کنید. برای نصب یا استفاده از هر یک از این نرم‌افزارها، ابتدا نسخه‌ای را انتخاب کنید که با سیستم‌عامل فعلی شما مطابقت دارد، و سپس آن را دانلود و نصب نمایید. اکثر این برنامه‌ها نحوه‌ی کار مشابهی دارند. برای آشنایی با نحوه کار با Rufus تصویر ۱-۵ را ملاحظه نمایید. همان‌طور که در تصویر نیز مشخص شده است، ابتدا فلش USB خود را مشخص کرده و سپس فایل ISO دانلود شده را انتخاب کنید و در انتها روی گزینه Start کلیک نمایید تا نوار سبز رنگ در پایین کادر برنامه پر شود.

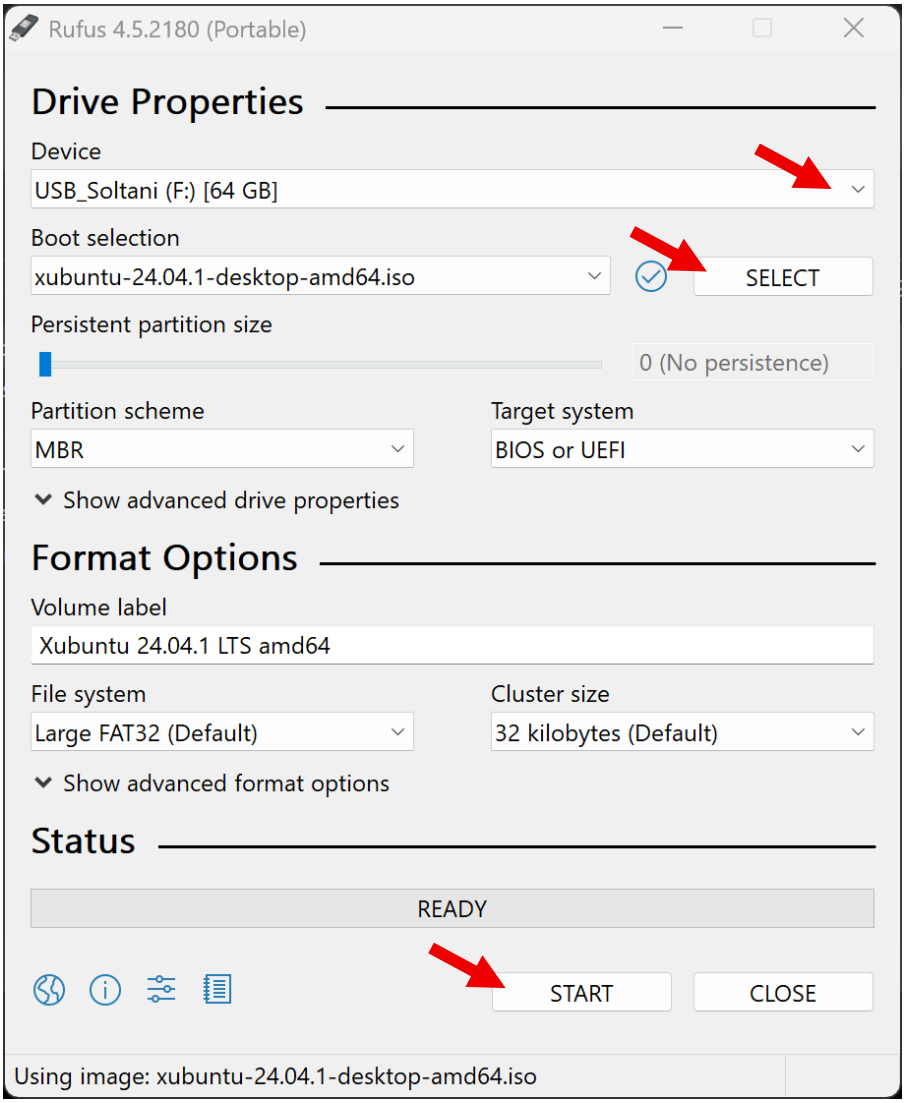
۳. بوت کردن از درایو USB

فلش را به سیستمی که می‌خواهید بر روی آن لینوکس را نصب کنید، متصل کرده و سیستم را راه‌اندازی مجدد نمایید. سیستم باید رسانه نصب را به طور خودکار تشخیص دهد. اگر مشکلی پیش آمد و آن را تشخیص نداد، F12 را در هنگام راه‌اندازی نگه دارید و دستگاه USB را از منوی بوت سیستم، انتخاب کنید.

نکته : کلید F12 رایج‌ترین کلید برای نمایش منوی بوت است، اما کلیدهای Escape، Del، F2 و F10 نیز ممکن است در لپ‌تاپ‌ها یا سیستم‌های مختلف برای این کار معرفی شده باشد.

---

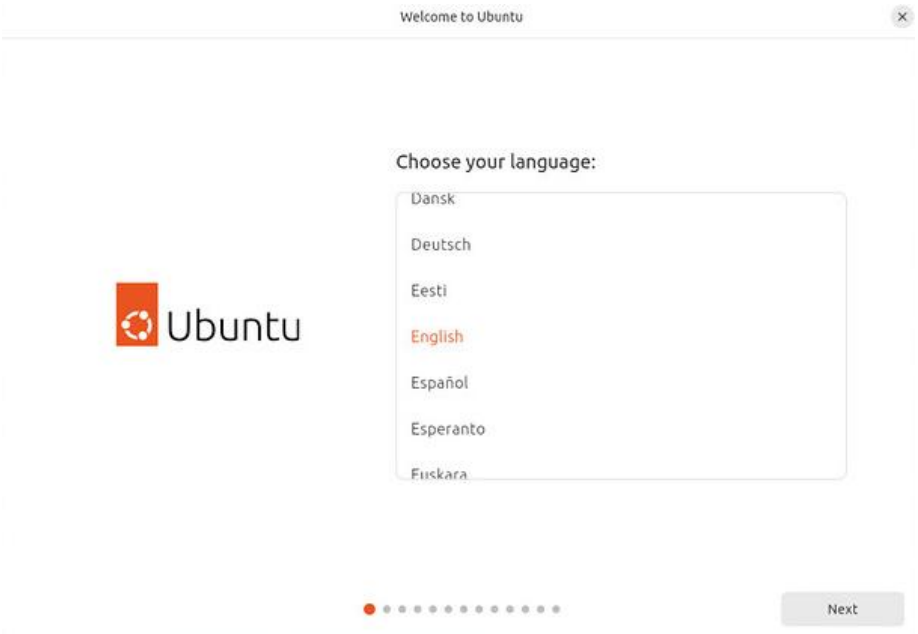
[22] Bootable Flash
[23] https://rufus.ie
[24] https://www.balena.io/etcher



تصویر ۱-۵- محیط نرم‌افزار Rufus

٤. تنظیمات نصب

هنگامی که نصاب اجرا شد، در اولین مرحله باید زبان خود را انتخاب کنید. همان‌طور که در تصویر ۱-۶ ملاحظه می‌نمایید، زبان پیش‌فرض، انگلیسی می‌باشد. می‌توانید کلید next را زده و بدون تغییر به مرحله بعدی بروید.



تصویر ۱-۶- انتخاب زبان در نصاب اوبونتو

در صفحه دوم، تنظیمات دسترس‌پذیری را مشاهده خواهید کرد. درصورتی‌که هنگام استفاده از کامپیوتر، نیاز به تسهیلات خاصی ندارید، در این مرحله نیز فقط بر روی next کلیک کنید.

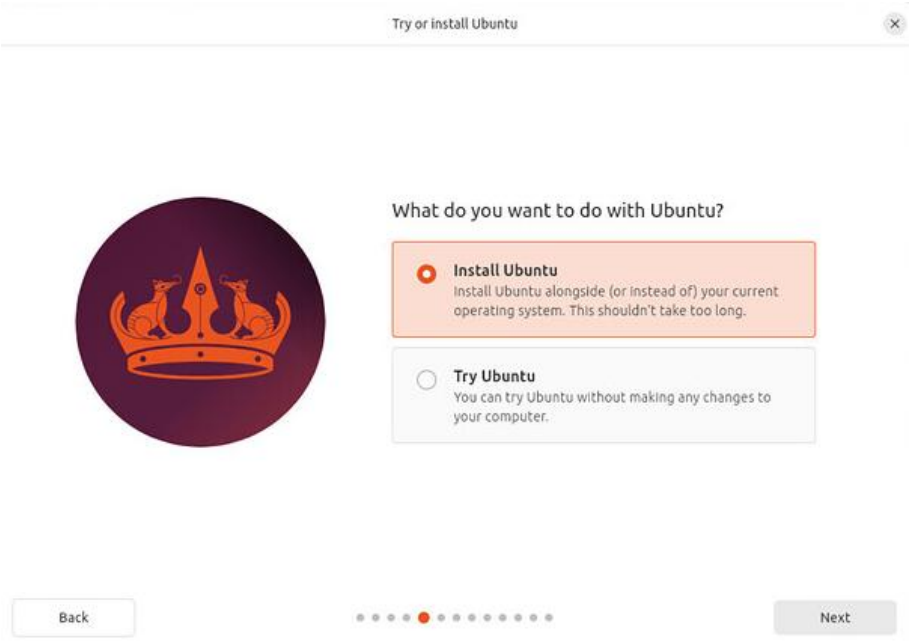
در صفحه سوم، چیدمان صفحه‌کلید[۲۵] را انتخاب خواهید کرد. این چیدمان، پس از اتمام نصب و از طریق بخش تنظیمات نیز قابل تغییر می‌باشد. بنابراین برای سادگی می‌توانید از این مرحله بدون اعمال هیچ تغییری عبور کنید.

در صفحه چهارم، می‌توانید با اتصال به شبکه، به سیستم‌عامل این اجازه را بدهید که به‌روزرسانی‌ها و درایورهایی مانند درایورهای گرافیک NVIDIA را در حین نصب، دانلود نماید. اما اگر نمی‌خواهید این اتفاق بیافتد، پیش از رفتن به مرحله بعدی، گزینه «عدم اتصال به اینترنت» را انتخاب کنید. این تنظیمات نیز پس از نصب و از داخل لینوکس، قابل تغییر می‌باشند.

---

[25] Keyboard Layout

در صفحه پنجم که در تصویر ۱-۷ نمایش داده شده است، امکان انتخاب بین دو گزینه «آزمودن» یا «نصب» لینوکس اوبونتو را دارید. برای ادامه، روی Install Ubuntu کلیک کنید. اگر روی Try Ubuntu کلیک کنید، می‌توانید نسخه‌ی زنده اوبونتو را بدون اینکه تغییر یا نصبی در سیستم شما انجام شود، امتحان نمایید.



تصویر ۱-۷- انتخاب بین نصب یا مشاهده نسخه زنده اوبونتو

نکته: برخی از کامپیوترها از Intel RST (فناوری ذخیره سازی سریع) استفاده می‌کنند که توسط اوبونتو پشتیبانی نمی‌شود. اگر سیستم شما نیز از این فناوری استفاده می‌کند، نمی‌توانید بدون غیر فعال کردن RST در منوی بایوس سیستم خود، از این مرحله جلوتر بروید.

در صفحه ششم، از شما خواسته می‌شود که از بین نصب تعاملی[۲۶] و نصب خودکار[۲۷] یکی را انتخاب کنید. گزینه تعاملی، مسیر استانداردتری است. از گزینه نصب خودکار نیز برای وارد کردن یک فایل پیکربندی[۲۸] از یک وب سرور، به جهت استانداردسازی نصب‌های متعدد و سفارشی‌سازی‌های بیشتر، استفاده می‌شود. در این آزمایش، نصب را با مسیر تعاملی پیش می‌بریم.

در مرحله هفتم، برای نصب نرم‌افزارها باید بین گزینه‌های «مجموعه پیش‌فرض[۲۹]» و «مجموعه گسترده[۳۰]» یکی را انتخاب کنید. بهتر است گزینه پیش‌فرض را برای شروع انتخاب کنید زیرا می‌توانید پس از اتمام نصب، با استفاده از «مرکز نرم‌افزار»[۳۱] موارد بیشتری را به سیستم خود اضافه و نصب نمایید.

در صفحه هشتم، از شما خواسته می‌شود که نرم‌افزارهای Third Party را نصب کنید که می‌توانند پشتیبانی و عملکرد سیستم را بهبود بخشند؛ مانند درایورهای گرافیکی یا شبکه و همچنین نرم‌افزارهای پشتیبانی از انواع فرمت رسانه‌ها. توصیه می‌شود تمام گزینه‌های موجود را تیک زده و برای نصب انتخاب نمایید.

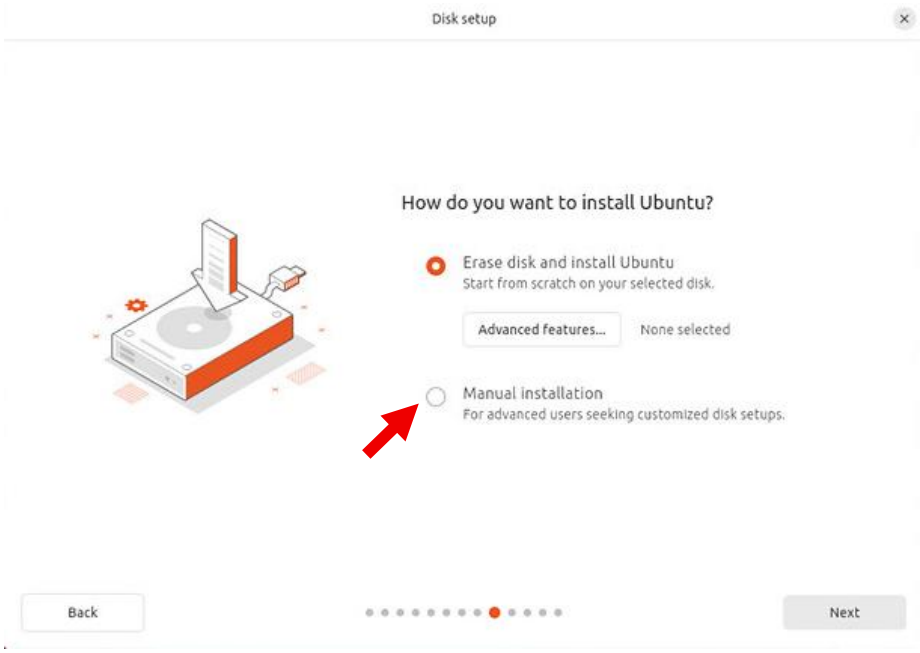
۵. انتخاب محل نصب

در این بخش که در تصویر ۱-۸ نمایش داده شده است، به دقت بیشتری نیاز دارید تا به اشتباه فایل‌ها یا سیستم‌عامل فعلی خود را پاک نکنید. درصورتی‌که محل نصب ابونتو را مطابق روشی که در ابتدای دستورکار آمده آماده کرده‌اید، گزینه Manual Installation را انتخاب کرده و مراحل بعدی را مطابق با دستورکار ادامه بدهید تا اوبونتو در کنار ویندوز، روی سیستم شما نصب شود.

---

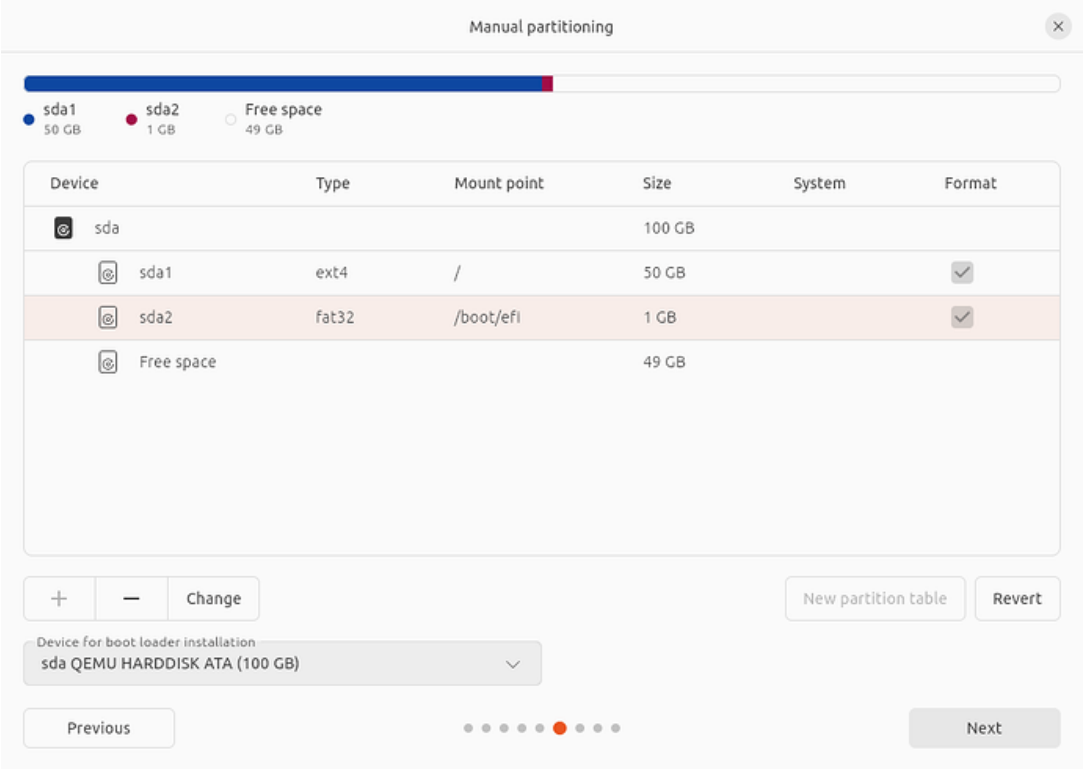
[26] Interactive Installation
[27] Automated Installation
[28] Configuration
[29] Default Selection
[30] Extended Selection
[31] App Center

هشدار: درصورتی‌که گزینه Erase Disk را بزنید کل هارد پاک خواهد شد. مگر اینکه سیستم دارای چند هارد دیسک مجزا باشد که دراین‌صورت نیز باید توجه کنید که دیسک صحیح و موردنظر را انتخاب کرده باشید. اما اگر در حال نصب بر روی ماشین‌مجازی هستید، انتخاب این گزینه هیچ مشکلی ایجاد نخواهد کرد.



تصویر ۱-۸- انتخاب نوع نصب

در صفحه بعدی، همان‌طور که در تصویر ۱-۹ مشاهده می‌کنید به قسمت Free space رفته، روی علامت + کلیک کرده و پارتیشن‌های ضروری را به شرح زیر ایجاد می‌نماییم. البته درصورتی‌که کل یک دیسک را به این کار اختصاص داده‌اید، ابتدا باید کلید New Partition Table را از پایین سمت چپ پنجره کلیک کنید تا این جدول روی دیسک ایجاد گردد و سپس با کمک کلید + پارتیشن‌های مورد نیاز را ایجاد کنید. پارتیشن‌های اساسی عبارتند از root، home، swap، EFI و boot که در ادامه به توضیح هر یک می‌پردازیم.



تصویر ۱-۹- پارتیشن‌بندی به‌صورت دستی

نکته : در این مرحله هرگاه کاری را به اشتباه انجام دادید، روی گزینه Revert کلیک کنید تا تنظیمات به حالت اولیه بازگردند. به ازای هر پارتیشن، یک بار کلید + را زده و مطابق تصویر ۱-۱۰ و توضیحاتی که در ادامه آمده، پیش بروید.

پس از کلیک روی علامت +، یک پنجره جدید شامل ۵ بخش باز خواهد شد. این بخش‌ها به ترتیب از بالا به پایین عبارتند از:

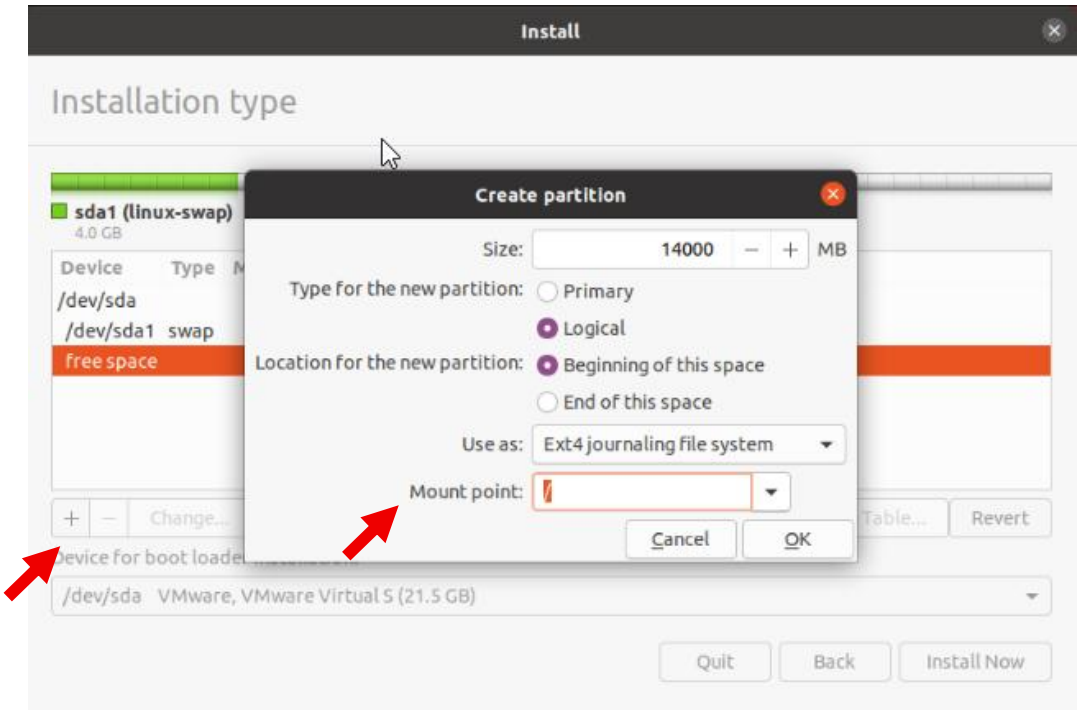
Size : اندازه پارتیشن بر حسب مگا بایت. یعنی پارتیشنی که می‌خواهید بسازید چه اندازه‌ای خواهد داشت.

Type : نوع پارتیشن. یعنی پارتیشن جدید از نوع اصلی است یا مجازی.

Location : محل قرارگیری پارتیشن بر روی دیسک. یعنی تخصیص فضا از ابتدای بخش خالی دیسک صورت بگیرد یا از انتهای آن.

Use as : نوع فایل‌سیستم پارتیشن؛ که در لینوکس از فرمت ext4 استفاده می‌شود.

Mount Point : دایرکتوری‌ای که به‌عنوان نقطه اتصال پارتیشن به سیستم معرفی خواهد شد.



تصویر ۱-۱۰- ایجاد پارتیشن‌های ضروری

در تصویر ۱-۱۰ تنظیمات مربوط به ایجاد پارتیشن ریشه را مشاهده می‌کنید. به این پارتیشن، بیشترین حجم ممکن را تخصیص بدهید. برای پارتیشن boot/ که مختص تنظیمات بوت‌لودر است، اندازه را روی ۱ گیگابایت یا بیشتر تنظیم کرده و در بخش Mount point، عبارت boot/ را از لیست انتخاب نمایید. دو پارتیشن «دایرکتوری خانگی[۳۲]» و «فضای

[32] Home

مبادله[۳۳]»، اختیاری هستند. اما در صورت تمایل، پس از انتخاب سایز دلخواه و نقطه اتصال مناسب می‌توانید آن‌ها را نیز ایجاد نمایید.

در سیستم‌های مدرن بسته به نوع میان‌افزار[۳۴] سیستم شما، نصاب ممکن است به پارتیشن دیگری نیز نیاز داشته باشد. اگر سیستم شما از نوع UEFI است، باید یک EFI System Partition ایجاد نمایید. برای این کار، مجددا free space را انتخاب کرده و کلید + را کلیک کنید. اندازه‌ی این پارتیشن را بین ۱۰۰ تا ۵۰۰ مگابایت در نظر بگیرید و سپس در بخش Use as گزینه‌ی EFI System Partition را انتخاب کنید. اما اگر سیستم شما Legacy BIOS با پارتیشن‌بندی GPT است، به جای آن باید یک Reserved BIOS boot area با اندازه‌ی ۱ تا ۲ مگابایت ایجاد کنید. بدین‌منظور کافی است در بخش Use as گزینه‌ی Reserved BIOS boot area را انتخاب نمایید.

نکته: در سیستم‌های Legacy BIOS با پارتیشن‌بندی MBR، نیازی به این پارتیشن‌ها نیست.

پس از ساخت تمام پارتیشن‌های ضروری، بر روی گزینه Install now کلیک کرده و در پنجره باز شده گزینه Continue را انتخاب نمایید. دقت کنید که پس از تایید نهایی، تغییرات روی دیسک اعمال خواهد شد و معمولا امکان بازگشت وجود ندارد.

دریافت هشدار فعال بودن Windows BitLocker

اگر سیستم‌عامل فعلی کامپیوتر شما ویندوز است، به احتمال زیاد Windows BitLocker Drive Encryption در آن فعال می‌باشد. رمزگذاری درایو BitLocker یک ویژگی محافظت از داده‌ها در سیستم‌عامل ویندوز است و هنگامی که فعال می‌شود، محتویات هارد دیسک‌ها را در ویندوز رمزگذاری می‌کند و به این ترتیب، داده‌ها بدون کلید رمزگشایی صحیح، غیرقابل دسترسی خواهند بود. در این حالت، اوبونتو نیز قادر نخواهد بود که اطلاعات درایو مورد نیاز خود را

---

[33] Swap Area
[34] Firmware

برای نصب امن لینوکس در کنار ویندوز، جمع‌آوری کند. بنابراین قبل از راه‌اندازی مجدد، از طرف نصاب اوبونتو یک درخواست برای غیرفعال کردن BitLocker دریافت خواهید کرد. در این مرحله باید از نصاب خارج شده و سیستم را راه‌اندازی مجدد نمایید. سپس وارد ویندوز شده و از مسیر زیر به بخش تنظیمات مربوطه مراجعه کرده و گزینه غیرفعال کردن BitLocker را انتخاب نمایید.

```
Control Panel > System and Security > BitLocker Drive Encryption
```

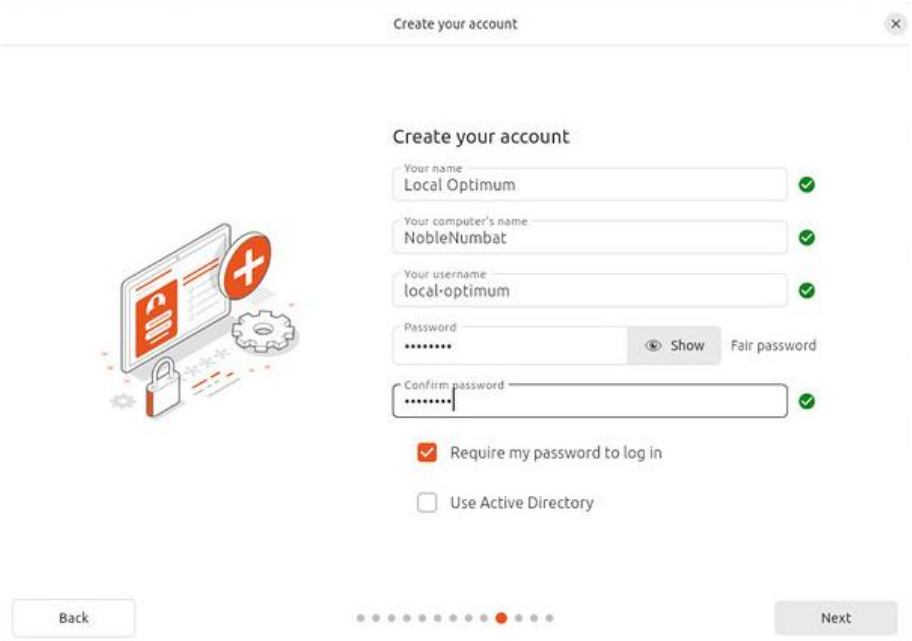
ممکن است این فرآیند کمی زمان‌بر باشد. اما پس از آن، می‌توانید با راه‌اندازی مجدد سیستم، دوباره نصاب اوبونتو را بارگذاری کرده و مراحل نصب را بدون مشکل، تکمیل نمایید.

نکته ۱: در مواردی که BitLocker فعال است اما هنوز رمزگذاری انجام نشده، لازم است ابتدا BitLocker را روشن و سپس دوباره خاموش کنید.

نکته ۲: در صورت پاک کردن کامل ویندوز یا استفاده از یک درایو جداگانه و رمزگذاری نشده برای نصب اوبونتو، نیازی به غیرفعال کردن BitLocker نیست.

۶. تنظیم اطلاعات ورود

در این بخش از شما خواسته می‌شود نام خود و کامپیوترتان را وارد کنید. همچنین یک نام کاربری انتخاب کرده و یک رمزعبور قوی ایجاد نمایید. به‌علاوه می‌توانید انتخاب کنید که هنگام ورود به سیستم، از کاربر رمزعبور درخواست بشود یا نه. اگر در حین سفر یا محیط‌های عمومی از دستگاه خود استفاده می‌کنید، توصیه می‌شود گزینه "Require my password to log in" را تیک بزنید.



تصویر ۱-۱۱- انتخاب نام کاربری و رمزعبور

در صفحه بعدی، لازم است منطقه جغرافیایی و زمانی خود را بر روی نقشه مشخص کرده و روی Continue کلیک نمایید. اگر به اینترنت متصل باشید، این اطلاعات به طور خودکار شناسایی می‌شوند.

۷. شروع نصب

در این مرحله با کلیک بر روی Next، خلاصه‌ای از پیکربندی نصب برای شما نمایش داده خواهد شد تا قبل از شروع نصب، تنظیمات خود را بررسی و تایید نمایید.

اگر در ابتدای فرآیند نصب، گزینه‌ی «پیکربندی نصب خودکار» را انتخاب کرده بودید، بلافاصله به این صفحه هدایت می‌شدید. اطلاعات این صفحه را بررسی کرده و در صورت تایید، بر روی install کلیک نمایید تا فرآیند نصب اوبونتو آغاز گردد.

۸. تکمیل فرآیند نصب

فرآیند نهایی نصب ممکن است مقداری زمان‌بر باشد. پس از اتمام نصب، پیام راه‌اندازی مجدد برای شما نمایش داده خواهد شد. گزینه Restart Now را انتخاب کنید. پس از راه‌اندازی مجدد و قبل از ورود به سیستم، از شما خواسته می‌شود که فلش یا هر رسانه نصبی که در سیستم قرار داده بودید را از دستگاه جدا کرده و پس از آن کلید ENTER را فشار دهید.



تصویر ۱-۱۲- منوی بوت پس از نصب اوبونتو

از این پس هر بار که کامپیوتر خود را روشن کنید، سیستم‌عامل‌های نصب شده روی سیستم توسط بوت لودر GRUB برایتان نمایش داده می‌شوند و می‌توانید انتخاب کنید که وارد ابونتو یا ویندوز شوید.

اوبونتو را انتخاب کنید تا صفحه ورود به سیستم برایتان نمایش داده شود. با وارد کردن نام کاربری و رمزعبور صحیح، وارد دسکتاپ اوبونتو خواهید شد. یک صفحه خوش‌آمدگویی برایتان نمایش داده می‌شود که می‌تواند در تنظیم برخی از گزینه‌های راه‌اندازی کمک کرده و شما را با محیط آشنا نماید. به‌عنوان مثال، می‌توانید تعیین کنید که تصمیم به ارسال اطلاعات سیستم خود به Canonical برای کمک به بهبود اوبونتو را دارید یا خیر. و همچنین شما را با «مرکز نرم‌افزار» برای دانلود برنامه‌های کاربردی بیشتر از «مخازن نرم‌افزاری اوبونتو[۳۵]» آشنا می‌کند.

---

[35] Ubuntu Software

۱-۵-۳- نصب اوبونتو در ماشین‌مجازی

۱. نصب مدیر ماشین‌مجازی

درصورتی‌که از قبل چنین نرم‌افزاری را بر روی سیستم خود ندارید، ابتدا آن را دانلود و نصب نمایید. ممکن است لازم باشد پس از اتمام نصب، سیستم خود را راه‌اندازی مجدد کنید. در این کتاب از VMware Workstation استفاده شده است. شما می‌توانید از Oracle Virtualbox، KVM یا هر برنامه مجازی‌سازی دیگر که با سیستم‌عامل میزبان و نیازهایتان هم‌خوانی دارد، استفاده نمایید.

۲. دانلود فایل ISO نصب لینوکس

از وب‌سایت[۳۶] رسمی اوبونتو، آخرین نسخه Desktop Ubuntu را دانلود کنید. معماری مناسب (۳۲ یا ۶۴ بیتی) را بر اساس نوع سیستم خود انتخاب نمایید. توجه داشته باشید که در نصب مجازی بر خلاف نصب محلی، نیازی به ایجاد فلش قابل بوت یا رسانه نصب نداریم. بلکه عملیات نصب به شکل مستقیم از فایل ISO انجام می‌گیرد.

۳. ایجاد یک ماشین‌مجازی جدید در VMware Workstation

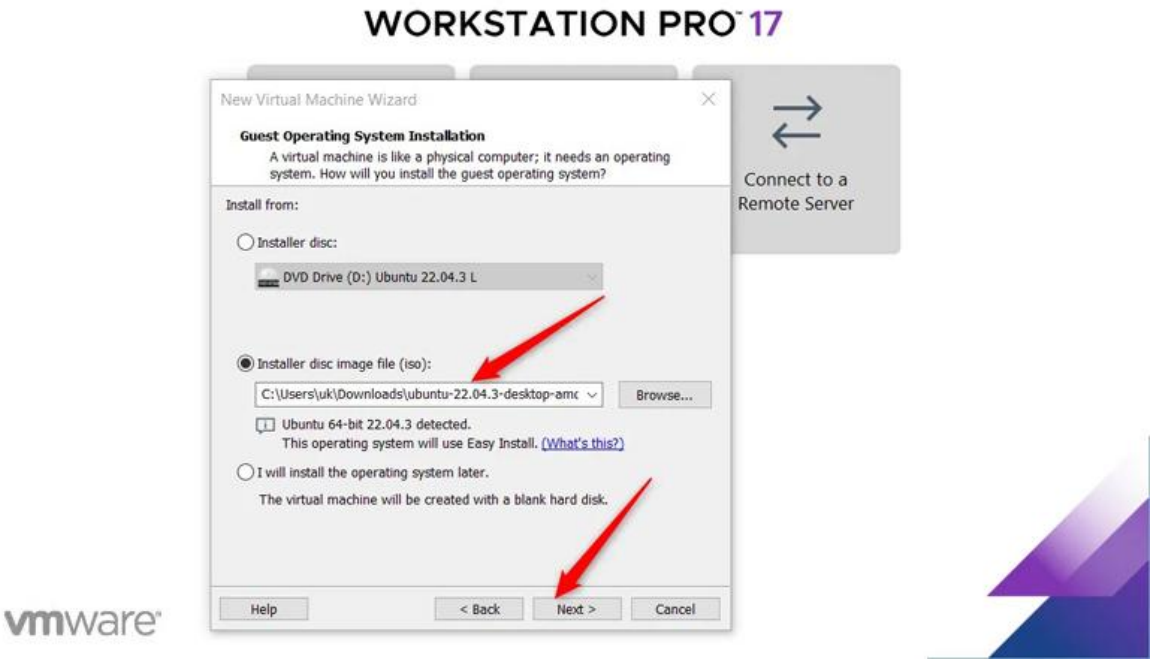
ابتدا VMware Workstation را اجرا کنید. سپس در پنجره اصلی برنامه، روی گزینه‌ی File از منوها کلیک کرده و New Virtual Machine را انتخاب نمایید. یک Wizard برای ایجاد ماشین‌مجازی جدید باز خواهد شد. در صفحه اول، گزینه Typical را انتخاب و روی Next کلیک کنید.

---

[36] https://ubuntu.com/download/desktop

٤. تعیین محل فایل ISO

در صفحه بعدی که مشابه تصویر ۱-۱۳ می‌باشد، ابتدا دکمه‌ی رادیویی Installer disc image file (iso) را انتخاب و روی Browse کلیک کنید. سپس به محلی که فایل ISO اوبونتو را در آن ذخیره کرده‌اید بروید و آن را انتخاب نمایید. برای ادامه روی Next کلیک کنید.



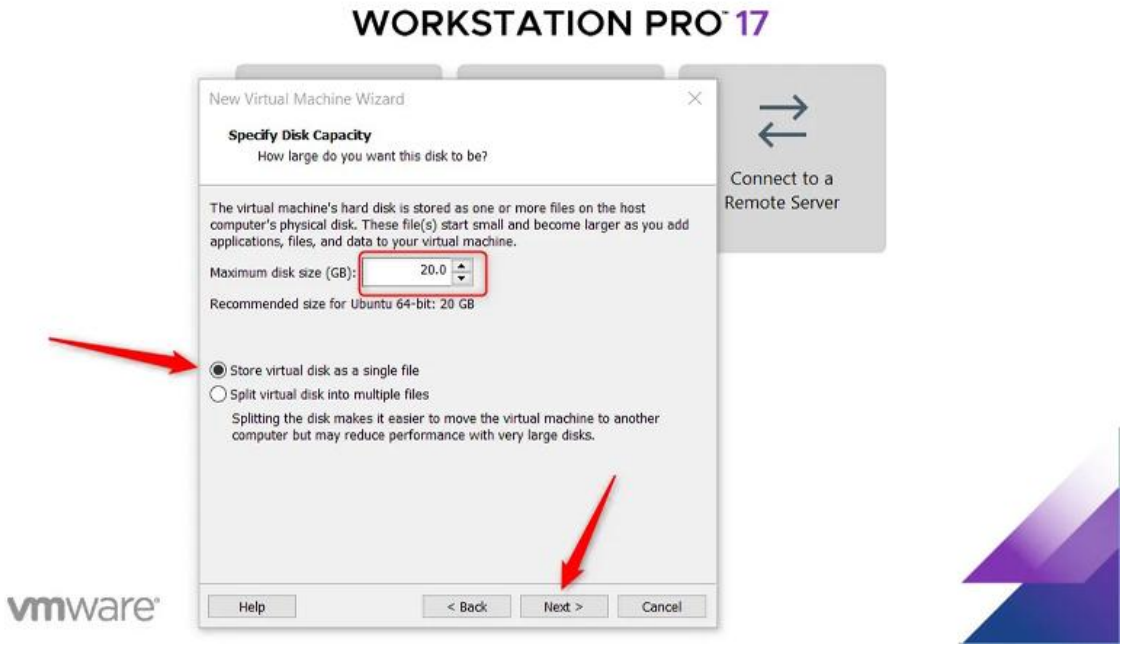
تصویر ۱-۱۳- انتخاب فایل نصب

٥. تعیین مشخصات سیستم‌عامل مهمان

معمولا این قسمت به‌طور خودکار شناسایی شده و با گزینه‌های Linux و Ubuntu پر می‌شود. درصورتی‌که توزیع دیگری از لینوکس را نصب می‌کنید که در لیست نسخه‌ها وجود ندارد، توزیع پایه را برای آن انتخاب نمایید. مثلا برای لینوکس مینت، اوبونتو را از لیست انتخاب کنید.

۶. انتخاب نام و تعیین فضای ذخیره‌سازی

ابتدا یک نام دلخواه برای ماشین‌مجازی خود وارد نمایید. در مرحله بعدی باید مکانی را برای ذخیره شدن فایل‌های ماشین‌مجازی خود انتخاب کنید و سپس مانند تصویر ۱-۱۴، ظرفیت دیسک را مشخص کنید (حداقل ۲۵ گیگابایت توصیه می‌شود). برای حفظ سرعت ماشین‌مجازی، گزینه Store virtual disk as a single file را انتخاب و برای ادامه روی Next کلیک کنید.



تصویر ۱-۱۴-تعیین فضای ذخیره‌سازی

۷. سفارشی‌سازی سخت‌افزار

این مرحله اختیاری است و در طی آن می‌توانید تنظیمات سخت‌افزاری ماشین‌مجازی را سفارشی کرده و تعداد پردازنده‌ها، حافظه و سایر تنظیمات را بر اساس قابلیت‌های سیستم خود تنظیم نمایید. کافی است روی هر گزینه

کلیک کرده و مقدار موردنظر خود را برای آن انتخاب نمایید. همچنین ممکن است بخواهید به برگه Options رفته و در صورت لزوم تنظیمات اضافی را نیز پیکربندی کنید.

۸. شروع نصب

در مرحله آخر، خلاصه‌ای از تنظیمات انتخابی شما نمایش داده خواهد شد. برای ایجاد ماشین‌مجازی روی finish کلیک کنید. ماشین‌مجازی از فایل ISO بوت می‌شود. دستورالعمل‌های روی صفحه را برای نصب اوبونتو دنبال کنید. درصورتی‌که در این بخش نیاز به راهنمایی دارید، به بخش ۱-۵-۲ مراجعه و راهنمای نصب را از مرحله ۴ به بعد دنبال نمایید.

۹. نصب ابزارهای ماشین‌مجازی

این بخش اختیاری است اما توصیه می‌شود که پس از پایان نصب به‌منظور ادغام و عملکرد بهتر، VMware Tools را نیز بر روی اوبونتو نصب کنید. در بخش منوهای VMware به VM > Install VMware Tools رفته و دستورالعمل‌های روی صفحه را در ماشین‌مجازی اوبونتو برای تکمیل نصب دنبال نمایید. یکی از امکاناتی که با نصب این ابزار در دسترس شما خواهد بود، بریده‌دان[۳۷] مشترک بین ویندوز میزبان و اوبونتوی مجازی است. به این ترتیب، به راحتی می‌توانید کدها و دستورالعمل‌ها را بین دو سیستم‌عامل با کپی و الصاق ساده جابه‌جا نمایید.

---

[37] Clipboard

۱۰. پشتیبان‌گیری از ماشین‌مجازی ایجاد شده

پس از اتمام مراحل نصب، حتما به یکی از روش‌های زیر، از ماشین‌مجازی خود پشتیبان[۳۸] تهیه نمایید. به این ترتیب هرگاه در حین اجرای دستورکارها، نصب بسته‌ها و یا انجام گزارش کارها، سیستم دچار مشکل جدی یا غیرقابل بازگشت شد، به جای نصب دوباره می‌توانید از نسخه پشتیبان استفاده کنید. همچنین اگر نسخه پشتیبان را پس از نصب بسته‌های مورد نیاز در لینوکس تهیه کرده باشید؛ در صورت بروز مشکل، از نصب و دانلودهای مکرر یا سنگین نیز جلوگیری کرده‌اید. برای ایجاد نسخه پشتیبان، دو روش زیر معرفی می‌شوند:

روش اول. ابتدا باید ماشین‌مجازی را از مسیر منوی زیر خاموش کنید.

```
VM > Power > Shut down Guest
```

درصورتی‌که ماشین‌مجازی روشن یا در حالت suspended باشد، پشتیبان‌گیری فعال نمی‌شود. حال از منوی VM گزینه Manage > Clone را انتخاب کرده و مطابق راهنمای خودش پیش بروید.

روش دوم. به محل ایجاد و ذخیره ماشین‌مجازی بر روی کامپیوتر خود مراجعه نمایید. مسیر پیش‌فرض ذخیره معمولا به‌صورت زیر می‌باشد. از پوشه‌ای[۳۹] که حاوی ماشین‌مجازی مدنظر شماست یک کپی گرفته و در محلی دیگر نگهداری نمایید.

```
C:\Users\UserName\Documents\Virtual Machines
```

---

[38] Backup
[39] Directory

تمرین

بر روی سیستم گروه خود در آزمایشگاه، یک توزیع اوبونتو یا مینت را به‌صورت راه‌اندازی دوگانه نصب کنید. فایل ISO مربوطه را می‌توانید از مسئول آزمایشگاه و یا فضای اشتراکی سایت دانشکده دریافت نمایید. مطابق دستورکار پیش بروید و دقت کنید تا سایر سیستم‌های عامل نصب شده بر روی سیستم شما آسیب نبینند.

گزارش کار

یک توزیع دلخواه و به‌روز از لینوکس (ترجیحا اوبونتو یا مینت) را به‌صورت مجازی و یا محلی بر روی سیستم شخصی خود نصب نمایید. توجه داشته باشید که برای انجام سایر گزارش کارها تا انتهای ترم، به این نصب نیاز خواهید داشت. درصورتی‌که سیستم شما از مجازی‌سازی پشتیبانی نمی‌کند و فضای کافی برای نصب به‌صورت محلی هم ندارد، می‌توانید از نسخه WSL[40] برای انجام گزارش کارها استفاده نمایید. WSL یک لایه‌ی سازگاری در ویندوز 10 و 11 است که به کاربر اجازه می‌دهد یک محیط لینوکسی واقعی را به شکل مستقیم از درون ویندوز اجرا کند. برای نصب باید از طریق Microsoft Store و یا وب‌سایت اوبونتو[41] اقدام نمایید. با استفاده از این نسخه، انجام اغلب گزارش کارها امکان پذیر خواهد بود؛ البته به غیر از هسته، ماژول‌ها و مجازی‌سازی. همچنین برای موضوعات تحلیل کارایی و فایل‌سیستم نیز با محدودیت‌های جدی مواجه است.

---

[40] Windows Subsystem for Linux

[41] https://ubuntu.com/desktop/wsl

منابع و مراجع:

- Fox R. *Linux with Operating System Concepts*, Chapman and Hall/CRC, 2021.
- Negus C. *Linux Bibl*, John Wiley & Sons, 2010.
- https://ubuntu.com/tutorials/install-ubuntu-desktop, visited 2026 Sep

آزمایش ۲ : تنظیمات و راهبری در سیستم‌های لینوکسی

اهداف:

- آشنایی با محیط، شیوه کار و تنظیمات در سیستم‌های لینوکسی

ابزار و مفاهیم مورد نیاز:

- سیستم‌عامل لینوکس
- مفاهیم دستورکار اول

محتوا:

- پایانه‌ها
- انواع کاربران
- فایل‌سیستم
- پارتیشن‌بندی
- سطوح اجرایی
- بوت لودر

۲-۱- قابلیت تغییر و تنظیم

از مدت‌ها پیش، توزیع‌های لینوکسی مفهوم «مخازن[1] نرم‌افزاری» را به سیستم‌های خود اضافه کرده‌اند. این منابع، فهرست‌های بزرگی از تعداد زیادی برنامه‌ی قابل نصب بر روی یک سیستم لینوکسی می‌باشند. به‌عنوان مثال، هرگاه بخواهیم برنامه گفتگوی Microsoft Teams را روی سیستم نصب کنیم، کافی است وارد برنامه مدیریت نرم‌افزارها[2] شده، Teams را جستجو کرده و با زدن تیک کنار آن و فشردن کلید Apply، به سیستم بگوییم که آن را از روی منابع نرم‌افزاری ویژه‌ی توزیع لینوکسی ما نصب کند. در عرض چند دقیقه، این برنامه دانلود شده و به شکل خودکار بر روی سیستم نصب خواهد شد.

برای انجام تنظیمات، در سیستم به‌دنبال گزینه System Setting بگردید. این گزینه حاوی بخش بزرگی از تنظیمات عمومی سیستم است. مثلا برای اضافه کردن زبان فارسی به صفحه‌کلید روی Keyboard Layout کلیک کرده و برگه Layout را انتخاب کنید. علامت + را زده، Persian را انتخاب کرده و کلید Add را بزنید.

حق انتخاب در گنو/لینوکس، چیزی بسیار فراتر از شخصی‌سازی رنگ‌ها و اندازه آیکون‌ها است به‌طوری‌که کاربر می‌تواند در عمیق‌ترین سطوح سیستم مانند هسته نیز تغییراتی اعمال کند. یا اگر بعد از نصب اوبونتو با میزکار Unity مواجه شد، به سادگی می‌تواند آن را با میزکار KDE که یک برنامه مدیریت محیط گرافیکی کاملا متفاوت است، جایگزین نماید. یا بسته به نیاز خود، از میزکارهایی مانند Awesome استفاده نماید که علاوه بر مدیریت خودکار پنجره‌ها در مانیتورهای متفاوت، اجازه نمی‌دهند هیچ فضایی روی هیچ نمایشگری به هدر برود و یا پنجره ای با رفتن زیر پنجره دیگر، از دید پنهان شود. به عبارت دیگر، کاربر لینوکس به معنای واقعی کلمه، حق انتخاب دارد.

---

[1] Repository
[2] Software Store

دلایل تنوع در لینوکس

برای درک این تنوع گسترده، لازم است که به تولید نرم‌افزار با دو دید متفاوت نگاه کنیم. در نگاه اول که دید تجاری دارد؛ عرضه‌کنندگان پیش از ارائه محصول تصمیم می‌گیرند که آن برنامه دقیقا چه کاری انجام دهد و سپس تیم برنامه‌نویسی آن ویژگی‌ها را پیاده‌سازی می‌نماید. ایده‌های جدید تنها در نسخه‌های بعدی قابل اعمال خواهند بود و انگیزه آن، افزایش فروش و حفظ رضایت مشتری می‌باشد. بنابراین، شرکت عرضه‌کننده بر پایه دیدگاه خود تصمیم می‌گیرد که امکان تنظیم برخی موراد درون برنامه را به کاربر بدهد یا نه. به‌عنوان مثال در ویندوز، کابران امکان تغییر تم‌ها و ظاهر سیستم را دارند درحالیکه سیستم‌عامل مک، کمترین امکان انجام تنظیمات غیرضروری را برای کاربران خود در نظر گرفته است.

همچنین تولیدکنندگان تجاری معمولا دو محصول متفاوت برای پاسخ به یک نیاز یکسان تولید نمی‌کنند. آن‌ها به سراغ بازارهای کم جمعیت نمی‌روند، مگر اینکه امکان افزایش قیمت تا سوددهی را داشته باشند. به همین دلایل است که مراحل پیش‌تولید طولانی می‌شود و در طی آن طراحان تلاش می‌کنند تا تمامی جنبه‌های محصول و نیازهای کاربران را پیش‌بینی نمایند.

اما در نگاه دوم که دید آزاد به توسعه نرم‌افزار دارد، نقطه شروع این است که «تولید کننده، مصرف کننده است». یعنی هر برنامه را شخصی می‌نویسد که خودش می‌خواهد از آن استفاده کند. در نتیجه کوشش می‌کند بهترین برنامه‌ی ممکن را ایجاد کرده و پیوسته آن را براساس نیازهای خود تغییر داده و بهبود ببخشد. هر چند که در این شیوه، با یک چالش در «طراحی اولیه» مواجه هستیم. و آن عبارت است از اینکه برنامه‌نویس در همان ابتدا، تمام جزئیات برنامه را طراحی نمی‌کند. در نتیجه ممکن است بسیاری از نیازهای کاربران آینده پیش‌بینی نگردد. اما در نقطه مقابل، در این روش یک مزیت بزرگ به نام «بقای اصلح» وجود دارد. همان‌طور که در طبیعت، گونه‌های زیستی پیوسته در حال تکامل هستند و در هر نسل فقط آن‌هایی باقی می‌مانند که سازگاری بیشتری با محیط داشته باشند، نرم‌افزارهای آزاد نیز مبتنی بر همین روش رشد می‌کنند.

برای توزیع‌های لینوکسی نیز همین‌گونه است. یعنی یک گروه، توزیعی را می‌سازند (مثلا دبیان) و گروه‌های دیگر، از روی آن شاخه‌های جدیدی پدید می‌آورند (مانند اوبونتو و پارسیکس). اما از این بین فقط توزیع‌هایی که به علت

سازگاری بالاتر و یا مفید بودن برای گروه‌های متنوع، مورد استقبال و استفاده بیشتری در بین مصرف‌کنندگان قرار بگیرند، به‌عنوان پایه توزیع جدید بعدی انتخاب خواهند شد. و این چرخه به همین شکل ادامه می‌یابد تا برخی توزیع‌ها از بین بروند و برخی دیگر به رشد و بقای خود ادامه دهند. البته در برخی موارد، محدودیت منابع انسانی و مالی برای ادامه توسعه و پشتیبانی نیز تبدیل به عاملی برای توقف رشد یک توزیع می‌گردد.

تولیدکنندگان آزاد می‌توانند برنامه‌های خود را با تغییرات جدید در جامعه پخش کنند تا با بررسی بازخوردها به سادگی دریابند که کدام ویژگی جدید توسط کاربران ترجیح داده شده است. این قابلیت، یک مزیت آشکار است. در حالیکه شرکت‌های تجاری همچنان به سیستم قدیمی خود که مبتنی بر «طراحی از قبل» است، وابسته هستند. به‌عنوان مثال، «مرکز نرم‌افزار اوبونتو» در نسخه ۹.۱۰ در سال ۲۰۰۹ با اعمال یک سری تغییرات در سیستم قدیمی نصب بسته‌ها به وجود آمد. بعد از دیدن علاقه کاربران، توزیع‌های بزرگ دیگر هم به سراغ آن رفتند در حالی که اپل در سال ۲۰۱۱ با معرفی Mac App Store و مایکروسافت هم در ۲۰۱۲ با Windows Store در ویندوز ۸، این ایده را پیاده‌سازی کردند.

۲-۲- خط فرمان

خط فرمان[۳] روشی برای ارتباط با سیستم‌عامل است و فقط با استفاده از متن کار می‌کند. در خط فرمان کاربر می‌تواند به‌صورت مستقیم دستورات لازم را برای کامپیوتر بنویسد تا بعد از زدن کلید Enter، کامپیوتر آن دستورات را اجرا نموده و در صورت نیاز، پاسخ را نیز به شکل متنی بازگرداند. این روش در نقطه مقابل روش گرافیکی قرار دارد که در آن، دستورات از طریق ماوس، منوها و کلیک کردن ارسال شده و پاسخ‌ها به‌صورت تصاویر و نمودارها نمایش داده می‌شوند.

---

[3] Command Line

همانگونه که بالاتر اشاره شد، دسترسی به خط فرمان، به‌صورت متنی و از طریق نوشتن دستورات و مشاهده نتایج آن‌ها انجام می‌پذیرد. برای این کار، به برنامه‌ای به نام شبیه‌ساز ترمینال[۴] نیاز داریم که برای سادگی، به آن ترمینال یا پایانه[۵] نیز گفته می‌شود.

توزیع‌های لینوکسی براساس فلسفه‌ای یکسان با سیستم‌های شبه‌یونیکسی بنا شده‌اند که می‌گوید: اول اینکه «همه چیز یا باید فایل باشد یا فرآیند[۶]» و دوم اینکه «همه تنظیمات باید در فایل‌های متنی ذخیره شوند». ترکیب این دو اصل، قدرت فوق العاده‌ی ترمینال را پدید می‌آورد. زیرا وقتی هر چیزی که در سیستم وجود دارد - از یک فایل معمولی گرفته تا سخت‌افزار یا حتی اتصال شبکه - به‌صورت یک فایل یا فرآیند نمایش داده شود، با استفاده از یک دستور ساده در ترمینال می‌توان به تمام آن بخش‌ها دسترسی پیدا کرد. از طرف دیگر، اگر همه‌ی تنظیمات در فایل‌های متنی ذخیره شوند، آن‌گاه می‌توان با ابزارهای ساده‌ی خط فرمانی، آن‌ها را خواند و تغییر داد. بنابراین فقط با استفاده از ترمینال، می‌توان به هر بخشی از سیستم دسترسی پیدا کرده و یا تغییرات لازم را در آن اعمال نمود.

کار کردن در ترمینال علاوه بر کارایی و انعطاف‌پذیری، سریع‌تر نیز هست. به‌عنوان مثال، فرض کنید بخواهیم یک فایل JPEG را بر روی دسکتاپ به PNG تغییر فرمت دهیم. در حالت گرافیکی نیاز به برنامه‌ای برای ویرایش تصاویر داریم. اما در حالت خط فرمانی، این کار فقط با یک خط دستور انجام خواهد شد. در آزمایش بعدی به تفصیل با دستورات خط فرمان آشنا خواهیم شد.

```
convert     Desktop/sample.jpg   Desktop/sample.png
```

ترمینال معمولا با کلیدهای میان‌بر مانند Alt+Ctrl+T قابل دسترسی است. به‌علاوه، خاصیت «کامل کردن خودکار[۷]» دستورات که با کلید Tab انجام می‌شود، سرعت نوشتن را بالا می‌برد.

---

[4] Terminal Emulator
[5] Terminal
[6] Process
[7] Auto Fill

برای مدیران سیستم و برنامه‌نویسانی که با سرورها کار می‌کنند، هنوز هم روش‌های گرافیکی کنترل از راه دور نتوانسته‌اند جای خط فرمان را بگیرند. زیرا اتصال به سرور از طریق یک شبیه‌ساز ترمینال روی کامپیوتر شخصی به‌منظور تعمیر، تنظیم یا به‌روزرسانی سرور نه تنها سریع‌تر که سبک‌تر نیز می‌باشد. در این حالت، سرور فقط باید حجم اندکی از اطلاعات متنی را رد و بدل کند، در حالی که انتقال یک محیط گرافیکی کامل، پهنای باند و منابع بیشتری می‌طلبد.

از سوی دیگر، یک سرور برای انجام وظایف واقعی خود مانند میزبانی وب، پردازش داده یا مدیریت پایگاه‌داده، هیچ نیازی به محیط گرافیکی ندارد. بنابراین با استفاده از حالت متنی، می‌توان از هدر رفتن قدرت پردازشی و حافظه سرورها جلوگیری نمود. همچنین ازآنجایی‌که افزایش لایه‌های نرم‌افزاری، احتمال بروز خطا و باگ را افزایش می‌دهد، با حذف محیط گرافیکی از سرورها، سطح خطاپذیری سیستم کاهش یافته و به این ترتیب، پایداری آن افزایش می‌یابد.

با این توضیحات به وضوح دیدیم که خط فرمان سریع‌تر بوده، در سرورها منابع کمتری را به هدر می‌دهد، دسترسی به هر آنچه که در یک سیستم هست را فراهم می‌کند و قابلیت کنترل بیشتری را در اختیار کسی که پشت یک سیستم گنو/لینوکسی نشسته است، قرار می‌دهد.

۲-۲- ۱- انواع پایانه در لینوکس

لینوکس دارای پایانه‌های مختلفی می‌باشد که هر یک از آن‌ها، کاربرد و جایگاه مشخصی در سازوکار ارتباط بین کاربر با سیستم دارند. این پایانه‌ها عبارتند از:

TTY[8] : در گذشته که کامپیوترها به شکل مین‌فریم در اتاق‌های مخصوص قرار داشتند و کاربران نمی‌توانستند به‌صورت مستقیم به آن‌ها دسترسی داشته باشند، از پایانه‌های فیزیکی[9] برای برقراری اتصال استفاده می‌شد. این پایانه‌ها،

---

[8] Teletypewriter

[9] Physical Terminal

Teletype نام داشتند - یک دستگاه، شبیه ماشین تحریر قدیمی که با کابل به کامپیوتر وصل می‌شد. کاربر با کمک این دستگاه درخواست خود را نوشته و به کامپیوتر منتقل می‌کرد. سپس پاسخ روی کاغذ چاپ می‌گردید.

در کامپیوترهای لینوکسی امروزی، پایانه فیزیکی به دو چیز اشاره دارد: یکی کنسول مجازی[10] مانند صفحه لاگین که هنگام ورود به سیستم از شما نام کاربری و رمزعبور را درخواست می‌کند و دیگری پورت‌های سریال. مثلا اگر یک کابل سریال را به کامپیوتر خود متصل کرده و با استفاده از آن کابل، یک دستگاه دیگر مثل یک ترمینال سخت‌افزاری یا لپ‌تاپ را به کامپیوتر خود وصل کنید، آن دستگاه هم یک پایانه فیزیکی محسوب می‌شود.

اما امروزه، به هر نوع پایانه متنی[11] در کامپیوتر نیز tty گفته می‌شود. به عبارت دیگر، هر tty یک نقطه‌ی ورود/خروج برای تعامل کاربر با سیستم از طریق خط فرمان محسوب می‌شود و یک نشست مستقل دارد. کنسول‌های tty توسط هسته ساخته شده و درون خود سیستم‌عامل قرار می‌گیرند. این کنسول‌ها به شکل مستقیم و بدون هیچ نرم‌افزار واسطه‌ای به سخت‌افزار (صفحه‌کلید و صفحه‌نمایش) متصل هستند؛ حتی با اینکه اصطلاح «مجازی» را برای آن‌ها به‌کار می‌بریم.

نکته: به‌صورت پیش‌فرض ۶ ترمینال مجازی متنی وجود دارد. از tty1 تا tty6 که با کلیدهای Ctrl+Alt+F1 تا F6 قابل دسترسی هستند. tty7 و یا گاهی tty8 نیز معمولا برای محیط گرافیکی رزرو می‌شوند. بنابراین اگر مثلا محیط گرافیکی دچار خرابی شود، می‌توانید با این کلیدهای میان‌بر به یک tty فیزیکی رفته و مشکل را برطرف نمایید.

PTY یا شبه‌پایانه[12]: برای دسترسی به پایانه در شرایطی که پایانه فیزیکی وجود ندارد، طراحی شده است. به‌عنوان مثال، وقتی از طریق SSH به یک سرور متصل می‌شوید یا یک شبیه‌ساز پایانه مانند GNOME Terminal یا Xterm را در محیط گرافیکی باز می‌کنید، در واقع از یک PTY استفاده می‌کنید .

---

[10] Virtual Console
[11] Text Terminal
[12] Pseudo-Terminal

PTS[13] **و PTMX** [14] : دو بخش تشکیل‌دهنده‌ی یک PTY مدرن هستند. فایل /dev/ptmx یک دستگاه کاراکتری[15] (major 5, minor 2) است که مثل یک دروازه عمل می‌کند. هر برنامه‌ای مثل sshd یا xterm که بخواهد یک شبه‌پایانه جدید بسازد، این دروازه را باز می‌کند. به محض باز شدن /dev/ptmx، سیستم‌عامل یک جفت شبه‌پایانه PTY جدید ایجاد می‌کند. یک سر این جفت، PTY master نام دارد و در اختیار همان برنامه قرار می‌گیرد. سر دیگر که PTY slave نام دارد، به‌صورت خودکار ساخته شده و در مسیر /dev/pts/N قرار می‌گیرد و در آن، N یک عدد صحیح مثبت است. مثلا /dev/pts/0 یا /dev/pts/1. این فایل، دقیقا مثل یک tty فیزیکی رفتار می‌کند. بنابراین وقتی یک پوسته مثل پوسته bash یا zsh آن را باز کند، فکر خواهد کرد که به یک ترمینال واقعی وصل شده است. به عبارت دیگر، /dev/pts/N همان شبه‌پایانه است که پوسته در آن اجرا می‌شود.

دستگاه‌های /dev/pts/N به‌صورت پویا ایجاد می‌شوند و هر بار که یک شبیه‌ساز پایانه یا یک اتصال SSH جدید باز شود، یک دستگاه جدید در سیستم ظاهر خواهد شد. ورودی‌هایی که کاربر می‌نویسد، از سمت master به slave می‌رسند و خروجی‌هایی که پوسته تولید می‌کند، از slave به master برمی‌گردند.

به‌عنوان مثال فرض کنید در لینوکس یک شبیه‌ساز ترمینال گرافیکی مثل xterm را باز می‌کنیم. xterm ابتدا فایل /dev/ptmx را باز می‌کند. بلافاصله، سیستم‌عامل یک جفت PTY می‌سازد که سر master آن، در دست xterm قرار می‌گیرد و سر slave به نام /dev/pts/0 ساخته می‌شود. حال xterm یک پوسته مثل bash را اجرا کرده و آن را به /dev/pts/0 متصل می‌کند. بنابراین اگر کاربر در همان ترمینال دستور tty را بزند، خروجی /dev/pts/0 خواهد بود. از این پس هرچیزی که کاربر تایپ کند، توسط شبیه‌ساز ترمینال (xterm) در سر PTY master نوشته می‌شود تا از آنجا به سر slave یا همان /dev/pts/0 ارسال شود و bash آن را بخواند. خروجی bash نیز از /dev/pts/0 به سر master برمی‌گردد تا xterm آن را خوانده و روی صفحه برای کاربر نمایش دهد.

نکته: در کل سیستم، فقط یک ptmx وجود دارد. درحالیکه به تعداد ترمینال‌های باز، PTS ساخته می‌شود.

---

[13] Pseudo-Terminal Slave
[14] Pseudo-Terminal Master
[15] Character Device

۲-۳-رفع مشکل در لینوکس

تقریبا تمام جنبه‌های یک سیستم گنو/لینوکسی دارای راهنما و مستندات بسیار کاملی هستند که صفحات راهنما[۱۶] نامیده می‌شوند. مثلا برای دیدن راهنمای کامل Firefox کافی است در ترمینال بنویسید man firefox تا تمام سوییچ‌ها و روش‌های ممکن برای اجرای فایرفاکس برای شما به نمایش درآید. برای خروج از راهنما کافی است کلید q را بزنید.

البته خواندن این راهنماها در ابتدا کار راحتی نیست، ولی تفاوت یک حرفه‌ای و یک تازه‌کار در همین توانایی برای خواندن راهنما می‌باشد. غیر از صفحات راهنما، برای حل مشکلات پیش آمده در حین راه‌اندازی یا کار با یک سیستم لینوکسی می‌توانید به سایت اصلی، فروم‌ها و یا صفحات ویکی مرتبط با توزیع خود مراجعه نمایید.

۲-۴- انواع کاربران

سلسله مراتب کاربری به لینوکس کمک می‌کند تا هم قابلیت استفاده و هم امنیت را به خوبی فراهم نماید. در ادامه انواع کاربران در لینوکس آمده است. با نحوه ایجاد و مدیریت کاربران در آزمایش بعدی آشنا خواهیم شد.

کاربر Root: کاربر ریشه یا Superuser نیز نامیده می‌شود و به منزله مدیر سیستم است. دسترسی کامل و بدون محدودیت به تمام سیستم دارد و می‌تواند هر کاری انجام دهد، حتی حذف فایل‌های سیستمی یا تغییر تنظیمات هسته. بنابراین اگر این کاربر مرتکب اشتباهی حیاتی شود، ممکن است سیستم غیرقابل boot گردد. کاربر ریشه، دارای شناسه کاربری[۱۷] برابر با ۰ می‌باشد.

کاربر Sudoer: کاربری با امتیازات ویژه است. در واقع یک کاربر عادی است که مجوز اجرای دستورات با امتیاز کاربر ریشه را دارد. بدین ترتیب، امنیت بهتری نسبت به ورود مستقیم به‌عنوان کاربر ریشه، تامین می‌شود. این نوع از کاربران از طریق فایل /etc/sudoers مدیریت می‌شوند.

---

[16] Man Pages
[17] User ID (UID)

کاربر Regular: یک کاربر عادی است که در سیستم ساخته می‌شود و فقط دسترسی‌های محدود به دایرکتوری خانگی و فایل‌های شخصی خود دارد. یک کاربر عادی نمی‌تواند تغییرات سیستمی ایجاد کند و معمولا دارای شناسه کاربری ۱۰۰۰ یا بالاتر می‌باشد. ساده‌ترین روش در اوبونتو برای ارتقای یک کاربر عادی به کاربر sudoer، افزودن کاربر به گروه sudo می‌باشد که در آزمایش بعدی، نحوه انجام آن را مشاهده خواهید کرد.

کاربر Service: کاربران سرویس یا سیستمی، کاربرانی هستند که توسط سیستم و با هدف اجرای سرویس‌ها و daemons بدون استفاده مستقیم از root، ساخته شده‌اند. میزان دسترسی‌هایشان بر اساس اصل «کمترین امتیاز[18]» تعیین می‌گردد و بسته به توزیع لینوکسی، معمولا شناسه کاربری بین ۱ تا ۹۹۹ دارند. با این کاربران، نمی‌توان به‌طور مستقیم وارد[19] سیستم شد.

لیست کاربرانی که در سیستم تعریف شده اند در یک فایل متنی ذخیره و نگهداری می‌شود. با دستور زیر می‌توانید محتویات این فایل را مشاهده نمایید.

```
cat   /etc/passwd
```

نکته‌ها: برای حفظ امنیت سیستم خود، بهتر است هرگز با کاربر ریشه، وارد سیستم نشوید. بلکه از sudo برای انجام وظایف مدیریتی خود استفاده نمایید. همچنین برای اجرای سرویس‌های مختلف، کاربران مجزا ایجاد کرده و دسترسی‌ها را بر اساس اصل کمترین امتیاز تنظیم نمایید. طبق این اصل، همیشه باید حداقل میزان موردنیاز از سطوح دسترسی و مجوزها را به کاربران تخصیص داد و از تخصیص مجوزهای غیرضروری اجتناب کرد. علاوه بر این موارد، همیشه سعی کنید برای کاربران رمزعبور قوی انتخاب نمایید.

---

[18] Least Privilege

[19] No Login Capability

۲-۵- فایل‌سیستم

فایل‌سیستم[۲۰]، روشی برای سازمان‌دهی فایل‌ها بر روی دیسک است. به عبارتی، یک ساختار داده است که به کمک آن یک سیستم‌عامل می‌تواند فایل‌های روی دیسک یا پارتیشن را ذخیره، بازیابی و رهگیری[۲۱] نماید. فایل‌سیستم لینوکس به‌صورت یک ساختار درختی وارونه سازماندهی شده است که در آن همه چیز از دایرکتوری ریشه[۲۲] شروع شده و به پایین گسترش می‌یابد. این ساختار استاندارد Filesystem Hierarchy Standard (FHS) نام دارد که آن را در تصویر ۲-۲ مشاهده می‌کنید.

مهم‌ترین دایرکتوری‌ها در فایل‌سیستم

/ : دایرکتوری ریشه که با نماد / نمایش داده می‌شود. ریشه کل فایل‌سیستم است. همه دایرکتوری‌های دیگر، زیرمجموعه این دایرکتوری هستند و فقط کاربر ریشه حق نوشتن در آن را دارد.

bin (Binary): شامل فایل‌های اجرایی ضروری سیستم و دستورات پایه است و در دسترس همه کاربران قرار دارد.

sbin (System Binaries): شامل دستورات حیاتی سیستم که بیشتر به‌منظور مدیریت و اداره سیستم توسط کاربر ریشه، طراحی شده‌اند. مانند دستور fdisk یا mkfs.

etc (Editable Text Configuration): شامل فایل‌های پیکربندی سیستم و برنامه‌ها است که معمولا از نوع فایل متنی هستند.

home: شامل دایرکتوری‌های شخصی کاربران می‌باشد. درون این دایرکتوری، هر کاربر یک دایرکتوری شخصی، هم‌نام با نام کاربری خودش دارد. به هر کاربر، فقط بر روی دایرکتوری خودش اجازه دسترسی کامل داده می‌شود. دقت داشته

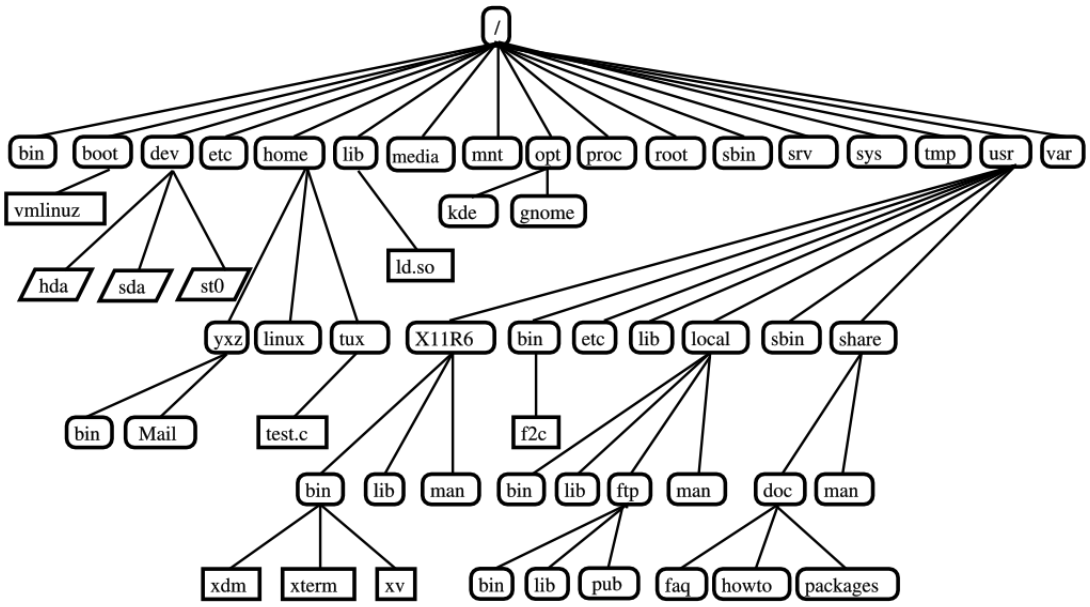
---

[20] File System
[21] Track
[22] Root Directory

باشید که دایرکتوری خانگی کاربر اصلی، به‌صورت مجزا و تحت نام root، خارج از این دایرکتوری (مستقیما در دایرکتوری /) قرار گرفته است.

dev (Devices): شامل فایل‌های دستگاه‌ها[۲۳] است. همان‌طور که در بخش ۲-۲ نیز اشاره شد می‌دانید که در لینوکس، هر سخت‌افزار به‌صورت یک فایل نمایش داده می‌شود.

proc (Process): فایل‌سیستم مجازی فرآیندها و اطلاعات سیستم می‌باشد و شامل اطلاعات بلادرنگ درباره فرآیندها و منابع سیستم است.



تصویر ۲-۲- فایل‌سیستم لینوکس

var (Variable): حاوی داده‌های متغیر سیستم است. مانند لاگ‌ها، spool صف چاپ و حافظه نهان[۲۴]

---

[23] Device files

[24] Cache

usr (User): شامل برنامه‌ها و منابع فقط خواندنی[۲۵] کاربران است. سه بخش اصلی آن عبارتند از: usr/bin/ که دستورات کاربران را در برمی‌گیرد. usr/lib/ که حاوی کتابخانه‌هاست و usr/share/ که داده‌های اشتراکی را شامل می‌شود.

boot: شامل فایل‌های ضروری برای فرآیند بوت سیستم می‌باشد. از جمله هسته لینوکس یعنی vmlinuz، فایل initramfs و فایل‌های پیکربندی بوت‌لودر مانند GRUB.

lib (Libraries): کتابخانه‌های اشتراکی حیاتی و ماژول‌های کرنل را در خود جای می‌دهد که برنامه‌های موجود در bin/ و sbin/ برای اجرا به آن‌ها نیاز دارند. در سیستم‌های ۶۴ بیتی، معمولا lib64/ نیز وجود دارد.

media: نقطه اتصال[۲۶] پیش‌فرض برای رسانه‌های قابل حمل مانند درایوهای USB، CD/DVD و کارت‌های حافظه در این دایرکتوری قرار می‌گیرد. سیستم‌های مدرن معمولا این رسانه‌ها را به‌صورت خودکار در این مسیر متصل یا به اصطلاح سوار[۲۷] می‌کنند. با استفاده از نقطه اتصال، امکان اتصال فایل‌سیستم‌های مختلف مثل فلش درایو، به درخت اصلی وجود دارد. در آزمایش سوم، توضیحات بیشتری در این باره داده شده است.

mnt (Mount): برخلاف media/ که برای ایجاد نقطه اتصال خودکار می‌باشد، مدیران سیستم از این دایرکتوری به‌عنوان محلی برای اتصال دستی و موقت فایل‌سیستم‌ها استفاده می‌نمایند.

opt (Optional): بسته‌های نرم‌افزاری اضافی و شخص ثالث[۲۸] که مدیربسته‌ی سیستم، آن‌ها را مدیریت نمی‌کند در این دایرکتوری نصب می‌شوند. معمولا هر بسته در یک زیردایرکتوری مجزا در این مسیر نصب می‌گردد.

run: یک فایل‌سیستم موقت در حافظه به نام tmpfs است که داده‌های زمان اجرای فرآیندها مانند فایل‌های PID و سوکت‌ها را نگهداری می‌کند و محتوای آن با راه‌اندازی مجدد پاک می‌شود.

srv (Service): داده‌های مربوط به سرویس‌های سیستم را نگهداری می‌کند، مانند فایل‌های وب‌سرور یا مخازن FTP. به عبارت دیگر، این دایرکتوری مخصوص داده‌هایی است که توسط سرویس‌ها به کاربران ارائه می‌شود.

---

[25] Read-Only
[26] Mount Point
[27] Mount
[28] Third Party

sys (System): شامل فایل‌سیستم مجازی هسته به نام sysfs است که اطلاعات مربوط به دستگاه‌های سخت‌افزاری و درایورهای هسته را به فضای کاربر ارائه می‌دهد. این دایرکتوری سازوکاری برای پیکربندی و نظارت بر سخت‌افزار فراهم می‌نماید.

tmp (Temporary): برای ذخیره فایل‌های موقتی که برنامه‌ها در حین اجرا ایجاد می‌کنند، مورد استفاده قرار می‌گیرد. محتوای این دایرکتوری معمولا پس از هر بار راه‌اندازی مجدد، پاک می‌شود و همه کاربران مجوز نوشتن در آن را دارند.

نکاتی درباره فایل‌سیستم

حساسیت به حروف بزرگ و کوچک: فایل‌سیستم نسبت به حروف بزرگ و کوچک حساس[۲۹] است. بنابراین File.txt با file.txt و آن هم با FILE.TXT متفاوتند و سه فایل متفاوت و مجزا را نشان می‌دهند.

آدرس‌دهی مطلق و نسبی: در فایل‌سیستم، آدرس‌دهی به دو شکل مطلق و نسبی امکان‌پذیر است. آدرس مطلق از root شروع می‌شود و در همه جای سیستم معتبر است. به‌عنوان مثال آدرس زیر، یک آدرس مطلق است.

```
/home/user/file
```

در هر کجای سیستم که قرار داشته باشید، این آدرس به درستی شما را به آیتمی به نام file خواهد رساند. اما آدرس نسبی از دایرکتوری جاری شروع می‌شود و فقط در همان دایرکتوری، معتبر و قابل استفاده می‌باشد. مثلا اگر در داخل دایرکتوری home/ باشید، برای دسترسی به file کافی است آدرس نسبی usr/file را ذکر نمایید.

---

[29] Case Sensitive

۲-۶- پارتیشن‌بندی

فضای دیسک سخت[۳۰] به بخش‌های مجزایی با عنوان پارتیشن تقسیم می‌شود. همان‌طور که در آزمایش قبلی هنگام نصب لینوکس مشاهده کردید، هر پارتیشن برای هدف خاصی مورد استفاده قرار می‌گیرد. مانند نگهداری از سیستم عامل، تنظیمات بوت، فایل‌سیستم، داده‌های کاربر و فضای swap. از جمله مزایای پارتیشن‌بندی دیسک، جداسازی داده‌های سیستمی از داده‌های کاربر است. به این ترتیب، در صورت بروز خرابی یا نیاز به نصب مجدد سیستم‌عامل، اطلاعات کاربران از بین نمی‌رود. علاوه بر این، پارتیشن‌بندی امکان استفاده از فایل‌سیستم‌های متفاوت را در یک دیسک واحد فراهم می‌سازد.

دستگاه	نام
اولین دیسک فلاپی (A)	/dev/fd0
دومین دیسک فلاپی(B)	/dev/fd1
اولین دیسک سخت از نوع IDE (کل درایو)	/dev/hda
پارتیشن اول از hda	/dev/hda1
دومین دیسک سخت از نوع IDE (کل درایو) یا CD-ROM	/dev/hdb
اولین دیسک سخت از نوع SCSI (کل درایو)	/dev/sda
پارتیشن اول از sda	/dev/sda1
پارتیشن دوم از sda	/dev/sda2
دومین دیسک سخت از نوع SCSI (کل درایو)	/dev/sdb
پارتیشن اول از sdb	/dev/sdb1
اولین درایو CD-ROM از نوع SCSI	/dev/scd0
دومین درایو CD-ROM از نوع SCSI	/dev/scd1

جدول ۲-۱- نام‌گذاری پارتیشن‌ها در لینوکس

[30] Hard Disk

همان‌طور که در جدول ۲-۱ مشاهده می‌کنید، دستگاه‌های ذخیره‌سازی در لینوکس با نام‌هایی مانند sda، sdb و ... شناخته می‌شوند. عبارت sd مخفف SCSI Disk است و حروف بعدی یعنی a، b، c و ... به ترتیب به اولین، دومین و سومین دیسک شناسایی‌شده توسط سیستم اشاره می‌کنند. سپس هر دستگاه به پارتیشن‌هایی تقسیم می‌شود؛ مثلا برای دستگاه sda، پارتیشن‌هایی با نام‌های sda1، sda2 و ... ایجاد می‌گردد. عدد انتهایی هر پارتیشن نشان‌دهنده شماره ترتیبی آن پارتیشن روی دیسک است.

نکته: عبارت hd به دیسک‌های سخت از نوع IDE/PATA اشاره دارد که امروزه مانند فلاپی‌ها منسوخ شده‌اند.

فرمت‌های رایج فایل‌سیستم در لینوکس

در یک دسته‌بندی کلی، فرمت‌های قابل شناسایی در لینوکس به ۵ دسته‌ی فایل‌سیستم‌های محلی[۳۱]، فایل‌سیستم‌های شبکه[۳۲]، فایل‌سیستم‌های خاص[۳۳]، فایل‌سیستم‌های ویندوز[۳۴] و فایل‌سیستم‌های مک[۳۵] تقسیم می‌شوند. با استفاده از ابزارهای خط فرمانی یا گرافیکی مدیریت دیسک، می‌توانید هر یک از پارتیشن‌ها در لینوکس را به دلخواه فرمت نمایید.

۱. فرمت‌های فایل‌سیستم محلی

Fourth Extended File System(ext4): پیش‌فرض اکثر توزیع‌های لینوکس است و از روش ژورنالینگ[۳۶]، برای بازیابی بهتر پس از ازکارافتادن ناگهانی سیستم، استفاده می‌کند. برای استفاده در پارتیشن root و home مناسب می‌باشد.

---

[31] Local File Systems
[32] Network File Systems
[33] Special Purpose
[34] Windows Compatible
[35] MacOS Compatible
[36] Journaling

XFS: یک فایل‌سیستم با کارایی بالا[۳۷] و مناسب برای فایل‌های بسیار بزرگ است که معمولا در سرورها و سیستم‌های ذخیره‌سازی[۳۸] مورد استفاده قرار می‌گیرد.

Btrfs (B-tree File System): یک فایل‌سیستم پیشرفته با قابلیت‌های مدرن است که قابلیت‌هایی نظیر Snapshots، Copy-on-Write، فشرده‌سازی خودکار، RAID نرم‌افزاری داخلی و Checksum برای داده و فراداده[۳۹] دارد.

۲. فرمت‌های فایل‌سیستم شبکه

NFS (Network File System): استانداردی برای اشتراک‌گذاری فایل در یونیکس/لینوکس است و مناسب محیط‌های شبکه‌ای می‌باشد.

SMB/CIFS: پروتکل پیش‌فرض اشتراک‌گذاری فایل و چاپگر در ویندوز است که در لینوکس نیز معتبر بوده و برای اشتراک‌گذاری بین سکوهای مختلف[۴۰] مورد استفاده قرار می‌گیرد.

۳. فرمت‌های رایج برای فایل‌سیستم‌های خاص

Swap: یک فضای مبادله است که به‌عنوان extension RAM استفاده می‌شود.

tmpfs: یک فایل‌سیستم سریع برای استفاده در حافظه موقت است.

procfs و sysfs: فایل‌سیستم‌های مجازی هستند که به‌عنوان رابط بین فضای هسته و فضای کاربر عمل می‌کنند. procfs که نقطه اتصالش در proc/ قرار دارد، حاوی اطلاعات فرآیندها و برخی اطلاعات سیستم است. sysfs هم با نقطه اتصال sys/ حاوی اطلاعات مربوط به سخت‌افزار و درایورهای هسته می‌باشد.

---

[37] High-Performance
[38] storage systems
[39] Metadata
[40] Cross-Platform

۴. فرمت‌های فایل‌سیستم ویندوز

NTFS: در لینوکس نیز پشتیبانی می‌شود و برای درایوهای اشتراکی با ویندوز مناسب است.

FAT: فرمتی با سازگاری بالا برای همه سیستم‌عامل‌هاست و بنابراین برای فلش‌درایوها و درایوهای قابل حمل مناسب می‌باشد. اما محدودیت حجم دارد و از فایل‌های بزرگتر از 4GB پشتیبانی نمی‌کند.

۵. فرمت‌های فایل‌سیستم مک

+HFS: در لینوکس نیز پشتیبانی می‌شود و برای درایوهای اشتراکی با MacOS مناسب است.

APFS: در لینوکس، تنها به شکل «فقط خواندنی» پشتیبانی می‌شود و برای استفاده، نیاز به درایورهای خاص دارد.

۲-۷- سطح اجرایی

سطوح اجرایی[۴۱]، حالت‌های عملیاتی مختلف در سیستم‌عامل لینوکس هستند که مشخص می‌کنند کدام سرویس‌ها و daemons در سیستم در حال اجرا باشند. به عبارتی هر سطح اجرا، مجموعه‌ای خاص از سرویس‌ها را فعال یا غیرفعال می‌کند. هفت سطح اجرایی استاندارد در لینوکس تعریف شده است. سطح عملیاتی تعریف شده در هر یک از این سطوح را در جدول ۲-۲ مشاهده می‌کنید. البته تعریف سطوح در توزیع‌های مبتنی بر دبیان با توزیع‌های مبتنی بر ردهت کمی متفاوت است.

---

[41] Runlevels

سطح اجرایی معادل	systemd target	توضیحات
۰	poweroff.target	خاموش کردن
۱	rescue.target	حالت rescue (تک‌کاربره بدون شبکه برای recovery یا maintenance)
۲, ۳, ۴	multi-user.target	حالت چندکاربره متنی
۵	graphical.target	حالت گرافیکی (X Window)
۶	reboot.target	راه‌اندازی مجدد

جدول ۲-۲- سطوح اجرایی و معادل آن‌ها در سیستم‌های جدید

سطوح اجرایی در سیستم‌های مدرن با نام target شناخته شده و در Systemd تعریف شده‌اند. فایل پیکربندی مرتبط با این سطوح در مسیر /etc/systemd/system/default.target قرار گرفته است. در حالیکه در سیستم‌های قدیمی‌تر، تعریف سطوح اجرایی در System V init بود و فایل پیکربندی مرتبط با آن‌ها در مسیر /etc/inittab قرار می‌گرفت. به همین دلیل در جدول ۲-۲، هم شماره سطح اجرایی و هم معادل جدید آن یعنی target آورده شده است.

برای مشاهده سطح اجرایی فعلی سیستم، از دستور زیر استفاده می‌شود. خروجی دستور، شماره سطح اجرایی و زمان فعال شدن آن را نمایش خواهد داد.

```
who   -r
```

برای تغییر سطح اجرایی، از دستور زیر استفاده می‌شود. در این مثال، سیستم به سطح اجرایی ۶ می‌رود که به معنی راه‌اندازی مجدد می‌باشد.

```
sudo init 6
```

به‌عنوان مثالی دیگر، هرگاه نتوانستید سیستم را به حالت عادی بوت نمایید کافی است با حالت rescue که به همین منظور طراحی گردیده است، وارد سیستم شده و مشکل را برطرف نمایید.

۲-۸- فرآیند بوت سیستم

فرآیند بوت شدن در سیستم‌عامل‌های لینوکسی، یک زنجیره منظم از اقدامات سخت‌افزاری و نرم‌افزاری است که سیستم را از حالت خاموش به حالت آماده‌به‌کار برای کاربر می‌رساند. این فرآیند به‌صورت مختصر شامل مراحل زیر می‌شود.

آغازگر بوت. پس از فشردن دکمه روشن و ورود جریان برق به سیستم، اولین کدی که اجرا می‌شود، سیستم ورودی/خروجی پایه[۴۲] (BIOS) یا در سیستم‌های جدیدتر، رابط میان‌افزار توسعه‌پذیر یکپارچه[۴۳] (UEFI) است. وظیفه اصلی این مرحله، اجرای فرآیند خودآزمایی هنگام روشن شدن[۴۴] است تا از سلامت اولیه سخت‌افزارهای حیاتی مانند حافظه اصلی و پردازنده اطمینان حاصل شود. سپس، BIOS/UEFI بر اساس ترتیب بوت تعیین‌شده، اولین دیسک قابل‌بوت - مثلا هارد دیسک - را شناسایی کرده و کنترل را به آن واگذار می‌کند.

نکته: در سیستم‌های UEFI، وجود یک پارتیشن سیستمی از نوع EFI[45] که به اختصار ESP نامیده می‌شود، برای ذخیره فایل‌های بوت‌لودر ضروری است.

بوت‌لودر. پس از شناسایی دیسک قابل بوت، نوبت به بوت‌لودر[۴۶] می‌رسد که در سیستم‌های لینوکس مدرن، GRUB2[47] رایج‌ترین گزینه است. GRUB2 در دیسک‌های قدیمی در MBR و در سیستم‌های جدید در پارتیشن سیستمی ESP

---

[42] Basic Input/Output System
[43] Unified Extensible Firmware Interface
[44] Power-On Self-Test (POST)
[45]EFI System Partition (ESP)
[46] Boot Loader
[47] GRand Unified Bootloader Version 2

نصب می‌شود و مسئولیت بارگذاری هسته لینوکس و فایل‌سیستمی موقت به نام initramfs را بر عهده دارد. GRUB2 با نمایش یک منوی گرافیکی یا متنی به کاربر، امکان انتخاب بین هسته‌های مختلف یا سایر سیستم‌عامل‌های نصب‌شده در سیستم را فراهم می‌آورد و سپس هسته انتخابی و فایل initramfs مرتبط را از دایرکتوری /boot در حافظه اصلی بارگذاری می‌نماید.

نکته ۱: معادل initramfs در سیستم‌های قدیمی‌تر، initrd نام دارد.

نکته ۲: linux loader (LILO) یکی از اولین بوت‌لودرهای لینوکس بود و تا سال‌ها به‌عنوان بوت‌لودر پیش‌فرض بسیاری از توزیع‌ها مورد استفاده قرار می‌گرفت. اما پس از معرفی GRUB که مزایای متعددی نسبت به LILO داشت، منسوخ گردید. مهمترین این مزایا عبارت بودند از: پشتیبانی از فایل‌سیستم‌ها و امکان ویرایش پیکربندی بدون نیاز به بازنویسی MBR.

هسته. پس از بارگذاری هسته در حافظه، کنترل سیستم به آن واگذار می‌شود. هسته ابتدا ساختارهای داخلی خود مانند مدیرحافظه و زمان‌بند را مقداردهی اولیه[۴۸] می‌کند. سپس هسته، ماژول‌های ضروری خود مانند درایورهای دیسک‌های سخت، سیستم‌های فایل RAID و LVM را که در فایل‌سیستم موقت initramfs قرار دارند، بارگذاری می‌کند تا با استفاده از آن‌ها بتواند به دیسک اصلی دسترسی پیدا کرده و فایل‌سیستم ریشه[۴۹] را متصل نماید[۵۰].

بررسی سلامت فایل‌سیستم. پیش از اتصال کامل دیسک سخت، سیستم‌عامل با استفاده از ابزار[51]fsck سلامت فایل‌سیستم‌ها را بررسی می‌کند تا از عدم وجود خطاهای ساختاری که می‌تواند به داده‌ها آسیب بزنند، اطمینان حاصل کند.

---

[48] Initialization
[49] Root Filesystem
[50] Mount
[51] File System Consistency Check

اولین فرآیند. پس از اتصال موفق فایل‌سیستم ریشه، هسته اولین فرآیند فضای کاربر[۵۲] را اجرا کرده و کنترل سیستم را به آن واگذار می‌نماید. این فرآیند با PID=1 و به‌عنوان والد همه فرآیندهای دیگر شناسایی می‌شود. در توزیع‌های مدرن لینوکس، این فرآیند systemd است که جایگزین init در سیستم‌های قدیمی‌تر شده است. وظیفه systemd این است که بر اساس سطح اجرایی فعال - یا همان هدف پیش‌فرض در سیستم‌های جدیدتر - تمام سرویس‌های مورد نیاز سیستم را به ترتیب و بر اساس وابستگی‌هایشان راه‌اندازی کند. مثلا هدف پیش‌فرض ممکن است graphical.target برای حالت گرافیکی و یا multi-user.target برای حالت متنی باشد.

اجرای محیط کاربری. پس از راه‌اندازی تمام سرویس‌های سیستمی، باید محیط برای کاربر آماده شود. در سیستم‌های گرافیکی، Display Manager مثلا LightDM یا GDM اجرا شده و صفحه ورود[۵۳] را نمایش می‌دهد. در سیستم‌های بدون واسط گرافیکی، سرویس‌های getty روی ترمینال‌های مجازی مانند tty1 اجرا می‌شوند تا کاربر بتواند از طریق خط فرمان وارد سیستم شود. در نهایت، با ورود موفق کاربر به سیستم، فرآیند بوت به اتمام می‌رسد.

---

[52] User Space
[53] Login Screen

تمرین

سیستم خود را در حالت گرافیکی بالا آورده و سپس از طریق system settings تنظیمات مرتبط با هر بخش را یافته و تنظیمات خواسته شده را اعمال نمایید:

- بخش About یا System Info را یافته و اطلاعات پردازنده، حافظه، نسخه سیستم‌عامل و نوع دیسک را استخراج کنید.
- به بخش Date & Time و Region & Language مراجعه و منطقه زمانی و زبان سیستم را به دلخواه خود تنظیم کرده و سپس تغییرات را در سیستم مشاهده نمایید.
- در بخش Startup Applications، لیست برنامه‌هایی که هنگام بالا آمدن سیستم اجرا می‌شوند را مشاهده کرده و یک برنامه غیرضروری را غیرفعال کنید. سپس سیستم را راه‌اندازی مجدد کرده و تاثیر آن را مشاهده نمایید.
- با استفاده از ابزار Disks، پارتیشن‌های موجود را مشاهده کرده و اطلاعاتی مانند نوع فایل‌سیستم، حجم و فضای آزاد هر پارتیشن را استخراج نمایید.
- به بخش Appearance رفته و تم، آیکون‌ها، و تصویر پس‌زمینه سیستم خود را تغییر دهید. سپس پنل پایین دسکتاپ یا همان نواروظیفه را با افزودن ابزارهای دلخواه، سفارشی کنید.

گزارش کار

- تنظیمات مربوط به بوت‌لودر GRUB در مسیر /etc/default/grub قرار دارد. فایل را با استفاده از ویرایشگر متنی گرافیکی پیش‌فرض باز کرده و تنظیمات جاری را مشاهده نمایید. تحقیق کنید که چگونه می‌توان زمان انتظار منوی بوت را به مقدار دلخواه و یا به بی‌نهایت، افزایش داد. توجه داشته باشید که بدون دسترسی به کاربر ریشه، امکان ثبت تغییرات در این فایل را نخواهید داشت.
- صفحه تنظیمات شبکه را در محیط گرافیکی یافته و سیستم خود را به شبکه محلی یا اینترنت متصل نمایید.
- با استفاده از App Centers یا Software Manager یک برنامه جدید مانند GIMP یا VLC را در سیستم خود نصب کنید.
- برنامه Update Manager را در توزیع خود یافته و سیستم را به‌روزرسانی نمایید.
- با استفاده از System Monitor، وضعیت لحظه‌ای پردازنده، حافظه اصلی، شبکه و دیسک را بررسی کنید.

منابع و مراجع:

- Garrels M. *Introduction to Linux*, Fultus Corporation, 2010.
- Fox R. *Linux with Operating System Concepts*, Chapman and Hall/CRC, 2021.
- Dalheimer MK, Welsh M. *Running Linux*, O'Reilly, 2002.
- https://www.gnu.org/software/grub/manual/grub, visited 2026 Sep.
- https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/index.html, visited 2022 Apr.

آزمایش ۳ : خط فرمان

اهداف:

- آشنایی با دستورات لینوکس

ابزار و مفاهیم مورد نیاز:

- Linux Terminal Emulator

محتوا:

- واسط خط فرمان
- شکل کلی دستورات خط فرمان
- SYNOPSIS
- Switches / Options
- راهبری
- مجوزها
- Sudo
- صفحات راهنما
- دستورات مدیریت فایل و دایرکتوری
- دستورات جستجو و مرتب سازی
- دستورات مدیریت و نظارت بر فرآیندها
- نصب، حذف و مدیریت بسته‌ها

۳-۱-دستورات خط فرمان

واسط خط فرمان (CLI[1])، مکانیزمی برای برقراری ارتباط با سیستم‌عامل یا برنامه‌ها و اجرای دستورات بوسیله نوشتن است.

- هر دستور، یک خلاصه[۲] دارد که نحوه کاربرد کلی دستور را مشخص می‌کند.
- دستورات لینوکس به بزرگی و کوچکی حروف حساس می‌باشند.
- حضور سوئیچ‌های[۳] مختلف، به هر دستور قابلیت‌های بیشتری می‌دهد و دستور را تعاملی و انعطاف‌پذیر می‌سازد.
- سوئیچ‌هایی که به‌صورت «تک حرفی» هستند اغلب به کمک یک علامت خط تیره[۴] «-» مشخص می‌شوند. معمولا می‌توان چند سوئیچ تک حرفی را به‌طور همزمان به‌کار برد که در آن صورت، همه سوئیچ‌ها بعد از یک علامت خط تیره‌ی مشترک قرار می‌گیرند.
- برخی دیگر از سوئیچ‌ها به‌صورت کلمه یا عبارت تعریف شده‌اند. این نوع سوئیچ‌ها بعد از دو علامت خط تیره‌ی پشت‌سرهم «--» قرار می‌گیرند. در این حالت هر سوئیچ، -- مجزا و مخصوص به خود را خواهد داشت.

۳-۲- راهبری[۵]

دستور pwd : سرحرف print working directory است و مسیر جاری را نمایش می‌دهد.

```
pwd    [options]
```

دستور cd : سرحرف کلمات change directory است و برای ورود به دایرکتوری و یا تغییر آن به‌کار می‌رود.

```
cd    [options]      نام و مسیر دایرکتوری
```

---

[1] Command Line Interface
[2] SYNOPSIS
[3] Switchs /Options
[4] Dash
[5] Navigation

مثال : ورود به دایرکتوری دسکتاپ

```
cd     Desktop
```

بازگشت به دایرکتوری خانگی کاربر جاری

```
cd\
```

توجه: در برخی توزیع‌ها برای اجرای دستور باید دوبار کلید ENTER را بزنید.

بازگشت به دایرکتوری بالاتر (معادل دکمه up درWindows Explorer)

```
cd  ..
```

بازگشت سریع به آخرین دایرکتوری (معادل دکمه back در Windows Explorer)

```
cd -
```

دستور clear : محتویات صفحه‌نمایش ترمینال را پاک می‌کند. برای مشاهده دستورات و محتوای قبلی، نوارلغزان[۶] پنجره ترمینال را به سمت بالا بکشید.

```
clear
```

دستور reboot : سیستم را راه‌اندازی مجدد می‌کند.

```
reboot
```

دستور halt : سیستم را shut down می‌کند.

```
halt
```

---

[6] Scroll bar

دستور ls : سرحرف list segments است و برای نمایش لیست فایل‌ها و دایرکتوری‌ها در مسیر جاری یا مسیر مشخص شده به‌کار می‌رود.

```
ls    [options]   [Path]
```

نکته: درصورتی‌که مسیر را مشخص نکنید، بر روی دایرکتوری جاری عمل خواهد کرد.

سوئیچ‌ها:

S- : مرتب کردن خروجی بر اساس اندازه (در حالت پیش‌فرض مرتب سازی بر اساس حروف الفبا است)

t- : مرتب کردن خروجی بر اساس زمان تغییر

l- : نمایش لیست با جزئیات بیشتر (long list format)

a- : فایل‌های مخفی را نیز در لیست نمایش می‌دهد.

نکته ۱: برای نمایش فایل‌های مخفی در حالت گرافیکی، کافی است گزینه «نمایش فایل‌های مخفی» را از منوی view در مرورگر فایل[۷] انتخاب کنید.

نکته ۲: در لینوکس برای مخفی کردن یک فایل کافی است در ابتدای نام فایل، یک کاراکتر «نقطه» اضافه نمایید.

مثال زیر از سه سوئیچ به‌طور همزمان استفاده می‌کند.

```
ls   -lth

total 62K
drwxr-xr-x.     2    root  root  4.0K  Dec 3 22:25     Documents
drwxr-xr-x.     2    root  root  4.0K  Dec 2 20:47     Downloads
-rw-r--r--.     1    root  root  46K   Nov 28 15:22    install.log
```

اطلاعاتی که در نتیجه‌ی اجرای دستور بالا در ترمینال نمایش داده شده‌اند، به ترتیب از چپ به راست عبارتند از:

---

[7] File Explorer

مجوزها[8]، تعداد لینک‌ها یا زیردایرکتوری‌های موجود در داخل دایرکتوری فعلی، نام کاربر مالک[9] فایل یا دایرکتوری، نام گروه. سایز برحسب بایت، تاریخ آخرین تغییر[10]، نام فایل یا دایرکتوری.

مجوزها:

هر مجوز از 4 بخش مجزا تشکیل شده است، به‌عنوان مثال، مجوزی که در سطر اول مثال قبل نمایش داده شده است را در نظر بگیرید.

d	rwx	r-x	r-x
بخش ۱	بخش ۲	بخش ۳	بخش ۴

بخش ۱ : اگر d بود یعنی آیتم موردنظر دایرکتوری است. اگر علامت - بود یعنی آیتم موردنظر فایل است. کاراکتر l نیز به منزله‌ی لینک یا فایل میان‌بر[11] است.

بخش ۲ : مجوزهای مربوط به کاربر مالک فایل یا دایرکتوری را نمایش می‌دهد.

بخش ۳ : مجوزهای مربوط به هم‌گروهی‌های کاربر مالک را نشان می‌دهد. در اینجا فقط اعضای گروه اصلی مالک، مدنظر هستند.

بخش ۴ : مجوزهای مربوط به سایر کاربران را نمایش می‌دهد.

هر بخش، از سه حرف تشکیل شده است. معنای هر یک از این حروف در زیر آمده است.

---

[8] Permissions
[9] Owner
[10] Modification
[11] Shortcut

r : اجازه خواندن (**r**ead) برای کاربر موردنظر وجود دارد.

w : اجازه نوشتن (**w**rite) برای کاربر موردنظر وجود دارد.

x : اجازه اجرا کردن (e**x**ecute) برای کاربر موردنظر وجود دارد.

- : اجازه موردنظر وجود ندارد.

۳-۳- مجوزها و کاربران

دستور chmod : سرحرف change mode است و برای تغییرمجوزهای فایل و دایرکتوری به‌کار می‌رود. مجوزها مشخص می‌کنند که چه افرادی چه مواردی را می‌توانند بخوانند، بنویسند و یا تغییر دهند.

```
chmod    [options]    value    آدرس و نام فایل موردنظر
```

علاوه بر حروف، هر مجوز را می‌توان با یک مقدار عددی نیز نمایش داد:

Read: 4, **W**rite: 2, E**x**ecute: 1

همان‌طور که دیدیم، مجوز هر فایل یا دایرکتوری از سه بخش (برای کاربر مالک، هم‌گروهی‌ها و سایر افراد) تشکیل شده است. برای مشخص کردن مجوزهای هر کاربر، مقدار عددی تمام مجوزهای مربوط به آن کاربر را با هم جمع کرده و در نهایت بخش مربوط به هر کاربر را فقط با یک مقدار عددی[۱۲] نمایش می‌دهیم.

به‌عنوان مثال، یک دایرکتوری با مجوز rwxr-x--x را در نظر بگیرید. می‌خواهیم مجوز این دایرکتوری را طوری تغییر بدهیم که کاربر مالک همان مجوزهای قبلی را حفظ کند، هم‌گروهی‌ها فقط مجوز خواندن داشته باشند ولی سایر افراد هیچ مجوزی نداشته باشند. ابتدا نمایش عددی مجوز جاری را محاسبه می‌کنیم.

---

[12] Numeric Value

کاربر مالک	گروه	سایر افراد	
r + w + x	r + x	x	
4+2+1=7	4+0+1=5	0+0+1=1	➔ 751

بنابراین مجوز فعلی این دایرکتوری، با مقدار عددی ۷۵۱ نمایش داده می‌شود. مجوز جدید عبارت است از rwxr----- که معادل عددی آن می‌شود: 740 = (0+0+0)(4+0+0)(4+2+1). برای تغییر مجوز به مقدار جدید، به‌صورت زیر عمل می‌کنیم.

```
chmod   740     Desktop/MyDirectory
```

سوئیچ‌ها:

-v : اطلاعات تغییر مجوز را نمایش می‌دهد.

-R : مجوز تعیین شده را به زیردایرکتوری‌ها و فایل‌ها نیز اعمال می‌کند.

مثال: دستور زیر تغییر مجوز را با حروف معادل، انجام می‌دهد و مجوز فایلی به نام myfile را برابر با ۷۵۴ تنظیم می‌نماید.

```
chmod           u=rwx,g=rx,o=r          myfile
```

کاراکتر u نماینده کاربرمالک، g نماینده هم گروهی‌ها و o نماینده سایر افراد است. مجوزها نیز با همان کاراکترهای r,w,x مشخص شده‌اند. با اجرای دستور بالا، مجوزهای قبلی فایل از بین رفته و مجوز جدید (۷۵۴) به آن داده می‌شود.

اگر فقط قصد تغییر مجوزهای یک دسته از کاربران را داشتید، به‌صورت زیر عمل کنید. به‌عنوان مثال، این دستور فقط مجوز سایر کاربران را به r,w تغییر می‌دهد، اما مجوز مالک و هم گروهی‌ها به همان صورتی که قبلا بود باقی خواهد ماند. به این ترتیب، مجوز کل برابر ۷۵۶ می‌شود.

```
chmod     o=rw          myfile
```

همچنین ممکن است بخواهید برای یکی از دسته‌های کاربری، فقط یکی از مجوزها را اضافه یا کم نمایید.بدون اینکه سایر مجوزهای آن کاربر تغییر کند. مثال زیر، فقط مجوز اجرا را به سایرکاربران اضافه می‌کند. مجوز خواندن و نوشتن برای این دسته از کاربران به همان‌صورتی که از قبل تعریف شده بود باقی می‌ماند. پس مجوز فایل از ۷۵۶ که در مثال قبل به آن رسیدیم به ۷۵۷ تغییر خواهد کرد.

```
chmod    o+x          myfile
```

مثال بعدی مجوز خواندن را از هم گروهی‌ها می‌گیرد. پس مجوز فایل به ۷۱۷ تغییر خواهد یافت.

```
chmod      g-r          myfile
```

برای تغییر مجوز همه‌ی کاربران به‌صورت یکسان، می‌توان از کاراکتر a[13] استفاده کرد. مثال زیر مجوز نوشتن را به همه کاربران اعطا می‌کند. مجوز خواندن و اجرا برای هر کاربر، به همان صورتی که بود باقی خواهد ماند. پس مجوز فایل برابر با ۷۳۷ خواهد شد.

```
chmod      a+w          myfile
```

نکته: برای مشاهده مجوزها در حالت گرافیکی، روی فایل یا دایرکتوری راست‌کلیک کرده و پس از انتخاب گزینه properties زبانه permissions را بینید.

دستور chown : برای تغییر مالک و گروه یک فایل به‌کار می‌رود.

```
sudo   chown   username:groupname     آدرس و نام فایل موردنظر
```

---

[13] All

دستور su : سرحرف switch user است و برای تغییر به کاربری دیگر در ترمینال به‌کار می‌رود.

```
su      username
```

پس از اجرای دستور، رمزعبور کاربر موردنظر از شما درخواست می‌شود؛ البته به غیر از زمانی که بخواهید از کاربر ریشه به یک کاربر دیگر سوئیچ کنید. برای بازگشت به کاربر قبلی، دستور exit را اجرا نمایید.

نکته ۱: در ترمینال هرگاه به کاربر ریشه سوئیچ کنید، اعلان[۱۴] به‌صورت کاراکتر # نمایش داده می‌شود اما برای کاربران غیرریشه، این علامت به‌صورت کاراکتر $ است.

نکته ۲: در اکثر توزیع‌های لینوکسی هنگام وارد کردن رمزعبور در ترمینال، هیچ کاراکتری مثل * یا ● به‌جای حروف رمز، ظاهر نمی‌شود. بنابراین فقط بادقت رمزعبور خود را وارد کرده و کلید Enter را بزنید.

دستور sudo : برای اجرای برخی دستورات یا وظایف مدیریتی[۱۵] لازم است که مجوزهای کاربر ریشه را داشته باشیم. اما به‌جای سوئیچ به این کاربر، می‌توان از دستور sudo بدین‌منظور استفاده نمود. این دستور، به کاربران ویژه اجازه می‌دهد تا برخی از دستورات مدیریتی را اجرا نمایند، بدون اینکه رمز کاربر ریشه را در اختیار داشته باشند و یا به آن سوئیچ کنند.

در توزیع‌های مبتنی بر ردهت، معمولا قابلیت sudo به‌صورت پیش‌فرض برای کاربرانی غیر از superusers فعال نیست. در مقابل، در اکثر توزیع‌های مبتنی بر دبیان مانند اوبونتو، کاربر ریشه به‌صورت پیش‌فرض فعال نیست و هنگام نصب، صرفا یک کاربر غیرریشه ایجاد می‌شود که قابلیت sudo به شکل پیش‌فرض و بدون نیاز به انجام تنظیمات خاصی، برایش فعال است.

چنین کاربری، هنگام اجرای دستورات مدیریتی در ترمینال باید دستور sudo را در ابتدای خط ذکر نماید. پس از زدن کلید Enter، رمزعبور کاربر از وی درخواست خواهد شد. مشابه همین اتفاق، هنگام انجام کارهای مدیریتی در حالت

---

[14] prompt

[15] Administrative Tasks

گرافیکی نیز وجود دارد. به عنوان مثال، وقتی با یک کاربر sudoer در حالت گرافیکی وارد سیستم شده و از برنامه «مدیریت نرم‌افزار» بسته‌ای را برای نصب انتخاب نمایید، یک پنجره باز شده و رمزعبورتان را درخواست خواهد کرد.

برای فعال کردن کاربر ریشه در توزیع‌های مبتنی بر اوبونتو، کافی است برای این کاربر رمزعبور تعریف کنیم. در مثال زیر، کاربر جاری که از نوع sudoer می‌باشد، student نام دارد. بنابراین بعد از اجرای دستور، ابتدا باید رمزعبور کاربر student را وارد کنید و سپس رمزعبور انتخابی برای کاربر ریشه را دو بار وارد نمایید.

```
student@oslab:~$ sudo  passwd  root
student@oslab:~$ [sudo] password for student:
New password:
Retype new password:
passwd: password updated successfully
```

از این پس، با یکی از دو دستور زیر می‌توانید به کاربر ریشه سوئیچ کنید.

```
su  -
```

یا

```
su root
```

دستور useradd : برای ایجاد کاربر جدید استفاده می‌شود.

```
sudo   useradd   [options]    نام کاربر جدید
```

سوئیچ‌ها:

g- : تعیین گروه اصلی کاربر

G- : افزودن کاربر به گروه‌های تکمیلی (نام گروه‌ها با ویرگول از هم جدا می‌شوند)

مثال: ایجاد یک کاربر با نام student و عضویت آن در گروه‌های sudo و adm.

```
sudo   useradd   -G  sudo,adm    student
```

نکته ۱: چون گروه اصلی کاربر را مشخص نکرده‌ایم، یک گروه جدید هم‌نام با کاربر ایجاد شده و کاربر به عضویت آن درخواهد آمد.

نکته ۲: پس از ایجاد کاربر - بویژه در سیستم‌های چندکاربره - حتما با استفاده از دستور passwd برایش رمزعبور تعیین نمایید.

دستور usermod : برای تغییر اطلاعات کاربر، مورد استفاده قرار می‌گیرد.

```
sudo    usermod    [options]    نام کاربر موردنظر
```

سوئیچ‌ها:

l- : تغییر نام کاربری

s- : تغییر پوسته پیش‌فرض

aG- : افزودن کاربر به گروه جدید بدون حذف گروه‌های قبلی

L- : قفل کردن حساب کاربری

U- : باز کردن قفل حساب کاربری

مثال: کاربر ایجاد شده در مثال قبل را به گروه docker اضافه کرده و پوسته پیش‌فرضش را به fish تغییر می‌دهیم.

```
sudo    usermod    -aG  docker    -s  /usr/bin/fish    student
```

توجه: گروه و پوسته مذکور، به‌صورت پیش‌فرض در سیستم وجود ندارند.

دستور id : برای مشاهده اطلاعات کاربر از جمله شناسه کاربری، شناسه گروه و لیست گروه‌هایی که کاربر عضو آن‌هاست مورد استفاده قرار می‌گیرد.

```
id      نام کاربر موردنظر
```

دستور groups : برای نمایش گروه‌های کاربر استفاده می‌شود.

```
groups      نام کاربر موردنظر
```

دستور userdel : برای حذف کاربران مورد استفاده قرار می‌گیرد.

```
sudo    userdel    [options]     نام کاربر موردنظر
```

به‌منظور حذف دایرکتوری خانگی و صندوق پستی[۱۶] محلی کاربر، سوئیچ r- را اضافه کنید.

مثال: برای پاک کردن کاربر ایجاد شده در مثال قبل همراه با داده‌های شخصی وی، دستور را به شکل زیر می‌نویسیم.

```
userdel     -r     student
```

دستور groupadd : به‌منظور ایجاد یک گروه جدید، به‌کار می‌رود.

```
sudo    groupadd     نام گروه جدید
```

دستور groupdel : برای حذف یک گروه موجود، از این دستور استفاده می‌شود.

```
sudo    groupdel     نام گروه موردنظر
```

دستور groupmod : با کمک این دستور می‌توان نام یک گروه را تغییر داد.

```
sudo    groupmod    -n     نام جدید گروه     نام قبلی گروه
```

---

[16] Email Spool

۳-۴- جستجو

دستور find : در فایل‌سیستم سلسله‌مراتبی لینوکس، از دستور find برای یافتن فایل‌ها استفاده می‌شود. به کمک این دستور می‌توان جستجو را را بر اساس نام، مالک، گروه، نوع، مجوزها، تاریخ و سایر مولفه‌های یک فایل انجام داد. جستجو در تمامی زیر شاخه‌های مسیر تعیین شده انجام خواهد گرفت.

```
find   [options]    [path]    file-expression
```

فرض کنید در دایرکتوری خانگی کاربر هستیم که شامل آیتم های زیر می‌باشد. پنج مثال بعدی، در این دایرکتوری اجرا خواهند شد.

```
ls

Desktop          linux.pdf        Pictures         test1
Downloads        LiNuX.pdf        test             test.txt
```

مثال ۱ : فایل‌های با نام test را در مسیر جاری و زیرشاخه‌هایش جستجو می‌کند. در بخش نتایج، فقط یک فایل با این مشخصات پیدا و نمایش داده شده است.

```
find  -name  test

./test
```

مثال ۲: تمام فایل‌هایی که بخش ابتدایی نامشان test است را می‌یابد. نتیجه شامل سه فایل است که با عبارت test آغاز شده‌اند.

```
find       -name           "test*"

./test
./test1
./test.txt
```

مثال ۳: فایلی با نام text.txt را در مسیر /home/student/ که همان دایرکتوری خانگی کاربر است، جستجو می‌کند. ملاحظه می‌نمایید که فقط یک نتیجه که مطابقت کامل با دستور جستجو دارد، نمایش داده شده است.

```
find   /home/student/     -name     text.txt

/home/student/text.txt
```

مثال ۴: فایل linux.pdf را بدون توجه به بزرگ و کوچک بودن حروف نام فایل، جستجو می‌کند.

```
find     -iname      linux.pdf

./linux.pdf
./LiNuX.pdf
```

مثال ۵: فایل‌های با حجم ۱۰ مگابایت یا بیشتر را جستجو می‌نماید.

```
find  -size  +10M

./linux.pdf
```

دستور grep : برای جستجوی یک عبارت الگو که می‌تواند یک کلمه یا رشته باشد - درون فایل‌ها استفاده می‌شود و تمام سطرهای داخل فایل را که حاوی عبارت مورد جستجو هستند، نمایش می‌دهد.

```
grep   [options]   pattern_word      آدرس و نام فایل(های) موردنظر برای جستجو
```

سوئیچ‌ها:

c- : فقط تعداد کل سطرهایی که حاوی عبارت موردنظر هستند را نمایش می‌دهد

i- : در هنگام جستجو، تفاوتی بین حروف بزرگ و کوچک[17] قائل نمی‌شود.

l- : فقط نام فایل‌هایی که حاوی عبارت مورد جستجو هستند را لیست می‌کند.

n- : شماره سطر را نیز نمایش می‌دهد.

color-- : عبارت مورد جستجو را در خروجی، به شکل رنگی نمایش می‌دهد.

^ : اگر در ابتدای عبارت الگو قرار بگیرد، تمام سطرهایی را نمایش می‌دهد که با عبارت موردنظر آغاز می‌شوند.

---

[17] Case Insensitive

مثال ۱: دستور زیر، کلمه configuration را در فایلی به نام linux-commands.txt که در دسکتاپ قرار دارد، جستجو کرده و تمام سطرهای درون فایل که حاوی این کلمه هستند را چاپ می‌نماید.

```
grep  --color    configuration    Desktop/linux-commands.txt
  Yum uses a configuration file at /etc/yum.conf.
  Additional configuration files are also read from the directories
  Also this configuration is set to it,
  The inclusion of external configuration files is supported.
```

مثال ۲: نتیجه اجرای مثال زیر، تمام سطرهایی خواهد بود که اولین کلمه آن‌ها there باشد.

```
grep  ^there    Desktop/linux-commands.txt
```

نکته ۱: اگر اولین کاراکتر در یک سطر، کاراکتر فاصله باشد، آن سطر در نتایج نمایش داده نخواهد شد.

نکته ۲: برای جستجوی همزمان در چند فایل، کافی است نام و مسیر هر فایل را با یک فاصله در ادامه دستور اضافه کنید و یا می‌توانید با کاراکتر * در بین تمام فایل‌ها با یک پسوند خاص و یا فایل‌هایی که بخشی از نامشان مشترک است و ... جستجو نمایید.

مثال ۳: دستور زیر در بین تمام فایل‌های موجود در دسکتاپ، به‌دنبال کلمه configuration می‌گردد. در نتایج جستجو، قبل از هر سطر، نام فایل مربوطه نیز ذکر خواهد شد.

```
grep       configuration         *
```

نکته: برای جستجوی یک رشته، آن را در بین دو علامت '' قرار دهید.

```
grep       'configuration file'        *
```

دستور sort : برای مرتب‌سازی خطوط یک فایل متنی مورد استفاده قرار می‌گیرد.

```
sort  [options]   [FILE]
```

دستور who : کاربرانی که در حال حاضر به سیستم وارد شده‌اند را نمایش می‌دهد.

```
who
```

دستور whereis : مکان قرارگیری فایل‌های باینری، کدمنبع و راهنماهای مربوط به یک برنامه یا دستور را نمایش می‌دهد.

```
whereis  [options]      نام دستور یا برنامه موردنظر
```

سوئیچ‌ها:

-m : فقط آدرس فایل‌های راهنما را نمایش می‌دهد.

-b : فقط آدرس فایل‌های باینری را نمایش می‌دهد.

-s : فقط آدرس کدمنبع را نمایش می‌دهد.

دستور which : مشخص می‌کند که با اجرای یک دستور یا برنامه، کدام فایل از روی دیسک اجرا می‌شود. یا به عبارتی، فایل اجرایی دستور یا برنامه موردنظر در کجا قرار دارد.

```
which     نام دستور یا برنامه موردنظر
```

۳-۵- مدیریت فایل و دایرکتوری

دستور mkdir : یک دایرکتوری[18] جدید ایجاد می‌کند.

```
mkdir       آدرس و نام دایرکتوری جدید
```

دستور rmdir : یک دایرکتوری موجود را پاک میکند.

```
rmdir       آدرس و نام دایرکتوری موردنظر
```

---

[18] Directory

توجه: این دستور فقط امکان پاک کردن دایرکتوری‌های خالی را دارد. بنابراین اگر دایرکتوری موردنظر حاوی زیردایرکتوری بود، باید از پاک کردن آن‌ها شروع کنیم. همچنین این دستور قادر به پاک کردن فایل‌ها نمی‌باشد.

دستور rm : برای پاک کردن فایل‌ها استفاده می‌شود.

```
rm      آدرس و نام فایل(های) موردنظر
```

برای پاک کردن یک دایرکتوری با تمام محتویاتش، از سوئیچ r- استفاده کنید.

دستور cp : امکان کپی کردن فایل‌ها و دایرکتوری‌ها را فراهم می‌آورد.

```
cp    [options]        آدرس مبدا و نام فایل موردنظر        آدرس مقصد و نام فایل
```

درصورتی‌که قصد کپی کردن یک دایرکتوری را دارید، از سوئیچ r- استفاده نمایید.

مثال: دستور زیر، فایل test.txt را از دایرکتوری خانگی کاربر تحت یک نام جدید به دسکتاپ وی کپی می‌کند.

```
cp    /home/student/test.txt      /home/Desktop/test_copy.txt
```

دستور mv : برای جابه‌جایی یک فایل در فایل‌سیستم مورد استفاده قرار می‌گیرد. امکان تغییر نام فایل همزمان با جابه‌جایی نیز وجود دارد. اما درصورتی‌که قصد تغییر نام فایل را ندارید، کافی است در بخش آخر دستور، فقط آدرس مقصد را ذکر نمایید.

```
mv      آدرس و نام قبلی فایل        آدرس و نام جدید فایل
```

چند نکته کاربردی

ویژگی «تکمیل خودکار[19]» معمولا در ترمینال فعال است. هنگام نوشتن یک دستور یا نام یک دایرکتوری، اگر بخشی از آن را نوشته و کلید tab را بزنید بقیه دستور یا نام، به‌صورت خودکار، پر می‌شود.

---

[19] Auto Fill

البته اگر چند دایرکتوری با حروف ابتدایی یکسان وجود داشته باشد، فقط تا پایان قسمت مشترک پر می‌شود و اگر دوبار پشت سر هم کلید tab را بزنید آن‌گاه تمام گزینه‌های موجود که با آن حروف آغاز می‌شوند را نمایش می‌دهد. همچنین اگر دستورات دیگری با همان حروف اولیه وجود داشته باشند، تمام دستوراتی که با آن حروف شروع می‌شوند نمایش داده خواهد شد.

اگر نام فایل یا دایرکتوری شامل کاراکتر فاصله باشد، هنگام نوشتن آن نام در ترمینال باید کاراکتر فاصله را پس از یک کاراکتر \ قرار بدهیم. درغیراین‌صورت، نام نوشته شده به منزله دو نام مجزا در نظر گرفته می‌شود.

دستور tar[20] : برای ایجاد و مدیریت فایل‌های آرشیو مورد استفاده قرار می‌گیرد. آرشیو کردن، امکان جمع‌آوری و بسته‌بندی چندین فایل را در یک فایل واحد و در عین حفظ ساختار دایرکتوری‌ها و فراداده‌ها فراهم می‌آورد.

```
tar [options]    new_name.tar    نام فایل‌ها و دایرکتوری‌های موردنظر
```

سوئیچ‌ها:

c- : ساخت[21] فایل آرشیو

f- : نگاشت آرشیو ساخته شده به یک فایل

u- : آپدیت فایل آرشیو موجود بر اساس تغییرات ایجاد شده در فایل اصلی

r- : الحاق فایل یا دایرکتوری جدید به آرشیو موجود. این سوئیچ، آیتم جدید را به داخل بالاترین دایرکتوری اضافه می‌کند.

x- : استخراج فایل آرشیو

---

[20] Tape Archive
[21] Create

مثال ۱: دو فایل file1 و file2 در یک فایل جدید به نام myarchive آرشیو شده و در مسیر /root/Documents ذخیره می‌گردد.

```
tar  -cf  Documents/myarchive.tar   Desktop/file1   Desktop/file2
```

مثال ۲: آیتم‌های آرشیو ایجاد شده در مثال ۱، استخراج می‌شوند.

```
tar   -xvf   myarchive.tar
```

ترکیب با فشرده ساز xz : برای اینکه همزمان با ساخت آرشیو، عملیات فشرده‌سازی نیز بر روی فایل‌ها صورت بگیرد به شکل زیر عمل می‌کنیم.

```
tar   -cJf   archive.tar.xz   files_name&path
```

دستور tar، قابلیت ترکیب با فشرده‌سازهای دیگری همچون gzip و bzip2 را نیز دارد.

۳-۶- پردازش متن

دستور cat : محتوای یک یا چند فایل متنی را به‌صورت پیوسته و پشت سر هم در ترمینال نمایش می‌دهد.

```
cat   [options]   آدرس و نام فایل(های) متنی
```

برای نمایش شماره سطر، سوئیچ n- را اضافه کنید.

دستور sed : این دستور یکی از ابزارهای کلیدی برای پردازش متن است که برای جستجو، جایگزینی، حذف و درج خطوط در یک فایل متنی یا خروجی یک دستور استفاده می‌شود. این ابزار یک ویرایشگر جریان[۲۲] است، یعنی به‌صورت خط‌به‌خط عمل می‌کند و داده‌ها را از ورودی خوانده، پردازش کرده و به خروجی می‌فرستد. بنابراین فایل اصلی بدون تغییر باقی خواهد ماند و تغییرات صرفا در نمایش متن خروجی، انجام می‌پذیرند؛ مگر اینکه از سوئیچ i- استفاده شود.

---

[22] Stream Editor

```
sed 's/old_text/new_text/g'  نام و آدرس فایل متنی موردنظر
```

کاراکتر s، فرمان جایگزینی را صادر می‌کند. در حالیکه پرچم g اعلام می‌کند که جایگزینی مذکور باید روی تمامی موارد[۲۳] صورت بپذیرد.

مثال: جایگزینی تمام کلمات error با warning در یک فایل به نام log.txt

```
sed   's/error/warning/g'   log.txt
```

دستور awk : این دستور یک زبان برنامه‌نویسی و ابزار پردازش متن است که برای استخراج، فیلتر، محاسبه و گزارش‌گیری از داده‌های ساختاریافته مانند فایل‌های CSV و لاگ‌ها، طراحی شده است. این ابزار بر اساس یک جداکننده مانند فاصله یا ویرگول، هر خط از فایل را به فیلدهایی تقسیم کرده و امکان دسترسی به هر فیلد را فراهم می‌کند. برای استخراج فیلدهای خاص از یک فایل به‌صورت زیر عمل می‌کنیم:

```
awk   '{print $1, $3}'    نام و آدرس فایل متنی موردنظر
```

در این دستور، $1 مشخص کننده فیلد اول و $3 بیانگر فیلد سوم است. بنابراین فقط فیلد اول و سوم از فایل استخراج شده و نمایش داده می‌شود.

مثال: برای استخراج نام کاربری و پوسته پیش‌فرض هر کاربر از فایل /etc/passwd که فیلدهایش با کاراکتر : از هم جدا شده‌اند، از دستور مذکور به شکل زیر استفاده می‌کنیم.

```
awk   -F: '{print $1, $7}'   /etc/passwd
```

سوئیچ F- برای مشخص کردن کاراکتر جداکننده فیلدها اضافه شده است. بلافاصله پس از این سوئیچ کاراکتر جداکننده که در اینجا «دونقطه روی هم» است، می‌آید.

---

[23] Global

۳-۷- مدیریت دیسک

دستور fdisk[24] : برای مدیریت جدول پارتیشن‌بندی دیسک‌ها به‌کار می‌رود و بر اساس استاندارد MBR [25] کار می‌کند. پارتیشن‌بندی دیسک یکی از مراحل اولیه برای آماده‌سازی دیسک‌ها به‌منظور ذخیره کردن داده می‌باشد و این ابزار fdisk است که امکان ایجاد، حذف، تغییر نوع و نمایش پارتیشن‌ها را در اختیار ما قرار می‌دهد.

```
sudo   fdisk   [options]   device
```

مثال ۱ : مشاهده جدول پارتیشن‌بندی یک دیسک

```
sudo   fdisk   -l   /dev/sda
```

مثال ۲ : نمایش جدول پارتیشن تمام دیسک‌های متصل

```
sudo   fdisk   -l
```

مثال ۳: ورود به حالت تعاملی برای مدیریت پارتیشن‌ها

```
sudo   fdisk   /dev/sdb
```

پس از ورود به حالت تعاملی، با استفاده از دستورات داخلی زیر می‌توانید پارتیشن‌ها را مدیریت کنید:

m : نمایش راهنما

p : نمایش جدول پارتیشن فعلی

n : ایجاد پارتیشن جدید

---

[24] Fixed Disk

[25] Master Boot Record

d : حذف یک پارتیشن

t : تغییر نوع پارتیشن

w : ذخیره تغییرات و خروج

q : خروج بدون ذخیره تغییرات

به‌عنوان مثال، مراحل ایجاد یک پارتیشن جدید را در ادامه می‌بینیم. اجرای این مراحل را می‌توانید در آزمایش شماره ۱۶ (فایل‌سیستم)، به‌طور کامل ملاحظه نمایید.

```
sudo fdisk /dev/sdb1

Command (m for help): n
Partition type: p
Partition number (1-4, default 1): 1
First sector (2048-...) : 2048
Last sector: +5G
Command (m for help): w
```

به‌روزرسانی هسته : ممکن است پس از ایجاد یا تغییر پارتیشن، هسته نیاز به به‌روزرسانی داشته باشد. برای اعمال تغییرات بدون نیاز به راه‌اندازی مجدد، دستور زیر را اجرا کنید.

```
sudo    partprobe    /dev/sdb
```

نکته ۱: fdisk در نسخه‌های جدید خود از دیسک‌های GPT نیز پشتیبانی می‌کند.

نکته ۲ : fdisk مستقیما با دیسک کار می‌کند و بنابراین در استفاده از آن باید بسیار محتاط بود زیرا یک اشتباه کوچک می‌تواند منجر به از دست رفتن داده‌ها گردد.

دستور df : برای بررسی فضای ذخیره‌سازی و دیسک به کمک شما می‌آید.

```
df    -h
```

برای نمایش خروجی به شکل ساده‌تر و با واحدهای اندازه‌گیری رایج، از سوئیچ h- استفاده می‌شود.

```
soltani@VM:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           387M  1.7M  386M   1% /run
/dev/sda3        20G   14G  5.0G  73% /
tmpfs           1.9G     0  1.9G   0% /dev/shm
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
/dev/sda2       512M  6.2M  506M   2% /boot/efi
tmpfs           387M  124K  387M   1% /run/user/1000
```

تصویر ۳-۱- خروجی دستور df

در خروجی این دستور، همانند تصویر ۳-۱، اطلاعات زیر را مشاهده خواهید کرد:

- Filesystem: نام هر درایو
- Size: اندازه‌ی هر درایو
- Used: میزان دیسک استفاده شده توسط آن درایو
- Avail: میزان دیسک آزاد در آن درایو
- Use%: درصد دیسک استفاده شده در آن درایو
- Mounted on: دایرکتوری که درایو بر آن سوار شده است.

دستور mount : یک فایل‌سیستم را به یک نقطه از درخت دایرکتوری موجود، ضمیمه[۲۶] می‌کند و به این ترتیب، به کاربران اجازه می‌دهد تا به فایل‌ها و دایرکتوری‌های موجود در آن فایل‌سیستم دسترسی داشته باشند. عملیات ضمیمه‌سازی می‌تواند هم بر روی یک دستگاه[۲۷] فیزیکی و واقعی و هم بر روی یک دستگاه مجازی[۲۸] انجام بشود.

```
mount   [options]   device   mount_point
```

---

[26] Attach
[27] Device
[28] Loopback device

نکته: به جای عبارت ضمیمه کردن، از اصطلاح متصل‌کردن و سوارکردن[۲۹] نیز استفاده می‌شود.

مثال ۱: برای اتصال یک دستگاه USB به سیستم به‌صورت زیر عمل می‌کنیم. فرض کنید نام دستگاه را قبلا با استفاده از دستور fdisk به دست آورده‌ایم.

```
sudo  mkdir  /media/usb
sudo  mount  /dev/sdb1   /media/usb
```

ابتدا یک دایرکتوری به‌عنوان نقطه اتصال ایجاد کرده و سپس دستگاه را به دایرکتوری ایجاد شده متصل می‌نماییم.

مثال ۲: برای اتصال یک فایل ISO و بررسی محتوای آن بدون نیاز به نوشتن[۳۰] اطلاعات بر روی دیسک، از دستورات زیر استفاده می‌نماییم.

```
sudo  mount  -o   loop   /path/to/ubuntu.iso   /mnt/iso
```

نکته: در این مثال، عملیات اتصال بر روی یک دستگاه مجازی انجام شده است. با این نوع از دستگاه‌ها در آزمایش مربوط به دیسک بیشتر آشنا خواهیم شد.

دستور umount : یک فایل‌سیستم را از نقطه اتصال خود جدا[۳۱] می‌کند. این عمل باعث می‌شود که تمام داده‌های تغییر یافته در حافظه، به دیسک نوشته[۳۲] شوند و سپس دسترسی به آن فایل‌سیستم قطع گردد. جدا کردن صحیح فایل‌سیستم، پیش از خارج کردن یک دیسک یا خاموش کردن سیستم، از خرابی و از دست رفتن داده‌ها جلوگیری می‌کند

```
umount   device_or_mount_point
```

---

[29] Mount
[30] Write
[31] Detach
[32] Flush

مثال ۱: برای جدا کردن دستگاه USB با استفاده از نقطه اتصال به‌صورت زیر عمل می‌کنیم.

```
sudo    umount    /media/usb
```

مثال ۲: عملیات جداسازی با استفاده از نام دستگاه، به‌صورت زیر انجام می‌گیرد.

```
sudo     umount     /dev/sdb1
```

نکته: در سیستم‌های امروزی و بویژه گرافیکی، با اتصال دستگاه‌های جدید مانند حافظه فلش یا با دابل کلیک روی یک پارتیشن، عملیات mount به‌طور خودکار توسط سیستم انجام می‌گیرد. اما پیش از جدا کردن حافظه‌های جانبی، لازم است به‌صورت دستی عملیات umount یا eject را انجام بدهید.

دستور dd : یک ابزار خط فرمانی برای کپی و تبدیل داده‌ها در سطح بایت و بلوک[۳۳] می‌باشد. یعنی به‌صورت مستقیم با بلوک‌های خام و دستگاه‌ها[۳۴] کار می‌کند و بنابراین برای کارهای سطح پایین مانند ایجاد فایل‌های خام، پشتیبان‌گیری از دیسک، و شبیه‌سازی هارد مورد استفاده قرار می‌گیرد. در ادامه با برخی از کاربردهای این دستور آشنا می‌شویم.

ایجاد یک فایل خالی با اندازه مشخص

```
dd  if=/dev/zero  of=file.img  bs=1M  count=100
```

بخش‌های مختلفی که در این دستور استفاده شده‌اند، به شرح زیر می‌باشند:

```
if = /dev/zero
```

---

[33] Block
[34] Devices

فایل ورودی[۳۵] را مشخص می‌کند. یعنی داده‌ها از مسیر /dev/zero خوانده می‌شوند که یک فایل ویژه در لینوکس است و هر بار که خوانده بشود، بایت‌های صفر (0x00) تولید خواهد کرد.

```
of = disk.img
```

فایل خروجی[۳۶] را مشخص می‌کند. یعنی داده‌های خوانده شده در این فایل ذخیره خواهند شد. اگر فایل وجود نداشته باشد، ایجاد می‌گردد.

عبارت bs = 1M، اندازه بلوک‌ها را مشخص می‌کند که در این مثال برابر با ۱ مگابایت در نظر گرفته شده است. بدین ترتیب، هر بار ۱ مگابایت داده خوانده و یا نوشته می‌شود. انتخاب مقادیر بزرگتر برای این گزینه، باعث افزایش سرعت کپی خواهد شد.

عبارت count = 100 نیز تعداد بلوک‌ها را مشخص می‌کند. یعنی چند بلوک از فایل ورودی خوانده شود که در اینجا ۱۰۰ بلوک ۱ مگابایتی در نظر گرفته شده است. در نتیجه ۱۰۰ مگابایت داده تولید خواهد شد.

در تصویر ۳-۲، خروجی دستور dd را مشاهده می‌نمایید که می‌گوید ۱۰۰ بلوک کامل خوانده شده (records in) و ۱۰۰ بلوک کامل نیز نوشته شده (records out) است. در ادامه نیز حجم کل داده‌های نوشته شده بر حسب بایت، زمان اجرا برحسب ثانیه و سرعت انتقال داده بر حسب مگابایت بر ثانیه نمایش داده شده است.

```
soltani@VM:~$ dd  if=/dev/zero  of=file.img  bs=1M  count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.188653 s, 556 MB/s
```

تصویر ۳-۲- ایجاد یک فایل خالی برای مقاصد آزمایشی یا آموزشی

---

[35] Input File
[36] Output File

سایر کاربردهای رایج این دستور عبارتند از:

۱- پشتیبان‌گیری از دیسک

```
dd   if=/dev/sda   of=backup.img   bs=4M
```

۲- نوشتن فایل ISO روی فلش

```
dd   if=image.iso   of=/dev/sdb   bs=4M   status=progress
```

۳- کپی یک فایل متنی همراه با تبدیل حروف

```
dd   if=input.txt   of=output.txt   conv=ucase
```

۴- ایجاد یک فایل با الگوی تصادفی

```
dd   if=/dev/urandom   of=random.img   bs=1M   count=10
```

۵- بازیابی یک پارتیشن که دچاز خرابی شده است

```
dd   if=/dev/sda1   of=/dev/sdb1   bs=4M   conv=noerror,sync
```

ازآنجایی‌که دستور dd به‌صورت مستقیم با دستگاه‌ها کار می‌کند، یک اشتباه کوچک می‌تواند منجر به از دست رفتن داده‌های روی دیسک شود. به‌عنوان مثال اگر در یکی از مثال‌های بالا قرار بدهید of=/dev/sda، کل دیسک بازنویسی خواهد شد.

۳-۸- مشاهده اطلاعات سیستم

دستور uname : اطلاعات سیستم را چاپ می‌کند.

```
uname   [options]
```

سوئیچ‌ها:

m- : نوع پلتفرم سخت‌افزاری[۳۷] و پردازنده را چاپ می‌کند. به کمک این دستور می‌توانید 32 یا 64 بیتی بودن سیستم را تشخیص بدهید.

r- : نسخه هسته را چاپ می‌کند.

s- : نام هسته لینوکس را چاپ می‌کند.

دستور arch : این دستور نیز معماری سیستم را نمایش می‌دهد.

```
arch
```

دستور systemctl : برای آغاز کردن، پایان دادن، راه‌اندازی مجدد و یا بررسی وضعیت یک سرویس در سیستم استفاده می‌شود.

```
systemctl   start/stop/restart/status   service_name
```

مثال زیر سرویس شبکه و اتصال به اینترنت را راه‌اندازی مجدد میکند.

```
systemctl   restart   NetworkManager
```

نکته: در نسخه‌های قدیمی‌تر، به جای این دستور، از دستور service استفاده می‌شد.

---

[37] Hardware Platform

۳-۹- مشاهده و مدیریت فرآیندها

دستور kill : به‌منظور ارسال سیگنال‌های مقتضی به یک فرآیند یا گروهی از فرآیندها، مورد استفاده قرار می‌گیرد. به عنوان مثال، برای ارسال سیگنال خاتمه به یک فرآیند و پایان بخشیدن به آن، پس از نام دستور، شناسه فرآیند موردنظر را می‌آوریم.

```
kill        [options]              PID(process ID)
```

برای مشاهده شناسه[۳۸] فرآیندهای فعال در سیستم، می‌توانید دستور top (که در ادامه به آن خواهیم پرداخت) را اجرا کنید.

دستور killall : یک فرآیند را با استفاده از نامش پایان می‌بخشد.

```
killall          [options]              نام فرآیند موردنظر
```

درصورتی‌که سوئیچ 9- را اضافه کنید، سیگنال شماره ۹ ارسال خواهد شد. این سیگنال برای از بین بردن اجباری[۳۹] فرآیند استفاده می‌شود، حتی اگر فرآیند در حال اجرا باشد.

دستور ps : وضعیت فرآیندها را گزارش می‌کند. این گزارش در لحظه اجرای دستور تولید می‌شود و تعاملی نیست. به عبارت دیگر، فقط در لحظه‌ی اجرای دستور، تصویری از وضعیت فرآیندها می‌گیرد.

```
ps      [options]
```

سوئیچ‌ها:

e- : نمایش تمام فرآیندها

f- : نمایش فرآیندها با جزئیات[۴۰] بیشتر

p Process-ID- : فقط فرآیندهایی با شناسه مشخص شده را لیست می‌کند

--ppid Parent-Process-ID : فقط فرآیندهایی که شناسه‌ی والدشان مشخص شده است را لیست می‌کند

---

[38] Process ID
[39] Force Terminate
[40] Full Format

مثال: می‌خواهیم فقط فرآیندهایی که شناسه‌ی والدشان ۲ است را مشاهده کنیم.

```
ps    -f     --ppid     2
```

دستور top : برنامه‌ای برای نظارت بر عملکرد سیستم است. این دستور، تمام فرآیندهای در حال اجرا و فعال را در یک لیست مرتب و به‌صورت بلادرنگ[۴۱] همراه با به‌روزرسانی‌های منظم، نمایش می‌دهد.

همان‌طور که در تصویر ۳-۳ مشاهده می‌نمایید، مواردی که با اجرای این دستور نمایش داده می‌شوند عبارتند از:

PID: شماره شناسه فرآیند

USER: مالک فرآیند

PR: اولویت[۴۲]، عددی است که هسته لینوکس از آن برای تصمیم‌گیری درباره تخصیص پردازنده استفاده می‌کند. این عدد معمولا در محدوده ۰ تا ۹۹ قرار دارد و توسط هسته مدیریت می‌شود. هرچه این عدد بزرگتر باشد، اولویت بالاتر است و بنابراین آن فرآیند مدت زمان پردازش بیشتری دریافت می‌کند.

NI: Niceness عددی است که کاربر برای تنظیم اولویت یک فرآیند تعیین می‌کند. این عدد در محدوده 20- تا 19+ قرار دارد. هرچه این عدد کمتر باشد، فرآیند اولویت بالاتری خواهد داشت.

VIRT: حافظه مجازی مصرفی شامل فضای مبادله و حافظه اشتراکی

RES: حافظه فیزیکی مصرفی (RAM)

SHR: حافظه اشتراکی با سایر فرآیندها

---

[41] Real Time
[42] Priority

S: وضعیت[۴۳] فرآیند که می‌تواند در یکی از این حالات باشد: در حال اجرا (Running: R)، در حال خواب (Sleeping: S)، زامبی (Zombie: Z) و یا متوقف شده (Stopped: T).

```
top - 05:08:13 up  7:04,  1 user,  load average: 0.01, 0.01, 0.00
Tasks: 296 total,   1 running, 295 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.3 sy,  0.0 ni, 99.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3868.2 total,   1650.8 free,   1332.5 used,   1128.5 buff/cache
MiB Swap:   2140.0 total,   2140.0 free,      0.0 used.   2535.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    968 root      20   0  428468 131692  63852 S   0.7   3.3   1:49.90 Xorg
   5040 soltani   20   0 2888220 299760 159484 S   0.3   7.6   0:03.09 firefox-bin
   5369 soltani   20   0   17404   6144   3968 R   0.3   0.2   0:00.33 top
   5372 root      20   0       0      0      0 I   0.3   0.0   0:00.08 kworker/1:2-mpt_poll_0
      1 root      20   0   22304  13336   9368 S   0.0   0.3   0:01.92 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.07 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   0:00.79 rcu_preempt
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.23 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
```

تصویر ۳-۳- خروجی دستور top

%CPU: درصد استفاده از پردازنده

%MEM: درصد استفاده از RAM

+TIME: زمان کل استفاده از پردازنده تاکنون

COMMAND: نام دستور یا فرآیند

---

[43] Status

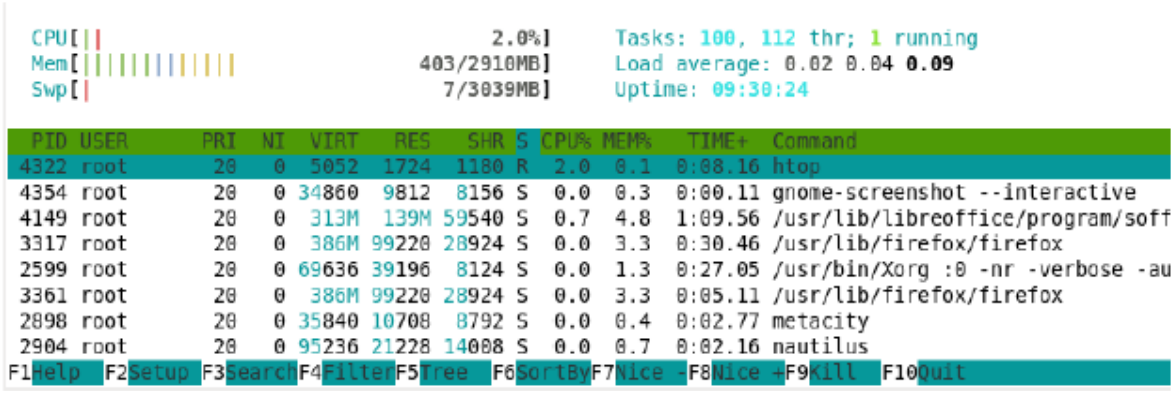
نکته ۱: دستوراتی مثل top، عموما اطلاعات خود را از فایل‌هایی همچون /proc/interrupts که حاوی اطلاعاتی درباره وقفه‌های سخت‌افزاری ایجاد شده توسط دستگاه‌های مختلف است و همینطور /proc/stats که شامل آمار کلی سیستم درباره پردازنده، فرآیندها، وقفه‌ها و موارد دیگر می‌باشد، می‌گیرند.

نکته ۲: /proc یک فایل‌سیستم مجازی است که توسط هسته لینوکس در حافظه ایجاد می‌شود و حاوی فایل‌هایی است که اطلاعات سیستم را به‌صورت زنده نمایش می‌دهند.

دستور htop : همان دستور top است که همراه با رابط کاربری بهتری ارائه شده است. ابزاری تعاملی است که به شکل بلادرنگ سیستم را مانیتور کرده و اطلاعاتی درمورد میزان مصرف پردازنده، حافظه و ... ارائه می‌دهد. درصورتی‌که این ابزار در سیستم شما نصب نیست می‌توانید به شکل زیر آن را نصب نمایید.

```
sudo   apt-get   install   htop
```

پس از نصب، در ترمینال دستور htop را اجرا کنید. این ابزار دارای واسط گرافیکی مستقل نیست و از ترمینال برای ارتباط با کاربر استفاده می‌کند. برای بالا و پایین رفتن در صفحه، از کلیدهای Page Up & Page Down و برای خروج، از کلید q استفاده کنید.



تصویر ۳-۴- خروجی دستور htop

در قسمت بالای تصویر، اطلاعات مربوط به پردازنده، اطلاعات حافظه و swap نمایش داده می‌شوند. معنای رنگ خطوط برای حافظه و swap متفاوت است:

- سبز: میزان حافظه مصرفی توسط فرآیندها
- آبی: میزان بافر مصرفی
- زرد: میزان cache
- بالا سمت راست، تعداد وظایف[۴۴] در حال اجرا یا فعال، میانگین بار و زمان uptime را می‌بینید. در قسمت میانگین بار، سه عدد وجود دارد. اولی میانگین در بازه یک دقیقه، دومی در پنج و سومی در پانزده دقیقه است.

همان ستون‌ها و اطلاعاتی که برای دستور top وجود داشت، در اینجا نیز در بخش وسط تصویر، قابل مشاهده است. با این تفاوت که به دلیل خاصیت تعاملی htop، روی هر کدام از ستون‌ها که کلیک کنید، اطلاعات براساس آن ستون مرتب خواهد شد.

دستور netstat : یک ابزار خط فرمانی برای نظارت بر آمار بسته‌ها و پورت‌های ورودی و خروجی شبکه است. این ابزار به‌منظور نظارت بر عملکرد شبکه و عیب‌یابی مشکلات مربوط به آن، برای مدیران سیستم بسیار مفید می‌باشد.

```
netstat   [options]
```

مثال

```
netstat   -a | more
```

خروجی دستور در تصویر زیر نمایش داده شده است.

---

[44] Tasks

```
soltani@OSLaboratory:~$ netstat -a | more
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 localhost:ipp           0.0.0.0:*               LISTEN
tcp        0      0 _localdnsstub:domain    0.0.0.0:*               LISTEN
tcp        0      0 _localdnsproxy:domain   0.0.0.0:*               LISTEN
tcp6       0      0 ip6-localhost:ipp       [::]:*                  LISTEN
udp        0      0 0.0.0.0:mdns            0.0.0.0:*
udp        0      0 OSLaboratory:52750      _gateway:domain         ESTABLISHED
udp        0      0 _localdnsproxy:domain   0.0.0.0:*
udp        0      0 _localdnsstub:domain    0.0.0.0:*
udp        0      0 OSLaboratory:bootpc     192.168.244.254:bootps  ESTABLISHED
--More--
```

تصویر ۳-۵- خروجی دستور netstat

نکته: در لینوکس‌های مدرن netstat با دستور ss جایگزین شده است.

دستور lsof : ابزاری قدرتمند برای نظارت[۴۵] بر عملکرد در سیستم‌عامل‌های لینوکسی و یونیکسی است که فهرستی از تمام فایل‌ها و فرآیندهای باز را به شما نمایش می‌دهد. این ابزار، اطلاعات کاملی از فایل‌های روی دیسک، سوکت‌های شبکه، پایپ‌ها، دستگاه‌های متصل و پردازش‌های در حال اجرا را ارائه می‌کند.

استفاده از این دستور در مقایسه با سایر ابزارهای نظارت بر عملکرد در لینوکس، بویژه زمانی که جداسازی[۴۶] دیسک از سیستم با خطاهای مربوط به عدم دسترسی یا نقص در فایل‌سیستم مواجه می‌شود، از مزایای قابل توجهی برخوردار است. این ابزار به شما امکان می‌دهد تا بدون نیاز به جداسازی دیسک، به اطلاعات مربوط به عملکرد آن دسترسی پیدا کنید و مشکلات را به سرعت شناسایی و حل نمایید. با این دستور به‌راحتی خواهید فهمید که چه فایل‌هایی در حال استفاده هستند و مانع جداسازی دیسک می‌شوند. نحوه استفاده از دستور به‌صورت زیر است.

```
lsof
```

---

[45] Monitoring
[46] Unmount

دستور chkconfig : برای مشاهده، تنظیم یا تغییر لیست سرویس‌هایی به‌کار می‌رود که در زمان راه‌اندازی[۴۷] سیستم، به شکل خودکار آغاز می‌شوند. اجرای دستور بدون هیچ‌گونه سوئیچی یا همراه با list-- سرویس‌های مذکور را لیست می‌کند.

```
chkconfig  --list
cpuspeed   0:off      1:on    2:on     3:on    4:on    5:on   6:off
vsftpd     0:off      1:off   2:off    3:off   4:off   5:off  6:off
network    0:off      1:off   2:on     3:on    4:on    5:on   6:off
```

اعداد ۰ تا ۶، سطوح اجرایی برنامه‌ها را مشخص می‌کنند.

۳-۱۰- راهنما و مستندات

دستور man : کتابچه راهنمای مربوط به دستور یا برنامه موردنظر را در صورت وجود نمایش می‌دهد.

```
man   [options]     نام دستور یا برنامه موردنظر
```

راهنما، در همان صفحه ترمینال نمایش داده می‌شود. برای جابه‌جا شدن در صفحه، از کلیدهای u و d و برای خروج، از کلید q استفاده کنید.

مثال: اجرای دستور زیر راهنمای مربوط به دستور ls را نمایش خواهد داد.

```
man    ls
```

سوئیچ h- یا help-- : این سوئیچ تقریبا برای تمام دستورات لینوکس تعریف شده است و راهنمای مربوط به آن دستور یا برنامه را نمایش می‌دهد.

```
نام دستور یا برنامه موردنظر        -h     یا      --help
```

مثال

```
ls     --help
```

---

[47] Sartup

۳-۱۱-مخازن نرم‌افزاری

مخازن[۴۸] (Repo)، منابع نرم‌افزاری هستند که می‌توان بسته‌های نرم‌افزاری را از آن‌ها دریافت و بر روی سیستم نصب کرد و یا بسته‌های موجود را به‌روزرسانی نمود. هر توزیع لینوکسی، مخازن مربوط به خود را دارد که معمولا متعدد و مختلفند. آدرس مخزن‌های اصلی در فایل /etc/apt/sources.list قرار می‌گیرد.

دستور apt-get یا apt

یک مدیر بسته است و برای نصب، حذف و مدیریت بسته‌های نرم‌افزاری در لینوکس‌های Debian-Based، مورد استفاده قرار می‌گیرد. توزیع‌هایی از قبیل Debian، Mint و Ubuntu عضوی از این گروه هستند. این دستور، معادل دستور yum در توزیع‌های Redhat-Based می‌باشد.

دستور apt-get به‌صورت چند فرمانی است. یعنی ابتدا apt-get (یا فقط apt) نوشته می‌شود و سپس دستور فرعی موردنظر مثل نصب یا حذف را ذکر می‌کنیم.

نصب بسته: برای نصب یک بسته نرم‌افزاری به‌صورت زیر عمل می‌کنیم.

```
apt-get  install   نام بسته موردنظر
```

به‌عنوان مثال نصب برنامه unrar که وظیفه استخراج فایل‌های rar را دارد به این ترتیب است.

```
$ sudo    apt-get        install    unrar
```

همان‌طور که از علامت اعلان در ابتدای خط پیداست، کاربر ریشه نیست. بنابراین از دستور sudo پیش از این دستور استفاده شده است.

---

[48] Repository

به همین ترتیب برای نصب هر بسته دیگر، فقط کافی است نام بسته را به اسم مدنظر خود تغییر دهید. مثلا برای نصب پوسته فیش به شکل زیر عمل می‌کنیم.

```
sudo   apt   install   fish
```

همان‌طور که ملاحظه می‌کنید، می‌توانید به جای apt-get از apt نیز استفاده نمایید.

حذف بسته: دستور زیر یک بسته نرم‌افزاری نصب شده را پاک می‌کند، اما فایل‌های پیکربندی آن را نگه می‌دارد.

```
apt-get   remove   نام بسته موردنظر
```

برای مثال برنامه unrar را پاک می‌کنیم.

```
sudo        apt-get        remove          unrar
```

برای اینکه فایل‌های پیکربندی نیز حذف شوند، سوئیچ زیر را اضافه کنید.

```
apt-get   --purge   remove   نام بسته موردنظر
```

تمرین

موارد زیر را به ترتیب و فقط از طریق خط فرمان انجام دهید:

- یک دایرکتوری با نام خودتان در دایرکتوری Downloads که در دایرکتوری خانگی کاربر جاری قرار دارد ایجاد کنید. به‌عنوان مثال اگر نام کاربری شما student باشد، مسیر مذکور به این شکل خواهد بود:

/home/student/Downloads

- فایل /etc/default/useradd را به دایرکتوری‌ای که در بخش اول ساختید، کپی کنید. درصورتی‌که این فایل در توزیع شما وجود ندارد، می‌توانید یک فایل متنی دیگر را انتخاب نمایید.
- محتوای این فایل متنی را به‌صورت مرتب شده در ترمینال نمایش دهید.
- با اجرای یک دستور، مکان فایل باینری مربوط به دستور ls را بیابید.

گزارش کار

- از طریق خط فرمان، در مسیر /etc/default به‌دنبال فایلی به نام grub بگردید.
- با استفاده از دستور grep و سوئیچ‌های مقتضی، از درون این فایل فقط سطرهایی را لیست کنید که با کاراکتر # شروع نمی‌شوند. (نکته: سطرهایی که با کاراکتر # آغاز می‌شوند، کامنت هستند.)
- همین کار را با استفاده از دستور sed نیز انجام بدهید.
- فقط با استفاده از خط فرمان، اطلاعاتی درباره دستورهای جدیدی مانند cut به دست بیاورید.

منابع و مراجع:

- Purcell J, Hat R. *Linux Complete Command Reference*, Sams, 1997.
- Dalheimer MK, Welsh M. *Running Linux*, O'Reilly, 2002.
- https://tldp.org/LDP/GNU-Linux-Tools-Summary/html/index.html, visited 2022.

آزمایش ۴ : اسکریپت‌نویسی پوسته

اهداف:

- آشنایی با مفاهیم اولیه اسکریپت‌نویسی پوسته

ابزار و مفاهیم مورد نیاز:

- CLI
- ابزارهای ویرایش متن
- پوسته bash یا sh

محتوا:

- آشنایی با مفاهیم و کاربردهای اسکریپت‌نویسی پوسته
- آشنایی با ابزارهای موردنیاز
  - ویرایشگرهای متنی
  - عملگرهای هدایتگر
  - مدیریت مجوز فایل و دایرکتوری در لینوکس
  - انواع پوسته
- ایجاد و اجرای اسکریپت‌های ساده
- متغیرها

۴-۱- معرفی

اسکریپت پوسته[1]، برنامه‌ای است که توسط مفسر[2] پوسته تفسیر و اجرا می‌گردد. پوسته، وظیفه دریافت اسکریپت‌ها و دستورات خط فرمان و اجرای آن‌ها را دارد. به طور کلی، اسکریپت‌ها توانایی انجام کارهایی مانند ایجاد تغییرات در فایل‌سیستم، اجرای برنامه‌ها، نمایش پیام‌ها و موارد مشابه را دارند.

هر سطر از یک اسکریپت پوسته، شامل یک دستور استاندارد پوسته یا لینوکس است. همان‌طور که در فصل قبل با دستورات خط فرمان لینوکس آشنا شدید، هر یک از این دستورات به تنهایی و به‌صورت مستقیم از خط فرمان نیز قابل اجرا هستند. دستورات پوسته نیز در ادامه معرفی خواهند شد. از جمله مزایای استفاده از اسکریپت‌ها این است که با اجرای یک فایل اسکریپت، تمامی دستورات موجود در آن به‌صورت خودکار اجرا می‌شوند و نیازی به نوشتن دوباره دستورات در تکرارهای بعدی نداریم. اسکریپت‌های پوسته نیازی به کامپایل ندارند، بلکه به شکل مستقیم توسط پوسته تفسیر و اجرا می‌گردند.

۴-۱-۱- چرا اسکریپت‌نویسی پوسته؟

آشنایی با مفاهیم اسکریپت‌نویسی برای مدیر سیستم[3] ضروری است. چرا که حتی اگر مدیر قصد اسکریپت‌نویسی به معنای واقعی نداشته باشد، بازهم برای تحلیل و درک رفتار سیستم به این دانش نیاز خواهد داشت. به‌عنوان مثال، هنگامی که یک سیستم لینوکسی راه‌اندازی می‌شود، اسکریپت‌های سیستمی موجود در مسیر /etc/rc.d اجرا می‌شوند که حاوی تنظیمات و سرویس‌های سیستم هستند. یک مدیر برای تحلیل رفتار سیستم باید با این اسکریپت‌های راه‌انداز[4] و طرز کار آن‌ها آشنایی داشته و در صورت لزوم بتواند آن‌ها را تغییر دهد. از اسکریپت‌نویسی پوسته بیشتر برای خودکارسازی کارهای روزمره، ایجاد گزارش خطا، نظارت[5] بر سیستم و همگام‌سازی استفاده می‌شود.

---

[1] Shell Scripting
[2] Interpretor
[3] Admin
[4] Startup Scripts
[5] Monitoring

۴-۱-۲- چه زمانی نباید از اسکریپت‌نویسی پوسته استفاده کرد؟

اسکریپت‌نویسی پوسته معمولا برای برنامه‌های سنگین - یعنی برنامه‌هایی که نیازمند منابع محاسباتی یا انجام عملیات ریاضی زیاد هستند - و همچنین برای ایجاد برنامه‌های پیچیده یا برنامه‌های گرافیکی، انتخاب مناسبی نیست.

۴-۲- پیش‌نیازها

برای شروع اسکریپت‌نویسی، علاوه بر دانشی که از بخش پیشین به دست آوردید، لازم است با برخی ابزارهای ضروری نیز آشنا شوید. در ادامه به توضیح و بررسی این ابزارها می‌پردازیم.

۴-۲-۱- ویرایشگرهای متنی

برای ایجاد و ویرایش فایل‌های متنی در لینوکس، هم برنامه‌های خط فرمانی و هم برنامه‌های گرافیکی مختلفی در دسترس می‌باشند. در ادامه با یک نمونه از هر دسته آشنا می‌شویم.

الف) ویرایشگرهای با واسط گرافیکی. ویرایشگرهایی همچون kwriter، emacs و gedit معمولا ظاهری شبیه Notepad ویندوز دارند اما در واقع قابلیت‌های بسیار بیشتری را ارائه می‌نمایند. به‌عنوان مثال، اگر از آن‌ها برای کدنویسی استفاده نمایید، کدها را تشخیص داده و همانند یک IDE بخش‌های مختلف کد را به‌صورت رنگی نمایش می‌دهند. به‌عنوان یک نمونه، در تصویر ۴-۱ محیط ویرایشگر xed را مشاهده می‌نمایید. در بیشتر توزیع‌های لینوکسی حداقل یک ویرایشگر پیش‌فرض در سیستم وجود دارد که با جستجوی عبارت Text Editor در منوی اصلی سیستم قابل دستیابی است.

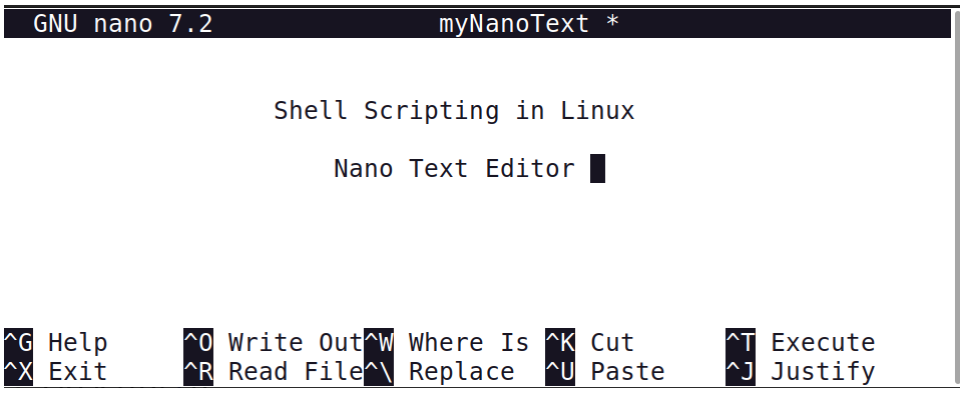


تصویر۴-۱- ویرایشگر xed

ب) ویرایشگرهای خط فرمانی. این ویرایشگرها از پنجره ترمینال به‌عنوان واسط کاربری استفاده می‌نمایند. مانند vim و nano. برای ورود به محیط ویرایشگر نانو، ابتدا دستور nano و سپس با یک فاصله، نام و مسیر فایل موردنظر خود را بنویسید. مثال زیر یک فایل متنی با نام myTxtFile در مسیر جاری ایجاد می‌نماید.

```
nano   myTxtFile
```

نکته: اگر فایلی با همین نام و در همین مسیر از قبل موجود باشد، فایل موجود باز می‌شود. درغیراین‌صورت، یک فایل جدید و خالی با این نام ایجاد خواهد شد. در تصویر ۴-۲ محیط ویرایشگر نانو را مشاهده می‌کنید. در نوار پایین ویرایشگر، راهنمای عملکردهای موجود در برنامه نمایش داده می‌شوند و قسمت بالای این منو، برای نوشتن متن در نظر گرفته شده است.



تصویر ۴-۲- ویرایشگر nano

بلافاصله پس از باز شدن فایل، می‌توانید شروع به نوشتن نمایید. برای خروج از ویرایشگر و بازگشت به ترمینال، باید از کلیدهای میان‌بری که در قسمت پایین ویرایشگر نمایش داده می‌شوند استفاده کنید. بدین‌منظور، ابتدا کلیدهای Ctrl+x را فشار دهید. پیامی در پایین صفحه، نمایش داده می‌شود که می‌پرسد «آیا قصد ذخیره تغییرات را دارید؟» در صورت پاسخ مثبت، کلید y و درغیراین‌صورت کلید n را بزنید. سپس پرسش دیگری مطرح می‌شود که در آن، مسیر و نام فایل نمایش داده شده است و به شما امکان Save As کردن فایل را می‌دهد. اگر قصد چنین کاری را ندارید، فقط کلید Enter را بزنید تا از ویرایشگر خارج شده و به ترمینال بازگردید. می‌توانید به آدرسی که فایل را در آن ایجاد کرده‌اید رفته و وجود فایل خود را بررسی نمایید.

ویرایشگر vim: برای ایجاد یک فایل جدید در محیط vim، دستور زیر را اجرا کنید. کار با این ویرایشگر، نیاز به ممارست بیشتری دارد. برای شروع، می‌توانید نکات کلیدی که در ادامه آمده را در نظر بگیرید.

```
vim   myTxtFile2.txt
```

در vim دو حالت «تایپ» و «دستور» وجود دارد. از حالت تایپ برای ویرایش متن و از حالت دستور برای فرمان دادن به برنامه - مثلا ذخیره کردن یا خروج از برنامه - استفاده می‌شود. برای فعال شدن حالت تایپ، پس از ورود به محیط vim یک‌بار کلید i را بزنید. قبل از زدن این دکمه، هر چه تایپ کنید چیزی نوشته نمی‌شود. برای ذخیره کردن

تغییرات و خروج از برنامه ابتدا کلید Esc را بزنید تا حالت دستوری فعال شود و سپس همزمان با پایین نگه داشتن کلید shift، دو بار کلید z را بزنید.

توجه داشته باشید که انتخاب ویرایشگر برای اسکریپت‌نویسی امری سلیقه‌ای است و نوع آن هیچ تاثیری در نتیجه و اجرای اسکریپت نخواهد گذاشت. بنابراین ویرایشگری را که با آن راحت‌تر هستید انتخاب نمایید.

۴-۲-۲- عملگرهای هدایتگر

با استفاده از عملگرهای هدایتگر[6] می‌توان خروجی یک دستور یا اسکریپت را به یک دستور دیگر و یا به یک فایل هدایت کرد. به این ترتیب، نتایج به جای نمایش در ترمینال، به دستور بعدی و یا فایل مشخص شده منتقل خواهند شد. سه هدایتگر پرکاربرد در ادامه معرفی شده‌اند.

الف) هدایتگر < : یک عملگر است که خروجی یک دستور را به یک فایل هدایت می‌نماید.

```
دستور  <  نام و مسیر فایل
```

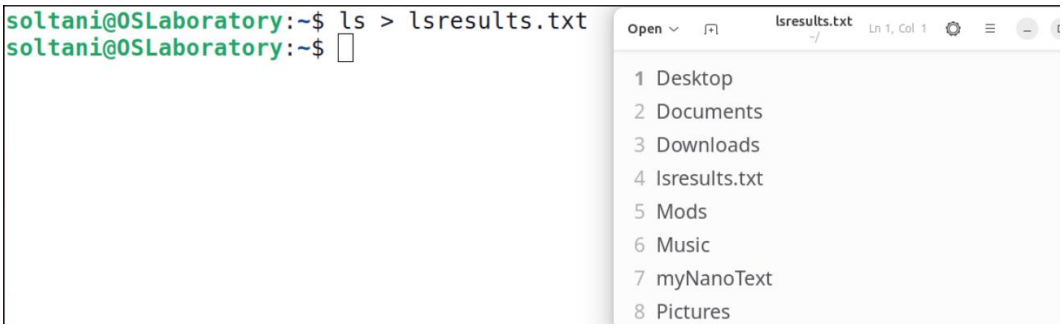
در مثال زیر، دستور ls محتویات دایرکتوری جاری - در اینجا دایرکتوری خانگی کاربر - را لیست کرده، اما همان‌طور که در تصویر ۴-۳ می‌بینید، چیزی در ترمینال نمایش داده نمی‌شود. زیرا خروجی در فایلی به نام lsresults.txt نوشته شده است.

```
ls > lsresults.txt
```

---

[6] Redirection

نکته : اگر محتوای فایل lsresults.txt را بررسی نمایید، متوجه خواهید شد که اسم خود فایل نیز در لیست نتایج قرار دارد. علت این امر این است که ابتدا فایل مذکور ایجاد می‌گردد و سپس دستور ls اجرا شده و خروجی در فایل نوشته می‌شود.



تصویر ۴-۳- استفاده از هدایتگر < و فایل حاصل

ب) هدایتگر < : این هدایتگر اطلاعات داخل یک فایل را خوانده و به‌عنوان ورودی به یک دستور تحویل می‌نماید.

```
دستور  <  نام و مسیر فایل
```

در مثال زیر، فایل lsresults.txt که در بخش قبل ایجاد کردیم را به‌عنوان ورودی به دستور wc می‌دهیم. این دستور با سوئیچ w– تعداد کلمات داخل فایل را شمرده و نمایش خواهد داد.

```
wc  -w  < lsresults.txt
```

نکته: ورودی دستور از فایل گرفته می‌شود ولی نتیجه‌ی دستور در ترمینال نمایش داده می‌شود.

در این مثال نیز همان فایل قبلی را به دستور sort داده‌ایم تا محتوای آن را به‌صورت مرتب شده بر اساس حروف الفبا، در ترمینال نمایش بدهد.

```
sort < lsresults.txt
```

ج) **هدایتگر** | : این هدایتگر، پایپ نام دارد و خروجی یک دستور را به‌عنوان ورودی به یک دستور دیگر هدایت می‌کند.

```
دستور۲  |  دستور۱
```

مثال. اگر دستور ls را در مسیر مشخص شده در مثال زیر اجرا کنید، یک لیست بسیار طولانی برای شما نمایش داده خواهد شد که برای مطالعه تمام آن، باید در پنجره ترمینال اسکرول نمایید.

```
ls   -l   /bin/ | more
```

اما با استفاده از دستور more و عملگر پایپ، همان‌طور که در تصویر ۴-۴ می‌بینید، نتایج به‌صورت صفحه‌به‌صفحه نمایش داده خواهد شد و با هر بار فشردن کلید Enter یا Space به سطر یا صفحه بعدی هدایت می‌شوید.

```
soltani@OSLaboratory:~$ ls -l /bin/ | more
total 240064
-rwxr-xr-x 1 root root       55744 Apr  5  2024 [
-rwxr-xr-x 1 root root       18744 Jul 15  2024 aa-enabled
-rwxr-xr-x 1 root root       18744 Jul 15  2024 aa-exec
-rwxr-xr-x 1 root root       18736 Jul 15  2024 aa-features-abi
-rwxr-xr-x 1 root root       22912 Apr  7  2024 aconnect
-rwxr-xr-x 1 root root        1622 Sep 27  2024 acpidbg
-rwxr-xr-x 1 root root       16422 Apr 18  2024 add-apt-repository
--More--
```

تصویر ۴-۴-عملگر پایپ

در مثال زیر نیز با کمک هدایتگر پایپ، نتایج حاصل از دستور سمت چپ (ls) به‌عنوان ورودی به دستور سمت راست (grep) داده می‌شود. این کار مانند این است که خروجی ls را در یک فایل متنی ذخیره کرده و سپس دستور grep را بر روی آن فایل اجرا نماییم. به این ترتیب، تمام سطرهایی که حاوی کلمه server هستند، در ترمینال نمایش داده خواهد شد.

```
ls   -l   /bin   | grep server --color
```

نکته: فقط خروجی حاصل از دستور دوم در ترمینال نمایش داده می‌شود.

دستور زیر را اجرا کرده و نتیجه‌ی آن را با مثال بالا مقایسه کنید.

```
find   /bin   -name   "*server*"
```

در مثال زیر نیز دستور sort محتوای همان فایل قبلی را پس از مرتب‌سازی، به‌عنوان ورودی به دستور دوم می‌دهد. دستور دوم، فقط ۵ سطر ابتدایی را در ترمینال نمایش خواهد داد.

```
sort  lsresults.txt  |  head  -n  5
```

نکته: درصورتی‌که دستور head بدون سوئیچ «تعداد سطرها» استفاده شود، به شکل پیش‌فرض، ۱۰ سطر اول از فایل ورودی را نمایش خواهد داد.

مثال ترکیبی ۱: در این مثال به‌صورت همزمان در یک خط دستور از دو هدایتگر استفاده شده است. خروجی ps به‌عنوان ورودی به sort داده شده و پس از مرتب‌سازی، در فایلی به نام pssort.txt ذخیره می‌گردد.

```
ps | sort > pssort.txt
```

مثال ترکیبی ۲: مثال زیر، سطر ۵ الی ۱۰ از نتایج مرتب شده حاصل از جستجوی کلمه information، در فایل‌هایی با پسوند conf. که در مسیر etc/ قرار دارند را در صورت وجود، نمایش می‌دهد.

```
grep  information  /etc/*.conf  | sort | head  -n  10 | tail  -n  5
```

مثال ترکیبی 3: این‌بار می‌خواهیم علاوه بر استفاده از هدایتگرها، چند دستور مجزا را پشت سرهم نوشته و درنهایت برای اجرا به مفسر ارسال نماییم. بدین‌منظور کافی است در انتهای هرمجموعه دستور، از علامت ";" استفاده نمایید.

```
sort lsresults.txt > temp; head -n 5 < temp; rm temp
```

بخش اول، محتویات داخل فایل lsresults.txt را مرتب کرده و در یک فایل دیگر به نام temp قرار می‌دهد. بخش دوم، محتویات temp را به‌عنوان ورودی به دستور head می‌دهد و بنابراین فقط ۵ سطر اول از فایل temp در ترمینال نمایش داده خواهد شد. در نهایت، فایل temp توسط بخش سوم پاک می‌شود. حال سعی کنید این مثال را فقط با استفاده از هدایتگرها پیاده سازی نمایید.

نکته: توجه داشته باشید که هنگامی که دستورات را با نقطه‌ویرگول از هم جدا می‌کنید، هر دستور به‌صورت مستقل عمل می‌کند و کاری به ورودی یا خروجی دستور قبلی ندارد، مگر اینکه با کمک فایل‌های موقت مانند مثال بالا و یا روش‌های دیگر، خروجی دستور قبلی را به دستور بعدی منتقل نماییم.

عملگر کنترلی & : در صورتی که از عملگر & در انتهای یک دستور استفاده نمایید، آن دستور در پس‌زمینه اجرا شده و شما بلافاصله به ترمینال بازخواهید گشت. استفاده از این عملگر برای اجرای سرویس‌های پس‌زمینه و یا انجام کارهای چندوظیفه‌ای[۷] سودمند است.

```
A_Time-consuming_Command_or_Process   &
```

نکته: وقتی فرآیند پس زمینه به اتمام برسد، شناسه آن فرآیند در ترمینال چاپ خواهد شد.

۴-۲-۳- پوسته‌ها

همان‌طور که قبلا دیدید، در لینوکس دستورها به وسیله‌ی ترمینال به سیستم‌عامل ارسال می‌شوند. اما در پشت سر ترمینال، یک مفسر فرمان[۸] وجود دارد که دستورها را ترجمه و تفسیر کرده و برای سیستم‌عامل قابل فهم می‌سازد. وقتی دستوری را نوشته و کلید Enter را می‌زنید، ترمینال دستور شما را دریافت کرده و از طریق شبه‌پایانه به مفسر

---

[7] Multi Tasking

[8] Command-line Interpreter

می‌فرستد. مفسر دستور را تفسیر کرده و برای اجرا به هسته سیستم‌عامل تحویل می‌دهد و سپس پاسخ و نتیجه اجرا را از هسته دریافت کرده و از طریق شبه‌پایانه به ترمینال بازمی‌گرداند تا روی صفحه برای کاربر نمایش داده شود.

در مورد اسکریپت‌ها نیز همینطور است. دستورات اسکریپت توسط مفسر خط فرمان تفسیر شده و سپس برای اجرا به سیستم‌عامل تحویل داده می‌شود. مفسر خط فرمان را پوسته[9] می‌نامند. زیرا مانند یک پوسته بین سیستم‌عامل و ترمینال قرار می‌گیرد. برای سیستم‌های لینوکسی، پوسته‌های مختلفی توسعه داده شده‌اند که از آن جمله می‌توان به موارد زیر اشاره کرد:

Bourne Shell : این پوسته با نام sh شناخته می‌شود. قدیمی‌ترین پوسته‌ی لینوکسی است و توسط آقای بورن در آزمایشگاه‌های بِل نوشته شده است. این پوسته فاقد ویژگی‌های تعاملی و ساختارهای پیچیده‌ی برنامه‌نویسی است. اما در طی سالیان تبدیل به استانداردی در این زمینه شده است.

BASH[10] : این پوسته که نسخه‌ی توسعه یافته‌ی پوسته‌ی بورن است، به‌عنوان پوسته‌ی پیش‌فرض در اکثر توزیع‌های لینوکسی عرضه می‌شود. بش از پروژه‌ی گنو با لینوکس همراه شده و به‌صورت بازمتن ارائه می‌شود. شباهت‌های بسیاری نیز با پوسته‌ی کورن دارد.

پوسته‌ی C : توسط آقای جوی در Berkeley UNIX ایجاد شده است و پس از بش و کورن سومین پوسته‌ی پرکاربرد می‌باشد.

zsh[11] : یک نسخه توسعه یافته از بش است که علاوه بر بهبود آن، برخی قابلیت‌ها و ویژگی‌های مفید از سایر زبان‌ها را نیز در خود جای داده است و تبدیل به پوسته‌ی پیش‌فرض در سیستم‌های MacOS که قبلا از بش استفاده می‌کردند، شده است.

---

[9] Shell
[10] Bourne Again Shell
[11] ZShell

۴-۳- اولین اسکریپت

در ادامه، با نحوه‌ی ایجاد و اجرای یک اسکریپت ساده آشنا خواهید شد. روند کلی به این صورت است که ابتدا در یک ویرایشگر، اسکریپت موردنظر را نوشته و ذخیره می‌نماییم. سپس مراحل لازم را برای تفسیر و اجرای آن انجام خواهیم داد. بنابراین، ابتدا ویرایشگر متنی مورد علاقه خود را باز کرده و سطرهای زیر را در آن بنویسید.

```
#!/bin/bash
echo “Hello World!”
exit 0
```

با اجرای این اسکریپت، فقط یک خط پیام در خروجی چاپ خواهد شد. ابتدا بررسی کنیم که هر یک از سطرهای اسکریپت چه معنایی دارند. در سطر آخر دستور exit بامقدار 0 به‌کار رفته است. دستور exit برای پایان دادن به اسکریپت است و مقدار 0 هم به سیستم اعلام می‌کند که اسکریپت با موفقیت به پایان رسیده است.

در سطر دوم، از دستور echo برای چاپ نتایج در خروجی استفاده شده است. در اسکریپت‌نویسی، برای چاپ پیام در خروجی، پس از دستور echo با یک فاصله پیام موردنظر را داخل علامت نقل قول می‌آوریم.

```
echo    "پیام موردنظر"
```

سطر اول با ترکیب کاراکترهای مربع و علامت تعجب (!#) شروع شده و در ادامه، مسیر پوسته‌ای را که اسکریپت توسط آن تفسیر و اجرا خواهد شد، مشخص کرده است. این ترکیب، به مفسر اعلام می‌کند که این سطر وظیفه‌ی مشخص کردن مفسر را بر عهده دارد و اگر آن را حذف کنید، از پوسته پیش‌فرض سیستم استفاده خواهد شد.

در آخر، فایل ایجاد شده را با نام دلخواه و با پسوند sh. ذخیره نمایید. برای تخصیص پسوند به فایل‌ها در لینوکس کافی است در انتهای نام فایل، یک نقطه گذاشته و سپس پسوند موردنظر خود را بنویسید. البته لینوکس برای تشخیص

نوع فایل‌ها از پسوند استفاده نمی‌کند، بلکه بدین‌منظور از ویژگی‌های فایل[12] بهره می‌برد. تخصیص پسوند در اینجا، بیشتر برای راحتی کاربر و امکان تشخیص نوع فایل توسط مفسر می‌باشد.

۴-۳-۱-اضافه کردن کامنت

برای اضافه کردن توضیحات، از یک علامت # در ابتدای سطر استفاده می‌شود. در مثال زیر، اضافه شدن کامنت به مثال قبل را می‌بینید.

```
#!/bin/sh

#  این سطر وسطر بعدی کامنت می‌باشند و از دید مفسر پنهان خواهد ماند
# hello.sh

echo "Hello World!"
exit 0
```

۴-۳-۲-اجرای اسکریپت

پس از نوشتن یک برنامه مثلا به زبان C، آن را کامپایل می‌کنید و سپس، فایل اجرایی با پسوند exe. (یا بدون پسوند در لینوکس) قابل اجرا بر روی سیستم خواهد بود. اما برای اجرای اسکریپت، روش متفاوتی وجود دارد؛ زیرا یک زبان مفسری است و نه کامپایلری. برای اینکه یک اسکریپت قابل اجرا شود، کافی است به آن مجوز اجرا بدهید. به‌عنوان مثال اسکریپتی را که در ابتدای همین بخش با نام hello.sh نوشتیم، در نظر بگیرید. ابتدا به این فایل مجوزهای لازم برای اجرا را تخصیص می‌دهیم و سپس اسکریپت را در خط فرمان، اجرا کرده و نتیجه آن را مشاهده می‌نماییم.

از آزمایش قبلی به‌یاد دارید که برای تخصیص مجوز اجرا باید به‌شکل زیر عمل کنیم. توجه داشته باشید که در این مثال، فایل در مسیر جاری قرار دارد. بنابراین تنها ذکر نام فایل کافی خواهد بود.

---

[12] File Attributes

```
chmod   a+x   hello.sh
```

نکته ۱: اینکه به کاربران مجوز خواندن و نوشتن هم تخصیص بدهید، به شما بستگی دارد؛ هر چند از لحاظ امنیتی توصیه نمی‌شود. اما به این نکته توجه کنید که حتما مجوز اجرا (برابر با عدد ۱) به کاربر موردنظر شما تخصیص داده شود. در مثال بالا، برای تمام کاربران مجوز اجرا اضافه شده است.

نکته ۲: ممکن است بخواهید در مورد یک اسکریپت خاص، فقط کاربر مالک امکان اجرای اسکریپت را داشته باشد. در آن صورت، در دستور بالا از u+x استفاده کنید.

حال که مجوزهای لازم را به فایل اسکریپت تخصیص دادیم، می‌توانیم آن را اجرا نماییم. هنگام اجرای اسکریپت، ممکن است یکی از دو حالت زیر پیش بیاید:

الف) فایل اسکریپت در دایرکتوری جاری باشد. آن‌گاه برای اجرای اسکریپت کافی است نام اسکریپت را بعد از علامت ./ بیاوریم:

```
./hello.sh
```

علامت ./ به مفسر اعلام می‌کند که در دایرکتوری جاری به‌دنبال اسکریپت بگردد و به سراغ دایرکتوی‌های پیش‌فرض سیستم - که اسکریپت‌های سیستمی در آن‌ها قراردارند و مسیرشان در متغیر محیطی PATH ذخیره شده است - نرود.

ب) فایل اسکریپت در مسیری متفاوت از دایرکتوری جاری باشد. آن‌گاه باید آدرس کامل و نام فایل را ذکر نماییم. در مثال بالا، اسکریپت در دایرکتوری خانگی کاربر قرار دارد.

```
/home/soltani/hello.sh
```

نکته: درصورتی‌که نوع پوسته در ابتدای اسکریپت تعیین نشده باشد و یا بخواهیم اسکریپت را با مفسر دیگری اجرا نماییم، ابتدا نام مفسر و سپس آدرس و نام اسکریپت را می‌آوریم. مثال:

```
sh        نام و آدرس اسکریپت موردنظر
```

یا

```
bash      نام و آدرس اسکریپت موردنظر
```

۴-۴- متغیرها

در Bash نیازی به اعلان متغیرها نمی‌باشد. هر گاه در یک اسکریپت، برای اولین بار از یک متغیر استفاده کنید یا آن را مقداردهی نمایید، متغیر به‌شکل خودکار اعلان می‌شود و در استفاده‌های بعدی بدون مشکل شناسایی خواهد شد. مقداردهی متغیرها به صورت زیر انجام می‌پذیرد.

```
مقدار موردنظر برای متغیر= نام متغیر
```

به‌عنوان مثال:

```
x=2
f=3.14
name='Ali'
lname="Irani"
```

نکته۱: مقادیر رشته‌ای و کاراکتری، باید داخل علامت نقل‌قول (" ") یا علامت نقل‌قول یکانی ('') قرار بگیرند.

نکته ۲: اکثر پوسته‌ها بیش از حد به کاراکتر فاصله حساس هستند. مثلا در مقداردهی متغیرها، نباید قبل و بعد از علامت مساوی هیچ کاراکتر فاصله‌ای قرار بگیرد. درغیراین‌صورت، هنگام اجرا با خطای مفسری مواجه خواهد شد.

چاپ در خروجی

برای چاپ پیام‌ها و متغیرها در خروجی، از دستور echo استفاده می‌شود. همان‌طور که در بخش قبل دیدید، هر آنچه را که در میان دو علامت نقل قول به دستور echo بدهید، عینا آن را در خروجی چاپ خواهد کرد.

چاپ متغیرها

برای چاپ مقدار متغیرها در خروجی، از اشاره‌گرها استفاده می‌کنیم تا بتوانیم به محتویات داخل متغیر دسترسی یابیم. برای اشاره به مقدار یک متغیر، قبل از نام متغیر یک علامت $ قرار بدهید.

```
echo    $x
```

چاپ ترکیبی پیام و متغیر در یک دستور

```
#!/bin/bash

چاپ نام  و مقدار متغیر #
x=2
echo   " x = $x "
exit 0
```

همان‌طور که در مثال بالا می‌بینید، عبارتی که در مقابل دستور echo نوشته شده، داخل علامت نقل‌قول قرار گرفته است. بنابراین انتظار داریم که عبارت عینا چاپ شود. در مورد x اول این اتفاق می‌افتد اما در مورد x دوم به‌علت وجود علامت $، مقدار متغیر x جایگزین شده و چاپ می‌شود. خروجی این اسکریپت به‌صورت زیر خواهد بود.

```
 x = 2
```

۴-۴-۲- دریافت از ورودی

برای دریافت مقدار متغیرها از ورودی توسط کاربر، از دستور read استفاده می‌شود.

```
read    نام متغیر
```

مثال. اسکریپتی بنویسید که با دریافت نام کاربر، یک پیام به نام وی چاپ نماید.

```
#!/bin/bash

# hello_you.sh
# با چاپ یک پیام از کاربر میخواهیم که نام خود را وارد نماید
echo "Please enter your name: "

# آنچه که کاربر وارد کرده را دریافت و در متغیری به نام name قرار می‌دهیم
read name

# چاپ پیام "سلام، نام کاربر!"
echo "hello, $name!"

exit 0
```

در تصویر ۴-۵ نتیجه‌ی اجرای اسکریپت بالا را مشاهده می‌کنید.



تصویر ۴-۵- دریافت متغیر از ورودی

برای دریافت دو یا چند متغیر با یک دستور، کافی است که نام متغیرها را به ترتیب و با فاصله از هم درج نماییم.

```
read   x   y
```

نکته: کاربر باید پس از نوشتن مقدار اول، یک کاراکتر فاصله قرار داده و سپس مقدار دوم را وارد نماید.

۴-۴-۳- محاسبات ریاضی بر روی متغیرها

در Bash برای انجام محاسبات ریاضی از چند روش استفاده می‌شود. حالت اول، استفاده از let است.

```
let    x=x+1
```

نکته ۱: در این حالت نیز، قبل و بعد از علامت + یا = نباید از کاراکتر فاصله استفاده کرد.

نکته ۲: دستور let در پوسته‌هایی همچون sh تعریف نشده است.

روش دوم استفاده از جفت پرانتز است. در این حالت وجود کاراکترهای فاصله‌ی اضافی، موجب بروز خطا نخواهد شد، البته تازمانی‌که بین جفت کمانه‌های پرانتز قرار نگیرد.

```
((x=5))
((x=x+1))
```

عملگرهای ریاضی تعریف شده در Bash، در جدول ۵-۱ آمده است.

عملگر	+	-	*	/	++	--	%	**
کاربرد	جمع	تفریق	ضرب	تقسیم	افزاینده	کاهنده	باقیمانده تقسیم	توان

جدول ۴-۱- عملگرهای ریاضی در پوسته Bash

در مثال زیر، اسکریپتی را ملاحظه می‌نمایید که ابتدا یک عدد صحیح مثبت را از کاربر دریافت کرده و عملیات ریاضی مختلفی روی آن انجام داده و در انتها نتایج را نمایش می‌دهد. در تصویر ۴-۶ نیز نتیجه اجرای اسکریپت برای یک عدد دلخواه را مشاهده می‌نمایید.

```
#!/bin/bash
# math_ops.sh : Math Operations

# دریافت عدد از کاربر
echo "Insert a positive integer number: "
read x

#محاسبه توان دوم عدد و ذخیره نتیجه در یک متغیر دیگر
let y=x**2
# محاسبه باقیمانده عدد تقسیم بر دو و محاسبه نتیجه در یک متغیر دیگر z
let z=x%2
# افزایش عدد به میزان یک واحد
let x++

# چاپ نتایج
echo "x^2 = $y"
echo "x%2 = $z"
echo "x+1 = $x"

exit 0
```

```
soltani@VM:~$ ./vars.sh
Insert a positive integer number:
12
x**2 = 144
x%2 = 0
x+1 = 13
```

تصویر ۴-۶- محاسبات ریاضی در اسکریپت‌نویسی

تمرین

- اسکریپتی بنویسید که تاریخ و زمان فعلی سیستم را به فرمت "%Y-%m-%d %H:%M:%S" نمایش بدهد.
- اسکریپتی بنویسید که یک رشته را از کاربر بگیرد. سپس با کمک دستور wc، تعداد کلمات و تعداد کاراکترهای آن را شمارش و چاپ نماید.
- اسکریپتی بنویسید که یک رشته متنی به انگلیسی را از خط فرمان دریافت کند. سپس با استفاده از دستور sed، تمام کاراکترهای فاصله را از رشته حذف کرده و تمام حروف آن را به حروف بزرگ[۱۳] تبدیل نماید. در نهایت نتیجه را در خروجی نمایش بدهد.

گزارش کار

اسکریپتی بنویسید که اطلاعات زیر را استخراج کرده و به‌صورت خلاصه چاپ نماید.

- اطلاعات پردازنده شامل model name و cpu cores از فایل /proc/cpuinfo
- اطلاعات حافظه شامل MemTotal ، MemFree و MemAvailable از فایل /proc/meminfo
- برای جستجوی اطلاعات از grep همراه با عبارات الگوی مذکور استفاده نمایید.
- با کمک دستور head، فقط خط اول هر نتیجه را نمایش بدهید.

اسکریپتی بنویسید که از فایل متنی /etc/passwd که شامل ستون‌هایی از اطلاعات کاربران بوده و از جداکننده «دونقطه» استفاده می‌کند، اطلاعات ستون اول (نام کاربری) و ستون سوم (شناسه کاربری) هر خط را با استفاده از دستور awk استخراج کرده و نتیجه را به‌صورت مرتب شده الفبایی و با فرمت Username : UID در خروجی چاپ نماید.

---

[13] Uppercase

منابع و مراجع:

- Matthew N, Stones R. *Beginning Linux Programming*, chapter 2. John Wiley & Sons, 2008.
- WikiBooks. *Bash Shell Scripting*, Wiki Books, last revised 2024 Nov.
- https://tldp.org/LDP/Bash-Beginners-Guide/html/index.html, visited 2022.

آزمایش ۵ : اسکریپت‌نویسی پیشرفته

اهداف:

- آشنایی تکمیلی با اسکریپت‌نویسی پوسته

ابزار و مفاهیم مورد نیاز:

- CLI
- ابزارهای ویرایش متن
- پوسته bash

محتوا:

- دستورات تکمیلی اسکریپت‌نویسی پوسته
- عملگرهای منطقی
- دستورات شرطی
- حلقه‌های تکرار
- متغیرهای محیطی

۵-۱- مرور مفاهیم پایه

در دستورکار قبل دیدیم که با استفاده از اسکریپت‌نویسی پوسته، می‌توان برنامه‌های نسبتا کوچک و سبکی نوشت که توسط مفسر خط فرمان که پشت ترمینال قرار دارد، اجرا می‌شوند. در این آزمایش، با مفاهیم و مثال‌های پیشرفته‌تری از اسکریپت‌نویسی پوسته آشنا خواهیم شد.

۵-۲- ساختار شرطی If

برای بررسی برقراری شرط‌ها، می‌توان از ساختار If استفاده کرد. شکل ساده این دستور به‌صورت زیر می‌باشد.

```
if     [     عبارت شرطی     ]
then
      دستورات ۱
else
      دستورات ۲
fi
```

نکته‌ها:

۱. درصورتی‌که عبارت شرطی صحیح باشد، دستورات ۱ و درغیراین‌صورت، دستورات ۲ اجرا خواهند شد.

۲. وجود بخشelse الزامی نیست.

۳. برای مشخص کردن بلوک دستورات، از کلمه کلیدی then استفاده می‌شود و بنابراین استفاده از تورفتگی[1]، ضروری نیست.

٤. عبارت fi پایان دهنده‌ی ساختار if است.

٥. وجود یک «کاراکتر فاصله» بین هر براکت و عبارت شرطی الزامی است و درصورتی‌که رعایت نشود، با خطای مفسر مواجه خواهد شد.

---

[1] Indention

عملگرهای مقایسه‌ای

عملگرهای مقایسه‌ای در عبارات شرطی کمی متفاوت از عملگرهای متداول ریاضی می‌باشند. مثلا در مورد مقایسه عبارات حسابی[۲] از عملگرهای زیر استفاده می‌شود.

عملگرهای مقایسه‌ای عبارات حسابی	
تساوی	expression1 -eq expression2
عدم تساوی	expression1 -ne expression2
بزرگتر	expression1 -gt expression2
بزرگتر مساوی	expression1 -ge expression2
کوچکتر	expression1 -lt expression2
کوچکتر مساوی	expression1 -le expression2

جدول ۵-۲- مقایسه دو عبارت حسابی با نام‌های expression1 و expression2

در جدول بالا مشاهده می‌کنید که برای مقایسه تساوی دو عبارت حسابی با نام‌های expression1 و expression2 از عملگر eq– استفاده می‌شود. اگر expression1 و expression2 مساوی باشند نتیجه‌ی این مقایسه True و درغیراین‌صورت False خواهد بود.

مثال. اسکریپتی بنویسید که بر اساس معدل، کلمه passed یا failed را چاپ نماید.

```
#!/bin/bash

echo "Enter the average: "
read ave
```

[2] Arithmetic Expressions

```
if [ $ave -ge 10 ]
then
  echo "passed"
else
  echo "failed"
fi
exit 0
```

۵-۲-۱- ساختار شرطی if ... elif

به‌منظور بررسی چند شرط مختلف، از ساختار زیر استفاده می‌کنیم.

```
if    [    عبارت شرطی ۱    ]
then
      دستورات ۱
elif    [    عبارت شرطی ۲    ]
then
      دستورات ۲
else
      دستورات ۳
fi
```

نکته: به‌منظور بررسی بیش از دو عبارت شرطی، می‌توان به تعداد مورد نیاز elif اضافه کرد.

مثال. اسکریپتی بنویسید که یک عدد را از ورودی دریافت کرده و بررسی نماید که آیا عدد مثبت، منفی یا صفر است.

```
#!/bin/bash
# if_numbers.sh

# با چاپ یک پیام از کاربر می‌خواهیم یک عدد وارد کند
echo "Enter a number: "

# عدد را دریافت کرده و در متغیر x قرار می‌دهیم
read x

#اگر x بزرگتر از ۰ باشد
if [ $x -gt 0 ]
then

# چاپ پیام مبنی بر اینکه عدد مثبت است
  echo "$x is positive!"
```

```
اگر x کوچکتر از ۰ باشد#
elif [ $x -lt 0 ]
then

چاپ پیام مبنی بر اینکه عدد منفی است #
  echo "$x is negative!"

در غیر این صورت عدد برابر ۰ است #
else
  echo "It's a zero!"

پایان بخش شرطی#
fi

exit 0
```

۵-۳- عملگرهای منطقی

ترکیب عبارات شرطی با استفاده از عملگرهای منطقی قابل انجام است. از زوج کاراکتر || به‌عنوان عملگر OR و از && به‌عنوان عملگر AND استفاده می‌شود. به‌عنوان مثال، اسکریپت زیر با دریافت دو عدد، مثبت یا منفی بودن حاصل‌ضرب آن‌ها را تشخیص می‌دهد.

```
#!/bin/bash

echo "Insert two integers: "
read x y

if [ $x -lt 0 ] && [ $y -lt 0 ]
then
  echo "Product of $x and $y is  positive!"

elif [ $x -eq 0 ] || [ $y -eq 0 ]
then
  echo "Product of $x and $y is  zero!"

elif [ $x -gt 0 ] && [ $y -gt 0 ]
then
  echo "Product of $x and $y is  positive!"

else
  echo "Product of $x and $y is negative!"

fi
```

```
exit 0
```

مقایسه رشته‌ها

در جدول ۵-۳، عملگرهایی که در اسکریپت‌نویسی برای مقایسه رشته‌ها مورد استفاده قرار می‌گیرند را ملاحظه می‌نمایید.

عملگر	مثال	توضیح
=	string1 = string2	تساوی
!=	string1 != string2	عدم تساوی
-n	-n string	نبودن رشته Null
-z	-z string	بودن رشته (رشته‌ی خالی) Null

جدول ۵-۳- عملگرهای مقایسه‌ای رشته‌ها

مثال. اسکریپتی بنویسید که پاسخ کاربر به یک سوال را دریافت کرده و سپس مثبت یا منفی بودن آن را بررسی نماید.

```
#!/bin/bash
# if_strings.sh

# درخواست از کاربر برای وارد کردن کاراکتر y یا n
echo "do u want to continue? (y/n)"

# قرار دادن پاسخ کاربر در متغیری به نام answer
read answer


# در کنار برکت‌ها باید فاصله قرار بدهیم
if [ $answer = "y" ]
then
echo "let's go"
elif [ $answer = "n" ]
then
echo "see you later"

# اگر کاربر کاراکتری غیر از y یا n وارد کند آن‌گاه
```

```
else
echo "please type y/n!"
fi

exit 0
```

نکته: یک بار دیگر اسکریپت را اجرا کنید. این بار در پاسخ به سوال، بدون اینکه کاراکتری را وارد کنید فقط کلید Enter را بزنید (پاسخ پوچ). چه اتفاقی می‌افتد؟ علت بروز خطا این است که وقتی کاربر هیچ کاراکتری را وارد نمی‌کند، برنامه باید یک متغیر با مقدار پوچ را با یک کاراکتر (y/n) مقایسه کند و بنابراین دچار مشکل می‌شود. برای حل این مشکل، در عبارات شرطی متغیر answer را داخل کوتیشین قرار بدهید.("$answer")

۵-۴- حلقه تکرار for

در این ساختار، دستور for به ازای هر یک از مقدارهایی که در بخش مقادیر برایش ذکر شود حلقه را تکرار خواهد کرد.

```
for     متغیر      in     مقادیر
 do
      دستورات
 done
```

مثال: با استفاده از حلقه for اعداد ۱ تا ۵ را به‌ترتیب در خروجی چاپ نمایید.

```
#!/bin/bash
# for.sh

for   x   in   1  2  3  4  5
 do      شروع دستورات حلقه#
      echo $x
 done    پایان حلقه#

exit 0
```

در هر تکرار، به ترتیب یکی از مقادیر، درون متغیر قرار می‌گیرد و دستورات حلقه به ازای آن مقدار، اجرا می‌شوند. در مثال بالا نیز، ابتدا مقدار ۱ درون متغیر x قرار گرفته و چاپ می‌شود، سپس مقدار ۲ و الی آخر.

تعیین شروع، پایان و گام حلقه

برای تکرارهای بیشتر می‌توان از یکی از ساختارهای زیر استفاده کرد. در ساختار اول، مقادیر شروع، پایان و طول گام را بین دو علامت قلاب مشخص کرده و از ".." به‌عنوان کاراکتر جداکننده استفاده می‌کنیم. این ساختار در نسخه‌ی 3.0 و بالاتر پوسته Bash قابل استفاده است.

```
#for i in {$start..$stop..$step}

for i in {0..10..2}
 do
    echo "Even Numbers: $i"
 done
```

در روش دوم که ساختاری مشابه قواعد زبان C دارد نیز، مقادیر آغازین حلقه، شرط پایان و طول گام را بین دو علامت پرانتز پشت‌سرهم قرار می‌دهیم. در مثال زیر از ۵ تا ۱۵ با گام ۳ پیش می‌رویم.

```
for ((i=5; i<=15; i+=3))
 do
    echo "$i"
 done
```

نکته: در هر دو ساختار، ترتیب نزولی نیز قابل تعریف است.

۵-۵- حلقه تکرار while

```
while [  شرط  ]
 do
   دستورات
 done
```

مثال: اسکریپتی بنویسید که با دریافت رمزعبور از کاربر، آن را با رمزعبور صحیح - که در اینجا، «secret» در نظر گرفته شده است - مقایسه نماید و تا دریافت رمزعبور صحیح، تکرار حلقه را ادامه بدهد.

```
#!/bin/bash
# while.sh

# دریافت رشته از کاربر و قراردادن در متغیری به نام pass
echo "Enter password: "
read pass


# تازمانی‌که ورودی کاربر با رمزعبور صحیح یکسان نباشد حلقه تکرار می‌شود
while [ "$pass" != "secret" ]
do     # شروع دستورات حلقه
  echo "Sorry, try again"
  read pass
done   # پایان حلقه

exit 0
```

مثال. اسکریپت «دریافت پاسخ از کاربر» را طوری تغییر بدهید که تازمانی‌که کاربر یکی از کاراکترهای مجاز y یا n را - چه به‌صورت حروف بزرگ و چه به‌صورت کوچک - وارد نکند، برنامه تکرار شده و از کاربر درخواست پاسخ نماید.

برای پیاده‌سازی چنین مثال‌هایی، رویکردهای مختلفی وجود دارد، یک روش را در زیر مشاهده می‌نمایید.

```
#!/bin/bash
# while_logic_operators.sh

# درخواست پاسخ با فرمت صحیح
while [ "$ans" != "y" ] && [ "$ans" != "Y" ] && [ "$ans" != "n" ]
      && [ "$ans" != "N" ]

 do      # شروع دستورات حلقه‌ی وایل
   read -p  "do u want to continue?(y/n)"  ans
 done    # پایان حلقه‌ی وایل

if [ "$ans" = "y" ] || [ "$ans" = "Y" ] # شروع بخش شرطی
  then
    echo "let's go!"

elif [ "$ans" = "n" ] || [ "$ans" = "N" ]
```

```
  then
    echo "see you later!"

fi     #پایان بخش شرطی

exit 0
```

نکته ۱: سوئیچ p- در پوسته sh تعریف نشده است و برای استفاده از آن، باید از bash استفاده نمایید.

نکته ۲: امکان استفاده از کلمات کلیدی break و continue برای کنترل حلقه‌ها، وجود دارد.

۵-۶- متغیرهای محیطی پوسته

متغیرهای محیطی[۳] مقادیری هستند که توسط سیستم و کاربران تعریف می‌شوند و بر رفتار سیستم و برنامه‌ها تاثیر می‌گذارند. این متغیرها اطلاعات مهمی مانند مسیرهای سیستم، تنظیمات کاربر و سایر پارامترهای محیطی را نگهداری می‌کنند و با استفاده از آن‌ها می‌توان محیط کاری در لینوکس را سفارشی‌سازی کرد. از جمله متغیرهای محیطی مهم، عبارتند از:

PATH: مسیرهای جستجو برای دستورات اجرایی

HOME: دایرکتوری خانگی کاربر

SHELL: پوسته پیش‌فرض کاربر

USER: نام کاربری جاری

PWD: دایرکتوری کاری فعلی

LANG: تنظیمات زبانی سیستم

---

[3] Environment Variables

برای دیدن تمام متغیرهای محیطی جاری یکی از دو دستور زیر را در خط فرمان اجرا نمایید.

```
env
printenv
```

درصورتی‌که می‌خواهید فقط مقدار یک متغیر خاص را مشاهده کنید، پس از دستور printenv نام آن متغیر را بیاورید. به‌عنوان مثال، هر یک از دو دستور زیر، فقط مقدار متغیر HOME را نمایش خواهند داد. دقت نمایید که نام این متغیرها تماما با حروف بزرگ تعریف شده‌اند.

```
printenv HOME
echo $HOME
```

برای نمایش مقادیر چند متغیر به‌طور همزمان نیز به روشی مشابه عمل می‌کنیم.

```
printenv   SHELL   HOME   PATH
```

۵-۶-۱- تعریف و تغییر مقدار متغیرهای محیطی

برای تعریف متغیرهای محیطی محلی، از دستور set استفاده می‌شود. متغیرهایی که با این دستور تعریف شوند، فقط در همان پوسته قابل دسترسی خواهند بود. مثال زیر یک متغیر به نام MY_VAR از نوع رشته‌ای و با مقدار پیش‌فرض Hello ایجاد می‌نماید.

```
set MY_VAR="Hello"
```

برای تعریف یا تغییر متغیرهای محیطی که در تمام زیرفرآیندها قابل دسترسی باشند، از دستور export استفاده می‌شود.

```
export MY_VAR2="World"
```

مثال: فرض کنید که یک دایرکتوری حاوی اسکریپت‌های شخصی در دایرکتوری خانگی کاربر جاری دارید. برای اضافه کردن این دایرکتوری به PATH، در دستور زیر به جای عبارت Username نام کاربری خودتان را جایگزین کرده و سپس آن را اجرا نمایید.

```
export PATH=$PATH:/home/username/my_scripts
```

از این پس، هنگام اجرای اسکریپت‌هایی که در دایرکتوری مذکور قرار دارند، دیگر نیازی به افزودن ./ به ابتدای اسم اسکریپت نخواهید داشت و فقط ذکر نام اسکریپت کافی می‌باشد. زیرا سیستم به‌طور خودکار دایرکتوری مذکور را نیز جستجو خواهد کرد.

نکته ۱: علامت ":" در دستور بالا، کاراکتر جداکننده مقادیر مسیرها در متغیر PATH می‌باشد.

نکته ۲: تغییراتی که با دستور export اعمال می‌شوند فقط در نشست[۴] جاری فعال خواهند بود. برای تغییرات دائمی باید فایل‌های پیکربندی پوسته را ویرایش کنید.

۵-۷- متغیرهای خاص[۵]

متغیرهای خاص در پوسته لینوکس، متغیرهای از پیش تعریف شده‌ای هستند که توسط سیستم عامل یا پوسته به‌صورت خودکار مدیریت و مقداردهی می‌شوند. این متغیرها اطلاعات حیاتی درباره محیط اجرا، پردازه‌ها، پارامترهای ورودی و وضعیت سیستم را در اختیار کاربر یا اسکریپت‌ها قرار می‌دهند. در ادامه، چند نمونه از هر دسته‌بندی را می‌بینیم.

۵-۷-۱- متغیرهای مرتبط با اسکریپت‌ها

متغیر $0 نام (و مسیر) اسکریپت را در خود نگهداری می‌کند. متغیرهای $1، $2 و ... حاوی آرگومان‌های موقعیتی می‌باشند. $# تعداد آگومان‌های ورودی را (بدون احتساب $0) نگهداری می‌کند و $@ نیز حاوی تمام آرگومان‌ها - به

---

[4] Session
[5] Special Variables

صورت یک لیست جداگانه - می‌باشد. دستور زیر را درون یکی از اسکریپ‌های پوسته که در این فصل نوشته‌اید اضافه کرده و نتیجه را مشاهده کنید.

```
echo "There are $# arguments in this scctipt($0). The first argument is: $1"
```

متغیر $$ شناسه فرآیند جاری را در خود نگهداری می‌کند. $? حاوی کد خروج آخرین دستور اجرا شده می‌باشد. $SECONDS نیز مدت زمان سپری شده از شروع اسکریپت را بر حسب ثانیه نگهداری می‌کند.

نکته: متغیرهای خاص از نوع فقط خواندنی هستند و نمی‌توان مقدار آن‌ها را مستقیما تغییر داد. همچنین در نظر داشته باشید که در اسکریپت‌های تودرتو، این متغیرها مقادیر مربوط به اسکریپت جاری را در خود نگهداری می‌کنند.

تمرین

اسکریپتی بنویسید که آیتم‌های داخل یک دایرکتوری دلخواه را چک کرده و مشخص کند که هر آیتم فایل است یا دایرکتوری. همچنین تعداد فایل‌ها و دایرکتوری‌ها را شمارش نموده و در خروجی چاپ نماید.

گزارش کار

- فرض کنید مسئول نگهداری یک سرور لینوکسی هستید. حجم لاگ‌های سیستم بسیار زیاد است و بررسی و مدیریت آن‌ها به‌صورت دستی زمان‌بر می‌باشد. بنابراین اسکریپتی به شرح زیر، برای «اتوماسیون مدیریت لاگ‌های سیستم» بنویسید.

- **آماده‌سازی فایل لاگ**. ابتدا محل ذخیره‌سازی فایل‌های لاگ را در توزیع خود پیدا کنید. در لینوکس مینت، فایل اصلی لاگ /var/log/syslog می‌باشد. اسکریپت باید، یک کپی از این فایل را در دایرکتوری خانگی شما قرار دهد و همزمان با کپی کردن، نام فایل مقصد را به نام شما با پسوند لاگ تغییر دهید، مثلا student.log. سپس پنج بار کلمه‌ی error را به انتهای این فایل کپی‌شده، اضافه نماید.
- **خلاصه‌سازی لاگ‌ها**. اسکریپت باید فایل لاگ کپی‌شده را خوانده و محتویات آن را بر اساس نوع پیام (مثلا Error, Warning, Info) دسته‌بندی و تعداد هر یک از انواع پیام را چاپ نماید. در جستجو، از حالت عدم حساس به حروف بزرگ و کوچک استفاده کنید.
- **مانیتورینگ و هشدار**. اگر تعداد پیام‌های Error از ۳ عدد بیشتر شد، آن‌گاه یک پیام هشدار با رنگ قرمز در ترمینال برای کاربر چاپ کند. برای تغییر رنگ متن از کدهای ANSI استفاده کنید. مثلا کد رنگ قرمز، \e[31m است.
- **ذخیره نتایج**. اسکریپت در هر اجرا، خروجی خود را علاوه بر چاپ در ترمینال، به انتهای یک فایل متنی به نام خودتان (مثلا student_summary.txt) الحاق می‌نماید.
- **مستندسازی**. برای افزایش خوانایی، در بخش‌های مختلف اسکریپت خود Comments مناسب اضافه نمایید.

- **اجرا و ثبت نتایج**. اسکریپت را به‌صورت یک فایل واحد نوشته و آماده اجرا نمایید. پس از اجرا، از مراحل کار و نتایج به‌دست آمده، از جمله فایل لاگ تغییریافته، نتایج چاپ شده در ترمینال و فایل student_summary.txt یک گزارش کار کامل تهیه نمایید.

▪ همان شرایط تمرین بالا را در نظر بگیرید. این بار پس از مرحله «آماده‌سازی فایل لاگ» و «خلاصه‌سازی لاگ‌ها»، مراحل زیر را به انجام برسانید.

- **ایجاد آرشیو**. اسکریپت پس از هر اجرا، فایل لاگ (همان فایل کپی‌شده در دایرکتوری خانگی) را با نامی حاوی تاریخ و زمان (مثلا log_20261026_1230.txt) در یک دایرکتوری مشخص آرشیو می‌کند.
- **زمان‌بندی اجرای خودکار**. درباره امکان اجرای زمان‌بندی شده‌ی وظایف با استفاده از systemd یا cron تحقیق کرده و سپس تنظیمات لازم را انجام دهید تا اسکریپت هر ۵ دقیقه یک‌بار به‌صورت خودکار و بدون نیاز به مداخله کاربر اجرا گردد. توجه داشته باشید که استفاده از cron ساده‌تر است، اما systemd timer روش مدرن‌تری محسوب می‌شود. بنابراین به دلخواه خود، یکی را برای انجام زمان‌بندی انتخاب نمایید.
- **مستندسازی**. برای افزایش خوانایی، در بخش‌های مختلف اسکریپت خود Comments مناسب اضافه کنید.
- **اجرا و ثبت نتایج**. اسکریپت را به‌صورت یک فایل واحد نوشته و آماده اجرا نمایید. برای اینکه تا اجرای خودکار در ۵ دقیقه‌ی بعد منتظر نمانید، می‌توانید زمان‌بندی را موقتا به ۱ دقیقه کاهش دهید. پس از اجرا، از مراحل کار و نتایج حاصل شده از جمله، فایل لاگ تغییریافته، لاگ اجرای خودکار اسکریپت، نتایج چاپ شده در ترمینال (در صورت وجود) و فایل آرشیو ایجاد شده، یک گزارش کار کامل تهیه نمایید.

نکته: پس از دریافت نتایج و تهیه‌ی گزارش کار، برای جلوگیری از کند شدن سیستم به‌علت اجرای زودبه‌زود اسکریپت به‌صورت خودکار، می‌توانید تنظیمات زمان‌بندی را حذف کنید.

- منابع و مراجع:

- Cooper M. *Advanced Bash Scripting Guide: An In-depth Exploration of the Art of Shell Scripting*, editor no identificat, 2014.
- Blum R. *Linux Command Line And Shell Scripting Bible*, John Wiley & Sons, 2008.
- Wiki Books, *Bash Shell Scripting*, Wiki Books, last revised 2024 Nov.

آزمایش ۶ : وقفه‌ها

اهداف:

- درک مفهوم وقفه
- انواع وقفه سخت‌افزاری و نرم‌افزاری
- نظارت، ایجاد و مدیریت وقفه‌ها

ابزار و مفاهیم مورد نیاز:

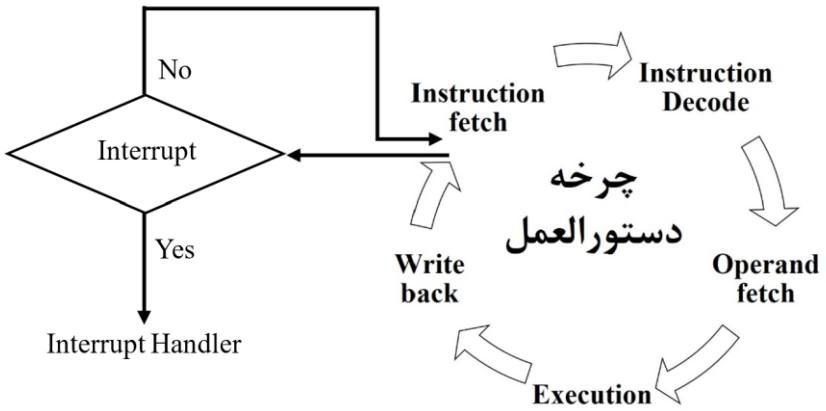
- ابزارهای کامپایل خط فرمانی یا IDE
- کتابخانه‌های ضروری

محتوا:

- مشاهده وقفه‌های سیستم
- فعال و غیرفعال‌سازی وقفه‌ها
- ارسال و نظارت بر فراخوانی‌های سیستمی
- تعریف مدیر وقفه سیگنال
- همگام‌سازی فرآیندها با استفاده از سیگنال

۶-۱- وقفه چیست؟

گاهی رویدادی در خارج یا داخل پردازنده رخ می‌دهد، بنابراین پردازنده روند عادی اجرای دستورالعمل‌های فرآیند جاری را موقتا قطع کرده و به آن رویداد پاسخ می‌دهد و سپس به اجرای ادامه فرآیند برمی‌گردد. چنین رویدادی، وقفه[1] نام دارد. پردازنده در پایان هر چرخه دستورالعمل، وجود وقفه را بررسی می‌کند و در صورت وجود، بلافاصله به آن رویداد می‌پردازد. پردازنده پس از مدیریت و رسیدگی به وقفه، دوباره به ادامه اجرای کاری که پیش از آن در دست داشت می‌پردازد.



تصویر ۶-۱- چرخه دستورالعمل و وقفه

پردازنده برای پاسخ به وقفه، باید رویه اداره‌کننده وقفه[2] یا روتین سرویس وقفه[3] (ISR) را اجرا کند.

---

[1] Interrupt
[2] Interrupt Handler
[3] Interrupt Service Routine

۶-۱-۱- انواع وقفه

وقفه‌ها به دو دسته سخت‌افزاری و نرم‌افزاری تقسیم می‌شوند. وقفه‌های سخت افزاری، منشا سخت افزاری دارند و به‌صورت یک سیگنال ناهمگام[۴] به پردازنده ارسال می‌شود. وقفه‌های نرم‌افزاری که تله[۵] نیز نامیده می‌شوند، توسط یک برنامه‌ی در حال اجرا در پردازنده تولید شده و از نوع همگام[۶] می‌باشند.

پردازنده با دریافت درخواست وقفه[۷] (IRQ)، متوجه وقوع وقفه شده و با ارسال تصدیق[۸] (INTA) دریافت وقفه را تایید می‌کند. سپس، شماره وقفه به پردازنده ارسال می‌شود و به این ترتیب، سخت‌افزاری که درخواست وقفه را ارسال کرده بود شناسایی می‌گردد. پردازنده، اطلاعات مورد نیاز از فرآیند جاری را ذخیره کرده و به مد هسته سوئیچ می‌کند. سپس با استفاده از بردار وقفه به آدرس ISR مربوطه مراجعه نموده و به وقفه پاسخ می‌دهد. و در نهایت دوباره به مد کاربر و ادامه اجرای فرآیند بازمی‌گردد.

از جمله وقفه‌های سخت‌افزاری می‌توان به این موارد اشاره کرد:

- وقفه تایمر[۹]: یکی از اساسی‌ترین وقفه‌های سخت‌افزاری است و توسط تایمر سیستم تولید می‌شود. هسته از وقفه‌ی تایمر برای مدیریت زمان‌بندی فرآیندها استفاده کرده و بدینوسیله از به انحصار گرفته شدن پردازنده توسط یک فرآیند، جلوگیری می‌نماید.
- وقفه I/O : مانند وقفه صفحه‌کلید یا پرینتر. این نوع وقفه هنگامی رخ می‌دهد که یک عملیات ورودی/خروجی، درخواست و یا تکمیل شود. وجود این وقفه باعث می‌شود که پردازنده بتواند به کارهای دیگر خود بپردازد بدون اینکه لازم باشد منتظر تکمیل عملیات I/O که بسیار کند است، بماند. کنترل‌کننده‌های وقفه I/O، داده‌ها را به بافرها منتقل کرده و سپس پردازنده را از طریق ارسال وقفه مطلع می‌سازند.

---

[4] Asynchronous
[5] Trap
[6] Synchronous
[7] Interrupt Request
[8] Interrupt Acknowledge
[9] Timer

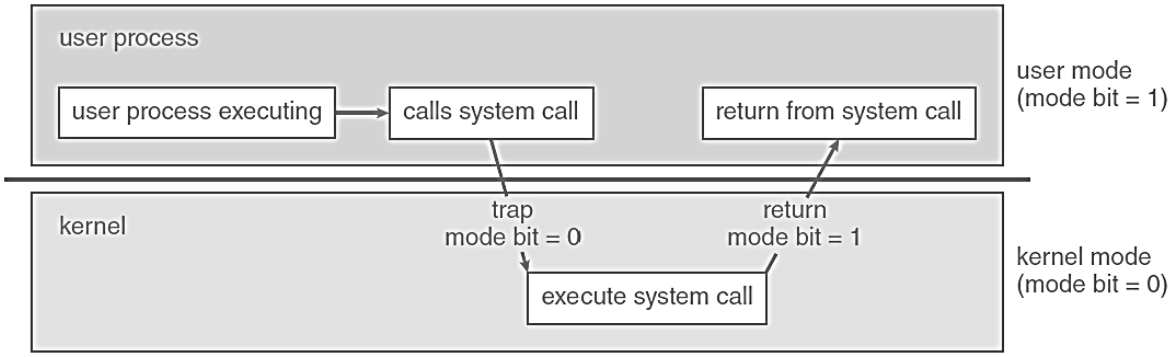
- وقفه خطا[10] یا نقص سخت افزاری[11]
- وقفه Restart

از جمله وقفه‌های نرم‌افزاری نیز می‌توان به این موارد اشاره کرد:

- فراخوانی سیستمی[12] : هرگاه یک برنامه سطح کاربر نیاز به یکی از سرویس‌های سطح هسته داشته باشد با استفاده از این فراخوان، یک وقفه نرم‌افزاری ایجاد می‌کند. این وقفه باعث می‌شود تا پردازنده از حالت کاربر به حالت هسته سوئیچ نماید. این مکانیزم امنیت را تضمین می‌کند زیرا به این ترتیب، برنامه‌های کاربر نمی‌توانند مستقیما به هسته دسترسی داشته باشند. به‌عنوان مثال، اگر یک فرآیند نیاز به دسترسی به دیسک داشته باشد مانند ذخیره کردن یک فایل بر روی دیسک یا باز کردن یک فایل، آن‌گاه با ارسال یک فراخوان سیستمی، درخواست خود را مطرح می‌نماید و پردازنده پس از سوئیچ به حالت هسته و برآورده ساختن درخواست وی به مد کاربر بازگشته و اجرای فرآیند را ادامه می‌دهد. در تصویر ۶-۲، این پروسه را مشاهده می‌نمایید.
- خطای برنامه[13] یا استثنا[14] : این وقفه زمانی رخ می‌دهد که فرآیند جاری سعی کند عمل غیرمجازی را انجام بدهد. مثل خطای سرریز[15] یا خطای «تقسیم عدد بر صفر».
- سیگنال[16]: یک وقفه نرم افزاری است که توسط یک فرآیند یا سیستم عامل یا کاربر، به یک یا چند فرآیند ارسال می‌شود. سیگنال‌ها برای مدیریت استثناها، ارتباط بین فرآیندی و کنترل فرآیندها استفاده می‌شوند.

---

[10] Machine Check
[11] Hardware Fault
[12] System Call
[13] Program Check
[14] Exception
[15] Overflow
[16] Signal



تصویر ۶-۲- فراخوان سیستمی

انواع فراخوان سیستمی

پروسه خواندن یک فایل از دیسک را در نظر بگیرید. این کار، یک درخواست دسترسی به I/O (دیسک) است ولی با یک فراخوان سیستمی از طرف فرآیند متقاضی آغاز می‌شود. ممکن است درخواست‌های دیگری نیز برای دسترسی به دیسک در صف قرار داشته باشند، که دراین‌صورت، برای جلوگیری از اتلاف زمان پردازنده، فرآیند متقاضی مسدود[۱۷] می‌شود. هرگاه دسترسی به دیسک به انجام رسید، کنترلر دیسک با استفاده از یک وقفه I/O این اتفاق را اعلام می‌کند.

اگر فرآیند متقاضی برای ادامه کار خود به پاسخ I/O نیاز داشته باشد، ناگزیر است که تا زمان دریافت پاسخ، مسدود گردد. اما گاهی برخی از وظایف فرآیند، به پاسخ I/O وابسته نیستند و بنابراین فرایند می‌تواند در طی زمان انتظار، به انجام آن وظایف بپردازد. بر این اساس دو نوع فراخوان سیستمی طراحی شده است:

فراخوان سیستمی مسدود کننده: فرآیند متقاضی، پس از ارسال فراخوان سیستمی به حالت مسدود درمی‌آید. پردازنده به فرآیند بعدی سوئیچ می‌کند. هرگاه درخواست فرآیند تکمیل شد، یک وقفه سخت افزاری I/O ارسال می‌گردد که منجر به خروج فرآیند از حالت انسداد خواهد شد.

---

[17] Block

فراخوان سیستمی غیرمسدود کننده: پس از ارسال فراخوان سیستمی، کنترل پردازنده بلافاصله به فرآیند متقاضی برمی‌گردد و فرآیند می‌تواند به سایر وظایفش بپردازد.

۶-۲- مشاهده وقفه‌های سیستم

برای بررسی انواع و تعداد وقفه‌های سیستم از دستورات زیر استفاده می‌کنیم. آمارکلی وقفه‌ها در فایلی به نام interrupts در مسیر proc/ در سیستم ذخیره می‌شود.

```
مشاهده آمار وقفه‌ها تا این لحظه  #
cat /proc/interrupts

مشاهده آمار وقفه‌های در حال رخ دادن به‌صورت زنده #
فقط ۲۰ سطر ابتدای فایل نمایش داده خواهد شد #
watch -n 1 'cat /proc/interrupts | head -n 20'
```

نتیجه اجرای دستور اول، در تصویر ۶-۳ نمایش داده شده است. همان‌طور که در تصویر نیز مشاهده می‌کنید، محتویات فایل وقفه‌ها به این صورت می‌باشد: اولین ستون، شناسه هر وقفه را نمایش می‌دهد. این شناسه می‌تواند یک عدد و یا یک کلمه اختصاری سه حرفی باشد. مثلا شناسه تایمر سیستم، برابر با عدد صفر است. شناسه تایمر محلی نیز با عبارت LOC مشخص شده است. سپس چند ستون وجود دارد که تعداد پردازنده یا هسته سیستم را مشخص می‌کنند. مثلا CPU0، CPU1، CPU2 و به همین ترتیب. در زیر هر یک از این ستون‌ها، تعداد وقفه‌های پردازش شده توسط آن پردازنده/ هسته نشان داده می‌شود. ستون آخر نیز بیانگر نام هر وقفه می‌باشد.

چند مورد از وقفه‌های مهم را که می‌توانید در این فایل مشاهده نمایید، در ادامه مورد بررسی قرار می‌دهیم.

وقفه شماره ۰: وقفه تایمر که تایمر اصلی و مرکزی سیستم را نشان می‌دهد و همیشه فعال است زیرا برای زمان‌بندی عمومی و کلی سیستم ضروری می‌باشد.

```
soltani@OSLaboratory:~$ cat /proc/interrupts
           CPU0       CPU1
  0:        431          0  IO-APIC    2-edge      timer
  1:          0       5435  IO-APIC    1-edge      i8042
  6:          0          2  IO-APIC    6-edge      floppy
  8:          0          0  IO-APIC    8-edge      rtc0
  9:          0          0  IO-APIC    9-fasteoi   acpi
 12:          0      90906  IO-APIC   12-edge      i8042
 14:          0          0  IO-APIC   14-edge      ata_piix
 15:          0          0  IO-APIC   15-edge      ata_piix
 16:     247639          0  IO-APIC   16-fasteoi   vmwgfx, snd_ens1371
 17:      37550          0  IO-APIC   17-fasteoi   ehci_hcd:usb2, ioc0
 18:         63          0  IO-APIC   18-fasteoi   uhci_hcd:usb1
 19:          0      20933  IO-APIC   19-fasteoi   ens33
```

تصویر ۶-۳- مشاهده وقفه‌ها

وقفه شماره ۱ و ۱۲: وقفه کنترلر صفحه‌کلید و ماوس که در معماری x86 با نام i8042 مشخص می‌شود. با فشردن هر کلید یا هر حرکت ماوس، تعداد این وقفه‌ها افزایش می‌یابد.

وقفه شماره ۱۷ و ۱۸: وقفه کنترلر دستگاه‌های یواس‌بی. کاربرد: هرگاه دستگاه USB به سیستم وصل شود، تعداد این وقفه افزایش خواهد یافت.

وقفه شماره ۸: وقفه ساعت واقعی[18] سیستم (rtc) است. معمولا مقادیر کمی دارد زیرا فقط برای به‌روزرسانی زمان مورد استفاده قرار می‌گیرد.

وقفه NMI: وقفه‌های سیستمی غیرقابل ماسک شدن[19]. این وقفه‌ها غیرقابل مسدودسازی بوده و در خطاهای سخت‌افزاری بحرانی کاربرد دارند.

---

[18] Real Time Clock

[19] Non-Maskable Interrupt

وقفه LOC: وقفه تایمرهای داخلی هر پردازنده[20]. هر پردازنده یک تایمر مستقل دارد و از آن برای زمان‌بندی پردازش‌های خود استفاده می‌نماید. این وقفه طبیعتا عدد بزرگی دارد. برای نظارت بر این وقفه به‌طور خاص، دستور زیر را اجرا نمایید.

```
مشاهده آمار وقفه‌های تایمر محلی هر پردازنده به‌صورت زنده #
watch -n 1 'cat /proc/interrupts | grep LOC'
```

نکته: غیر از وقفه تایمر، شناسه‌ی عددی سایر وقفه‌ها ممکن است در سیستم‌های مختلف، مقادیر متفاوتی داشته باشد.

۶-۳- برنامه‌نویسی به زبان C در لینوکس

در اکثر دستوکارهای آتی، برنامه‌هایی به زبان C خواهیم نوشت. بنابراین در این بخش به اختصار به روش کامپایل و اجرای این برنامه‌ها در لینوکس می‌پردازیم. برای کد نویسی در لینوکس می‌توانید یک محیط توسعه نرم‌افزار[21] مانند Code::Blocks، Visual Studio Code، NetBeans و یا Eclipse CDT را بر روی سیستم خود نصب کنید و یا به سادگی مانند این کتاب، از ویرایشگرهای متنی برای نوشتن کد و از ابزارهای خط فرمانی مانند gcc برای کامپایل کد استفاده نمایید. درصورتی‌که gcc به‌صورت پیش‌فرض روی توزیع لینوکسی شما نصب نشده است با دستور زیر آن را نصب نمایید.

```
sudo   apt   install   gcc
```

۶-۳-۱- کدنویسی

برای شروع، یک فایل متنی ساده با نام MyCode.c ایجاد کرده و یک کد ساده به زبان C مانند زیر در آن بنویسید. این کد صرفا یک پیام را در خروجی نمایش می‌دهد.

---

[20] Local Timer Interrupts

[21] Integrated Development Environment (IDE)

```
// MyCode.c
#include <stdio.h>
int main(){
     printf ("Hello world! Testing C Compiler");
     return 0;   }
```

۶-۳-۲- کامپایل و اجرای کد

اگر از یک محیط توسعه برای کد نویسی استفاده کرده‌اید، می‌توانید به‌سادگی از کلیدهای کامپایل و اجرا برای دیدن نتیجه‌ی کد خود استفاده نمایید. اما اگر از ویرایشگرهای متنی استفاده می‌کنید، ابتدا در ترمینال به دایرکتوری حاوی کد رفته و سپس دستور زیر را برای کامپایل کد اجرا نمایید.

```
gcc    MyCode.c   -o    MyProg
```

gcc نام کامپایلر زبان C در لینوکس است. پس از آن باید نام و آدرس کدمنبع را ذکر کنید. سوئیچ o- از کامپایلر می‌خواهد که کد را کامپایل کرده، فایل‌های کتابخانه‌ای را به آن لینک نموده و فایل آبجکت اجرایی نهایی را بسازد. در انتها، نام و آدرس مدنظرتان برای ذخیره فایل نهایی را ذکر کنید. توجه داشته باشید که برای فایل اجرایی هیچ پسوندی قرار ندهید.

در اینجا، چون فرض کردیم که در دایرکتوری کد قرار داریم، فقط نام فایل را ذکر کرده‌ایم. فایل اجرایی نیز در همان دایرکتوری و تحت نام MyProg ذخیره خواهد شد. درصورتی‌که در کدمنبع خطایی وجود داشته باشد، هر خطا همراه با شماره سطر و توضیحاتش در ترمینال برایتان نمایش داده خواهد شد. اما اگر کد شما فاقد خطا باشد، علامت اعلان - بدون هیچ توضیح اضافه‌ای - در سطر بعد پدیدار می‌گردد.

برای اجرا و دیدن نتایج، اگر در دایرکتوری فایل اجرایی هستید، پس از علامت /. که اشاره به مسیر جاری دارد، نام فایل را بنویسید.

```
./MyProg
```

و درغیراین‌صورت، نام و آدرس محل قرارگیری فایل را ذکر کنید. مثلا اگر فایل اجرایی در دایرکتوری‌ای به نام Codes باشد که آن هم در دایرکتوری خانگی کاربر جاری قرار دارد، به‌صورت زیر عمل می‌کنیم.

```
/home/student/Codes/MyProg
```

برای کامپایل و اجرای سایر مثال‌های زبان C نیز دقیقا به همین صورت عمل کنید.

۶-۴- غیرفعال‌سازی وقفه تایمر

در این بخش با مدیریت و نظارت بر وقفه‌ها در سیستم عامل آشنا می‌شویم. ازآنجایی‌که عملیات غیرفعال کردن وقفه‌ها باید در مد هسته اجرا شود، بنابراین لازم است این کد را به‌صورت یک ماژول هسته طراحی و به هسته وارد نماییم. به همین دلیل، کامپایل و اجرای کد در این مثال با آنچه که در قسمت قبل برای کدهای C آموختیم متفاوت خواهد بود. بعدا در یک دستورکار مجزا به‌طور کامل به مبحث ماژول‌ها خواهیم پرداخت.

این ماژول پس از بارگذاری در هسته، تمام وقفه‌های سیستم را با دستور local_irq_save() غیرفعال کرده و به مدت ۵ ثانیه به حالت انتظارمی‌رود. در این فاصله می‌توانید مثلا با فشردن کلیدهایی از صفحه‌کلید، چند وقفه به سیستم ارسال نمایید. اما ازآنجایی‌که وقفه‌ها غیرفعال هستند، وقفه‌های ارسالی شما پردازش نخواهند شد و در عوض در صف وقفه‌های معلق قرار می‌گیرند. سرانجام وقفه‌های سیستم دوباره فعال می‌شوند و وقفه‌هایی که معلق مانده بودند، پردازش خواهند شد. بنابراین به کمک این کد علاوه بر مدیریت وقفه تایمر، وقفه‌های ورودی/ خروجی را نیز مشاهده خواهیم کرد.

```
//IRQ_Monitor.c

// ماژول پایش فعال و غیرفعال‌سازی وقفه‌ها در لینوکس
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/interrupt.h>
#include <linux/delay.h>

//تعریف یک پرچم برای نگهداری وضعیت وقفه‌ها
static unsigned long flags
```

```
//تابع ورود ماژول
static int __init monitor_init(void){
    printk(KERN_INFO "Interrupts disabled for 5 seconds. Press some
keys \n");

    // ذخیره وضعیت فعلی و غیرفعال کردن وقفه‌ها
    local_irq_save(flags);

    // Busy waiting
    mdelay(5000);

    // فعال‌سازی مجدد وقفه‌ها
    local_irq_restore(flags); }
    printk(KERN_INFO "Interrupts enabled again\n);
    return 0;   }

//تابع خروج ماژول
static void __exit monitor_exit(void) {
      printk(KERN_INFO "Module unloaded.\n");  }

module_init(monitor_init);
module_exit(monitor_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("OSLaboratory of Urmia University");
MODULE_DESCRIPTION("Interrupt Monitor");
```

مراحل کامپایل و وارد کردن ماژول به هسته به‌صورت زیر است. ابتدا فایل Makefile را با محتوای زیر در کنار فایل کد اصلی ایجاد کرده و سپس دستور make را اجرا نمایید.

```
obj-m += irq_monitor.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
     $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
     $(MAKE) -C $(KDIR) M=$(PWD) clean
```

قبل از اینکه ماژول را اجرا کنید، یک ترمینال دیگر باز کرده و دستور زیر را برای نظارت بر وقفه‌های صفحه‌کلید و ماوس، اجرا نمایید.

```
watch  -n  1  'cat /proc/interrupts | grep i8042'
```

حال با دستور زیر ماژول را در هسته وارد و اجرا کنید.

```
sudo   insmod   irq_monitor.ko
```

به محض اینکه رمز را وارد نمایید (به‌دلیل استفاده از دستور sudo) با شروع اجرای ماژول، به مدت ۵ ثانیه سیستم کاملا قفل می‌شود. با وجود حرکت نشانگر یا فشردن کلیدهای صفحه‌کلید نه تنها اثری در سیستم مشاهده نمی‌شود، بلکه در ترمینال دوم نیز مشاهده می‌کنید که تعداد وقفه‌های مربوطه هیچ تغییری نمی‌یابند.

پس از سپری شدن مدت زمان مذکور، تمام کاراکترهایی که وارد کرده بودید در ترمینال ظاهر خواهند شد. این یعنی وقفه‌های معلقی که در صف مانده بودند، اکنون پردازش شدند.

برای مشاهده اطلاعات بیشتر درباره وقایعی که در طی اجرای کد اتفاق افتاده‌اند، دستورات زیر را یکی یکی اجرا و نتایج هر یک را بررسی نمایید.

```
sudo dmesg | tail -n 10     //مشاهده ۱۰ سطر انتهایی از پیام‌های لاگ کرنل
dmesg | grep "irq_monitor"  //مشاهده پیام‌های لاگ کرنل به‌صورت فیلترشده بر اساس کلمه موردنظر
```

و در نهایت برای حذف ماژول از هسته دستور زیر را اجرا کنید.

```
sudo rmmod irq_monitor
```

توجه داشته باشید که در این آزمایش برای سادگی فقط از تابع ()mdelay که از نوع مسدودکننده[۲۲] می‌باشد استفاده کردیم. ازآنجایی‌که غیرفعال ماندن وقفه‌ها برای مدت طولانی می‌تواند باعث کندی سیستم و از دست رفتن داده‌ها شود؛ کرنل با استفاده از مکانیزم‌های نظارتی خود، هشدارهایی را در لاگ ثبت می‌کند. این هشدارها، بین پیام ورود و خروج ماژول در لاگ هسته قابل مشاهده هستند. برای پیاده‌سازی پیشرفته‌تر، باید از روش‌های غیرمسدودکننده مانند تایمر و Workqueue استفاده نمود.

۶-۵- فراخون سیستمی

گاهی یک برنامه نیاز به انجام کاری دارد که خود، مجوز انجام آن کار را ندارد و بنابراین سیستم عامل باید آن کار را از طرف او به انجام برساند. در چنین مواقعی، برنامه درخواست خود را در قالب یک فراخوان سیستمی به سیستم‌عامل منتقل می‌کند. در این بخش به ردیابی نمونه‌هایی از این فراخوان‌های سیستمی می‌پردازیم. کد زیر، چند فراخوانی سیستمی مختلف را اجرا کرده و اطلاعات سیستم را جمع‌آوری می‌نماید. همچنین با چاپ پیام‌هایی، کاربر را از انتقال بین حالت کاربر و حالت هسته مطلع می‌سازد.

```
// Sys_Call.c

// ردیابی فراخوانی‌های سیستمی و سوئیچ بین مد کاربر و مد هسته
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/time.h>
#include <sys/resource.h>
#include <sys/sysinfo.h>
#include <time.h>

// ردیابی فراخوانی‌های سیستمی
void trace_system_calls() {

    // فراخوانی سیستمی برای دریافت شناسه فرآیند جاری
    printf("getpid() system call:\n");
    pid_t pid = getpid();
    printf("   PID: %d\n", pid);
```

[22] Blocking

```
    // فراخوانی سیستمی برای دریافت میزان مصرف منابع
    printf("\ngetrusage() system call:\n");
    struct rusage usage;
    getrusage(RUSAGE_SELF, &usage);
    printf("   User time: %ld.%06ld\n",
           usage.ru_utime.tv_sec, usage.ru_utime.tv_usec);
    printf("   System time: %ld.%06ld\n",
           usage.ru_stime.tv_sec, usage.ru_stime.tv_usec);

    // فراخوانی سیستمی برای دریافت اطلاعات سیستم
    printf("\nsysinfo() system call:\n");
    struct sysinfo info;
    sysinfo(&info);
    printf("   Free memory: %lu KB\n", info.freeram / 1024);
    printf("   Number of processes: %d\n", info.procs); }

//نمایش سوئیچ بین مد کاربر و مد هسته
void mode_switching() {
    printf("Running in user mode. Performing some simple
mathematical calculations. \n");
    int result = 0;
    for (int i = 0; i < 1000; i++) {
        result += i * i;            }
    printf("   Calculation result: %d\n", result);

    // Perform a system call that triggers mode switch
    printf("\nSwitching to kernel mode by executing nanosleep()
system call\n");

    struct timespec ts;
    ts.tv_sec = 0;
    ts.tv_nsec = 100000000; // 100ms

    // تابع nanosleep یک فراخوانی سیستمی است، بنابراین نیاز به مداخله هسته دارد.
    nanosleep(&ts, NULL);
    printf("\nReturned to user mode\n");   }

int main() {
    printf("\n___System Call Examples___\n");
    trace_system_calls();
    printf("\n___User/Kernel Mode Switching Example___\n");
    mode_switching();
    return 0;   }
```

توضیح بخش‌های مختلف کد

بخش اول؛ تابع ()getpid یک فراخوان سیستمی است[۲۳] و شناسه فرآیند جاری را برمی‌گرداند. در واقع فرآیند با اجرای این تابع، شماره شناسه خود را از هسته می‌پرسد. هسته برای پاسخگویی به این درخواست ابتدا پردازنده را از حالت کاربر به حالت هسته می‌برد. سپس به حافظه هسته منتقل شده و شناسه فرآیند را از ساختارهای داخلی پیدا می‌کند. مجددا به حالت کاربر برمی‌گردد و نتیجه را به فرآیند متقاضی تحویل می‌دهد.

بخش دوم؛ این بخش از کد با استفاده از تابع getrusage که یک فراخوان سیستمی است، اطلاعات دقیقی درباره میزان منابع سیستمی که برنامه مصرف کرده است، جمع‌آوری می‌کند. وقتی این تابع فراخوانی شود، سیستم عامل آمار کاملی را از نحوه استفاده برنامه از پردازنده، حافظه و دیگر منابع، در اختیار ما قرار می‌دهد. در این کد فقط دو مورد از مواردی که این تابع برمی‌گرداند، دریافت و چاپ شده‌اند که عبارتند از: «زمان کاربر» و «زمان سیستم».

زمان کاربر، به مدت زمانی اشاره دارد که پردازنده مستقیما در حال اجرای دستورات برنامه بوده است - یعنی زمانی که صرف انجام محاسبات و عملیات داخلی برنامه شده است. برای مثال، وقتی برنامه در حال انجام محاسبات ریاضی یا پردازش داده‌ها است، این زمان در بخش کاربر محاسبه می‌شود. در مقابل، زمان سیستم نشان‌دهنده مدت زمانی است که پردازنده - به درخواست برنامه - برای اجرای دستورات سیستم عامل صرف کرده است. این زمان، مدتی است که سیستم عامل برای پاسخ‌گویی به فراخوان‌های سیستمی برنامه صرف می‌کند. مانند خواندن فایل از دیسک یا برقراری ارتباط با سخت‌افزار.

اگر برنامه‌ای «زمان سیستم» بالایی داشته باشد، نشان می‌دهد که برنامه به کرات فراخوان سیستمی زده و از سیستم عامل درخواست سرویس داشته است. مثلا شاید به دلیل کار با فایل‌های زیاد یا ایجاد فرآیندهای جدید. در مقابل، برنامه‌ای با «زمان کاربر» بالا معمولا مشغول محاسبات سنگین است و کمتر به سیستم عامل مراجعه می‌کند. در مثال بالا، خروجی این بخش اعداد کوچکی را نشان می‌دهد چون برنامه ساده است و منابع زیادی مصرف نمی‌کند.

بخش سوم؛ فراخوان سیستمی sysinfo اطلاعاتی درباره وضعیت کلی سیستم شامل حافظه، فرآیندها و بار کاری ارائه می‌دهد. دو موردی که در این کد دریافت و نمایش داده می‌شوند عبارتند از: حافظه آزاد و تعداد فرآیندهای در حال

---

[۲۳] در واقع این تابع، یک wrapper کتابخانه C حول فراخوانی سیستمی واقعی SYS_getpid است. به این معنی که وقتی ()getpid را صدا می‌زنید، این تابع از کتابخانه C فراخوانی شده و syscall مربوطه را اجرا می‌کند.

اجرا. حافظه آزاد، مقدار حافظه موجود برای استفاده توسط برنامه‌های جدید را نشان می‌دهد. این عدد بر حسب بایت است که در اینجا به کیلوبایت تبدیل شده تا خوانایی بهتری داشته باشد. تعداد فرآیندها نیز نشان می‌دهد که چند برنامه و سرویس مختلف به‌صورت همزمان روی سیستم در حال اجرا هستند. این عدد شامل همه فرآیندهای سیستمی، سرویس‌های پس‌زمینه و برنامه‌های کاربری می‌باشد.

کامپایل و اجرا

کد را مطابق آنچه در بخش ۶-۳ دیدیم، کامپایل و اجرا می‌کنیم.

```
gcc   Sys_Call.c   -o   sys_call
./sys_call
```

همچنین می‌توانید از ابزار strace در هنگام اجرای برنامه استفاده کنید. این دستور، یک ابزار خط فرمانی در لینوکس است که فراخوانی‌های سیستمی و سیگنال‌های اجرا شده توسط یک برنامه را نمایش می‌دهد.

```
ردیابی فراخوانی‌های سیستمی یک فرآیند  #
strace ./sys_call

آمار فراخوانی‌ها #
strace -c ./sys_call
```

۶-۶- سیگنال

در این بخش، یک سیستم مدیریت و ردیابی سیگنال ایجاد خواهیم کرد که از مکانیزم ارسال، دریافت و مدیریت سیگنال‌ها استفاده نماید. بنابراین ابتدا یک برنامه ایجاد می‌کنیم که سیگنال‌های مختلف را دریافت کرده و نسبت به هر یک از آن‌ها واکنش مناسب نشان دهد. در مثال زیر از سه سیگنال SIGUSR1، SIGUSR2 و SIGTERM استفاده شده است.

درک مفهوم سیگنال‌ها گاهی دشوار می‌شود، زیرا سیگنال‌ها رویدادهایی ناهمزمان هستند و رفتار آن‌ها گاهی غیرقابل پیش‌بینی به نظر می‌رسد. به‌علاوه، وضعیت سیگنال‌ها به‌صورت مستقیم قابل مشاهده نیست. بنابراین در کد زیر ترتیبی اتخاذ می‌کنیم که برنامه شناسه خود را نمایش دهد تا کاربران بدانند به چه مقصدی باید سیگنال ارسال نمایند. برنامه در یک حلقه بی‌نهایت منتظر خواهد ماند تا از طرف کاربر سیگنالی دریافت نماید.

در درون برنامه، توابع مدیریت سیگنال[۲۴] را ثبت[۲۵] می‌کنیم. این توابع واکنش به هر سیگنال را مشخص می‌کنند. توابع کتابخانه‌ای که برای ثبت سیگنال در دو مثال بعدی مورد استفاده قرار گرفته‌اند، ()signal و ()sigaction نام دارند و در فایل سرآیند signal.h تعریف شده‌اند. وقتی یکی از این دو تابع فراخوانی شود، سیستم‌عامل در ساختار داده‌های داخلی خود یک ورودی[۲۶] ایجاد کرده و آدرس تابع مدیریت تعریف شده توسط کاربر را در آن ذخیره می‌کند تا هنگام دریافت سیگنال، به آن آدرس مراجعه نماید.

```
// Signal_Test.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

// تابع مدیریت سیگنال؛ هرگاه سیگنالی دریافت شود، این تابع فراخوانی خواهد شد
void signal_handler(int sig) {
    printf("\nSignal received: %d (%s)\n", sig, strsignal(sig));  }

int main() {
    pid_t pid = getpid();

    // Register signal handlers for different signals
    signal(SIGUSR1, signal_handler);  // User-defined signal 1
    signal(SIGUSR2, signal_handler);  // User-defined signal 2
    signal(SIGTERM, SIG_DFL);         // Termination request
    signal(SIGINT, SIG_DFL);          // Interrupt (Ctrl+C)

    // Instructions for testing
```

---

[24] Signal Handlers
[25] Register/ Notify
[26] Entry

```
    printf("\nSend these signals from another terminal.\n");
    printf("   To send SIGUSR1:    kill -USR1 %d\n", pid);
    printf("   To send SIGUSR2:    kill -USR2 %d\n", pid);
    printf("   To send SIGTERM:    kill -TERM %d\n", pid);
    printf("   To interrupt:       Ctrl+C in this terminal \n");
    printf("   Check status:cat /proc/%d/status | grep Sig\n", pid);
    printf("\nWaiting for a signal.\n");

    // Infinite loop: waiting for signals
    while(1) {
       // Use pause function to suspend the program until a signal
is received
        pause();
        printf("Received and processed a signal. Now waiting for the
next signal.\n"); }
    return 0; }
```

برنامه تازمانی‌که سیگنالی دریافت نکرده با استفاده از تابع pause - که در فایل سرآیند signal.h تعریف شده است - در حالت انتظار می‌ماند. وقتی سیگنالی می‌رسد، تابع مدیریت کننده مربوطه فراخوانی می‌شود. برای دو سیگنال TERM و INT مقادیر پیش‌فرض سیستمی در نظر گرفته شده اما برای دو سیگنال دیگر (User Defined)، تابع هندلر تعریف شده است.

حال یک ترمینال دیگر باز کنید و با استفاده از دستور kill سیگنال‌های تعریف شده را مانند زیر، به برنامه ارسال نمایید. در دستورات زیر به‌جای [PID] باید شناسه فرآیند خود را که در ترمینال اول نمایش داده می‌شود، جایگزین کنید.

```
# Send SIGUSR1
kill -USR1 [PID]

# Send SIGUSR2
kill -USR2 [PID]

# Send SIGTERM to terminate
kill -TERM [PID]
```

برای ارسال سیگنال INT، باید در همان ترمینال اول کلیدهای ترکیبی Ctrl+c را بفشارید.

برای مشاهده اطلاعاتی کامل درباره وضعیت لحظه‌ای سیگنال‌های فرآیند، از فایل /proc/[PID]/status/ استفاده می‌شود. این فایل شامل موارد زیر است:

SigPnd: سیگنال‌های معلق[۲۷]

SigBlk: سیگنال‌های مسدود شده[۲۸]

SigIgn: سیگنال‌های نادیده گرفته شده[۲۹]

SigCgt: سیگنال‌های مدیریت شده[۳۰]

جلوی هر یک از این عبارت‌ها یک عدد هگزادسیمال مشاهده خواهید کرد که یک بیت‌ماسک[۳۱] است. هر بیت از این عدد، نماینده یک شماره سیگنال است. اگر بیت مربوط به یک سیگنال برابر با ۱ باشد، یعنی آن سیگنال در آن دسته قرار دارد و اگر صفر باشد، یعنی آن سیگنال در آن دسته نیست.

به‌عنوان مثال اگر SigCgt = 0000000000000a00 باشد، ابتدا عدد را به مبنای دو تبدیل می‌کنیم که برابر می‌شود با 0001010000000. حال در این عدد باینری، شماره بیت‌های روشن را پیدا می‌کنیم که عبارتند از بیت‌های شماره ۹ و ۱۱. ازآنجایی‌که شماره سیگنال برابر است با شماره بیت روشن به اضافه‌ی یک، بنابراین سیگنال‌های شماره ۱۰ یعنی SIGUSR1 و سیگنال شماره ۱۲ یعنی SIGUSR2، توسط این فرآیند دریافت و مدیریت می‌شوند.

```
# مشاهده اطلاعات سیگنال‌های فرآیند
cat /proc/[PID]/status | grep Sig
```

این اطلاعات به محض دریافت یا پردازش هر سیگنال، در فایل مذکور بازنویسی می‌شود و بنابراین برای مشاهده سیگنال‌های معلق یا مسدود فرصت بسیار کمی داریم. بنابراین می‌توانید از روش‌های مختلفی استفاده کنید. مثلا اضافه کردن یک خط در هندلر برای ذخیره خودکار وضعیت در یک فایل لاگ، استفاده از کتابخانه inotify برای تشخیص

---

[27] Pending Signals
[28] Blocked Signals
[29] Ignored Signals
[30] Caught Signals
[31] Bitmask

بلادرنگ تغییرات فایل و یا ایجاد یک برنامه نظارت جداگانه که مثلا هر ۱۰۰ میلی‌ثانیه وضعیت فایل را چک کرده و تغییرات را ثبت کند.

سیگنال‌های همگام‌سازی

سیگنال‌ها برای ایجاد هماهنگی بین فرآیندها نیز مورد استفاده قرار می‌گیرند و از جمله روش‌های کلاسیک همگام‌سازی بین فرآیندها هستند. روش‌های جدیدتری مانند سمافور نیز بدین‌منظور وجود دارند که در دستورکارهای بعدی مورد بررسی قرا خواهند گرفت.

از جمله مواردی که می‌توان از سیگنال‌ها برای ایجاد هماهنگی استفاده کرد، عبارت است از: همگام‌سازی دو فرآیند در دسترسی به یک منبع مشترک. معمولا از سیگنال‌های کاربر (SIGUSR1, SIGUSR2) برای ایجاد ارتباط بین‌فرآیندی و انجام همگام‌سازی استفاده می‌شود. در این بخش دو برنامه مستقل داریم که یکی تولیدکننده و دیگری مصرف‌کننده نام دارد. تولیدکننده، داده‌ای را تولید کرده و در یک منبع مشترک فرضی قرار می‌دهد، سپس به مصرف‌کننده اطلاع می‌دهد که داده آماده مصرف می‌باشد. درنهایت، مصرف‌کننده از داده استفاده کرده و به تولیدکننده اطلاع می‌دهد که داده مصرف شده است و بنابراین امکان قرار دادن داده جدید در منبع مشترک وجود دارد. به این ترتیب، تولیدکننده و مصرف‌کننده با یکدیگر همگام شده و از یکدیگر جلو نمی‌افتند. به عبارت دیگر، تولید کننده تازمانی‌که به وی اعلام نشود که داده آماده مصرف است، سراغ منبع نمی‌رود و بنابراین داده‌های باقیمانده از مراحل قبل را به اشتباه مجددا مصرف نخواهد کرد. تولیدکننده نیز تا اعلام مصرف‌کننده، منتظر می‌ماند و بنابراین داده‌هایی که هنوز مورد استفاده قرار نگرفته‌اند را با داده‌های جدید بازنویسی نخواهد کرد.

کد تولیدکننده

در این کد، تولیدکننده از طریق ارسال و دریافت سیگنال‌هایی، با برنامه مصرف‌کننده همگام می‌شود. تولیدکننده، شناسه مصرف‌کننده را دریافت کرده و هندلرهای سیگنال خود را ثبت می‌کند. سپس منتظر می‌ماند تا مصرف‌کننده آماده شود و این آمادگی را با ارسال سیگنال SIGUSR1 اعلام نماید. پس از آن، داده‌ای را تولید می‌کند (از کاربر

دریافت می‌کند) و با ارسال SIGUSR1 به مصرف‌کننده اطلاع می‌دهد که داده، آماده مصرف است. در نهایت، تولیدکننده منتظر SIGUSR2 می‌ماند. این سیگنال، نشانگر تایید مصرف‌کننده مبنی بر پردازش داده دریافتی توسط وی می‌باشد. پس از دریافت تایید، برنامه خاتمه می‌یابد.

نکته: در این کد، سیگنال‌ها برای همگام‌سازی استفاده می‌شوند ولی هیچ داده واقعی ارسال نمی‌گردد. در برنامه‌های واقعی، داده از طریق مکانیزم‌هایی مانند حافظه مشترک یا شبه‌فایل منتقل می‌شود. در دستورکار فرآیندها، این موارد را بررسی خواهیم کرد.

```
// Producer.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

//تعریف متغیرهای پرچم برای ایجاد هماهنگی بین فرآیند تولیدکننده Error! Bookmark not defined.
و هندلرهای سیگنال
volatile sig_atomic_t ready_for_data = 0;
volatile sig_atomic_t data_processed = 0;

// تعریف تابع مدیریت کننده (هندلر) برای سیگنال اعلام آمادگی از طرف مصرف‌کننده
void consumer_ready_handler(int sig) {
    ready_for_data = 1;
    printf "Producer: Consumer is ready to receive data.\n\t
Insert an Integer number as data: ");   }

// تعریف تابع مدیریت کننده برای تایید پردازش اطلاعات
void data_processed_handler(int sig) {
    data_processed = 1;
    printf("Producer: Consumer confirmation received.\n");  }

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <consumer_pid>\n", argv[0]);
        return 1;   }

    // تبدیل آرگومان[۱] برنامه (شناسه مصرف‌کننده) از کد اسکی به عدد صحیح
    pid_t consumer_pid = atoi(argv[1]);
```

```
    // نمایش شناسه تولیدکننده و مصرف‌کننده
    printf("Producer PID: %d \n", getpid());
    printf("Consumer PID: %d\n", consumer_pid);

    // ثبت هندلرهای مدیریت سیگنال
    struct sigaction sa_ready, sa_processed;

    // Handler for SIGUSR1 (consumer ready)
    memset(&sa_ready, 0, sizeof(sa_ready));
    sa_ready.sa_handler = consumer_ready_handler;
    sigaction(SIGUSR1, &sa_ready, NULL);

    // Handler for SIGUSR2 (data processed)
    memset(&sa_processed, 0, sizeof(sa_processed));
    sa_processed.sa_handler = data_processed_handler;
    sigaction(SIGUSR2, &sa_processed, NULL);

    // Wait for consumer ready signal
    printf("Producer: Waiting for consumer to be ready.\n");
    while (!ready_for_data) {
        pause(); // Wait for signal   }

    // بخش تولید داده
   // در یک برنامه واقعی، از حافظه مشترک یا پایپ برای ارسال داده به فرآیند مصرف‌کننده استفاده می‌شود
    int data_packet;
    scanf("%d", &data_packet);

    // Send "data ready" signal
    kill(consumer_pid, SIGUSR1);
    printf("Producer: Data Ready signal sent. Waiting for processing
confirmation from consumer.\n");

    // انتظار برای تایید پردازش
    while (!data_processed) {
        pause();            }

    return 0;   }
```

توضیح بخش‌های کد

در ابتدای برنامه، دو متغیر سراسری ready_for_data و data_processed از نوع volatile sig_atomic_t تعریف شده‌اند که به‌عنوان پرچم برای هماهنگی بین فرآیند تولیدکننده و هندلرهای سیگنال عمل می‌کنند. کلمه کلیدی

volatile به کامپایلر می‌گوید که این متغیر ممکن است خارج از جریان عادی برنامه - مثلا توسط هندلر سیگنال - تغییر کند، بنابراین نباید آن را در ثبات کش کند بلکه باید هر بار مقدارش را از حافظه بخواند. sig_atomic_t نیز یک نوع داده صحیح است که خواندن و نوشتن روی آن حالت اتمیک دارد. یعنی این عملیات، فقط در یک مرحله واحد انجام می‌شود و امکان قطع شدن در میانه‌ی کار وجود ندارد. بنابراین هندلر سیگنال می‌تواند بدون نگرانی از ایجاد شرایط رقابتی، این نوع متغیرها را تغییر دهد.

هرگاه مصرف‌کننده آماده دریافت داده باشد، پرچم ready_for_data توسط هندلر مربوطه برابر با یک می‌شود و زمانی که مصرف‌کننده داده را پردازش کند، پرچم data_processed توسط هندلر مربوط به خودش برابر با یک می‌گردد.

در تابع اصلی برنامه، دو ساختار از نوع sigaction برای ثبت هندلرهای سیگنال تعریف شده‌اند. این ساختار، جایگزین تابع قدیمی‌تر ()signal است و کنترل بیشتری بر رفتار سیگنال فراهم می‌کند. در بخش ثبت هندلرها، در ابتدا با فراخوانی memset کل ساختار مذکور صفر می‌شود تا اطلاعات قبلی از آن پاک گردد. سپس در هر ساختار، فیلد sa_handler به تابع هندلر موردنظر اختصاص داده می‌شود. مثلا برای سیگنال SIGUSR1، تابع consumer_ready_handler به‌عنوان هندلر ثبت شده است. از این پس، هر بار که فرآیند تولیدکننده، سیگنال SIGUSR1 را دریافت کند، این تابع اجرا شده و پرچم ready_for_data را برابر یک می‌کند.

کد مصرف کننده

این کد یک برنامه مصرف‌کننده را تعریف کرده و آن را از طریق سیگنال، با تولیدکننده هماهنگ می‌نماید. ابتدا مصرف‌کننده، هندلرهای سیگنال را ثبت می‌کند: یک سیگنال برای دریافت داده و یک سیگنال دیگر برای همگام‌سازی. سپس شناسه فرآیند تولیدکننده را از کاربر گرفته و با ارسال SIGUSR1 به تولیدکننده اطلاع می‌دهد که آماده دریافت داده است. پس از آن، منتظر می‌ماند تا تولیدکننده نیز با ارسال SIGUSR1 اعلام کند که داده آماده است. سپس داده را به‌طور فرضی پردازش می‌کند (این کار با تابع sleep شبیه‌سازی شده است). در نهایت، مصرف‌کننده با ارسال SIGUSR2 به تولیدکننده اطلاع می‌دهد که کارش پایان یافته است.

```
// Consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

//تعریف متغیر پرچم برای ایجاد هماهنگی بین فرآیند مصرف‌کننده و هندلرهای سیگنال
volatile sig_atomic_t data_ready = 0;


//  تعریف هندلر برای سیگنال اعلام «آماده بودن داده» از طرف تولیدکننده
void data_ready_handler(int sig) {
    data_ready = 1;
    printf("Consumer: Data is prepared by producer.\n");    }

int main() {
    printf("Consumer PID: %d\n", getpid());

    //  دریافت شناسه تولیدکننده برای ارسال سیگنال
    pid_t producer_pid;
    printf("Enter producer PID: ");
    scanf("%d", &producer_pid);


    //  ثبت هندلرهای مدیریت سیگنال
    struct sigaction sa_data;


    // Handler for SIGUSR1 (data ready)
    memset(&sa_data, 0, sizeof(sa_data));
    sa_data.sa_handler = data_ready_handler;
    sigaction(SIGUSR1, &sa_data, NULL);

    //  ارسال سیگنال اعلام آمادگی از مصرف‌کننده به تولیدکننده
    kill(producer_pid, SIGUSR1);
    printf("Consumer: Ready signal sent. waiting for data
preparation by Producer.\n");


    //  انتظار برای دریافت سیگنال اعلام «آماده بودن داده» از طرف تولیدکننده
    while (!data_ready) {
        pause();        }


    //  پردازش فرضی روی داده
    printf("Consumer: Press Enter to process data. \n");
    getchar();   getchar();
    sleep(1); // Simulate processing


    //  ارسال سیگنال تایید پردازش
    kill(producer_pid, SIGUSR2);
```

```
    printf("Consumer: Data Processed. Confirmation sent to
Producer.\n");

    return 0;   }
```

نکته: در این مثال به جهت سادگی، شناسه هر فرآیند به‌صورت دستی به فرآیند دیگر داده می‌شود. اما در برنامه‌های واقعی، معمولا فرآیندها از روش‌های خودکار برای کشف یکدیگر استفاده می‌کنند. مثلا از طریق فایل‌های پیکربندی، پایپ بانام، سوکت یا D-Bus. به‌علاوه، اگر فرآیند گیرنده‌ی سیگنال نیاز به شناسایی فرستنده داشته باشد، می‌تواند با استفاده از پرچم SA_SIGINFO در هندلر سیگنال، شناسه‌ی فرآیند فرستنده را به‌صورت خودکار استخراج کرده و پاسخ خود را به همان فرآیند فرستنده ارسال نماید.

برای کامپایل کدها به روش زیر عمل کنید.

```
gcc   Producer.c   -o   producer
gcc   Consumer.c   -o   consumer
```

برای اجرا دو ترمینال باز کنید و هر برنامه را در یک ترمینال مجزا اجرا نمایید. توجه کنید که ابتدا باید فرآیند مصرف کننده اجرا شود. چون فرآیند تولیدکننده بر اساس شناسه، مصرف کننده را شناسایی و پیدا خواهد کرد.

```
ترمینال اول: اجرای برنامه مصرف‌کننده #
پس از اجرا، شناسه نمایش داده شده را یادداشت نمایید #
./consumer

ترمینال دوم: اجرای برنامه تولیدکننده #
شناسه یادداشت شده از مرحله قبل را به‌عنوان آرگومان وارد نمایید #
./producer PID [مصرف‌کننده]
```

تمرین

کد برنامه signal_test را که در بخش ۶-۶ همین دستورکار آمده است، تغییر دهید تا یک هندلر یا مدیریت‌کننده برای سیگنال SIGINT (معادل فشردن کلیدهای Ctrl+C در ترمینال) ثبت کند. به‌طوری‌که برنامه هنگام دریافت این سیگنال، به جای رفتار پیش‌فرض که «خروج از برنامه» است، یک پیام مناسب نمایش داده و همچنان به‌صورت یک فرآیند در حال اجرا باقی بماند تا کاربر بتواند چندین بار ارسال سیگنال را امتحان نماید. یک سیگنال دیگر نیز برای خروج از برنامه طراحی کنید. به‌علاوه، در راهنمای متنی برنامه خود نیز این تغییرات را اعمال نمایید.

گزارش کار

فرض کنید یک بانک نیاز به توسعه یک سامانه مدیریت سیگنال دارد که بتواند در حین انجام عملیات حساس مالی، رفتار کنترل‌شده‌ای در برابر سیگنال‌های خارجی داشته باشد. این عملیات حساس در دنیای واقعی می‌تواند انتقال بین بانکی وجوه با مبالغ بالا و یا تولید گزارش‌های مالی دوره‌ای باشد. نمی‌خواهیم در حین انجام چنین عملیاتی بروز یک وقفه، باعث نیمه‌کاره ماندن عملیات بشود. بنابراین لازم است سامانه‌ای طراحی کنیم که در حین انجام عملیات حساس، سیگنال‌های دریافتی را مسدود کرده و پس از اتمام عملیات، به‌صورت خودکار آن‌ها را پردازش نماید. بر این اساس، برنامه دو رفتار از خود بروز می‌دهد:

۱- **حالت عادی**: در این حالت هر سیگنالی دریافت شود، به‌صورت بلادرنگ پردازش خواهد شد.

- SIGUSR1: برنامه با دریافت این سیگنال، مقدار یک متغیر پرچم با نام mode_change را برابر با ۱ قرار می‌دهد که به منزله فعال شدن حالت ویژه می‌باشد.
- SIGUSR2: چاپ یک پیام که وضعیت برنامه (عادی/ ویژه) را نمایش می‌دهد.

۲- **حالت ویژه**: در این حالت، برنامه در حین انجام یک عملیات حساس قرار دارد. این عملیات را می‌توانید با استفاده از تابع sleep با یک مدت زمان مشخص مثلا ۵ ثانیه‌ای، شبیه‌سازی نمایید. در این حالت، فقط سیگنال SIGUSR2 با استفاده از فراخوانی سیستمی sigprocmask مسدود می‌شود ولی سایر سیگنال‌ها پردازش خواهند شد.

- SIGUSR1: مقدار متغیر پرچم mode_change را برابر با ۰ قرار می‌دهد که به منزله خروج از حالت ویژه است. این سیگنال را به‌عنوان سیگنال اضطراری در نظر گرفته‌ایم. بنابراین با وجود اینکه در حالت ویژه هستیم، این سیگنال در ماسک مسدودسازی قرار نگرفته و پردازش خواهد شد.

نکته: سیگنال SIGUSR1 مانند یک کلید عمل می‌کند که در حالت عادی باعث ورود به حالت ویژه و در حالت ویژه باعث خروج از آن می‌شود.

- SIGUSR2: ازآنجایی‌که در حالت ویژه هستیم، این سیگنال با استفاده از ماسک، مسدود می‌شود و هیچ پیامی چاپ نخواهد شد. تنها پس از خروج از حالت ویژه است که سیگنال معلق پردازش شده، چاپ پیام انجام می‌پذیرد و واقعه‌ی «مسدود شدن سیگنال» نیز در یک فایل لاگ متنی ثبت می‌گردد.

برنامه را اجرا کرده و عملکرد آن را در هر دو حالت عادی و ویژه، با ارسال سیگنال‌های تعریف شده بیازمایید.

منابع و مراجع:

- Silberschatz A. *Operating System Concepts*, Wiley,10th Edition, 2018.
- Tanenbaum AS. *Modern Operating Systems*, Pearson Education, 2015.
- Madieu J. *Linux Device Driver Development*, Chapter 3. Packt Publishing, 2017.
- Billimoria KN. *Linux Kernel Programming*, Part 2. Packt Publishing, 2021.

آزمایش ۷ : فرآیندها

اهداف:

- درک مفهوم فرآیندهای والد و فرزند
- ایجاد، نظارت و مدیریت فرآیندها
- ایجاد ارتباط بین فرآیندها

ابزار و مفاهیم مورد نیاز:

- ابزارهای کامپایل خط فرمانی یا IDE
- کتابخانه‌های ضروری

محتوا:

- مشاهده مشخصات فرآیندها
- فرآیند والد و فرزند
- ایجاد و جایگزینی کد فرآیندهای فرزند
- ارتباط بین فرآیندی
  - پایپ
  - حافظه مشترک
  - سوکت

۷-۱- فرآیند چیست؟

فرآیند[1] عبارت است از یک برنامه در حال اجرا با فضای آدرس و منابع مخصوص به خود. هر فرآیند یک شناسه[2] منحصربه‌فرد دارد. هرگاه یک فرآیند توسط فرآیند دیگری ایجاد شده باشد، فرآیند ایجادشده را فرزند[3] و فرآیند ایجادکننده را والد می‌نامیم. فرآیند فرزند علاوه بر شناسه اختصاصی خود، شناسه فرآیند والد[4] را نیز در لیست مشخصاتش خواهد داشت.

وقتی لینوکس بوت می‌شود، اولین فرآیند به نام systemd و با شناسه ۱ توسط هسته ایجاد می‌گردد. در سیستم‌های قدیمی این فرآیند init نام داشت و در توزیع‌های خاص نیز با نام‌های متفاوتی شناخته می‌شود. فرآیند مذکور، والد تمام فرآیندهای دیگر در سیستم می‌باشد. برای دیدن اولین فرآیند در توزیع خود، دستور زیر را اجرا نمایید.

```
ps   -p   1
```

از جمله مهمترین فرآیندهایی که از اولین فرآیند منشعب می‌شوند، می‌توان به این موارد اشاره کرد:

- systemd-logind : برای مدیریت ورود کاربران به سیستم بوده و والد دو فرآیند دیگر با نام‌های login shells برای مدیریت ورود در پوسته‌ها و user sessions برای مدیریت نشست‌های کاربران، می‌باشد.
- فرآیند getty@tty1 برای مدیریت ترمینال مجازی است و امکان ورود کاربران به سیستم را فراهم می‌آورد. این فرآیند وظیفه نمایش اعلان ورود[5] را دارد. سپس یک فرآیند فرزند به نام Login ایجاد می‌کند تا هویت کاربر را احراز نماید. این فرآیند نیز به‌نوبه خود یک فرزند دیگر ایجاد می‌کند که همان پوسته پیش‌فرض سیستم است و محیط خط فرمان را در اختیار کاربر قرار می‌دهد.

---

[1] Process
[2] Process ID (PID)
[3] Child Process
[4] Parent Process ID (PPID)
[5] Login Prompt

۷-۲- مشاهده فرآیندها

برای مشاهده تمام فرآیندهای جاری سیستم، دستورزیر را اجرا نمایید.

```
ps   -aux
```

نکته: در خروجی این دستور ttyها را نیز می‌توانید مشاهده کنید. البته برای برخی فرآیندها عبارت pts را می‌بینید که مربوط به شبه‌ترمینال‌هایی است که برای اتصالات شبکه‌ای، ورود از راه دور و ترمینال‌های شبیه‌سازی شده، می‌باشد. در حالیکه ttyها مربوط به کنسول‌های مستقیم و ترمینال‌های کلاسیک فیزیکی یا مجازی سیستم هستند.

همان‌طور که در دستورکار خط فرمان دیدیم، برای مشاهده تعاملی فرآیندها می‌توانید از دستور top یا htop نیز استفاده نمایید. دستور pstree امکان مشاهده ساختار درختی فرآیندها را فراهم می‌آورد. برای به دست آوردن شناسه یک فرآیند براساس نامش، می‌توانید دستور زیر را اجرا کنید.

```
pgrep     نام فرآیند موردنظر
```

۷-۳- ایجاد و مدیریت فرآیندها

در ادامه برنامه‌هایی با محوریت فرآیندها و به زبان C خواهیم نوشت. برای شروع، یک فایل متنی ساده با نام proc1.c ایجاد کرده و کد زیر را در آن بنویسید. این برنامه یک فرآیند ایجاد می‌کند. سپس با استفاده از یک فراخوان سیستمی، شناسه‌ی فرآیند ایجاد شده و والدش را استخراج کرده و نمایش می‌دهد.

```
// Create_Proc.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
      pid_t pid, ppid;
      pid = getpid();
      ppid = getppid();
      printf("Current Process PID: %d, PPID: %d\n", pid, ppid);
      return 0;   }
```

کامپایل و اجرای کد

```
gcc    Create_Proc.c   -o    p1
```

اجرا و دیدن نتایج

```
./p1
```

۷-۴- ایجاد فرآیند فرزند

در مثال دوم، با استفاده از تابع ()fork یک فرآیند فرزند ایجاد می‌کنیم. این تابع به فرآیند فرزند، عدد صفر و به فرآیند والد، شناسه‌ی فرآیند فرزند را برمی‌گرداند. اما در صورت بروز خطا یک عدد منفی برخواهد گرداند.

```
//Create_Child_Proc.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork ();

    if (pid == 0)  {
        //  کد فرآیند فرزند در این قسمت قرار می‌گیرد
        printf("Child Process PID: %d, PPID: %d\n", getpid(),
getppid());}
    else {
        //  کد فرآیند والد در این قسمت قرار می‌گیرد
        printf("Parent Process PID: %d, Child Process PID: %d \n",
getpid(), pid);

        //والد تازمانی‌که فرآیند فرزند خاتمه یابد، منتظر می‌ماند
        wait(NULL); }

    return 0;   }
```

مراحل کامپایل و اجرا را همانند مثال اول انجام دهید.

جایگزینی کد فرزند

در این مثال، فرآیند فرزند با استفاده از ()execlp، کدی را که از فرآیند والد به ارث برده، با کد جدید دلخواه خود - در اینجا دستور ls - جایگزین خواهد کرد. بدین‌منظور از ()execve نیز می‌توان استفاده کرد، اما به‌دلیل ساختار ساده‌تر از تابع اول استفاده شده است.

```
// Child_Code_Replacement.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork ();

    if (pid == 0) {
        // فرآیند فرزند
        printf("Child process will run ls command \n");
        execlp("ls", "ls", "-l", NULL);
        return 1; }
    else {
        // فرآیند والد
        printf("Parent Process: waiting for child termination\n");
        wait(NULL);
        printf("Child Process Has terminated\n");  }

    return 0;   }
```

قاعده کلی تابع ()execlp به‌صورت زیر می‌باشد.

```
execlp(program, arg0, arg1, arg2, ..., NULL);
```

پارامتر اول که با عبارت program مشخص شده است، نام برنامه مدنظر برای جایگزینی با برنامه والد می‌باشد. این نام، در مسیرهای متعارف سیستم (PATH)، مورد جستجو قرار می‌گیرد و اگر پیدا شد، جایگزین برنامه والد می‌شود.

پارامتر دوم(arg0)، نامی است که ما به برنامه فرزند اختصاص می‌دهیم. این نام می‌تواند با نام اصلی برنامه یکسان یا با آن متفاوت باشد. به‌عنوان مثال، هر دو تابع زیر یک کار یکسان را انجام می‌دهند. تنها تفاوتشان در این است که در اولی، برنامه نام خودش را ls می‌داند ولی در دومی، برنامه خود را با نام myprog می‌شناسد.

```
execlp("ls", "ls", "-l", NULL);
execlp("ls", "myprog", "-l", NULL);
```

برای مشاهده تفاوت دو حالت بالا؛ در هر دو تابع، آرگومان سوم را با عبارت "--invalid-option" که یک سوئیچ فرضی و اشتباه است، جایگزین نمایید تا برنامه خطا بدهد. پیام خطا در حالت اول، برنامه را با نام ls و در حالت دوم با نام myprog معرفی خواهد کرد.

پارمتر سوم تا ماقبل آخر، آرگومان‌های ارسالی به برنامه مقصد هستند. در مثال زیر دستور ls همراه با سوئیچ های l، a و h در متن برنامه جایگزین و اجرا خواهد شد.

```
execlp("ls", "ls", "-l", "-a", "-h", NULL);
```

پارمتر آخر(NULL)، پایان آرگومان‌ها را مشخص می‌کند و اگر ذکر نشود، برنامه از کار می‌افتد.

۷-۵- ارتباط بین فرآیندی

مفهوم ارتباط بین فرآیندی[6] (IPC) در سیستم‌های عامل مدرن، به دلیل چندوظیفه‌ای بودن و در نتیجه نیاز فرآیندهای مستقل به تبادل اطلاعات و هماهنگی، شکل گرفته است. ارتباط بین فرآیندها معمولا به دلایل مختلفی مانند همگام‌سازی، انتقال داده یا کنترل منابع مشترک، صورت می‌گیرد. مکانیزم‌های مختلف IPC عبارتند از: حافظه مشترک، پایپ، سوکت، صف پیام و ... .

در روش حافظه مشترک، فرآیندها بخشی از فضای آدرس خود را به اشتراک می‌گذارند. این روش بسیار کارآمد و سریع است اما نیاز به همگام‌سازی دقیق دارد. در مقابل، روش‌های مبتنی بر تبادل پیام مانند پایپ، سوکت یا صف پیام به دلیل کپی کردن پیام‌ها، کندترند اما مدیریت ساده‌تری داشته و ایمن‌تر می‌باشند.

---

[6] Inter-Process Communication

۷-۵-۱- پایپ

یکی از ساده‌ترین و قدیمی‌ترین روش‌های ارتباط بین فرآیندی، پایپ یا شبه‌فایل است. پایپ از طریق ایجاد یک کانال ارتباطی یک‌طرفه، دو فرآیند را به یکدیگر متصل می‌نماید. به این ترتیب، خروجی یک فرآیند به‌عنوان ورودی فرآیند دیگر استفاده می‌شود. پایپ‌ها در دو نوع بی‌نام[۷] و بانام[۸] وجود دارند.

پایپ‌های بی‌نام فقط بین فرآیندهای مرتبط (مانند فرآیند والد و فرزند) کاربرد دارند و پس از پایان فرآیندها از بین می‌روند. این نوع پایپ از طریق وراثت در دسترس فرآیند فرزند قرار می‌گیرد. در مقابل، پایپ‌های بانام که از مکانیزم FIFO برای نوشتن و خواندن داده‌ها استفاده می‌کنند، به‌صورت یک فایل در فایل‌سیستم ظاهر شده و به فرآیندهای غیرمرتبط اجازه ارتباط می‌دهند. این پایپ‌ها تا زمان حذف از فایل‌سیستم، پایدار می‌مانند.

چالش و محدودیت اصلی در این روش، یک‌طرفه بودن ارتباط است. به‌علاوه، اگر از پایپ بی‌نام استفاده کنیم فقط به فرآیندهای مرتبط با آن پایپ محدود خواهیم بود. با این وجود، پایپ‌ها در پردازش‌های خط فرمانی و برای اتصال و انتقال خروجی یک دستور به ورودی دستور دیگر، بسیار مفید و پرکاربرد هستند.

در مثال زیر، با استفاده از شبه‌فایل پایپ بین فرآیندهای والد و فرزند ارتباط برقرار می‌کنیم. از طرف فرآیند والد پیامی به فرآیند فرزند ارسال می‌شود. فرآیند فرزند با استفاده از ()read که یک فراخوانی مسدود کننده[۹] است، منتظر می‌ماند تا داده‌ای برای خواندن وجود داشته باشد، سپس آن را در خروجی چاپ می‌نماید.

```
// Pipe_IPC.c

ارتباط بین فرآیندی با استفاده از پایپ //
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int fd[2];
    char message[] = "Hello from Parent!\n";
    char buffer[100];
    pipe(fd);        //Create pipe

    pid_t pid = fork ();
```

---

[7] Unnamed
[8] Named
[9] Blocking Call

```
    if (pid == 0) {
        // فرآیند فرزندError! Bookmark not defined.
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer)); //Read from pipe
        printf("Child Process received this message:%s\n", buffer);
        close(fd[0]); } // Close the Read End
    else {
        // فرآیند والد
        close(fd[0]);
        write(fd[1], message, strlen(message)+1); //write in pipe
        close(fd[1]); // Close the Write End
        wait(NULL);    }

    return 0;   }
```

۷-۵-۲- حافظه مشترک

در این روش دو یا چند فرآیند، بخشی از حافظه را با یکدیگر به اشتراک می‌گذارند. این فرآیندها می‌توانند داده‌ها را مستقیما در این ناحیه بنویسند یا از آن بخوانند. البته برای جلوگیری از شرایط رقابتی[۱۰]، لازم است که بین فرآیندها یک سری همگام‌سازی‌ها صورت بگیرد. ازآنجایی‌که در این روش نیازی به کپی‌کردن داده‌ها وجود ندارد، به‌عنوان سریع‌ترین شکل IPC شناخته می‌شود.

در مثال زیر، دو فرآیند با نام‌های تولیدکننده و مصرف‌کننده داریم که از طریق حافظه مشترک با یکدیگر ارتباط برقرار می‌کنند. ابتدا فرآیند تولیدکننده را ایجاد می‌کنیم. این فرآیند، داده را از کاربر گرفته و در یک حافظه مشترک قرار می‌دهد. بدین‌منظور ابتدا یک ناحیه حافظه مشترک در سیستم تعریف شده و سپس یک رشته متنی از کاربر دریافت و در این حافظه مشترک نوشته می‌شود. سپس، فرآیند تولیدکننده منتظر خواهد ماند تازمانی‌که فرآیند مصرف‌کننده داده را بخواند و با قرار دادن علامت ستاره در حافظه، پایان کار را اعلام کند. در انتها، حافظه مشترک از سیستم حذف می‌شود.

```
// Shared_Memory_IPC_Producer.c

کد فرآیندتولیدکننده برای ارتباط بین فرآیندی با استفاده از حافظه مشترک //
```

---

[10] Race Condition

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#define SHM_SIZE 1024  // Size of shared memory segment

int main () {

// ایجاد فایلی که در تولید کلید مورد استفاده قرار خواهد گرفت
FILE *f = fopen("shmfile", "w");
if (f) fclose(f);

// ایجاد کلید یکتا برای حافظه مشترک
  key_t key = ftok("shmfile", 65);

// ایجاد سگمنت حافظه مشترک
  int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

// الحاق حافظه مشترک به فضای آدرس فرآیند
  char *str = (char*) shmat(shmid, NULL, 0);

// اطلاعات ورودی را از کاربر دریافت کرده و در حافظه مشترک می‌نویسیم
  printf("Producer Process; Enter data: ");
  fgets(str, SHM_SIZE, stdin);

  printf("Data is written to shared memory: %s\n", str);

// منتظر می‌مانیم تا مشتری داده را بخواند
// Consumer will signal by writing '*' to first byte
  printf("Waiting for consumer to read data...\n");
  while (*str != '*') {
    sleep(1);  // Sleep for 1 second  }

// Detach from shared memory
  shmdt(str);

// Mark shared memory segment for destruction
  shmctl(shmid, IPC_RMID, NULL);

  return 0; }
```

تابع ftok() یک کلید منحصربه‌فرد تولید می‌کند که از آن برای شناسایی حافظه مشترک بین دو فرآیند استفاده می‌شود. آرگومان اول، نام فایلی است که در دستور قبلی ایجاد کردیم و سیستم‌عامل از اطلاعات این فایل - مثلا شماره iNode آن - برای تولید کلید استفاده می‌کند. آرگومان دوم یک عدد (معمولا بین ۱ تا ۲۵۵) است که به‌عنوان شناسه پروژه شناخته شده و آن هم در تولید کلید یکتا مورد استفاده قرار می‌گیرد. ضروری است که این فایل و شماره، در هر دو فرآیند متقاضی برقراری ارتباط، یکسان انتخاب شوند تا کلید یکسانی برای هر دوی آن‌ها تولید گردد.

در بخش بعدی، ایجاد یا اتصال به حافظه مشترک توسط تابع shmget() به انجام می‌رسد. این اتصال بر اساس کلید یکتا، اندازه تعیین شده برای حافظه و مجوز خواندن و نوشتن برای تمامی کاربران انجام می‌شود. حاصل اجرای این تابع، شناسه عددی حافظه مشترک است که برای عملیات بعدی در متغیر shmid ذخیره می‌گردد.

حال باید حافظه اشتراکی ایجاد شده را به فضای آدرس فرآیند متصل کنیم. این کار توسط تابع shmat() و با استفاده از شناسه حافظه، یک آدرس دلخواه (NULL) توسط سیستم و بدون هیچ‌گونه پرچم اضافه، در کد بالا انجام شده است. مقدار بازگشتی از این تابع عبارت است از اشاره‌گری از نوع void که به ابتدای حافظه مشترک، در فضای آدرس فرآیند اشاره می‌کند. این مقدار را پس از تبدیل نوع به اشاره‌گر کاراکتری، در یک متغیر اشاره‌گر ذخیره کرده و در ادامه مورد استفاده قرار می‌دهیم. پس از تبدیل نوع، حافظه مشترک همچون یک آرایه کاراکتری برای کامپایلر به نظر می‌رسد و کار کردن با آن آسان‌تر خواهد بود.

در بخش دوم، فرآیند مصرف‌کننده را ایجاد می‌کنیم که باید داده را از حافظه مشترک خوانده و پردازش نماید. مصرف‌کننده ابتدا باید به ناحیه حافظه مشترک که توسط فرآیند تولیدکننده ایجاد شده بود، متصل شود. سپس منتظر بماند تا داده‌ای در حافظه قرار بگیرد و هرگاه داده موجود شد، آن را از حافظه مشترک خوانده و نمایش دهد. در پایان با قرار دادن علامت ستاره در حافظه، به فرآیند تولیدکننده سیگنال می‌دهد که کارش پایان یافته است. کد را به‌صورت زیر می‌نویسیم.

```
// Shared_Memory_IPC_Consumer.c
```

```
// کد فرآیند مشتری برای ارتباط بین فرآیندی با استفاده از حافظه مشترک
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/ipc.h>

#define SHM_SIZE 1024  // Size of shared memory segment

int main() {
    // ایجاد کلید یکسان با تولیدکننده
    key_t key = ftok("shmfile", 65);

    // دریافت سگمنت حافظه مشترک که قبلا در سرور ایجاد شده بود
    int shmid = shmget(key, SHM_SIZE, 0666);

    // الحاق حافظه مشترک به فضای آدرس فرآیند
    char *str = (char*) shmat(shmid, NULL, 0);

    // Wait for producer to write data
    // Check if first byte contains data (not null)
    printf("Consumer Process; Waiting for data...\n");
    while (*str == 0) {
        sleep(1);  // Sleep for 1 second
    }

    // خواندن و نمایش داده از حافظه مشترک
    printf("Data read from shared memory: %s\n", str);

    // Signal producer by '*' that writing is finished
    *str = '*';

    // Detach from shared memory
    shmdt(str);
    return 0; }
```

در انتها، برای کامپایل کدها به‌صورت زیر عمل می‌کنیم:

```
gcc   Shared_Memory_IPC_Producer.c   -o   producer
gcc   Shared_Memory_IPC_Consumer.c   -o   consumer
```

پس از کامپایل، دو ترمینال باز کنید و هر یک از فرآیندها را در یکی از ترمینال‌ها به شکل زیر اجرا نمایید. در ترمینال اول:

```
./producer
```

و در ترمینال دیگر:

```
./consumer
```

۷-۵-۳- سوکت

سوکت یک مکانیزم IPC مبتنی بر شبکه است که از مدل کلاینت سروری پیروی می‌کند. این روش برای ایجاد ارتباط بین فرآیندهای روی یک ماشین یا ماشین‌های مختلف مورد استفاده قرار می‌گیرد. و از پروتکل‌های مختلف مانند TCP یا UDP پشتیبانی می‌نماید. در ادامه این بخش، دو فرآیند به نام‌های سرور و کلاینت ایجاد خواهیم کرد که از طریق ایجاد سوکت با یکدیگر به مبادله پیام می‌پردازند.

ابتدا به فرآیند سرور می‌پردازیم که یک سوکت محلی از نوع UNIX Domain ایجاد کرده و آن را به یک آدرس محلی متصل می‌کند. این آدرس محلی مقدار ثابتی است که در فایل سرآیند sys/socket.h تعریف شده است و با کمک آن می‌توانیم بین فرآیندهایی که روی یک ماشین قرار دارند ارتباط برقرار نماییم. برای ارتباط بین سوکت‌های شبکه‌ای می‌توانیم سوکت را از خانواده AF_INET تعریف نماییم.

این سوکت سرور، به اتصالات کلاینت‌ها گوش فرا داده و منتظر درخواست اتصال از کلاینت می‌ماند. وقتی کلاینت متصل شد، پیام ارسالی از طرف کلاینت را دریافت کرده و نمایش می‌دهد. سپس یک پاسخ به کلاینت ارسال کرده و در نهایت ارتباط را می‌بندد.

```
// Socket_IPC_Server.c

//کد فرآیند سرور برای ارتباط بین فرآیندی با استفاده از سوکت
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
```

```
#include <sys/un.h>

#define SOCKET_PATH "/tmp/my_socket"  // Path for UNIX domain socket

int main () {
  //تعریف توصیف‌گر فایل برای سوکت کلاینت و سرور
  int server_fd, client_fd;

  // تعریف ساختارهای آدرس
  struct sockaddr_un server_addr, client_addr;
  //Define client Address size
  socklen_t client_len = sizeof(client_addr);
  char buffer[256];   // buffer to put data in

   // ایجاد توصیف‌گر فایل سوکت
   // AF_UNIX: Unix domain socket, SOCK_STREAM: TCP-like reliable
socket
  server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
  if (server_fd == -1) {
    perror("Socket creation failed");
    exit(1); }

   // تنظیمات ساختار آدرس سرور
  memset(&server_addr, 0, sizeof(server_addr));
  server_addr.sun_family = AF_UNIX;
  // server socket address = your socket address
  strcpy(server_addr.sun_path, SOCKET_PATH);

   // حذف فایل سوکتی که از اجراهای قبلی باقیمانده است (در صورت وجود)
  unlink(SOCKET_PATH);

   // Bind socket to the specified address
  if (bind(server_fd, (struct sockaddr*)&server_addr,
sizeof(server_addr)) == -1) {
    perror("Socket bind failed");
    exit(1); }

   // رفتن به حالت انتظار برای درخواست‌های اتصال
   // Backlog queue size: 5
  if (listen(server_fd, 5) == -1) {
    perror("Socket listen failed");
    exit (1); }
  printf("Server: Listening for connections.\n");

   // پذیرش اتصال ورودی
```

```
  client_fd = accept(server_fd, (struct sockaddr*)&client_addr,
&client_len);
  if (client_fd == -1) {
    perror("Socket accept failed");
    exit(1); }

  printf("Server: Client connected.\n");

    // خواندن داده از کلاینت
  int n = read(client_fd, buffer, sizeof(buffer)-1);
  if (n > 0) {
    buffer[n] = '\0';  // Null-terminate the string
    printf("Server: Received: %s\n", buffer);

     // Send response to client
    char response[256];
    printf("Enter server's response: ");
    fgets(response, sizeof(response), stdin);
    write(client_fd, response, strlen(response)); }

    // قطع اتصالات ایجاد شده و پاک‌سازی
  close(client_fd);
  close(server_fd);
  unlink(SOCKET_PATH);  // Remove socket file

  return 0; }
```

ایجاد سوکت با استفاده از تابع socket() انجام می‌گیرد. همان‌طور که اشاره شد در این مثال، سوکت از نوع محلی است و از پروتکل TCP استفاده خواهد کرد. سایر آرگومان‌ها را با مقادیر پیش‌فرض (۰) رها کرده‌ایم. پس از دریافت آدرس سرور، آن را در فایلی که در ابتدای کد با عنوان my_socket ایجاد کرده بودیم، ذخیره می‌کنیم و سپس با تابع bind() سوکت را به آدرس مشخص شده متصل می‌نماییم.

حال سرور به سوکت گوش فرا می‌دهد تا در صورت رسیدن درخواست اتصال، متوجه بشود. در این مثال، حداکثر امکان حضور ۵ اتصال در صف در نظر گرفته شده است. هرگاه درخواستی به سرور برسد، با تابع accept() آن را پذیرفته و یک توصیفگرفایل جدید برای آن کلاینت ایجاد می‌نماید. سپس با کمک read() اطلاعات دریافتی را خوانده و در متغییر buffer می‌ریزد. حال با تابع write() پاسخی برای کلاینت می‌نویسد. در نهایت، سوکت را بسته و حذف می‌نماید.

در فرآیند کلاینت نیز یک سوکت محلی ایجاد کرده و به همان آدرس سرور محلی متصل می‌کنیم. پس از برقراری ارتباط موفق، یک پیام به سرور ارسال کرده و منتظر پاسخ سرور می‌مانیم. پس از دریافت پیام، آن را نمایش می‌دهیم و در پایان، ارتباط با سرور قطع می‌شود.

```
// Socket_IPC_Client.c

//کد فرآیند کلاینت برای ارتباط بین فرآیندی با استفاده از سوکت
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

//تعریف مسیر سوکت (باید با مسیری که در کد سرور تعریف کردیم، یکی باشد)
#define SOCKET_PATH "/tmp/my_socket"

int main() {
    int sockfd;
    struct sockaddr_un server_addr;
    char buffer]256[;

   // ایجاد توصیف‌گر فایل سوکت
    sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sockfd == -1) {
        perror("Socket creation failed");
        exit(1); }

   // تنظیم ساختار آدرس سرور
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sun_family = AF_UNIX;
    strcpy(server_addr.sun_path, SOCKET_PATH);

   // اتصال به سرور
    if (connect(sockfd, (struct sockaddr*)&server_addr,
sizeof(server_addr)) == -1) {
        perror("Socket connect failed");
        exit(1); }
```

```
    printf("Client: Connected to server\n");

   // ارسال پیام به سرور
    char message[256];
    printf("Enter message for server: ");
    fgets(message, sizeof(message), stdin);
    write(sockfd, message, strlen(message));

   // خواندن پاسخ سرور
    int n = read(sockfd, buffer, sizeof(buffer)-1);
    if (n > 0) {
        buffer[n] = '\0';  // Null-terminate the string
        printf("\nClient: Server response: %s\n", buffer); }

   // قطع اتصال سوکت
    close(sockfd);

    return 0; }
```

در انتها برای کامپایل و اجرای این دو فرآیند، مطابق معمول به شکل زیر عمل کنید:

```
gcc   Socket_IPC_Server.c   -o   server
gcc   Socket_IPC_Client.c   -o   client
```

دو ترمینال باز کرده و هر یک از فرآیندها را در یکی از ترمینال‌ها اجرا نمایید.

```
./server
./client
```

تمرین

- یک فرآیند فرزند ایجاد کرده و از ایجاد شدن آن اطمینان حاصل نماید. یک بخش کنترل خطا به کد والد اضافه نمایید تا در صورت بروز خطا در ایجاد فرآیند فرزند، با یک پیام به کاربر اطلاع دهد.
- برنامه‌ای بنویسید که ابتدا یک فرآیند فرزند ایجاد کرده و سپس کد آن را با دستور grep با شرایط زیر جایگزین نماید.
  - نام شما را به‌عنوان نام برنامه به فرآیند فرزند اختصاص بدهد.
  - دستور grep با حداقل ۳ سوئیچ رایج اجرا شود و کلمه error را در فایل /var/log/syslog/ جستجو نماید. (ممکن است فایل لاگ توزیع شما در مسیر متفاوتی قرار داشته باشد.)
- در تمرین قبل، یک بخش کنترل خطا به کد والد اضافه نمایید تا در صورت بروز خطا در اجرای فرآیند فرزند، والد از طریق wait و کد خروجی فرزند، از این موضوع مطلع گردد. برای ایجاد خطا می‌توانید در هنگام جایگزینی کد فرزند، نام دستور جایگزین را همراه با اشتباه املایی وارد کرده و یا از سوئیچ‌های نامعتبر به‌عنوان آرگومان استفاده کنید. همچنین می‌توانید یک مسیر نامعتبر برای فایل لاگ در نظر بگیرید. کد خود را در هر دو حالت «اجرای صحیح فرآیند فرزند» و «خطا در اجرای فرآیند فرزند» اجرا و نتایج را مشاهده کنید.
- یک فرآیند یتیم یا زامبی ایجاد کرده و با دستور top یا ps همراه با سوئیچ‌های مقتضی، وجود آن را بررسی نمایید. در صورت نیاز از تابع sleep استفاده نمایید تا فرصت کافی برای بررسی داشته باشید.
  - فرآیند یتیم[11]: فرآیند والد باید بدون فراخوانی ()wait خاتمه یابد در حالیکه فرآیند فرزند هنوز در حال اجراست.
  - فرآیند زامبی[12]: فرآیند فرزند باید قبل از اینکه فرآیند والد ()wait را فراخوانی کند، خاتمه یابد. به این ترتیب، رکورد PCB مربوط به فرزند در جدول فرآیندها باقی می‌ماند.

---

[11] Orphan Process
[12] Zombie Process

گزارش کار

- یک سیستم فرضی پردازش تراکنش را شبیه‌سازی نمایید که تراکنش‌ها را زمان‌بندی و بر اساس اولویتشان مدیریت نماید. این سیستم شامل دو نوع تراکنش است: تراکنش‌های عادی و تراکنش‌های فوری که اولویت بالاتری نسبت به تراکنش‌های عادی دارند. برای پیاده‌سازی چنین سیستمی، از مکانیزم اولویت‌بندی فرآیندها استفاده کنید. به‌عنوان مثال، دو فرآیند فرزند با دو اولویت مذکور برای فرآیند اصلی ایجاد نمایید. این دو فرآیند باید به ترتیب اولویتشان اجرا شوند.
  - برای تعیین اولویت فرآیندها از فراخوانی سیستمی setpriority استفاده کنید.
  - زمان شروع و پایان هر فرآیند را ثبت کرده و زمان اجرا و ترتیب اتمام آن‌ها را با هم مقایسه کنید.
  - برای مشاهده‌ی اولویت فرآیندها از ستون NI در دستور ps -l و برای مشاهده زمان‌بندی و ترتیب اجرای آن‌ها از دستور top یا هر ابزار مانیتورینگ مناسب دیگر استفاده نمایید.

- در کد تولیدکننده-مصرف‌کننده از دستورکار وقفه‌ها، بخش‌های مقتضی را به‌شرح زیر تغییر دهید تا دو برنامه بتوانند از یک داده واقعی مشترک استفاده نمایند:
  - در برنامه تولیدکننده یک حافظه مشترک ایجاد کرده و به جای ذخیره داده در متغیر محلی، آن را در حافظه مشترک بنویسید. سپس با ارسال سیگنال به مصرف‌کننده، اعلام کنید که داده آماده است.
  - در برنامه مصرف‌کننده، به همان حافظه مشترک متصل شده و پس از دریافت سیگنال آمادگی، داده را از حافظه مشترک خوانده و چاپ نمایید. سپس با ارسال سیگنال به تولیدکننده، تایید کنید که پردازش به اتمام رسیده است.
  - در نهایت، هر دو برنامه را به‌صورت همزمان اجرا کرده و صحت انتقال داده از طریق حافظه مشترک را بررسی نمایید.

منابع و مراجع:

- Silberschatz A. *Operating System Concepts*, Wiley,10th Edition, 2018.
- Tanenbaum AS. *Modern Operating Systems*, Pearson Education, 2015.
- Tanenbaum AS. *Operating Systems: Design And Implementation*. Prentice Hall, 2006.

آزمایش ۸ : رشته‌ها

اهداف:

- درک مفاهیم چندرشته‌ای
- برنامه‌نویسی چندرشته‌ای با استفاده از POSIX pthread

ابزار و مفاهیم مورد نیاز:

- ابزارهای کامپایل خط فرمانی یا IDE
- کتابخانه‌های ضروری

محتوا:

- اهمیت چندرشته‌ای
- برنامه‌نویسی چندرشته‌ای همگام
- ایجاد و خاتمه رشته‌ها با استفاده از توابع رایج pthread
- ارسال آرگومان به رشته
- جمع‌آوری نتایج از رشته‌ها
- اشتراک منابع بین رشته‌ها
- رشته‌های تودرتو

۸-۱- چرا چند رشته‌ای؟

در سیستم‌های عامل مدرن، رشته[1] به‌عنوان سبک‌ترین موجودیت قابل برنامه‌ریزی[2] تعریف می‌شود که در یک فرآیند قرار دارد و حافظه و منابع را با سایر رشته‌های درون همان فرآیند، به اشتراک می‌گذارد. هر رشته دارای شمارنده برنامه[3]، ثبات‌ها[4] و پشته مخصوص به خود است، اما منابع دیگری مانند فضای آدرس، فایل‌های باز و سیگنال‌ها را با سایر رشته‌های موجود در فرآیند، به اشتراک می‌گذارد. این اشتراک‌گذاری منابع، ایجاد و مدیریت رشته‌ها را بسیار سبک‌تر از فرآیندها می‌کند و تعویض متن[5] بین رشته‌ها در یک فرآیند، بسیار کم هزینه‌تر از تعویض متن بین فرآیندهاست. برنامه‌نویسی چندرشته‌ای، علاوه بر اشتراک‌گذاری منابع، مزایایی همچون پاسخگویی سریع‌تر، مقیاس‌پذیری، امکان بهره‌گیری از قابلیت‌های چندپردازشی سخت‌افزار و استفاده‌ی بهینه از معماری‌های چندهسته‌ای را فراهم می‌کند.

از سوی دیگر، اشتراک‌گذاری داده بین رشته‌ها، چالش‌های جدیدی مانند شرایط رقابتی[6]، بن‌بست[7] و نیاز به همگام‌سازی را به‌وجود می‌آورد که مدیریت صحیح آن‌ها، یک مهارت کلیدی در برنامه‌نویسی سیستم‌عامل محسوب می‌گردد. در این آزمایش، مفاهیم اولیه ایجاد و مدیریت رشته‌ها را با استفاده از کتابخانه استاندارد لینوکس مطالعه خواهیم کرد.

---

[1] Thread
[2] Schedulable Entity
[3] Program Counter (PC)
[4] Registers
[5] Context Switch
[6] Race Condition
[7] Deadlock

۸-۲- ایجاد و از بین بردن رشته‌ها

اولین مرحله برای درک چگونگی ساخت یک برنامه چند رشته‌ای، آشنایی با نحوه ایجاد و از بین بردن رشته‌ها است. معمولا علاوه بر ایجاد یک رشته، می‌خواهیم پارامترهایی را نیز به آن رشته ارسال کنیم و هنگامی که رشته به پایان رسید، مقادیر بازگشتی از رشته را جمع‌آوری نماییم. این‌ها از جمله مواردی هستند که در این بخش به آن‌ها خواهیم پرداخت.

ابتدا فایل کتابخانه‌ای مورد نیاز برای کار با رشته‌ها را نصب می‌نماییم. کامپایلر را در دستورکار قبل نصب کرده‌ایم.

```
sudo   apt-get   install   libpthread-stubs0-dev
```

سپس یک فایل متنی ساده با نام thread1.c ایجاد کرده و کد زیر را در آن قرار می‌دهیم. در بخش‌های بعدی، هر یک از قسمت‌های این کد توضیح داده شده است.

```
// thread1.c

ایجاد یک رشته درون رشته اصلی برنامه //
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

تابعی که توسط رشته جدید اجرا خواهد شد //
void* thread_function(void* arg) {
printf("Hello from new thread!\n");
return NULL;
}

int main() {

pthread_t thread_ID;

printf("Hello from main thread!\n");

ایجاد رشته جدید //
pthread_create(&thread_ID, NULL, thread_function, NULL);

// The main program continues while the other thread executes.
printf("Main thread is running parallel with the new thread !\n");
```

```
// انتظار برای خاتمه رشته جدید
pthread_join(thread_ID, NULL) ;

// از اینجا به بعد، فقط رشته اصلی برنامه در حال اجرا می‌باشد
printf("Now, only the main thread is running.\n");
return 0;
}
```

۸-۲-۱- ایجاد رشته

برای ایجاد یک رشته جدید، از تابع ()pthread_create استفاده می‌شود که در کتابخانه pthread.h تعریف شده است. اولین پارامتر این تابع، متغیری به نام thread_ID است که از نوع pthread_t تعریف شده و شناسه‌ی رشته‌ی جدید را در خود نگهداری می‌کند. این متغیر در سایر فراخوانی‌ها نیز قابل استفاده می‌باشد.

پارامتر دوم، یک اشاره‌گر به یک Thread Attribute Object است که می‌توان از آن برای تنظیم ویژگی‌های رشته‌ی جدید استفاده کرد. این ویژگی‌ها عبارتند از: اندازه پشته هر رشته، متصل یا منفصل بودن رشته و تعیین اولویت برای رشته‌ها بویژه در برنامه‌نویسی بلادرنگ[8] و ... . در اینجا از اشاره‌گر تهی به‌جای این پارامتر استفاده شده است که به معنای استفاده از ویژگی‌های پیش‌فرض می‌باشد. در اکثر موارد، استفاده از ویژگی‌های پیش‌فرض، کافی و مناسب است. به همین دلیل، در مثال‌های بعدی نیز این پارامتر را با NULL مقداردهی کرده‌ایم.

سومین پارامتر، یک اشاره‌گر به تابعی است که رشته‌ی ایجاد شده، باید آن را اجرا کند. تابع مذکور که thread_function نام دارد، قبل از تابع main، تعریف شده است.

پارامتر آخر، آرگومان ارسال شده به تابع رشته می‌باشد. در این مثال، این پارامتر نیز مقدار NULL دارد، یعنی چیزی به تابع رشته ارسال نشده است. در مثال‌های بعدی، این پارامتر را مقداردهی خواهیم نمود.

---

[8] Real Time

۸-۲-۲- تابع رشته

تابع رشته، با نام thread_function و دقیقا مانند یک تابع معمولی بالاتر از تابع main تعریف شده است. تابع رشته، از نوع اشاره‌گر تهی تعریف شده و هیچ مقداری برنمی‌گرداند، بلکه صرفا یک خط پیام را برای کاربر چاپ می‌کند. به این ترتیب، کاربر متوجه می‌شود که رشته‌ی جدید، ایجاد شده و در حال اجرا است.

برای این تابع، فقط یک پارامتر از نوع اشاره‌گر تهی در نظر گرفته شده است. زیرا این نوع اشاره‌گر را به طور بالقوه می‌توان به هر نوع داده‌ای اختصاص داد، مشروط بر اینکه castهای مناسب اعمال شوند.

اگر عملیاتی که درون این دو رشته انجام می‌پذیرد را سنگین‌تر کنید و سپس کد را چندبار اجرا نمایید، امکان تغییر ترتیب اجرای رشته‌ها وجود دارد. بررسی کنید علت این امر چیست؟

۸-۲-۳- خاتمه رشته و پیوستن به رشته اصلی

پس از ایجاد رشته جدید، هر دو رشته (اصلی و جدید) به‌صورت موازی یا شبه موازی به اجرای خود در پردازنده ادامه خواهند داد. اما در نقطه‌ای از برنامه باید منتظر بمانیم تا رشته‌ی جدید پایان یابد و رشته اصلی با فراخوانی pthread_join() نتیجه‌ای را که رشته جدید تولید کرده، جمع‌آوری نماید. تابع مذکور، دو پارامتر دریافت می‌کند. پارامتر دوم مقدار بازگشتی از تابع رشته است که در مثال بعدی درباره آن صحبت خواهیم کرد. پارامتر اول شناسه‌ی رشته است. اگر چندین رشته داشته باشیم و بخواهیم منتظر پایان و جمع‌آوری نتیجه‌ی تمام آن‌ها بمانیم، باید به ازای هر شناسه، یک‌بار متد pthread_join فراخوانی شود. پس از آن، تنها رشته‌ی در حال اجرا، رشته‌ی اصلی خواهد بود. در مثال بالا وقتی رشته اصلی به این مرحله می‌رسد، یک پیام چاپ می‌نماید.

۸-۲-۴- رشته‌های منفصل

روش دیگر این است که یک رشته منفصل[9] ایجاد کنید. یک رشته منفصل، حالت اجرایی مستقل دارد و نیازی به همگام‌سازی با رشته والد از طریق Join ندارد. نتایج بازگردانده شده توسط چنین رشته‌هایی دور ریخته می‌شوند. یک رشته منفصل پس از اتمام اجرا، منابع سیستمی خود را به طور خودکار و بدون نیاز به فراخوانی صریح ()pthread_join، آزاد می‌کند.

اما مشکل رشته‌های منفصل این است که هرگز از زمان تکمیل شدن آن‌ها مطمئن نیستیم، مگر اینکه ترتیبات خاصی اتخاذ نماییم. معمولا می‌خواهیم قبل از اینکه کل فرآیند - با بازگشت نتیجه به تابع ()main - پایان یابد، مطمئن شویم که تمام رشته‌های جدید، کاملا خاتمه یافته‌اند. زیرا وقتی در تابع ()main خط return اجرا گردد، تمام رشته‌های در حال اجرا، به طور ناگهانی قطع خواهند شد. اگرچه ممکن است این کار در برخی موارد مناسب باشد، اما این خطر را به‌همراه دارد که کارهای مهمی که توسط یک رشته در حال انجام بودند، تکمیل نشده باشند.

در چنین رشته‌هایی، وضعیت رشته به‌صورت خودکار توسط سیستم‌عامل پیگیری می‌شود و هنگامی که رشته خاتمه یافت، بلافاصله منابعش آزاد می‌گردد. بنابراین نیازی به مداخله برنامه‌نویس برای پاک‌سازی وجود نخواهد داشت. رشته والد و فرزند کاملا از هم مجزا هستند و هیچ وابستگی اجرایی ندارند و هر یک به‌صورت مستقل زمان‌بندی می‌شوند. در این حالت، به دلیل استفاده از مدل حافظه غیراشتراکی، امکان بازگشت داده از فرزند به والد از طریق return value وجود نخواهد داشت. و همچنین وضعیت خروج[10] رشته فرزند نیز برای والد قابل دریافت نمی‌باشد.

برای ایجاد چنین رشته‌هایی، یا باید از ابتدا رشته را به‌صورت منفصل ایجاد کرد که از طریق تغییر در attributes رشته قابل انجام است و یا می‌توان پس از ایجاد رشته، آن را با تابع زیر به حالت منفصل درآورد.

```
pthread_detach(thread);
```

نکته: رشته‌ای که منفصل شود دیگر امکان join نخواهد داشت.

---

[9] Detached Thread
[10] Exit Status

رشته‌های منفصل برای مواردی همچون کارهای پس‌زمینه که نیازی به نظارت ندارند، سرویس‌های مستقل، وظایفی که نتیجه‌ای برنمی‌گردانند و یا هندلرهای رویداد در برنامه‌های شبکه‌ای مناسب هستند. اما در مواردی که نیاز به جمع‌آوری نتایج رشته‌ها، همگام‌سازی بین آن‌ها و یا بررسی وضعیت پایان رشته‌ها داریم، قابل استفاده نخواهند بود.

۸-۲-۵- لغو کردن

اگر می‌خواهید یک رشته را قبل از اینکه تابع رشته به طور معمول بازگردد، از بین ببرید می‌توانید از pthread_cancel() استفاده کنید. هر چند که این روش مشکلاتی دارد. مثلا باید مطمئن شوید که رشته قبل از اینکه واقعا بمیرد، منابعی را که به دست آورده بود، آزاد کرده باشد. زیرا اگر حافظه‌ای که دیگر به آن نیازی نیست به سیستم بازگردانده نشود، منجر به کاهش تدریجی حافظه موجود و کند شدن برنامه یا حتی کل سیستم می‌شود. به عبارتی، آزادسازی حافظه پس از خاتمه رشته‌ها، از هدر رفتن منابع جلوگیری می‌کند.

به‌عنوان مثال، اگر به یک رشته مقداری حافظه به‌صورت پویا تخصیص داده باشید ولی قبل از اینکه رشته بتواند حافظه را آزاد کند آن را لغو کنید، برنامه شما دچار نشت حافظه[11] خواهد شد. البته این حالت، متفاوت از زمانی است که یک فرآیند را به‌طور کامل از بین ببرید زیرا معمولا سیستم‌عامل منابع خاصی را که توسط یک فرآیند، معلق مانده‌اند پاک می‌کند. به ویژه، فضای آدرس فرآیند که به‌طور کامل به سیستم بازگردانده می‌شود.

اما سیستم‌عامل این کار را برای رشته‌ها انجام نمی‌دهد، زیرا رشته‌های یک فرآیند، از منابع به‌صورت اشتراکی استفاده می‌کنند. سیستم‌عامل می‌داند که حافظه اختصاص داده شده توسط یک رشته توسط رشته دیگر نیز استفاده خواهد شد. به همین علت، لغو صحیح رشته‌ها دشوار می‌شود.

---

[11] Memory leak

۸-۲-۶- کامپایل و اجرای کد

مطابق روال قبل، به‌منظور کامپایل و اجرای کد به ترتیب دستورات زیر را اجرا نمایید.

```
gcc    thread1.c   -o    thread1
./thread1
```

در مثال بعدی، نشان می‌دهیم که چگونه می‌توان یک عدد صحیح را به‌عنوان آرگومان به تابع رشته فرستاد. البته در عمل ممکن است یک اشاره‌گر به یک ساختار[۱۲] که حاوی چندین آرگومان است، به رشته ارسال شود. به‌علاوه، نحوه برگرداندن نتایج از توابع رشته را خواهیم دید.

۸-۳- ارسال آرگومان به رشته و برگرداندن نتایج از آن

در این مثال متغیری به نام number از نوع عدد صحیح تعریف شده است و پس از دریافت مقدار از کاربر، به‌عنوان آرگومان چهارم به تابع pthread_create ارسال شده تا از این طریق و به‌عنوان آرگومان ورودی، به تابع رشته تحویل داده شود. اما ازآنجایی‌که پارامتر تابع رشته، از نوع اشاره‌گر تهی تعریف شده است، لازم است ابتدا در داخل تابع، یک تبدیل نوع به «اشاره‌گر صحیح» انجام شده و سپس مقدار عددی آن، برای استفاده در تابع استخراج گردد.

```
//thread2.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// تابعی که توسط رشته جدید اجرا خواهد شد
void* thread_function(void* arg) {
int number = *((int*)arg);          // Cast the argument
int* result = malloc(sizeof(int)); // Allocate memory for the result
*result = number * 2;      // Example computation: double the number
return (void*)result; // Return the result as a void pointer
}
```

---

[12] Struct

```
int main() {
pthread_t thread_ID; // Declare a thread identifier
int number;          // The integer to be passed to the thread
int* thread_result;  // Pointer to store the result from the thread

// دریافت یک عدد صحیح از کاربر
printf("Enter an integer: ");
scanf("%d", &number);

// ایجاد یک رشته که تابع 'thread_function' را اجرا خواهد کرد
pthread_create(&thread_ID, NULL, thread_function, &number) ;

// انتظار برای پایان رشته و دریافت نتایج
pthread_join(thread_ID, (void**)&thread_result) ;

// چاپ نتایج
printf("Thread returned value: %d\n", *thread_result);

// آزادسازی حافظه تخصیص‌یافته
free(thread_result);

return 0;   }
```

توجه داشته باشید توابع رشته طوری اعلان شده‌اند که یک اشاره‌گر تهی برگردانند. برخی مشکلات در استفاده از این اشاره‌گر وجود دارد و فقط روی ماشین‌هایی کار می‌کند که درآن‌ها بتوان اعداد صحیح را به یک اشاره‌گر تبدیل کرد و سپس بدون از دست دادن اطلاعات به عدد صحیح برگشت. در برخی از ماشین‌ها، چنین تبدیل‌هایی خطرناک هستند. در واقع این روش در تمام مواردی که فرد تلاش می‌کند یک شی، مانند ساختاری که بزرگتر از یک اشاره‌گر است را برگرداند، شکست خواهد خورد.

در مثال بالا، باید یک عدد صحیح از تابع رشته دریافت کنیم. بدین‌منظور یک اشاره‌گر عدد صحیح به نام thread_result در تابع mian تعریف کرده‌ایم. اما مقدار بازگشتی از تابع رشته، یک اشاره‌گر تهی است. بنابراین از یک متغیر اشاره‌گر به اشاره‌گر استفاده می‌کنیم.

```
pthread_join(thread_ID, (void**)&thread_result) ;
```

thread_result یک اشاره‌گر است و &thread_result، حاوی آدرس مربوط به آن اشاره‌گر می‌باشد. *void اشاره‌گری است که می‌تواند به هر نوع داده‌ای اشاره کند. و با استفاده از **void، می‌توان متغیری از نوع آدرس را به یک «اشاره‌گر به اشاره‌گر» با نوع داده‌ی نامشخص تبدیل کرد.

۸-۴- چند رشته با تابع مشابه

در این بخش، دو رشته ایجاد خواهیم کرد که هر دوی آن‌ها از یک تابع رشته یکسان استفاده می‌کنند؛ اگرچه ممکن است مقادیر آرگومان‌های دریافتی و نتایج بازگشتی برای هر رشته متفاوت باشد.

```
//thread3.c

// ایجاد دو رشته که هر دو یک تابع یکسان را اجرا خواهند کرد
#include <stdio.h>
#include <pthread.h>

// تابعی که توسط رشته‌ها اجرا خواهد شد
void* print_message(void* arg) {
char* message = (char*)arg;
printf("%s\n", message);
return NULL;   }

int main() {
pthread_t thread1, thread2;

// پیامی که توسط رشته‌ها چاپ خواهد شد
char* message1 = "Hello from Thread 1!";
char* message2 = "Hello from Thread 2!";

// ایجاد دو رشته
pthread_create(&thread1, NULL, print_message, (void*)message1);
pthread_create(&thread2, NULL, print_message, (void*)message2);

// انتظار برای پایان رشته‌ها
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

printf("Both threads have finished execution.\n");

return 0;   }
```

برای ایجاد تعداد رشته‌های بیشتر، معمولا از حلقه استفاده می‌کنیم و شناسه‌ها را در یک آرایه قرار می‌دهیم.

۸-۴-۱- بافر ایستا

در مثال زیر، به جای چاپ در خروجی از یک بافر ایستا برای نوشتن پیام رشته‌ها استفاده شده است. بافر ایستا حتی پس از پایان تابع رشته، به وجود خود ادامه خواهد داد و چون از نوع سراسری تعریف شده، توسط تمامی رشته‌ها قابل دسترسی می‌باشد. البته این روش به دلیل استفاده همزمان دو رشته از یک منبع مشترک و بروز شرایط رقابتی دچار مشکل تداخل خواهد شد.

```
//thread4.c

// چند رشته با یک تابع مشترک و بافر ایستا
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>

#define BUFFER_SIZE 100

// تعریف بافر ایستا
static char buffer[BUFFER_SIZE];

// تعریف تابعی که توسط رشته‌ها فراخوانی خواهد شد
void* write_to_buffer(void* arg) {
char* str = (char*)arg; //arg is a ptr to string

for (int i = 0; i < BUFFER_SIZE - 1; ++i) {

//بافر را با کاراکترهای آرگومان رشته‌ای پر می‌کنیم. هر کاراکتر از آرگومان، یک سلول از بافر را اشغال می‌کند.
buffer[i] = str[i % strlen(str)];

// برای شبیه‌سازی کار واقعی، رشته را به مدت ۱۰۰ میلی ثانیه به خواب می‌بریم. به این ترتیب، شانس تعویض متن
افزایش یافته و مشکلات هم‌روندی در چندرشته‌ای نمایان می‌شود.
usleep(100);    }

// آخرین کاراکتر بافر را برابر با NULL قرار می‌دهیم که نشانگر انتهای رشته در بافر است.
buffer[BUFFER_SIZE - 1] = '\0';

printf("Buffer content from thread: %s\n", buffer);
```

```
return NULL;    }

int main() {
pthread_t thread1, thread2;

// ایجاد دو رشته
pthread_create(&thread1, NULL, write_to_buffer, "Thread1");
pthread_create(&thread2, NULL, write_to_buffer, "Thread2");

// انتظار برای پایان رشته‌ها
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

// چاپ محتوای نهایی بافر
printf("\nFinal buffer content: %s\n", buffer);

return 0;    }
```

نکته: از تابع ()usleep برای شبیه سازی زمان اجرای واقعی رشته‌ها و درنتیجه افزایش شانس سوئیچ شدن بین رشته‌ها استفاده کرده‌ایم.

پس از پایان اجرای کد، محتوای چاپ شده را چک کنید. هر رشته در بخشی از بافر نوشته، سپس به خواب رفته یا به علت تعویض متن، نوبت را به رشته دیگر داده و آن‌گاه رشته دوم در بافر نوشته است. مجددا نوبت رشته اول شده و از محلی که نیمه‌کاره مانده بود، نوشتن را ادامه داده است و الی آخر. بدین ترتیب محتوای بافر، تلفیقی تصادفی از محتوای نوشته شده توسط هر دو رشته خواهد بود. علت این است که هر دو رشته از یک منبع مشترک (بافر ایستا) استفاده می‌کنند که منجر به ایجاد شرایط رقابتی می‌شود. برای پیشگیری از این شرایط، باید از روش‌های همگام‌سازی استفاده کرد که در دستورکار سمافور بدان پرداخته شده است.

۸-۴-۲- بافر پویا

این بار، هر رشته با استفاده از تابع Malloc برای خودش بافر پویا[۱۳] تخصیص می‌دهد. در این حالت دیگر محتوای دو رشته با هم تلفیق یا رونویسی نمی‌شوند زیرا دیگر منبع مشترکی وجود ندارد. هرچند دسترسی به این نوع بافر به محض اتمام رشته، از بین می‌رود اما باز هم احتمال نشت حافظه - بویژه اگر تابع اصلی بدون انتظار برای خاتمه رشته‌های درون خود، به پایان برسد - وجود دارد. زیرا همان‌طور که قبلا هم اشاره شد با پایان یافتن تابع main، تمام رشته‌های در حال اجرا ناگهان قطع شده و آنچه که در حال انجامش بودند نیمه‌کاره می‌ماند.

```
//thread5.c

// چند رشته با یک تابع مشترک و تخصیص حافظه پویا
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>
#define BUFFER_SIZE 100

//  تابعی که توسط رشته‌ها اجرا خواهد شد
void* write_to_buffer(void* arg) {
char* str = (char*)arg;
char* buffer = (char*)malloc(BUFFER_SIZE * sizeof(char));
if (buffer == NULL) {
perror("Failed to allocate memory");
return NULL;   }

for (int i = 0; i < BUFFER_SIZE - 1; ++i) {
buffer[i] = str[i % strlen(str)];

//  برای شبیه‌سازی کار واقعی و افزایش احتمال تعویض متن، رشته را به حالت خواب می‌بریم.
usleep(100);   }

buffer[BUFFER_SIZE - 1] = '\0';
printf("Buffer content from thread %s: %s\n", str, buffer);

free(buffer);
return NULL;   }

int main() {
```

---

[13] Dynamic Buffer

```
pthread_t thread1, thread2;

// ایجاد دو رشته
pthread_create(&thread1, NULL, write_to_buffer, "Thread1");
pthread_create(&thread2, NULL, write_to_buffer, "Thread2");

// انتظار برای پایان هر دو رشته
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

return 0;   }
```

۸-۵- رشته‌های تودرتو[۱۴]

در این مثال، یک رشته داخلی دیگر توسط تابع رشته اول ایجاد می‌شود.

```
//thread6.c

//nested threads: one thread creates another thread
#include <stdio.h>
#include <pthread.h>

// تابعی که توسط رشته داخلی اجرا خواهد شد
void* nested_thread_function(void* arg) {
printf("Hello from the nested thread!\n");
return NULL;   }

// تابعی که توسط رشته اول اجرا خواهد شد
void* main_thread_function(void* arg) {
pthread_t nested_thread;

// ایجاد رشته داخلی
pthread_create(&nested_thread, NULL, nested_thread_function, NULL);

// انتظار برای پایان رشته داخلی
pthread_join(nested_thread, NULL);

printf("Nested thread has finished execution.\n");
return NULL;   }

int main() {
pthread_t main_thread;
```

---

[14] Nested Threads

```
// ایجاد رشته اول
pthread_create(&main_thread, NULL, main_thread_function, NULL);

// انتظار برای پایان رشته اول
pthread_join(main_thread, NULL);

printf("Main thread has finished execution.\n");
return 0;   }
```

تمرین

برنامه‌ای به زبان C بنویسید که با استفاده از یک حلقه، ۱۰ رشته ایجاد کند. از رشته‌ها بخواهید که یک تابع یکسان را اجرا کنند ولی به هر رشته یک عدد منحصربه‌فرد به‌عنوان شماره رشته، از طریق آرگومان ارسال کنید. از آرایه‌ای از اشیاء pthread_t برای نگهداری شناسه‌های رشته‌های مختلف استفاده کنید. مطمئن شوید برنامه تازمانی‌که تمام رشته‌ها کامل نشده‌اند، خاتمه نمی‌یابد. در نهایت، هر رشته باید **سه بار** پیام زیر را چاپ نماید که در آن، x با **شماره رشته** جایگزین می‌گردد.

```
Hello from thread x
```

گزارش کار

برنامه قبلی را به این صورت گسترش دهید که ابتدا یک بافر ایستا به‌صورت سراسری و با اندازه ۱۰۰ بایت تعریف شده و به‌عنوان منبع مشترک بین تمام رشته‌ها مورد استفاده قرار بگیرد. تابع هر رشته، بافر ایستا مشترک را با عدد منحصربه‌فرد خود پر کرده و سپس، شماره خود و در ادامه، محتوای بافر را چاپ خواهد کرد. پیام چاپ شده توسط هر رشته، باید قالبی مشابه پیام زیر داشته باشد که در آن x با شماره رشته و s با محتوای بافر جایگزین می‌گردد.

```
Thread x. Content of shared buffer: s
```

در نهایت، محتوای چاپ شده را بررسی کرده و گزارش نمایید که آیا هر رشته محتوای اختصاصی خود را دارد یا نه؟ چرا خروجی در اجراهای مختلف متفاوت است و این تفاوت ناشی از چه پدیده‌ای می‌باشد؟

منابع و مراجع:

- Silberschatz A. *Operating System Concepts*, Wiley,10th Edition, 2018.
- Tanenbaum AS. *Modern Operating Systems*, Pearson Education, 2015.
- Love R. *Linux System Programming: Talking Directly to The Kernel And C Library*, O'Reilly Media, 2013.

آزمایش ۹ : سمافور

اهداف:

- درک مفاهیم رقابت، همگام‌سازی و سمافور

ابزار و مفاهیم مورد نیاز:

- ابزارهای خط فرمانی کامپایل
- کد برنامه‌های چندرشته‌ای از آزمایش قبل

محتوا:

- انواع سمافور
  - شمارشی باینری
  - شمارشی عددصحیح
- تعریف، مقداردهی اولیه و پاکسازی سمافور
- مسئله تولید کننده-مصرف کننده

۹-۱- ساختار سمافور

سمافور یک مکانیزم همگام‌سازی[1] است که برای کنترل دسترسی به منابع مشترک در سیستم‌های همزمان مانند سیستم‌های عامل چندوظیفه‌ای مورد استفاده قرار می‌گیرد. سمافورها برای اجتناب از شرایط رقابتی و کسب اطمینان از عدم دسترسی همزمان چندین فرآیند یا رشته به بخش‌های بحرانی[2] کدها استفاده می‌شوند. ساختار داده سمافور شامل یک صف برای نگهداری فرایندها یا رشته‌هایی که روی آن سمافور به انتظار نشسته‌اند و یک شمارنده برای نگهداری سیگنال‌های wake up می‌باشد که هر گاه صفر شود، فرآیند به حالت خواب می‌رود.

عملیات قابل دسترسی برای کاربر بر روی یک سمافور عبارتند از:

- انتظار (P / Wait / Call Down): مقدار شمارنده سمافور را کاهش می‌دهد. اما اگر مقدار از قبل صفر باشد، فرآیند تازمانی‌که مقدار سمافور بزرگتر از صفر شود، به صف انتظار رفته و مسدود می‌گردد.
- سیگنال (V / Post / Signal / Call Up): مقدار شمارنده سمافور را افزایش می‌دهد. اما اگر فرآیندهای مسدود شده‌ای در صف وجود داشته باشند، یکی از آنها رفع انسداد می‌شود.

متد wait، تلاش می‌کند شمارنده را کاهش دهد در حالیکه متد post، در تلاش برای افزایش شمارنده است. چیزی که سمافورها را خاص می‌کند این است که اگر یک رشته سعی کند روی سمافور صفر منتظر بماند، مسدود می‌شود. بعدا وقتی رشته دیگری سمافور را پست کند، رشته مسدود شده فعال می‌شود و مقدار سمافور روی صفر باقی می‌ماند. در واقع، post باعث می‌شود که شمارنده سمافور افزایش یابد و درنتیجه، رشته مسدود شده‌ای که در تلاش برای انجام wait بود، اجازه می‌یابد تا ادامه پیدا کند و این‌کار باعث می‌شود که شمارنده سمافور بلافاصله دوباره به صفر کاهش یابد؛ به‌طوری‌که گویی از ابتدا در همان مقدار صفر باقی مانده بود.

---

[1] Synchronization
[2] Critical Section

اگر چندین رشته در صف انتظار یک سمافور مسدود شده باشند، سیستم یکی را برای رفع انسداد انتخاب می‌کند. اینکه چگونه این انتخاب انجام شود، به‌طورکلی به سیستم بستگی دارد. اما در مثال‌های آموزشی معمولا فرض می‌شود که ترتیب دسترسی بر اساس FIFO است. ترتیب مسدود شدن رشته‌ها اغلب موضوع نگران کننده‌ای نیست. اگر در برنامه‌ای این نگرانی وجود داشته باشد، احتمالا به این معنی است که آن برنامه خیلی خوب طراحی نشده است.

۹-۲- انواع سمافور

- **سمافور باینری**: به اسم mutex نیز شناخته می‌شود و تنها می‌تواند دو مقدار ۰ یا ۱داشته باشد. معمولا برای مدیریت دسترسی به یک منبع واحد استفاده می‌گردد.
- **سمافور شمارشی غیرمنفی**: می‌تواند هر مقدار صحیحی از ۰ تا مثبت بی‌نهایت داشته باشد و برای مدیریت دسترسی چندین رشته به یک مخزن از منابع، استفاده می‌شود.
- **سمافور شمارشی عددصحیح**: می‌تواند مقدار منفی نیز بگیرد. در این حالت، مقدار منفی بیانگر تعداد رشته‌های منتظر در صف سمافور خواهد بود.

هنگامی که یک رشته بخواهد وارد ناحیه بحرانی خود شود و به یک ساختار داده مشترک دسترسی پیدا کند، یک عملیات انتظار بر روی سمافور انجام می‌دهد. اگر رشته دیگری در ناحیه بحرانی خود نباشد، سمافور همان مقدار اولیه یک را خواهد داشت و بنابراین رشته مدنظر منتظر نخواهد ماند و بلافاصله وارد ناحیه بحرانی می‌گردد و مقدار سمافور صفر می‌شود. در طی این مدت، اگر رشته دیگری سعی کند روی سمافور منتظر بماند، مسدود خواهد شد. هنگامی که رشته اول اجرای ناحیه بحرانی خود را پایان می‌دهد، یک عملیات post روی سمافور به انجام می‌رساند. با این کار، یکی از رشته‌های منتظر در صف، از حالت انسداد خارج می‌شود. ولی اگر رشته‌ی منتظری وجود نداشته باشد، سمافور به مقدار اولیه خود یعنی یک افزایش می‌یابد. سمافور مورد استفاده در این روش، سمافور باینری نامیده می‌شود زیرا دقیقا دو حالت دارد.

اما ازآنجایی‌که سمافورها اعداد صحیح هستند، می‌توانند مقادیر بزرگتر از یک نیز بگیرند. به این ترتیب می‌توان از آن‌ها برای شمارش منابع کمیاب نیز استفاده کرد. به‌عنوان مثال، یک رشته ممکن است روی یک سمافور منتظر بماند تا بتواند یک آیتم از یک منبع را مصرف کند. هرگاه هیچ آیتمی باقی نماند، سمافور صفر می‌شود و با انجام عملیات انتظار، رشته مسدود می‌گردد. سپس هنگامی که یک رشته‌ی دیگر، یک آیتم به همان منبع اضافه کند، متد post را فراخوانی خواهد کرد. به این ترتیب یا تعداد آیتم‌های موجود افزایش می‌یابد یا به یک رشته مسدود، اجازه دسترسی به آیتم موجود داده می‌شود. سمافور مورد استفاده در این روش را سمافور شمارشی غیرمنفی می‌نامند.

نصب کتابخانه های مورد نیاز

```
sudo apt update
sudo apt install build-essential
sudo apt install manpages-dev
sudo apt install gcc
sudo apt install libc-dev
sudo apt install libpthread-stubs0-dev
```

کامپایل و اجرا

در کامپایلرهای هوشمند مانند gcc که مورد استفاده ما است، به روال قبلی دستور زیر برای کامپایل کافی است.

```
gcc  Semaphore_Code.c    -o    Semaphore_Prog
```

اما در برخی کامپایلرها لازم است که کتابخانه های pthread و rt (real-time) به‌صورت دستی در زمان کامپایل لینک شوند.

```
gcc  Semaphore_Code.c   -lpthread    -lrt   -o   Semaphore_Prog
```

اجرا

```
./Semaphore_Prog
```

۹-۲-۱- سمافور باینری

مثال زیر فرم کلی یک برنامه برای تعریف، مقداردهی اولیه، پاک‌سازی و استفاده از سمافور POSIX را نشان می‌دهد. از سمافور ایجاد شده استفاده می‌کنیم تا دسترسی دو رشته را به یک ناحیه بحرانی در کد، کنترل نماییم. هدف این کد، حصول اطمینان از این است که در هر لحظه، فقط یکی از رشته‌ها امکان ورود به آن ناحیه بحرانی را داشته باشد تا بدین ترتیب، از ایجاد شرایط رقابتی پیشگیری شود.

```
// Mutex.c
#include <stdio.h>          // Standard input-output library
#include <pthread.h>        // POSIX thread library
#include <semaphore.h>      // Semaphore library
#include <unistd.h>         // POSIX operating system API

//تعریف یک سمافور باینری با نام میوتکس
sem_t mutex;

// تعریف تابع رشته
void* thread(void* arg) {

// کاهش مقدار سمافور، اگر ۰ بود، رشته مسدود می‌شود
sem_wait(&mutex);

// چاپ پیام ورود رشته به ناحیه بحرانی
printf("\n The thread has entered the critical section \n");

// ۴ ثانیه خواب برای شبیه سازی مدت زمانی که اجرای رشته در واقعیت طول می‌کشد
sleep(4);

// چاپ پیام خروج رشته از ناحیه بحرانی
printf("\nExiting thread\n");

// با افزایش مقدار سمافور به رشته های دیگر اجازه می‌دهیم که وارد ناحیه بحرانی بشوند
sem_post(&mutex);
}

int main() {

// به میوتکس مقدار اولیه ۱ می‌دهیم
sem_init(&mutex, 0, 1);

// تعریف دو متغیر از نوع رشته
pthread_t  t1, t2;
```

```
// ایجاد رشته اول
pthread_create(&t1, NULL, thread, NULL);
sleep(2);
// ایجاد رشته دوم
pthread_create(&t2, NULL, thread, NULL);

// انتظار برای خاتمه یافتن رشته‌ها
pthread_join(t1, NULL);
pthread_join(t2, NULL);

// پاک‌سازی سمافور
sem_destroy(&mutex);

return 0;   }
```

۹-۲-۲- سمافور شمارشی

تفاوت دیگر بین mutex و سمافور شمارشی این است که برخلاف mutex، یک سمافور شمارشی می‌تواند در یک رشته متفاوت از رشته‌ای که عملیات انتظار را انجام می‌دهد post شود. به عبارتی در یک رشته عملیات wait و در رشته‌ای دیگر عملیات post روی همان سمافور انجام می‌پذیرد. در واقع، این کار هنگامی که از سمافور برای شمارش نمونه‌های یک منبع کمیاب استفاده می‌کنیم، ضروری است.

برای مشاهده استفاده از سمافورهای شمارشی، مثال کلاسیک تولید کننده - مصرف کننده را در نظر بگیرید. در این مساله یک رشته در حال تولید آیتم‌ها است (مثلا اشیایی از نوع void * که ممکن است به اشیاء دیگری با پیچیدگی دلخواه اشاره کنند) در حالی که رشته دوم، آن آیتم‌ها را مصرف می‌کند. بنابراین از سمافورها برای همگام‌سازی دسترسی[3] به این منبع اشتراکی استفاده خواهیم کرد. در این مثال، منبع اشتراکی یک بافر است که آیتم‌ها را هنگام عبور از یک رشته به رشته دیگر نگهداری می‌کند.

---

[3] Synchronize Access

تعیین تعداد و نوع مناسب برای سمافورها بر حسب مسئله

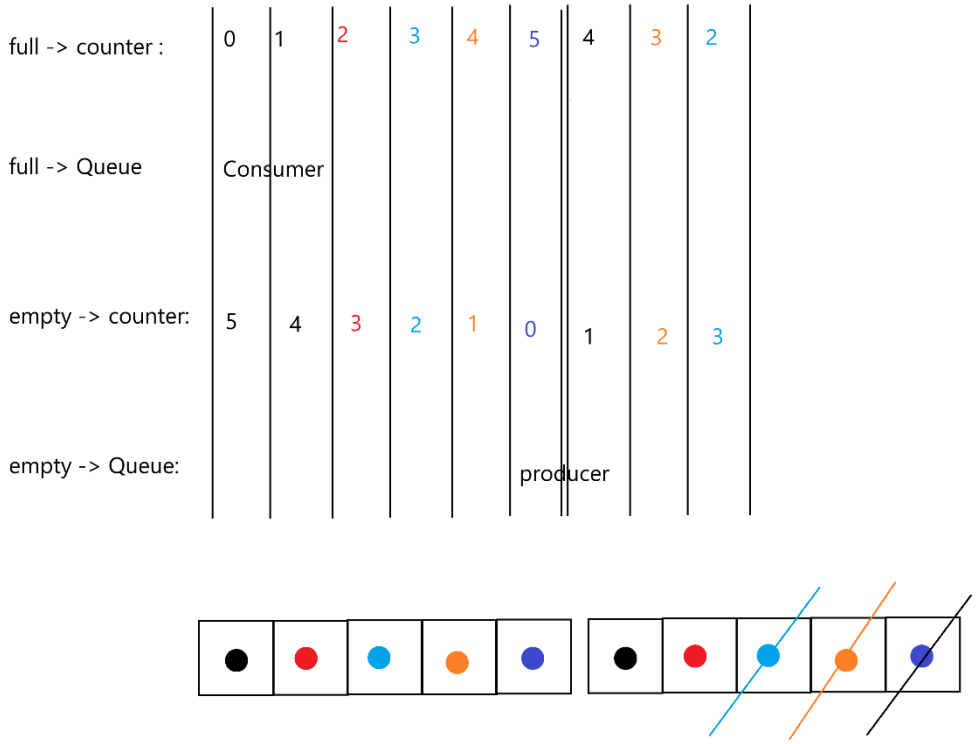
با کمک سمافورها اطمینان حاصل می‌کنیم که تولید کننده و مصرف‌کننده در دسترسی به بافر با یکدیگر همگام هستند و بنابراین شرایط رقابتی پیش نخواهد آمد. بر این اساس، تولیدکننده و مصرف‌کننده نباید به‌صورت همزمان به بافر دسترسی داشته باشند. در غیراین صورت، عملیات خواندن و نوشتن با هم تداخل می‌کنند. به عبارت دیگر، هنگامی که یکی از رشته‌ها می‌خواهد در بافر بنویسد یا از آن چیزی را بخواند، بافر را قفل می‌کنیم تا مطمئن شویم که هیچ رشته دیگری نمی‌تواند به‌صورت همزمان در بافر تغییری ایجاد کند. بنابراین، حداقل به یک سمافور باینری برای کنترل دسترسی همزمان به بافر که منبع اشتراکی در این مسئله است، نیاز داریم.

همچنین تولید کننده و مصرف کننده نباید از یکدیگر پیشی بگیرند. یعنی تولیدکننده زمانی که بافر پر است نباید آیتم جدیدی تولید کند اگرنه روی آیتم های موجود رونویسی[4] می‌شود. مصرف‌کننده هم نمی‌تواند زمانی که بافر خالی است و آیتمی وجود ندارد، چیزی از بافر بردارد. بر این اساس به چند سمافور نیاز داریم؟

اگر فقط یک سمافور مشترک داشته باشیم، این اتفاق خواهد افتاد: برای اینکه تولیدکننده به حالت انتظار برود، باید سمافور مقدار 0 داشته باشد که این نشان دهنده پر بودن بافر است. از طرف دیگر برای اینکه مصرف‌کننده به حالت انتظار برود، باید سمافور مقدار 0 داشته باشد که نشان دهنده خالی بودن بافر است! بنابراین تناقض ایجاد می‌شود. از طرفی اگر مقدار سمافور 0 شود، تولیدکننده به حالت انتظار می‌رود. مصرف‌کننده نیز چون از همان سمافور اشتراکی استفاده می‌کند، به حالت انتظار خواهد رفت. در نتیجه، هر دو رشته روی یک سمافور مسدود می‌شوند و بن‌بست رخ می‌دهد.

---

[4] Overwrite



تصویر ۹-۱- دو سمافور شمارشی غیرمنفی برای مسئله تولیدکننده – مصرف کننده

بنابراین حداقل به 2 سمافور دیگر نیز نیاز داریم که یکی پر بودن و دیگری خالی بودن بافر را برایمان مشخص کند. یا به عبارت دیگر، یکی برای شمارش تعداد اسلات‌های آزاد در بافر و دیگری برای شمارش تعداد اسلات‌های استفاده شده. در ابتدا، بافر خالی است و هیچ آیتمی برای مصرف وجود ندارد. بنابراین مقدار اولیه سمافور full را 0 در نظر می‌گیریم تا مصرف کننده در همان ابتدا به حالت انتظار برود. اما وقتی بافر خالی باشد، تولیدکننده به اندازه سایز بافر اجازه تولید دارد. بنابراین سمافور empty را با BUFFER_SIZE مقداردهی اولیه می‌کنیم. وقتی تعداد «جاهای خالی» در بافر صفر شود، تولید کننده دیگر اجازه تولید ندارد و به حالت انتظار خواهد رفت. بنابراین در کل به ۳ سمافور برای حل این مسئله نیاز داریم.

```
// Producer_Consumer.c

#include <stdio.h>
```

```
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

// تعیین اندازه بافر
#define BUFFER_SIZE 5

// تعریف یک بافر آرایه‌ای
int buffer[BUFFER_SIZE];

// شمارنده آیتم های درون بافر
int count = 0;

// تعریف ۳ سمافور با نام های متناسب برای حالت‌های خالی، پر و انحصار متقابل
sem_t empty, full, mutex;

// تعریف تابع برای رشته تولید کننده
void* producer(void* arg) {

// تعریف حلقه برای تولید ۱۰ آیتم
for (int i = 0; i < 10; i++) {

// سمافور خالی را کاهش می‌دهد و در صورت پر بودن بافر، تولیدکننده مسدود می‌شود
sem_wait(&empty);

// میوتکس برای ورود به ناحیه بحرانی کاهش داده می‌شود
sem_wait(&mutex);

// یک آیتم به بافر اضافه کرده و شمارنده را افزایش می‌دهد
buffer[count++] = i;

// شماره آیتم تولید شده را چاپ می‌کند
printf("Produced: %d\n", i);

// ایجاد تاخیر یک ثانیه‌ای (اختیاری)
sleep (1);

// میوتکس برای خروج از ناحیه بحرانی افزایش داده می‌شود
sem_post(&mutex);

// سمافور پر را افزایش می‌دهد که نشان دهنده تولید یک آیتم است. اگر مصرف کننده مسدود بود، بیدارش می‌کند
sem_post(&full);
 }   }

// تعریف تابع برای رشته مصرف کننده
void* consumer(void* arg) {
for (int i = 0; i < 10; i++) {
```

```
// سمافور پر را کاهش می‌دهد و در صورت خالی بودن بافر، مصرف‌کننده مسدود می‌شود
sem_wait(&full);

// کاهش میوتکس برای ورود به ناحیه بحرانی
sem_wait(&mutex);

// یک آیتم را از بافر حذف کرده و شمارنده را کاهش می‌دهد
int item = buffer[--count];

// آیتم مصرف شده را چاپ می‌کند
printf("Consumed: %d\n", item);
sleep (1);

// افزایش میوتکس برای خروج از ناحیه بحرانی
sem_post(&mutex);

// سمافور خالی را افزایش می‌دهد که یعنی یک آیتم مصرف شده است. بنابراین تولیدکننده Error! Bookmark not defined. از انتظار بیرون می‌آید
sem_post(&empty);
 }   }

int main() {

// اعلان دو متغیر از نوع رشته برای تولیدکننده و مصرف‌کننده
pthread_t prod, cons;

// مقداردهی اولیه سمافور خالی به اندازه بافر
sem_init(&empty, 0, BUFFER_SIZE);

// مقداردهی اولیه سمافور پر با صفر
sem_init(&full, 0, 0);

// مقداردهی اولیه سمافور میوتکس با ۱
sem_init(&mutex, 0, 1);

// ایجاد رشته تولید کننده و مصرف کننده
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);

// انتظار برای پایان رشته‌ها
pthread_join(prod, NULL);
pthread_join(cons, NULL);

// پاک‌سازی سمافورها
sem_destroy(&empty);
sem_destroy(&full);
sem_destroy(&mutex);
```

```
return 0;    }
```

نکته: سمافورهای POSIX، برای همگام‌سازی دو یا چند فرآیند مجزا نیز قابل استفاده می‌باشند. به این‌صورت که یک فرآیند یک سمافور را تحت یک نام خاص ایجاد می‌نماید، آن‌گاه سایر فرآیندها نیز می‌توانند به آن سمافور با نام مربوطه‌اش دست پیدا کنند.

تمرین

- گزارش کار مربوط به تخصیص بافر ایستا از دستورکار «رشته‌ها» را با استفاده از سمافور باینری حل نمایید، به‌طوری‌که دیگر تداخل محتوای رشته‌ها اتفاق نیافتد.
- مسئله Reader-Writer را با استفاده از سمافور شمارشی حل کنید. حداقل سه عدد Reader و دو عدد Writer ایجاد کنید. در این مسئله باید دسترسی به یک منبع مشترک را همگام نمایید به‌طوری‌که Readerها بتوانند به طور همزمان بخوانند اما writerها باید به‌صورت انحصاری به منبع دسترسی داشته و در آن بنویسند.

گزارش کار

راه حل مسئله غذاخوردن فیلسوف‌ها[5] را با استفاده از سمافور و الگوریتم Dijkstra برای ۵ فیلسوف، پیاده سازی نمایید. در این الگوریتم، هر فیلسوف می‌تواند در یکی از سه حالت Thinking، Hungry و یا Eating قرار بگیرد. هر فیلسوف تنها زمانی اجازه غذا خوردن دارد که همسایه‌هایش در حالت Eating نباشند. بنابراین، علاوه بر تعریف یک سمافور اختصاصی برای هر فیلسوف، لازم است وضعیت همسایه‌ها را در یک آرایه نگهداری کرده و با استفاده از یک سمافور عمومی از این «آرایه وضعیت» محافظت کنید. در نهایت با بررسی وضعیت همسایه‌ها، از ایجاد بن‌بست جلوگیری نمایید.

---

[5] Dining Philosophers Problem

منابع و مراجع:

- Silberschatz A. *Operating System Concepts*, Wiley,10th Edition, 2018.
- ArpaciDusseau RH. *Operating Systems: Three Easy Pieces*. Madison, 2018.
- Haghighat A, *Operating Systems with Problem Solving Approachs*, Pars Resaneh, 2023.
- https://pubs.opengroup.org/onlinepubs/9699959399/basedefs/semaphore.h.html, visited 2022 Jun.
- https://man7.org/linux/man-pages/man7/sem_overview.7.html, visited 2022 Jun.

آزمایش ۱۰ : تحلیل کارایی با Perf

اهداف:

- آشنایی با نصب و تنظیمات اولیه
- یادگیری دستورات فرعی و گزینه‌های ابزار Perf
- نحوه جمع‌آوری اطلاعات کارایی، تحلیل و تفسیر نتایج

ابزار و مفاهیم مورد نیاز:

- ابزار Perf نصب شده روی سیستم‌عامل لینوکس محلی
- کدمنبع چند برنامه آزمایشی جهت تحلیل کارایی

محتوا:

- Perf و سایر ابزارهای تحلیل کارایی در لینوکس
- دستورات فرعی
  - list, top, stat, record, report, annotate, script
- سوئیچ‌ها
- تحلیل کارایی بر اساس داده‌های حاصل از annotate و report

۱۰-۱- معرفی

پروفایلینگ[1] عبارت است از فرآیند اندازه‌گیری و تحلیل رفتار یک برنامه‌ی در حال اجرا به‌منظور شناسایی نقاط حساس، گلوگاه‌ها و الگوهای ناکارآمد در استفاده از منابع سخت‌افزاری. پروفایلینگ معمولا به‌منظور تحلیل کارایی، عیب‌یابی و بهینه‌سازی انجام می‌گیرد. همان‌طور که در شکل ۱۰-۱ مشاهده می‌کنید، ابزارهای گوناگونی - با حوزه‌های عملکردی مختلف - بدین‌منظور طراحی شده‌اند. از جمله top، vmstat، iostat، netstat، perf، starce.

با برخی از این ابزارها قبلا آشنا شدیم. مثلا با استفاده از top می‌توانیم یک نمای کلی از میزان استفاده از پردازنده و حافظه توسط فرآیندها به دست بیاوریم. با iostat آمار ورودی یا خروجی دیسک‌ها و پارتیشن‌ها را به دست آورده و آن‌ها را از نظر میزان مشغول یا منتظر بودن بررسی می‌کنیم. با strace فراخوانی‌های سیستمی یک فرآیند را ردیابی کرده و متوجه می‌شویم آیا برنامه عملیات I/O زیادی دارد یا گیر[2] کرده است. با netstat اتصالات شبکه، سوکت‌ها و پورت‌های باز را بررسی می‌نماییم. اما برای تحلیل‌های عمیق‌تر مانند خطاهای حافظه پنهان، تعداد دستورالعمل‌ها در هر چرخه[3] (IPC) یا پیش‌بینی‌های نادرست شاخه‌ها[4] به ابزارهای تخصصی‌تری مانند perf نیاز داریم.

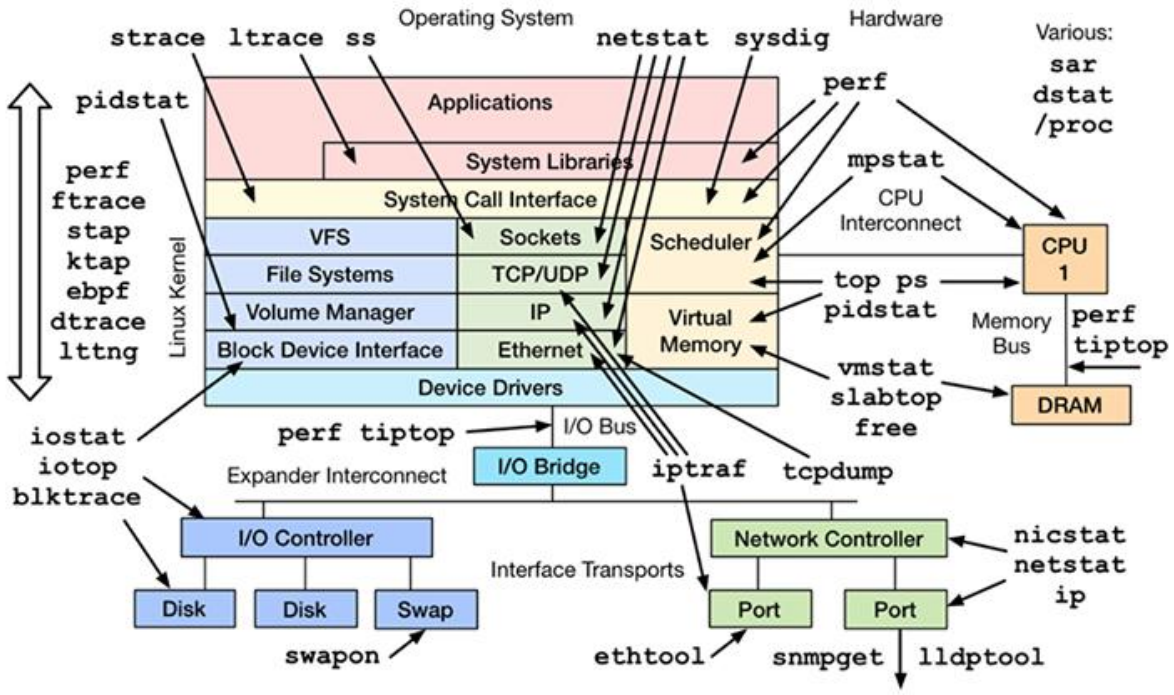
perf یک چهارچوب متن‌باز به زبان C می‌باشد که تحت مجوز GNU GPL منتشر گردیده و با یک رابط خط فرمانی ساده ارائه می‌شود. این ابزار قدرتمند، برای پروفایلینگ در سطح سیستم عامل مورد استفاده قرار می‌گیرد و توسط خود توسعه‌دهندگان هسته لینوکس ساخته و درون آن تعبیه شده است. perf از هسته لینوکس نسخه 2.6.31 از سال 2009 به بعد، در دسترس می‌باشد.

این ابزار نظارتی، می‌تواند به‌طور مستقیم با سخت‌افزار و هسته لینوکس ارتباط برقرار کرده و رویدادهای سخت‌افزاری سطح پایین را بدون نیاز به تغییر در کد برنامه یا کامپایل مجدد، شمارش و ثبت نماید. به‌علاوه، از ردیابی پویا و

---

[1] Profiling
[2] Hang
[3] Instructions Per Cycle/Clock
[4] Branch Mispredictions

شمارنده‌های کارایی سخت‌افزار، tracepoints، شمارنده‌های کارایی نرم‌افزار (مثلا hrtimer) و Probeهای پویا (مانند kprobes یا uprobes) پشتیبانی می‌کند. به عبارت دیگر، perf یک شمارنده کارایی کامل برای لینوکس[5] است.



تصویر ۱۰-۱- ابزارهای مشاهده کارایی در لینوکس

پیاده سازی

برای درک بهتر عملکرد perf، باید بدانیم که این ابزار با شمارنده‌های واحد نظارت بر عملکرد[6] پردازنده (PMU)، که در پردازنده‌های مدرن وجود دارد، به‌صورت مستقیم تعامل می‌کند. PMU یک زیرسیستم سخت‌افزاری است که می‌تواند رویدادهای سطح پایین پردازنده را بدون کاهش سرعت اجرا شمارش کند. perf با برنامه‌ریزی شمارنده‌های

---

[5] Performance Counters for Linux (PCL)

[6] Performance Monitoring Unit

PMU، داده‌های خام سخت‌افزاری را جمع‌آوری می‌کند. سپس آن‌ها را به‌صورت قابل فهم برای کاربر تبدیل کرده و نمایش می‌دهد.

رابط بین ابزار perf و هسته، فقط از یک فراخوانی سیستمی تشکیل شده است و از طریق یک توصیف‌گر فایل و یک منطقه حافظه‌ی نگاشت شده انجام می‌پذیرد. حتی به service daemons هم نیازی ندارد، زیرا بیشتر قابلیت‌هایش در هسته یکپارچه شده‌اند. هنگامی که بافر پر می‌شود، ابزار perf داده‌های خام را از بافر map شده، به دیسک تخلیه می‌کند. در کل، پروفایل کردن توسط perf هزینه سربار بسیار کمی دارد.

کاربردها

از جمله رویدادهایی که می‌توان با perf آن‌ها را مشاهده کرد، عبارتند از: تعداد چرخه‌های پردازنده، تعداد دستورات اجرا شده، تعداد دسترسی‌های موفق و ناموفق به داده‌ها یا صفحات حافظه در حافظه پنهان، پیش‌بینی اشتباه در شرط‌ها، تعداد دفعات جابه‌جایی بین فرآیندها در پردازنده و غیره.

با توجه به اینکه perf قادر است کل سیستم (هم هسته و هم کدهای سمت کاربر) را به‌صورت آماری پروفایل کند، از آن در حوزه های متنوعی همچون بهینه‌سازی کدها و مقایسه الگوریتم‌ها، تحلیل هسته لینوکس، بررسی ناهماهنگی و شرایط رقابتی در دسترسی به حافظه سیستم، شناسایی الگوهای غیرعادی اجرا و تحلیل امنیتی سیستم، یافتن دلایل کندی ناگهانی سرور و موارد دیگری ازا این دست، استفاده می‌شود.

۱۰-۲-نصب و تنظیمات

Perf همراه با سایر بسته‌های متداول لینوکس در مخازن مربوطه عرضه می‌شود. برای نصب این ابزار باید به کاربر ریشه و یا یک کاربر sudoer دسترسی داشته باشید. توصیه می‌شود قبل از نصب، لیست مخزن سیستم را به‌روزرسانی نمایید.

```
sudo   apt   update
```

سپس با دستور aptitude به‌صورت زیر، ابزارهای رایج لینوکس را روی دستگاه خود نصب کنید. این دستور نیز به سطح دسترسی ریشه نیاز دارد به همین دلیل از sudo در ابتدای دستور استفاده شده است. هنگامی که نصب به پایان رسید، می‌توانید بسته‌های Perf را در دایرکتوری /usr/bin/perf پیدا کنید.

```
sudo apt install  linux-tools-common  linux-tools-generic
```

ازآنجایی‌که Perf از جمله بسته‌های مشترک لینوکس است، لازم است اطمینان حاصل کنیم که با هسته سیستم‌عامل سازگار است. دستور زیر، نسخه هسته را شناسایی کرده و سپس ابزارهای لینوکسی مناسب هسته سیستم را نصب می‌نماید.

```
sudo   apt   install   linux-tools-$(uname -r)
```

می‌توانید برای اطمینان از نسخه Perf نصب شده، دستور زیر را اجرا کنید. خواهید دید که نسخه Perf و هسته یکسان است.

```
perf   --version
```

۱۰-۲-۱- تنظیم مجوز کاربران برای استفاده از ابزار Perf

استفاده از دستور perf به شکل پیش‌فرض نیاز به امتیاز sudo دارد. برای اینکه یک کاربر بتواند از perf استفاده کند، مراحل زیر را طی کنید. ابتدا به کاربر روت سوئیچ کنید.

```
sudo su -
```

سپس دستور زیر را اجرا کنید. این دستور به کاربران اجازه می‌دهد در نشست حاضر از ابزار perf استفاده نمایند.

```
echo 0 > /proc/sys/kernel/perf_event_paranoid
```

برای دسترسی‌های بیشتر می‌توانید در دستور بالا، 0 را با 1- جایگزین کنید. حال با دستور زیر به کاربر قبلی برگردید.

```
exit
```

برای اینکه تغییراتی که ایجاد کرده‌اید، دائمی شود می‌توانید به‌صورت زیر عمل کنید. ابتدا فایل تنظیمات sysctl را با یک ویرایشگر متنی باز کنید.

```
sudo   nano   /etc/sysctl.conf
```

سپس، خط زیر را به انتهای فایل اضافه نمایید.

```
kernel.perf_event_paranoid = 0
```

تغییرات را ذخیره کرده و فایل را بندید.

۱۰-۳- دستورات فرعی

ابزار Perf عملکردی مشابه Git دارد. این ابزار به‌عنوان رابطی برای دستورهای فرعی و فعالیت‌های مختلف عمل می‌کند. اجرای دستور بدون هیچ سوئیچ یا آرگومانی، لیست دستورات فرعی موجود را نمایش خواهد داد.

```
usage: perf [--version] [--help] [OPTIONS] SUBCOMMAND [ARGS]
```

برای دریافت اطلاعات بیشتر در مورد هر دستور فرعی، می‌توانید دستور زیر را اجرا کنید.

```
perf   help   <SUBCOMMAND>
```

دستورات فرعی نیز به نوبه خود دارای سوئیچ‌های اضافی می‌باشند. برای دیدن سوئیچ‌های هر دستور فرعی:

```
perf   <SUBCOMMAND>   -h
```

در جدول زیر، دستورات فرعی[7] پرکاربرد Perf را همراه با توضیحی مختصر برای هر یک، مشاهده می‌نمایید. در بخش بعدی نیز کاربردهای مهم‌ترین دستورات فرعی را به‌طور مشروح در قالب چند سناریو و برای مقاصد خاص خواهیم دید.

دستور	توضیح
annotate	اطلاعات perf.data را خوانده و کد اسمبلی توابع را همراه با درصد زمان هر دستور اسمبلی نمایش می‌دهد.
archive	با فایل objects حاوی شناسه‌های توکار که در فایل perf.data قرار دارند، آرشیو ایجاد می‌کند.
config	متغیرها را در یک فایل پیکربندی دریافت و تنظیم می‌کند.
daemon	record sessions را در پس زمینه اجرا می‌کند.
iostat	معیارهای کارایی I/O را نشان می‌دهد.
list	همه رویدادهای قابل شمارش در سیستم را فهرست می‌کند.
mem	دسترسی به حافظه را پروفایل می‌کند.
record	یک دستور یا برنامه را اجرا کرده و پروفایل آن را در perf.data ضبط می‌کند.
report	فایل perf.data را که توسط record ایجاد شده است، خوانده و پروفایل را نمایش می‌دهد.
script	خروجی خام فایل perf.data را به‌صورت متنی برای پردازش‌های بیشتر نمایش می‌دهد.
stat	یک دستور یا برنامه را اجرا کرده و خلاصه رویدادها را نمایش می‌دهد.
top	پرمصرف‌ترین توابع روی سیستم را به‌صورت بلادرنگ نمایش می‌دهد.
version	نسخه perf را نمایش می‌دهد.

جدول ۱۰-۱- لیست دستورات فرعی perf

۱۰-۴- پروفایل کردن پردازنده

برای اینکه با استفاده از perf یک پردازنده را پروفایل کنیم، معمولا چنین روندی طی می‌شود: ابتدا با استفاده از دستور فرعی list، رویدادهای قابل دسترس در سیستم را لیست می‌کنند و با stat، آماری خلاصه و سریع پیرامون رویدادها

---

[7] Subcommand

نمایش می‌دهند. البته در این حالت، این اطلاعات هیچ‌جا ذخیره نمی‌شوند. در صورت نیاز به دریافت اطلاعات بیشتر و همچنین امکان ذخیره‌سازی، از دستور record استفاده می‌شود. این دستور رویدادهای مشخص شده را در فایل perf.data ذخیره می‌کند. سپس دستور report اجرا می‌شود که فایل مذکور را خوانده و گزارشی تعاملی نمایش می‌دهد. در صورت نیاز به داده‌های خام برای پردازش‌های بیشتر، می‌توان از دستور script استفاده کرد. این دستور، خروجی خام نمونه‌های ضبط شده را به‌صورت متنی نمایش می‌دهد. این داده‌ها برای تحلیل خارج از perf مثلا با پایتون یا اکسل، یا تشخیص الگوهای تکراری مثل دسترسی به یک آدرس خاص، و یا مصورسازی‌های حرفه‌ای مانند FlameGraph و مواردی از این دست، مورد استفاده قرار می‌گیرند.

در حالت کلی، خروجی دستورات بسته به سیستم و منابع محلی موجود متفاوت خواهد بود. چون پردازنده‌های مختلف رویدادهای متفاوتی دارند و از طرف دیگر نیز، رویدادهای قابل دسترسی در سیستم با توجه به سطح دسترسی کاربر، متفاوت خواهد بود. در ادامه، دستورات ضروری برای پروفایل کردن پردازنده اجرا شده‌اند.

لیست کردن تمام رویدادهای قابل اندازه‌گیری

```
sudo   perf   list
```

دستور list، تمام رویدادهای پشتیبانی‌شده در سیستم را صرف‌نظر از نوع آن‌ها، در خروجی فهرست می‌کند. هنگام ارجاع به این رویدادها در دستورات فرعی دیگر، از سوئیچ e- استفاده کنید و به‌دنبال آن، نام رویداد مدنظرتان را از ستون اول انتخاب نمایید. به‌عنوان مثال، تصویر ۱۰-۲ یک نمونه از خروجی دستور بالا را نشان می‌دهد. لیست رویدادهای از پیش تعریف شده را در ستون چپ تصویر مشاهده می‌کنید.

```
soltani@LIFEBOOK:~$ sudo perf list

List of pre-defined events (to be used in -e or -M):

  branch-instructions OR branches                    [Hardware event]
  branch-misses                                      [Hardware event]
  bus-cycles                                         [Hardware event]
  cache-misses                                       [Hardware event]
  cache-references                                   [Hardware event]
  cpu-cycles OR cycles                               [Hardware event]
  instructions                                       [Hardware event]
  ref-cycles                                         [Hardware event]
  stalled-cycles-frontend OR idle-cycles-frontend    [Hardware event]
  alignment-faults                                   [Software event]
  bpf-output                                         [Software event]
  cgroup-switches                                    [Software event]
  context-switches OR cs                             [Software event]
  cpu-clock                                          [Software event]
  cpu-migrations OR migrations                       [Software event]
  dummy                                              [Software event]
  emulation-faults                                   [Software event]
  major-faults                                       [Software event]
  minor-faults                                       [Software event]
  page-faults OR faults                              [Software event]
  task-clock                                         [Software event]
  duration_time                                      [Tool event]
  user_time                                          [Tool event]
```

تصویر ۱۰-۲- خروجی دستور فرعی لیست

برای محدود کردن لیست بر اساس نام رویداد (ستون اول) یا نوع رویداد (ستون دوم)، پارامتر فیلتر موردنظر را بعد از دستور لیست اضافه می‌کنیم. توجه داشته باشید درصورتی‌که دستور لیست را در ماشین‌مجازی اجرا نمایید، در ستون دوم، عبارت Hardware event را اصلا مشاهده نخواهید کرد. زیرا ماشین‌مجازی در دسترسی مستقیم به منابع سخت افزاری سیستم میزبان با محدودیت‌هایی روبرو است. اما در حالت کلی، برای اینکه فقط رویدادهای سخت‌افزاری نمایش داده شوند، دستور فرعی لیست باید همراه با آگومان زیر اجرا گردد.

```
sudo   perf   list   hardware
```

همچنین برای نمایش رویدادهای مرتبط با پردازنده، از آرگومان متناسب با آن استفاده می‌شود.

```
sudo   perf   list   cpu
```

در خروجی، نتیجه فیلتر شده بر اساس پارامتر یا پارامترهای مشخص‌شده، نمایش داده خواهد شد.

```
soltani@LIFEBOOK:~$ sudo perf list cpu

List of pre-defined events (to be used in -e or -M):

  cpu-cycles OR cycles                                [Hardware event]
  cpu-clock                                           [Software event]
  cpu-migrations OR migrations                        [Software event]

cpu:
  L1-dcache-loads OR cpu/L1-dcache-loads/
  L1-dcache-load-misses OR cpu/L1-dcache-load-misses/
  L1-dcache-stores OR cpu/L1-dcache-stores/
  L1-dcache-store-misses OR cpu/L1-dcache-store-misses/
  L1-dcache-prefetch-misses OR cpu/L1-dcache-prefetch-misses/
  L1-icache-load-misses OR cpu/L1-icache-load-misses/
  LLC-loads OR cpu/LLC-loads/
  LLC-stores OR cpu/LLC-stores/
  LLC-prefetches OR cpu/LLC-prefetches/
  dTLB-loads OR cpu/dTLB-loads/
  dTLB-load-misses OR cpu/dTLB-load-misses/
  dTLB-stores OR cpu/dTLB-stores/
  dTLB-store-misses OR cpu/dTLB-store-misses/
  iTLB-loads OR cpu/iTLB-loads/
  iTLB-load-misses OR cpu/iTLB-load-misses/
  branch-loads OR cpu/branch-loads/
  branch-load-misses OR cpu/branch-load-misses/
  branch-instructions OR cpu/branch-instructions/     [Kernel PMU event]
```

تصویر ۱۰-۳- خروجی دستور فرعی لیست فقط برای رویدادهای پردازنده

ازآنجایی‌که رویدادهای سخت‌افزاری مستقیما از پردازنده گرفته می‌شوند، دقیق و سریع هستند. اما رویدادهای نرم‌افزاری توسط هسته لینوکس شمارش می‌شوند و کمی کندترند.

مشاهده بلادرنگ مشخصات پردازنده

این دستور فرعی، مشابه دستور top در لینوکس عمل می‌کند و توابع نمونه‌برداری شده را به‌صورت بلادرنگ، نمایش می‌دهد.

```
sudo   perf   top
```

در خروجی، سه ستون به ترتیب از چپ به راست چاپ می‌شوند که به شرح زیر هستند:

ستون اول. میزان استفاده از پردازنده که به‌صورت تابعی بر حسب درصد بیان می‌شود.

ستون دوم. کتابخانه یا برنامه‌ای که از تابع استفاده می‌کند.

ستون سوم. نماد و نام تابع. در این ستون، [k] بیانگر فضای هسته و [.] بیانگر فضای کاربر است.

به طور پیش‌فرض، دستور perf top تمام پردازنده‌های آنلاین را رصد می‌کند. اما سوئیچ‌های دیگری نیز وجود دارد. مثلا سوئیچ a- برای نظارت بر تمام پردازنده‌ها از جمله پردازنده‌های بیکار، یا C- برای نظارت بر پردازنده‌های خاص و یا F- برای کنترل فرکانس نمونه‌برداری. برای دیدن سوئیچ های اضافی، در حالی که دستور perf top در حال اجراست، کلید h را فشار دهید. همچنین برای خروج و بازگشت به ترمینال، کلید q را از صفحه‌کلید بزنید.

۱۰-۵- پروفایل کردن برنامه‌های سطح کاربر

در بخش قبل، با پروفایل پردازنده در سطح هسته، به مشاهده‌ی آمار و رویدادهایی که مستقیما توسط هسته و سخت‌افزار تولید می‌شوند، پرداختیم. در این بخش می‌خواهیم یک برنامه در سطح کاربر - یعنی برنامه‌ای که در فضای کاربر اجرا می‌شود - را پروفایل کنیم. فرض کنید یک برنامه برای فشرده‌سازی تصاویر نوشته‌ایم که کند کار می‌کند. می‌خواهیم بفهمیم مشکل از کجاست؟ از پردازنده، حافظه پنهان یا دسترسی به حافظه. کد زیر، چنین برنامه‌ای را شبیه‌سازی می‌کند و طوری نوشته شده است که در آن «عدم برخورد حافظه نهان[8]» رخ بدهد تا بتوانیم آن را با perf ثبت وبررسی نماییم. توجه داشته باشید که این یک کد ساختگی است و هیچ عملیات فشرده‌سازی واقعی بر روی تصویر انجام نخواهد داد، بلکه صرفا آرایه‌ای از پیکسل‌ها را به دو شیوه متفاوت پیمایش می‌نماید.

```
// Profiling.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define WIDTH 4096
#define LENGTH 4096
#define ITERATIONS 100
```

[8] Cache Miss

```
// Cache Unfriendly (column-major)
void bad_access_pattern(unsigned char *pixel) {
    unsigned char temp[LENGTH];
    for (int i = 0; i < LENGTH; i++) {
        temp[i] = pixel[i * WIDTH]; }  // big stride : cache-miss
}
// Cache Friendly (row-major)
void good_access_pattern(unsigned char *pixel) {
    unsigned char temp[WIDTH];
    for (int i = 0; i < WIDTH; i++) {
        temp[i] = pixel[i];  }        // sequential (cache-hit)
}
int main(int argc, char *argv[]) {
    // تخصیص حافظه پویا به پیکسل‌ها
    unsigned char *image = malloc(WIDTH * LENGTH);
    memset(image, 128, WIDTH * LENGTH); // assign 128 to every pixel
    for (int iter = 0; iter < ITERATIONS; iter++) {
        if (iter % 2 == 0) {
            bad_access_pattern(image);      // results in cache miss
        } else {
            good_access_pattern(image);  }  // results in cache hit
        }
    // آزادسازی حافظه تخصیص‌یافته
    free(image);
    return 0;   }
```

توضیح کد: تابع اول، یک اشاره گر به نام pixel را دریافت می‌کند که به آرایه ای از پیکسل ها به طول ۴۰۹۶ در عرض ۴۰۹۶ اشاره می‌کند. فرض کنید این ابعاد تصویری است که قرار است کد ما آن را فشرده کند.

```
WIDTH * LENGTH =  ۴۰۹۶×  ۴۰۹۶ = ۱۶ مگابایت
```

خروجی این تابع، یک آرایه موقت به نام temp می‌باشد که مقدار pixel[i * WIDTH] را ذخیره می‌نماید. در این تابع، نحوه دسترسی به پیکسل ها به‌صورت پیمایش ستونی در نظر گرفته شده است. در نتیجه هر بار باید به فاصله WIDTH (۴۰۹۶ بایت) پرش کند. مثلا؛ اول pixel[0]، سپس pixel[4096]، بعد pixel[8192] و به همین ترتیب. این فاصله به‌طور معمول، از اندازه یک خط حافظه نهان[9] بزرگ‌تر است و بنابراین باعث می‌شود داده‌ها در خطوط کش مختلف

---

[9] Cache Line

قرار بگیرند. در نتیجه با هر دسترسی، یک «عدم برخورد» جدید ایجاد شده و آمار کلی «عدم برخورد حافظه نهان» بالا خواهد رفت.

اما در تابع دوم، پیمایش به‌صورت خطی صورت می‌گیرد؛ یعنی به ترتیب از ابتدا تا انتهای آرایه پیمایش می‌شود و بنابراین، نحوه دسترسی به‌صورت پشت‌سر هم است. یعنی اول pixel[0]، سپس pixel[1]، بعد pixel[2] و به همین ترتیب. این الگوی دسترسی بسیار سازگار با حافظه نهان است. به عبارتی، داده‌های مجاور در یک خط کش قرار می‌گیرند و در نتیجه آمار « عدم برخورد حافظه نهان» بسیار پایین خواهد بود. به این ترتیب، فقط اولین دسترسی دچار «عدم برخورد» می‌شود ولی برای بقیه دسترسی‌ها، «برخورد حافظه نهان[10]» رخ خواهد داد.

این دو الگوی دسترسی متفاوت در دو تابع، به perf اجازه می‌دهند تا تفاوت حاصل را اندازه‌گیری نماید. به‌علاوه این کد، بدون عملیات چاپ در حلقه طراحی شده است تا دخالت I/O که عملیاتی زمان‌بر است، کاهش یافته و perf را به اشتباه نیاندازد.

در تابع main ابتدا به آرایه‌ی پیکسل‌ها، حافظه‌ای برابر با ۱۶ مگابایت به‌صورت پویا تخصیص می‌یابد. و سپس همه بایت‌ها با عدد ۱۲۸ مقداردهی اولیه می‌شوند. عدد ۱۲۸، بیانگر رنگ خاکستری متوسط است.

سپس حلقه، ۱۰۰ بار تکرار می‌شود که در تکرارهای زوج، تابع اول (دسترسی نامناسب) و در تکرارهای فرد، تابع دوم (دسترسی مناسب) را فراخوانی می‌نماید و در نتیجه، به هر دو روش به آرایه دسترسی پیدا می‌کند.

کامپایل برنامه

در ادامه با کمک perf، تفاوت نرخ « عدم برخورد حافظه نهان» و «تعداد دستورالعمل‌ها در هر چرخه» بین دو این تابع را مشاهده کرده و دو الگوی دسترسی را مقایسه خواهیم نمود. اما اول باید کد را کامپایل کنیم.

```
gcc   Perf_Test.c   -O0   -g   -o   perf_test
```

سوئیچ‌های مورد استفاده در این دستور کامپایل، عبارتند از:

---

[10] Cache Hit

O0- : به معنای بهینه‌سازی در سطح صفر می‌باشد. یعنی از کامپایلر می‌خواهیم که هیچ‌گونه بهینه‌سازی بر کد اعمال ننماید. به این ترتیب، کد اسمبلی مستقیما معادل کد C می‌شود و بنابراین نتایج تحلیل perf annotate دقیق خواهد بود. درصورتی‌که از سطوح بالاتر بهینه‌سازی استفاده کنید؛ مثلا O2- یا O3- ممکن است کامپایلر به‌منظور بهینه شدن کد، تغییراتی در آن بدهد. مثلا حلقه‌ها را باز کند، ترتیب دستورات را تغییر بدهد، توابع را درون‌خطی کند، متغیرهای بدون استفاده را حذف نماید و تغییرات دیگری از این دست. این کار باعث می‌شود که ابزار perf در تعیین درصد زمانی صرف شده توسط برخی از کدهای C دچار مشکل بشود.

در کد ما نیز سطوح بالای بهینه‌سازی ممکن است حتی منجر به حذف کامل تابع bad_access_pattern بشود و در نتیجه، perf report درصدهای اشتباهی را به قسمت های مختلف کد نسبت بدهد. به این ترتیب، هدف آموزشی کد که مشاهده «عدم برخورد حافظه نهان» بود نیز از بین خواهد رفت.

g- : این سوئیچ، اطلاعات دیباگ (DWARF) را به فایل خروجی اضافه می‌کند. به این ترتیب، نام سمبل‌ها، شماره خطوط و نوع متغیرها ذخیره می‌شوند و در نتیجه دستور perf report می‌تواند در خروجی خود، نام توابع C را به جای آدرس آن‌ها نمایش بدهد.

اجرای برنامه با perf

پس ازکامپایل برنامه با شرایط مذکور می‌توانیم به سراغ perf و تحلیل عملکرد برنامه با آن برویم. ابتدا برنامه را بدون هیچ تحلیلی، اجرا کرده و زمان اجرای آن را می‌سنجیم.

```
time ./perf_test
```

خروجی

```
real    0m0.015s
```

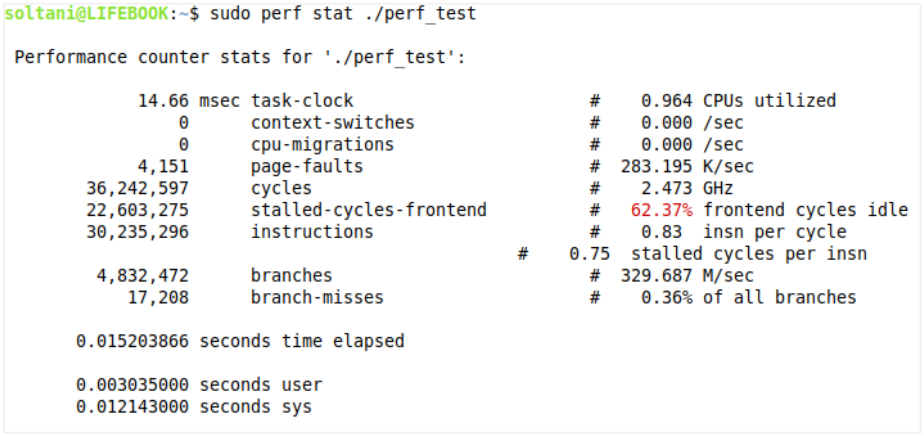
که یعنی اجرای برنامه حدود ۰.۰۱۵ ثانیه طول کشیده است.

برای دریافت آمار کلی، از دستور فرعی stat استفاده می‌کنیم.

```
sudo    perf      stat     ./perf_test
```

خروجی این دستور را می‌توانید در تصویر ۱۰-۳ مشاهده نمایید.

نکته: همان‌طور که قبلا نیز در این بخش اشاره شد، برخی دستورات perf بویژه دستورات مرتبط با سخت افزار ممکن است در لینوکس نصب شده در ماشین‌مجازی امکان اجرا نداشته باشند.



تصویر ۱۰-۳- خروجی دستور stat

تفسیر نتایجی که در تصویر بالا مشاهده می‌کنید، بدین شرح می‌باشد:

سطر اول نتایج که به‌صورت زیر می‌باشد، نشان می‌دهد که زمان اجرای واقعی برنامه حدود ۱۵ میلی‌ثانیه بوده است. بخش دوم در این سطر می‌گوید که از یک هسته پردازنده به مقدار 0.96 درصد استفاده شده است که نزدیک به استفاده کامل و نتیجه خوبی است.

```
14,66   msec task-clock      # 0.964 CPUs utilized
```

سطر دوم نتایج، نشان می‌دهد که هیچ تعویض متن یا مهاجرت بین هسته‌ها رخ نداده و برنامه بدون وقفه و روی یک هسته اجرا شده است.

```
0   context-switches         # 0.000 /sec
```

```
0   cpu-migrations            # 0.000 /sec
```

در سطر سوم، تعداد خطاهای صفحه نشان داده شده است که تعداد زیادی هم دارد. خطاهای صفحه، به دلیل وجود malloc و memset رخ داده‌اند؛ یعنی تخصیص حافظه و نوشتن در آن. بنابراین این خطاها مربوط به زمان راه‌اندازی برنامه است، نه حلقه اصلی که دو تابع را فراخوانی و اجرا می‌کند.

```
4,151   page-faults          # 283.195 K/sec
```

سطر چهارم، فرکانس اندازه‌گیری‌شده برای پردازنده را نمایش می‌دهد.

```
36,242,597   cycles          # 2.473 GHz
```

سطر پنجم، می‌گوید که ۶۲٪ از چرخه‌های پردازنده در frontend، یعنی در واحد fetch/decode منتظر مانده‌اند. دلیل اصلی این اتفاق، دسترسی کند به حافظه در تابع bad_access_pattern است که باعث می‌شود خط‌لوله[11] خالی شود و frontend نتواند دستور بعدی را بردارد[12].

```
22,603,275   stalled-cycles-frontend   # 62.37% frontend cycles idle
```

در سطر ششم، مشاهده می‌کنیم که در هر چرخه به طور میانگین ۰.۸۳ دستور اجرا شده که رقم پایینی است. در حالت کلی اعداد بالای ۲ در این بخش، خوب محسوب می‌شوند.

```
30,235,269   instructions    # 0.83  insn per cycle
```

دلیل این اتفاق نیز مانند سطر قبلی، همان توقف[13] طولانی است که جلوی اجرای دستورات را می‌گیرد.

---

[11] Pipeline
[12] Fetch
[13] Stalls

در سطر هفتم، با دیدن نرخ پایین پیش‌بینی‌های اشتباه (کمتر از 0.4%) متوجه می‌شویم که در این مورد مشکلی در کد ما وجود ندارد و مشکل کندی اجرا از حافظه نهان می‌باشد.

```
17,208          branch-misses
4,832,472   branches         # 329.687 M/sec
```

نتیجه کلی که از خروجی دستور perf stat می‌توانیم بگیریم، این است که عدم اصابت حافظه نهان که ناشی از گام‌های بزرگ در تابع bad_access_pattern است، باعث می‌شود که بیش از ۶۲٪ از چرخه‌های پردازنده در frontend pipeline منتظر بمانند و این موضوع باعث کاهش تعداد دستورالعمل‌ها به ۰.۸۳ در هر چرخه می‌شود. به این ترتیب با کمک دستور stat موفق شدیم گلوگاه اصلی در کد را بیابیم.

حال که اطلاعاتی کلی درباره وضعیت برنامه به دست آوردیم، می‌توانیم به سراغ دستور record برویم تا رویدادهای برنامه را با جزئیات بیشتری ضبط و تحلیل نماییم. در این دستور، از سوئیچ e- استفاده شده است تا فقط رویداد عدم برخورد حافظه نهان - که مسئول کندی برنامه ما است - ضبط بشود.

```
perf        record    -e    cache-misses    ./perf_test
```

درصورتی‌که بخواهیم اطلاعات درخت فراخوانی[14] نیز ثبت بشود، از سوئیچ g- استفاده می‌کنیم. خروجی دستور بالا به شکل زیر خواهد بود.

```
soltani@LIFEBOOK:~$ sudo perf record -e cache-misses ./perf_test
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.016 MB perf.data (83 samples) ]
```

تصویر ۱۰-۴- دستور ضبط داده‌ها

[14] Call Graph

با اجرای دستور مذکور، یک فایل با نام perf.data در دایرکتوری جاری ایجاد می‌شود. برای دیدن اطلاعات ضبط شده از دستور فرعی report استفاده می‌شود و برای نمایش نتایج به‌صورت تعاملی، می‌توان از سوئیچ stdio-- همراه با دستور report استفاده کرد. خروجی دستور، در تصویر ۱۰-۵ نمایش داده شده است.

```
perf      report
```

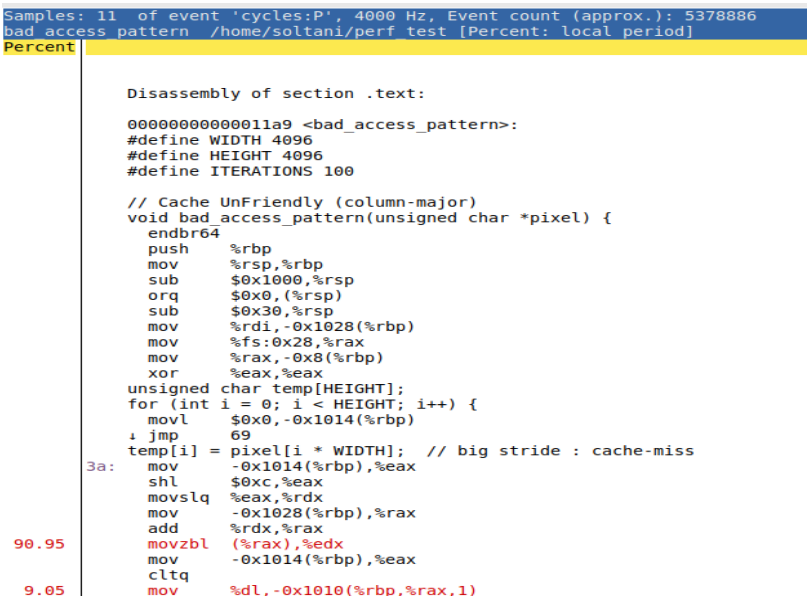
```
Samples: 83  of event 'cache-misses', Event count (approx.): 484405
Overhead  Command    Shared Object          Symbol
  39.80%  perf_test  perf_test              [.] bad_access_pattern
  37.01%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8edb1817
   3.36%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e0bbbd7
   2.74%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8edb1819
   2.50%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e03cf53
   2.32%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e0bbbe0
   1.70%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e0c1245
   1.17%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e023abd
   1.16%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e04d078
   1.15%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8dde327d
   1.15%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e04dc45
   1.15%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e0c121e
   1.14%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e0c3dca
   1.03%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e0c120d
   0.99%  perf_test  [kernel.kallsyms]      [k] 0xffffffff8e08984f
   0.46%  perf_test  ld-linux-x86-64.so.2   [.]  dl lookup symbol x
```

تصویر ۱۰-۵- نمایش اطلاعات آماری جمع‌آوری شده

دستور record، تعداد عدم برخورد در حافظه نهان را شمارش کرده و مشخص می‌نماید که کدام قسمت کد باعث بیشترین عدم برخورد شده است. همان‌طور که در تصویر بالا مشخص است، تابع bad_access_pattern به شکل مستقیم مسئول 39.8٪ از این اتفاق است. غیر از آن، تابع مذکور باعث شده که پردازنده به صفحات جدید حافظه نیاز پیدا کند و در نتیجه هسته لینوکس برای مدیریت خطاهای نقص صفحه و عدم برخورد در جدول TLB وارد عمل شده است. در نتیجه و در مجموع، حدود 55% (… + 37.01+3.36+2.74) از عدم برخوردها به حافظه نهان، توسط هسته ایجاد گردیده است. هر چند که «تابع با دسترسی بد» مسئول غیرمستقیم این حادثه می‌باشد.

حال با استفاده از دستور زیر، کد را در سطح اسمبلی تحلیل می‌کنیم. خروجی را در تصویر ۱۰-۶ مشاهده می‌نمایید که نشان می‌دهد دستور movzbl در پردازنده، مسئول ایجاد عدم برخورد در حافظه نهان است.

```
perf annotate    -i     perf.data       --symbol = bad_access_pattern
```



تصویر ۱۰-۶- تحلیل در سطح اسمبلی

۱۰-۶- کاربردهای تکمیلی

در ادامه، مجموعه‌ای تکمیلی از دستورات فرعی perf را مشاهده می‌نمایید که هر یک برای مقصودی متفاوت طراحی شده‌اند. در هر مورد، سوئیچ‌های مختلفی نیز معرفی شده است که به کمک آن‌ها می‌توان خروجی را متناسب با نیاز تنظیم نمود.

مشاهده آمار عملکرد پردازنده، برای تمام رویدادهای سخت‌افزاری و نرم‌افزاری در سطح پردازنده

در خروجی دستور زیر، گزارش مفصلی از کل سیستم که طی پنج ثانیه جمع‌آوری شده است، نمایش داده خواهد شد. اگر عبارت sleep 5 را حذف کنید، سیستم تازمانی‌که با فشردن CTRL+C به کارش خاتمه ندهید، به عملیات اندازه‌گیری ادامه خواهد داد.

```
sudo   perf   stat   -a   sleep   5
```

بررسی آمار عملکرد پردازنده برای یک دستور خاص

با کمک این دستور می‌توان آمار عملکرد پردازنده را تنها برای یک برنامه‌ی مشخص اندازه‌گیری نمود، نه برای کل سیستم. این کار به کاربر امکان می‌دهد تا مصرف منابع سخت‌افزاری یک برنامه‌ی خاص را به‌صورت دقیق و جداگانه بررسی نماید.

```
sudo   perf   stat   <command>
```

به‌عنوان مثال، برای بررسی دستور ls، به‌این‌صورت عمل می‌کنیم.

```
sudo   perf   stat   ls
```

کل زمان صرف شده برای اجرای یک دستور، با عنوان «زمان سپری شده[15]» در انتهای خروجی، نمایش داده خواهد شد.

مشاهده عملکرد پردازنده برای یک فرآیند خاص در حال اجرا

بدین‌منظور از سوئیچ p- استفاده کرده و پس از آن شناسه فرآیندرا ارائه می‌نماییم.

```
sudo   perf  -p  <PID>   sleep  5
```

شمارش فراخوانی‌های سیستمی هسته لینوکس بر اساس نوع رویداد

---

[15] Time Elapsed

با این دستور می‌توان تمام فراخوانی‌های سیستمی را که در طول یک بازه‌ی زمانی مشخص توسط برنامه‌های مختلف انجام می‌شود، بر اساس نوع آن‌ها شمارش نمود. این قابلیت برای تحلیل رفتار هسته، شناسایی فراخوانی‌های پرتکرار و عیب‌یابی عملکرد سیستم بسیار مفید است. در مثال زیر پس از پنج ثانیه، تمام فراخوانی‌های سیستمی و تعداد آن‌ها در خروجی نمایش داده خواهد شد.

```
sudo   perf   stat  -e  'syscalls:sys_enter_*'  -a   sleep  5
```

ثبت چرخه‌های پردازنده.

چرخه‌های پردازنده از جمله رویدادهای سخت‌افزاری هستند. برای ثبت این چرخه‌ها، از دستور فرعی record استفاده کنید و نام رویداد را با سوئیچ e- مشخص نمایید.

```
sudo  perf  record  -e  cycles  sleep  10
```

از دستور perf record می‌توان برای ثبت پروفایل کارایی یک فرآیند موجود نیز استفاده کرد. به این طریق می‌توان رفتار و ویژگی‌های کارایی یک فرآیند در حال اجرا را بدون نیاز به راه‌اندازی مجدد و یا ایجاد وقفه در آن، تجزیه و تحلیل نمود. ابتدا شناسه‌ی فرآیند را به دست آورده و سپس دستور زیر را اجرا می‌کنیم.

```
sudo  perf  record  -p  pid   sleep 5
```

دستور فرعی record، داده‌ها را در یک فایل به نام perf.data ذخیره می‌کند و پس از اتمام فرآیند ضبط، یک خروجی کوتاه شامل میزان پیشرفت دستور، اندازه فایل ایجاد شده و تعداد نمونه‌های داده موجود در آن را نشان می‌دهد.

```
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.001 MB perf.data(3612 samples> ]
```

مشاهده نتایج عملکرد در فایل perf.data

دستور فرعی report به خواندن فایل perf.data کمک می‌کند و تمام رویدادها و آمار جمع‌آوری‌شده را نمایش می‌دهد. برای خروج از نمایش گزارش، باید CTRL+C را فشار دهید.

```
sudo   perf   report
```

تغییر قالب خروجی

برای مشاهده خروجی نمونه در قالب خروجی استاندارد، دستور زیر را اجرا کنید.

```
sudo   perf   report  --stdio
```

تغییرات قابل اعمال بیشتر، عبارتند از: نمایش شماره نمونه برای هر رویداد با استفاده از n- و یا نمایش ستون‌های خاص با استفاده از <sort < column name--. به‌عنوان مثال با اجرای دستور زیر، در خروجی یک ستون برای شماره نمونه و اطلاعات دستور و نماد، اضافه می‌شود.

```
sudo   perf   report  -n  --sort  comm,symbol  --stdio
```

نمایش خروجی ردیابی[16] ثبت شده در perf.data با استفاده از دستور فرعی script

با استفاده از دستور فرعی script، جزئیات perf.data به ترتیب زمان در خروجی چاپ می‌شود. از این دستور، به‌عنوان داده پساپردازش استفاده می‌گردد.

```
sudo   perf   script
```

نمایش اطلاعات سربرگ[17].

برای اینکه علاوه بر تمامی رویدادهای perf.data، اطلاعات بیشتری از سربرگ ردیابی نیز نمایش داده شود، دستور زیر را اجرا کنید. در خروجی، اطلاعات سربرگ فایل نمایش داده می‌شود. از جمله، زمان شروع ردیابی، مدت زمانی که طول کشیده، اطلاعات پردازنده و همچنین دستوری که داده‌ها را واکشی کرده است. پس از اطلاعات سربرگ نیز، لیست رویدادها قرار می‌گیرند.

```
sudo  perf  script  --header
```

---

[16] Trace
[17] Header

استخراج داده‌های خام

برای تخلیه داده‌های خام از فایل perf.data در قالب hex، از سوئیچ D- استفاده کنید. آنچه در خروجی خواهید دید شامل اطلاعات خام ردیابی رویداد در قالب ASCII است. این سوئیچ برای اشکال‌زدایی رویدادها بسیار مفید می‌باشد.

```
sudo perf script -D
```

حاشیه‌نویسی[18] داده‌ها و جداسازی بیشتر با استفاده از دستور فرعی annotate

سوئیچ v- همراه با این دستور، خروجی دقیقی را در اختیارتان قرار می‌دهد که در آن، کدمنبع و جداسازی[19] رویدادها را خواهید دید.

```
sudo  perf  annotate  --stdio  -v
```

مشاهده مستندات

برای مشاهده مستندات کامل ابزار perf، از دستور man استفاده کنید.

```
man   perf
```

---

[18] Annotate
[19] Disassembly

تمرین

ابزار Perf را روی توزیع لینوکسی خود نصب کنید. با استفاده از آن، پروفایل پردازنده سیستم خود را - هم از لحاظ آمار کلی و هم به‌صورت بلادرنگ - بررسی و تحلیل نمایید.

گزارش کار

یک برنامه ساده بنویسید که در یک حلقه با تعداد تکرار زیاد (مثلا ده هزار بار) بچرخد. می‌توانید دستوراتی مثل چاپ در هر صد تکرار، یا دستورات متناسب دیگری را نیز در آن اضافه نمایید.

- برنامه را همراه با سوئیچ‌های مناسب برای «اطلاعات دیباگ» و « بدون بهینه‌سازی» کامپایل کرده و آماده اجرا نمایید.
- با کمک perf record، برنامه را اجرا و عملکردش را ضبط کنید.
- سپس با استفاده از perf report، زمان صرف شده در بخش‌های مختلف کد را نمایش داده و بررسی نمایید کدام بخش از کد، بیشترین زمان اجرا را به خود اختصاص داده است؟
- چرا printf با اینکه فقط در 1% از تکرارها اجرا می‌شود، ممکن است بیشترین سربار را داشته باشد؟
- با استفاده از perf stat، آمار کلی cycles و instructions را مشاهده و مقایسه نمایید. آیا تفاوت معناداری در این دو مشاهده می‌کنید؟ این تفاوت چه چیزی را نشان می‌دهد؟

منابع و مراجع:

- Gregg B. *BPF Performance Tools*, Addison-Wesley Professional, 2019.
- https://perf.wiki.kernel.org/index.php/Tutorial, visited 2024 Aug.
- https://www.brendangregg.com/linuxperf.html, visited 2024 Aug.
- https://thelinuxcode.com/linux-perf-commands, visited 2024 Aug.

آزمایش ۱۱ : کامپایل هسته لینوکس

اهداف:

- آشنایی با انواع هسته
- کامپایل دستی هسته لینوکس

ابزار و مفاهیم مورد نیاز:

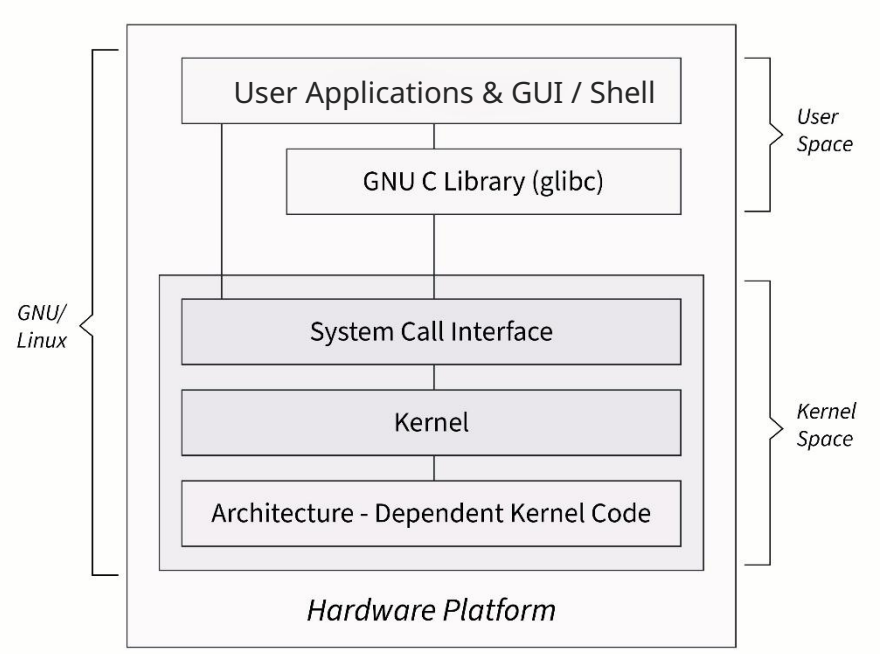
- سیستم‌عامل لینوکس
- کاربر ریشه یا sudoer
- کدمنبع هسته لینوکس
- ابزارهای کامپایل مانند gcc و make
- حداقل ۱۲ گیگابایت فضای آزاد روی دیسک

محتوا:

- معرفی هسته لینوکس
  - مفاهیم monolithic، microkernel و modular
- تنظیمات هسته با ابزار make menuconfig
- کامپایل و نصب هسته جدید
- به‌روزرسانی خودکار هسته

۱۱-۱- هسته لینوکس

همان‌طور که در تصویر ۱۱-۱ نیز مشاهده می‌کنید، هسته لینوکس جزء مرکزی سیستم‌عامل گنو/لینوکس است که مسئولیت مدیریت منابع سخت‌افزاری و ارائه خدمات اساسی به برنامه‌های کاربردی سطح کاربر را دارد. یکی از مهم‌ترین خدماتی که توسط هسته ارائه می‌گردد، مدیریت فرآیندها می‌باشد. هسته لینوکس، ایجاد، زمان‌بندی و خاتمه فرآیندها را مدیریت کرده و استفاده کارآمد از منابع پردازنده را تضمین می‌نماید.



تصویر ۱۱-۱- معماری لایه‌ای سیستم‌عامل

هسته لینوکس، از الگوریتم‌های زمان‌بندی مختلفی برای مدیریت اجرای فرآیندها استفاده می‌کند. اما در توزیع‌های مدرن، معمولا زمان‌بندی کاملا منصفانه[1](CFS)، الگوریتم پیش‌فرض می‌باشد. هدف این الگوریتم، تخصیص متعادل و

[1] Completely Fair Scheduler

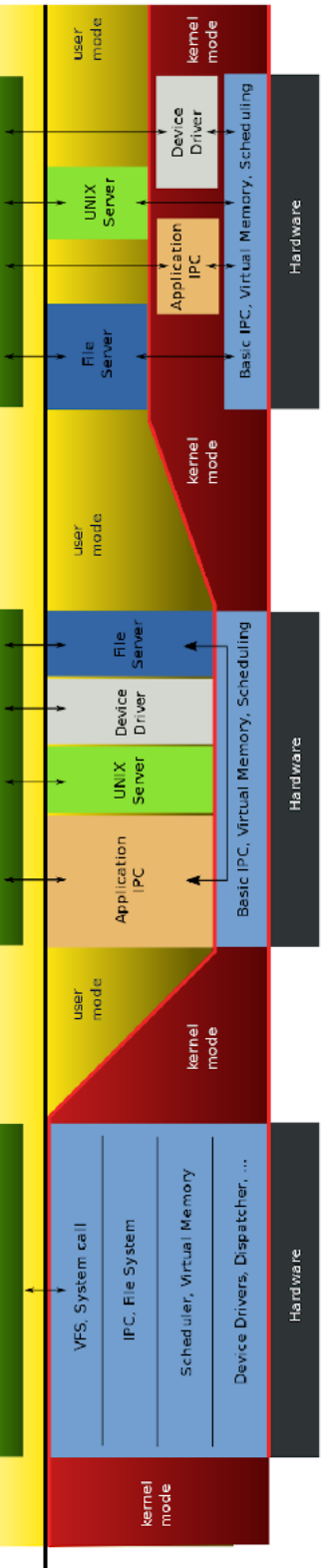
منصفانه زمان پردازنده به فرآیندها، بهبود پاسخگویی و کارایی کلی سیستم است. علاوه بر این، هسته لینوکس از چند وظیفه‌ای و چند رشته‌ای پشتیبانی می‌کند و به چندین فرآیند و رشته اجازه می‌دهد تا به‌طور همزمان اجرا شوند.

با توجه به استفاده گسترده از لینوکس در سیستم‌های حیاتی، امنیت یک نگرانی اساسی برای هسته محسوب می‌شود. هسته دارای چندین مکانیسم امنیتی برای محافظت در برابر آسیب‌پذیری‌ها و حملات است. از جمله، چارچوب‌های کنترل دسترسی مانند SELinux[2]، که کنترل‌های اجباری دسترسی را اعمال می‌کند و همچنین AppArmor که امنیت در سطح برنامه‌ها را فراهم می‌نماید. به‌علاوه، هسته از الگوریتم‌های رمزنگاری مختلف و پروتکل‌های ارتباطی امن برای محافظت از یکپارچگی و محرمانه بودن داده‌ها پشتیبانی می‌کند. با این وجود، همیشه ممیزی[3] امنیتی و به‌روزرسانی مداوم برای مقابله با تهدیدات نوظهور و حفظ انعطاف‌پذیری هسته، امری ضروری است.

در ابتدا، هسته لینوکس به‌عنوان یک هسته یکپارچه[4] طراحی شد، به این معنی که تمام عملکردهای اصلی، از جمله درایور دستگاه‌ها، فایل‌سیستم و پروتکل‌های شبکه، در یک فایل اجرایی بزرگ که در مد هسته اجرا می‌شد، ادغام شده بودند. این طراحی مزیت بهره‌وری و کارایی را فراهم می‌کرد، زیرا همه اجزا می‌توانستند مستقیما در یک فضای آدرس یکسان، با هم ارتباط برقرار کنند و بنابراین، هزینه‌های سربار مرتبط با ارتباطات بین فرآیندی کاهش می‌یافت. اما در این حالت، هسته پیچیده‌تر و نگهداری از آن سخت‌تر می‌شد، زیرا هر تغییری مستلزم کامپایل مجدد کل هسته بود. با گذشت زمان، نیاز به انعطاف‌پذیری بیشتر و ماژولار بودن، منجر به معرفی ماژول‌های هسته قابل بارگیری شد. به‌طوری‌که امروزه یکی از نقاط قوت اصلی هسته لینوکس، معماری ماژولار آن است که امکان بارگذاری و حذف پویای ماژول‌های هسته را فراهم می‌کند. در تصویر ۱۱-۲ می‌توانید مراحل گذار از حالت یکپارچه به ماژولار را ببینید.

---

[2] Security-Enhanced Linux
[3] Audit
[4] Monolithic



تصویر ۱۱-۲- مراحل گذار هسته لینوکس از یکپارچه به ماژولار

۱۱-۲- مزایای هسته ماژولار

به‌طور کلی انگیزه‌های تغییر به سمت هسته ماژولار و مزایای حاصل از آن را می‌توان به‌شرح زیر دانست:

- **انعطاف‌پذیری بیشتر**: ماژولار بودن کاربران را قادر می‌سازد بدون کامپایل مجدد کل هسته، ویژگی‌هایی[5] را حذف یا اضافه کنند. ماژول هسته می‌تواند شامل درایورهای دستگاه[6]، فایل‌سیستم و پروتکل‌های شبکه و سایر اجزا باشد.
- **کارایی بهبودیافته**: هسته ماژولار در هر زمان تنها به ماژول‌های لازم اجازه بارگذاری می‌دهد و به این ترتیب، سیستم می‌تواند استفاده از منابع را بهینه کرده و کارایی را بهبود بخشد، به خصوص در سیستم‌هایی با منابع سخت‌افزاری محدود.
- **مقیاس پذیری**: هسته‌های ماژولار به راحتی می‌توانند مقیاس شوند تا طیف وسیعی از دستگاه‌ها و معماری‌ها را - از سرورهای با کارایی بالا گرفته تا سیستم‌های توکار - پشتیبانی نمایند.
- **مشارکت جامعه**: طراحی ماژولار، مشارکت جامعه‌ی بازمتن را تسهیل کرده است، زیرا توسعه‌دهندگان می‌توانند بدون نیاز به درک کل هسته، فقط بر روی ماژول‌های خاصی تمرکز کنند.
- **قابلیت نگهداری بهتر هسته**: با جداسازی عملکردهای مختلف در ماژول‌های جداگانه، توسعه‌دهندگان می‌توانند بر روی اجزای جداگانه کار کنند بدون اینکه کل سیستم را تحت تاثیر قرار دهند. بنابراین اشکال‌زدایی و به‌روزرسانی ساده‌تر می‌شود.
- **افزایش امنیت**: هسته‌های ماژولار می‌توانند مقدار کدهای در حال اجرا در فضای هسته را به حداقل برسانند، به این ترتیب سطح حملات کاهش یافته و امنیت سیستم افزایش می‌یابد.

---

[5] Features
[6] Device Drivers

انتقال از هسته یکپارچه به یک هسته ماژولار در لینوکس، تدریجی و شامل چندین پیشرفت کلیدی بود:

- **ماژول‌های هسته قابل بارگیری** [۷]: که در اوایل دهه ۱۹۹۰ معرفی شدند، اجازه بارگذاری و حذف بخش‌هایی از هسته را در حین اجرا می‌دادند. این نوآوری امکان افزودن قابلیت‌های جدید مانند درایورهای دستگاه را بدون راه‌اندازی مجدد سیستم فراهم کرد.
- **ابزارهای پیکربندی هسته**: ابزارهایی مانند make menuconfig و make xconfig رابط‌های کاربرپسندی را برای پیکربندی هسته ارائه دادند که به کاربران امکان می‌داد بر اساس نیاز خود انتخاب کنند که کدام ماژول‌ها، اضافه یا حذف شوند.
- **بهبود مدیریت ماژول‌ها**: با گذشت زمان، هسته لینوکس مکانیسم‌های بهتری را برای مدیریت وابستگی بین ماژول‌ها در خود گنجانده است و اطمینان حاصل می‌کند که ماژول‌ها می‌توانند با خیال راحت و به‌طور کارآمد بارگیری و حذف گردند.

۱۱-۳- چالش‌های هسته ماژولار

با وجود مزایای ذکر شده، رویکرد ماژولار چالش‌هایی را نیز به همراه دارد که عبارتند از:

- **پیچیدگی**: مدیریت وابستگی‌ها و اطمینان از سازگاری بین ماژول‌ها می‌تواند پیچیده بوده و به ابزارها و فرآیندهای قوی نیاز داشته باشد.
- **سربار کارایی**: در حالی که ماژولار بودن می‌تواند کارایی را افزایش دهد، به دلیل نیاز به ارتباط بین ماژول‌ها، می‌تواند سربار نیز ایجاد کند.
- **خطرات امنیتی**: بارگذاری پویای ماژول‌ها، به طور بالقوه و در صورت عدم مدیریت صحیح، می‌تواند باعث ایجاد آسیب‌پذیری‌های امنیتی شود.

---

[7] Loadable Kernel Modules (LKM)

در مجموع می‌توان گفت که هسته لینوکس پایه‌ای قدرتمند و انعطاف‌پذیر برای طیف گسترده‌ای از برنامه‌ها فراهم آورده است. تکامل آن از یک معماری یکپارچه به معماری ماژولار، انعطاف‌پذیری و قابلیت نگهداری آن را به مقدار زیادی افزایش داده است. معماری ماژولار، همراه با مدیریت کارآمد فرآیندها و ویژگی‌های امنیتی قوی، هسته لینوکس را به گزینه‌ای ارجح برای توسعه‌دهندگان تبدیل کرده است. ماهیت مشارکتی توسعه‌ی آن نیز تضمین می‌کند که هسته به تکامل خود ادامه داده، با فناوری‌های جدید سازگار شده و به نیازهای محاسباتی که بطور دائم در حال تغییرند، پاسخ بدهد.

۱۱-۴- کامپایل و آپدیت دستی هسته

۱. **دانلود کدمنبع**. از وب‌سایت رسمی کرنل[8] که در تصویر ۱۱-۳ نمایش داده شده است، آخرین نسخه هسته را دانلود کنید. توجه داشته باشید که باید شماره نسخه موجود در دستورکار را با آخرین نسخه موجود در سایت جایگزین نمایید. فایل دانلود شده حاوی کدمنبع فشرده است.

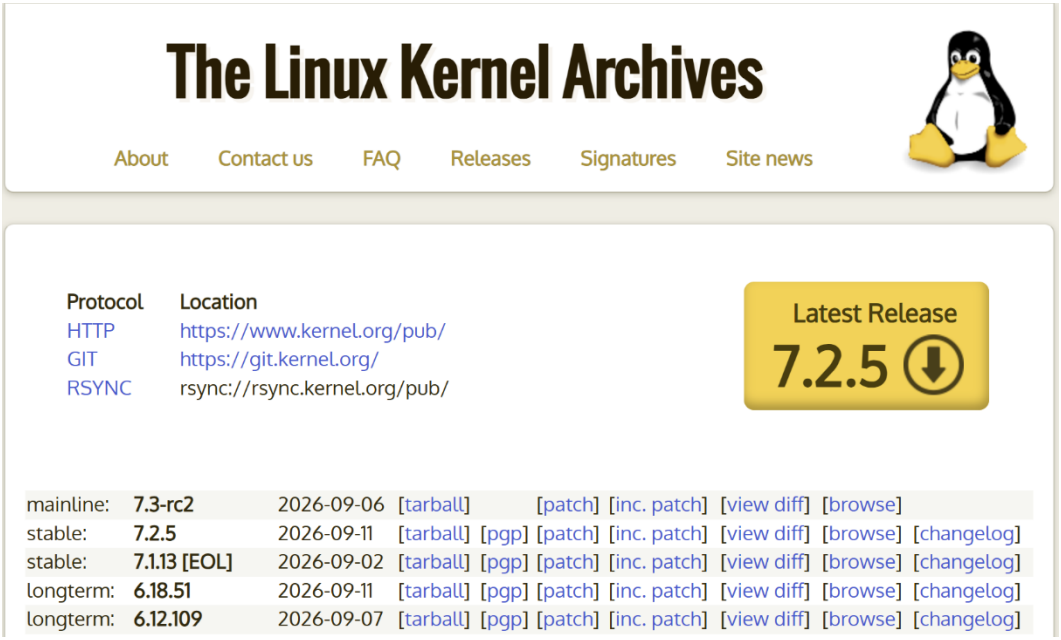
از طریق ترمینال و اجرای دستور wget نیز می‌توانید کدمنبع هسته لینوکس را دانلود نمایید. فقط توجه داشته باشید که نام و شماره نسخه را به درستی وارد کنید.

```
wget    https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-
7.2.5.tar.xz
```

پس از اتمام دانلود، پیام saved در خروجی نمایش داده می‌شود.

---

[8] https://www.kernel.org/



تصویر ۱۱-۳- وب‌سایت هسته

۲. **خروج ار حالت فشرده.** وقتی فایل دانلود شد، از دستور tar برای استخراج کدمنبع استفاده نمایید. فایل‌های کدمنبع استخراج شده، در خروجی نمایش داده می‌شوند.

```
tar   xvf   linux-7.2.5.tar.xz
```

۳. **نصب بسته‌های ضروری.** قبل از شروع به ساخت هسته، باید بسته‌های مورد نیاز را نصب کنید.

```
sudo apt-get   install   git   fakeroot   build-essential
ncurses-dev   xz-utils   libssl-dev   bc   flex   libelf-dev   bison
```

اجرای این دستور، بسته‌های زیر را نصب خواهد کرد.

git: در صورت نیاز به ایجاد نمونه همسان[9]، از این دستور برای دریافت کدمنبع هسته از مخازن Git، استفاده می‌شود. این دستور، امکان رهگیری و ثبت تمام تغییرات کدمنبع در حین توسعه و همچنین امکان برگرداندن تغییرات را فراهم می‌کند.

fakeroot: یک محیط ریشه جعلی ایجاد می‌کند که هنگام ساخت بسته‌های deb. نهایی برای نصب هسته جدید، دسترسی ریشه را شبیه سازی نماید. بنابراین نیازی به دسترسی واقعی ریشه وجود نخواهد داشت.

build-essential: ابزارهای توسعه شامل کامپایلر gcc، g++، make و سایر ابزارهای پایه برای ساخت برنامه‌های C و C++ را نصب می‌کند.

ncurses-dev: این دستور برای ترمینال‌های متنی، API ارائه می‌دهد. در اینجا نیز استفاده از رابط گرافیکی menuconfig را برای تنظیم گزینه‌های هسته در ترمینال، میسر می‌سازد. درصورتی‌که قصد استفاده از menuconfig را – که در مرحله ۴ بدان اشاره شده است – ندارید، می‌توانید از نصب این بسته صرف‌نظر نمایید.

xz-utils: فشرده‌سازی و خروج از حالت فشرده سریع را برای فایل‌های tar.xz. فراهم می‌کند. این فرمت، قالب پیش‌فرض فشرده‌سازی هسته می‌باشد.

libssl-dev: از SSL و TLS پشتیبانی می‌کند که داده‌ها را رمزگذاری کرده و اتصال اینترنت را ایمن می‌کنند. همچنین برای ماژول‌های هسته امکان ایجاد امضای دیجیتال را فراهم می‌نماید.

bc (Basic Calculator): برای محاسبات ریاضی در اسکریپت‌های Kbuild هنگام ساخت هسته مورد نیاز است.

flex (Fast Lexical Analyzer Generator) و bison: تحلیل‌گرهای واژگانی لغوی و نحوی[10] مورد نیاز در کامپایل هسته را تولید می‌کنند که کاراکترها را به توکن تبدیل می‌نمایند.

libelf-dev: یک کتابخانه مشترک برای مدیریت و پردازش فایل‌های ELF (فایل‌های اجرایی، فایل‌های خالی هسته و کد اشیا) ایجاد می‌کند که در هنگام کامپایل هسته و همچنین لینک کردن ماژول‌ها مورد نیازند.

---

[9] Clone
[10] Lexer & Parser

٤. **پیکربندی.** کدمنبع هسته لینوکس با پیکربندی پیش‌فرض ارائه می‌شود. اما می‌توان آن را بر اساس نیازهای خود نیز تنظیم کرد. بدین منظور، ابتدا به دایرکتوری ایجاد شده در مرحله ۲ بروید. (در دستور زیر شماره نسخه را بر اساس نسخه دانلودی خودتان تصحیح کنید)

```
cd    linux-7.2.5
```

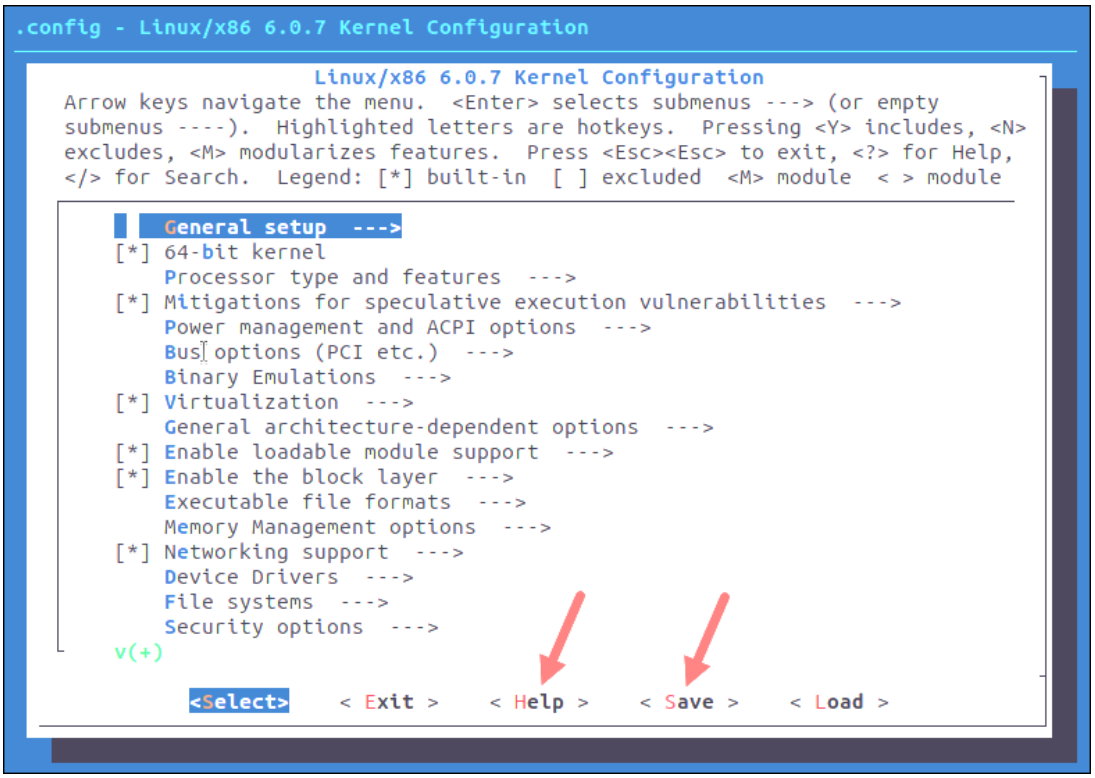
یک کپی از فایل پیکربندی هسته فعلی لینوکس تهیه نمایید.

```
cp     -v    /boot/config-$(uname -r)   .config
```

برای ایجاد تغییرات در فایل پیکربندی، دستور زیر را اجرا کنید.

```
make  menuconfig
```

این فرمان چندین اسکریپت را اجرا و در نهایت منوی پیکربندی را باز می‌کند. منو، شامل گزینه‌هایی مانند تنظیمات سیستم‌عامل، فایل‌سیستم، شبکه و حافظه است. از «کلیدهای جهت» در کیبورد، برای حرکت روی گزینه‌ها استفاده کنید. همچنین می‌توانید با انتخاب راهنما، اطلاعات بیشتری درباره گزینه‌های موجود به دست بیاورید. پس از اتمام تغییرات، ذخیره را انتخاب کرده و از منو خارج شوید.



تصویر ۱۱-۴- منوی تنظیمات

توجه: تغییر تنظیمات برای برخی از گزینه‌ها می‌تواند منجر به یک هسته‌ی غیر کاربردی شود. بنابراین اگر مطمئن نیستید که چه چیزی را تغییر می‌دهید، تنظیمات را به حالت پیش‌فرض رها کنید.

۵. **کامپایل کدمنبع**. با اجرای دستور زیر شروع به ساخت هسته کنید. فرآیند کامپایل و ساخت کامل هسته لینوکس مدتی طول می‌کشد. در این حین، ترمینال تمام اجزای هسته لینوکس را فهرست می‌کند. مانند: مدیریت حافظه، درایورهای دستگاه سخت‌افزار، درایورهای فایل‌سیستم، درایورهای شبکه و مدیریت فرآیند.

```
make
```

نکته: اگر در حال کامپایل هسته در اوبونتو هستید، ممکن است خطای زیر را دریافت کنید که فرآیند ساخت را مختل می‌کند.

```
No rule to make target 'debian/canonical-certs.pem
```

در صورت مواجهه با چنین خطایی، با اجرای دو دستور زیر، گواهینامه‌های امنیتی را که منجر به ایجاد تداخل شده‌اند، غیرفعال نمایید. سپس فرآیند ساخت را دوباره با make شروع کرده و برای تایید گزینه‌های پیش‌فرض برای تولید گواهی‌های جدید، مکررا Enter را فشار دهید.

```
scripts/config --disable SYSTEM_TRUSTED_KEYS
scripts/config --disable SYSTEM_REVOCATION_KEYS
```

نکته: اجرای دستورات مذکور هیچ خروجی ندارد.

پس از اتمام ساخت هسته، ماژول‌های مورد نیاز را با این دستور نصب کنید.

```
sudo  make  modules_install
```

٦. **نصب.** در نهایت، هسته را با دستور زیر نصب کنید. پس از اتمام کار، عبارت done در خروجی نمایش داده خواهد شد.

```
sudo  make  install
```

٧. **به‌روزرسانی بوت‌لودر.** انجام این مرحله اختیاری می‌باشد و به دو روش قابل انجام است.

به‌روزرسانی خودکار، که به‌سادگی و فقط با یک دستور قابل انجام است.

```
make   install
```

به‌روزرسانی دستی، که در طی دو مرحله، به این شکل انجام می‌شود: ابتدا initramfs را به نسخه‌ی هسته نصب شده، به‌روز کنید.

```
sudo   update-initramfs  -c  -k  7.2.5
```

سپس، بوت لودر GRUB را با دستور زیر به‌روزرسانی نمایید. در ترمینال، مراحل کار و پیام تایید چاپ خواهد شد.

```
sudo   update-grub
```

۸. **راه‌اندازی مجدد**. پس از تکمیل مراحل بالا، دستگاه را مجددا راه‌اندازی کرده و پس از بوت شدن سیستم، نسخه هسته را با استفاده از دستور uname تایید کنید. باید شماره نسخه‌ی به‌روزرسانی شده‌ی هسته، در ترمینال چاپ شود.

```
uname   -mrs
```

۱۱-۵- آپدیت خودکار هسته لینوکس

روش اول: استفاده از فرآیند آپدیت سیستم

فرآیند به‌روزرسانی سیستم، راهی ساده برای به‌روزرسانی همه بسته‌ها، از جمله هسته لینوکس، به آخرین نسخه‌ی موجود است. قبل از شروع، نسخه هسته فعلی را با دستور uname بررسی کنید.

```
uname   -r
```

خروجی این دستور، عبارتی مشابه 6.2.0-37-generic خواهد بود که در آن، بخش 6.2.0 به شماره نسخه هسته و 37 به شماره انتشار یا ساخت خاص هسته، اشاره دارد. عبارت genertic نیز نشان دهنده نوع هسته است که یک انتخاب رایج برای سیستم‌های دسکتاپ و سرورهای معمولی می‌باشد.

اکنون، با استفاده از apt فهرست بسته‌های محلی را به‌روزرسانی کنید. این دستور اطلاعات مربوط به آخرین نسخه‌ی بسته‌های موجود را از مخازن تنظیم‌شده روی سیستم، دریافت می‌کند.

```
sudo   apt   update
```

اگر نسخه هسته جدیدتری وجود داشته باشد، این دستور آن را پیدا کرده و برای دانلود و نصب علامت گذاری می‌کند. با این حال، این دستور به خودی خود هیچ بسته‌ای را نصب یا ارتقا نمی‌دهد.

برای ارتقاء بسته‌های نصب شده از جمله هسته، به آخرین نسخه‌های موجود در مخازن، دستور زیر را اجرا کنید.

```
sudo   apt   upgrade
```

این دستور، همه برنامه‌های سیستم را که امکان به‌روزرسانی دارند لیست خواهد کرد. در این لیست، عباراتی همچون linux-image و یا Linux-modules به بسته‌های مرتبط با هسته اشاره دارند. در صورت تایید شما، سیستم به‌روزرسانی شده و آخرین نسخه هسته لینوکس نصب می‌شود. تغییرات حاصل را با دستور uname -r بررسی کنید. البته اگر از قبل، هسته در آخرین نسخه خود بوده باشد، خروجی دستور نسبت به حالت قبل از ارتقا، تغییری نخواهد کرد.

نکته: اگر به غیر از به‌روزرسانی هسته، می‌خواهید سیستم خود را نیز به نسخه جدید اوبونتو ارتقا بدهید (به‌عنوان مثال، از اوبونتو ۲۰.۰۴ به ۲۲.۰۴)، از دستور زیر استفاده کنید. این دستور تغییرات در پیش‌نیازها و ارتقاهای عمده سیستم را نیز مدیریت می‌کند.

```
sudo   apt-get   dist-upgrade
```

روش دوم: استفاده از ابزار Mainline

برای ارتقاء به جدیدترین نسخه از هسته (حتی نسخه بتا یا تست نشده که خطرات احتمالی نیز به همراه دارد)، روش دیگری وجود دارد که از Mainline استفاده می‌کند.

این ابزار کاربرپسند، به‌روزرسانی هسته لینوکس را ساده‌تر می‌کند. اما در اکثر سیستم‌های لینوکس از پیش نصب نشده است. بنابراین اگر Mainline را روی سیستم خود ندارید، ابتدا به‌شکل زیر، PPA نگهداری شده توسط Cappelikan را به فهرست منابع یا مخازن نرم‌افزاری خود اضافه کنید.

```
sudo   add-apt-repository   ppa:cappelikan/ppa
```

این PPA توسط کاربر cappelikan نگهداری می‌شود و حاوی نصاب هسته Mainline است. افزودن این PPA باعث می‌شود که نرم‌افزار Mainline برای نصب در سیستم اوبونتو در دسترس باشد.

سپس لیست بسته‌ها و مخازن را با استفاده از دستور زیر، به‌روزرسانی نمایید.

```
sudo   apt   update
```

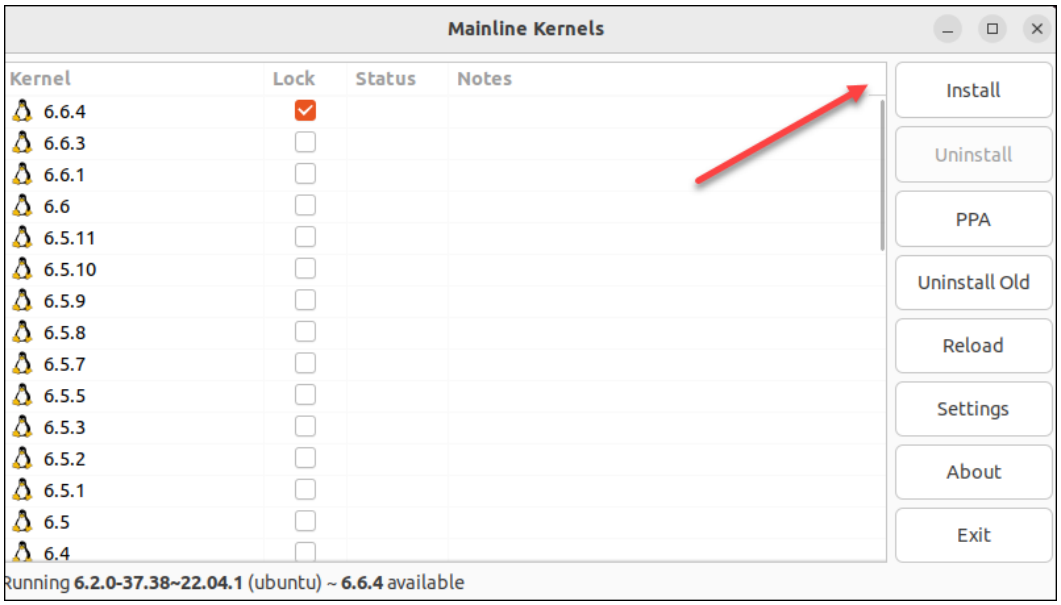
و در انتها، Mainline را نصب کنید.

```
sudo   apt   install   mainline
```

حال با دستور زیر، Mainline را اجرا کنید تا رابط برنامه نمایش داده شود. با زدن سوپرکلید (کلید ویندوز) و جستجوی Mainline نیز می‌توانید به برنامه دسترسی پیدا کنید. داخل برنامه، نسخه‌های موجود هسته لینوکس نمایش داده می‌شوند.

```
sudo   mainline
```

حال برای دریافت یک هسته خاص، آن را در لیست پیدا کرده و انتخاب کنید. سپس روی دکمه Install در سمت راست‌کلیک کرده و پس از اتمام نصب، سیستم را راه‌اندازی مجدد نمایید.

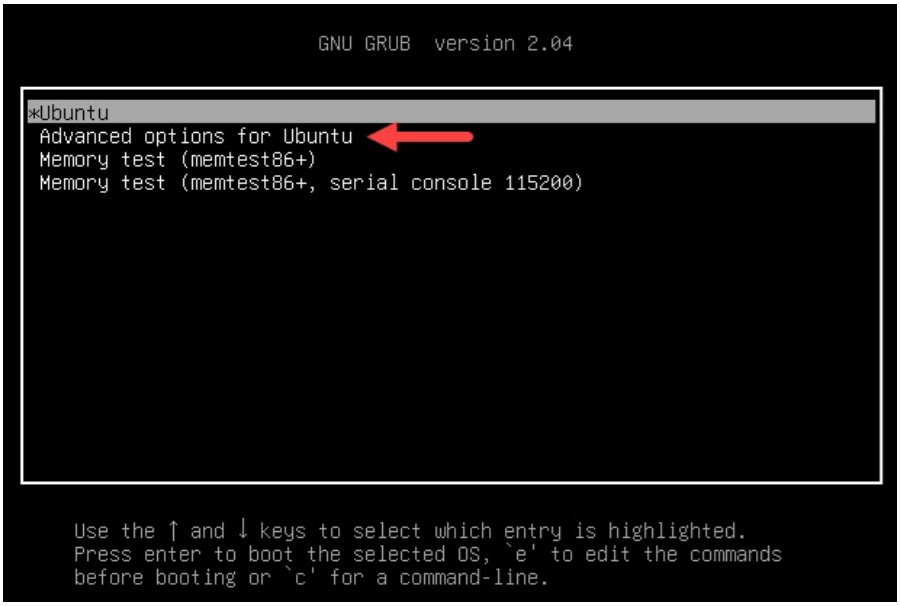


تصویر ۱۱-۵- نصب هسته با استفاده از Mainline

۱۱-۶-بازگشت به هسته قبلی و حذف هسته‌های اضافه

در صورت بروز مشکل می‌توانید به شکل زیر، نسخه قبلی هسته را از منوی بوت انتخاب کنید تا سیستم با آن بالا بیاید.

- ابتدا سیستم را راه‌اندازی مجدد کنید و بسته به توزیع لینوکس، کلید Shift یا Esc را نگه دارید.
- سپس گزینه Advanced options for Ubuntu را انتخاب کنید.
- سپس هسته قبلی را (که با شماره نسخه مشخص شده است) انتخاب نمایید.



تصویر ۱۱-۶- نمایش هسته‌های سیستم در منوی بوت

برای پاک کردن هسته‌های قدیمی و آزادسازی فضای دیسک نیز می‌توان از همان Mainline استفاده کرد. کافی است روی نسخه‌های هسته اضافی، کلیک کرده و کلید Uninstall را در سمت راست بزنید.

تمرین

توصیه می‌شود برای این تمرین و گزارش کار، از ماشین‌مجازی استفاده نمایید تا در صورت بروز مشکل، سیستم اصلی شما آسیب نبیند. درصورتی‌که کلا از ماشین‌مجازی استفاده می‌کنید، با مراجعه به انتهای بخش ۱-۵-۳، از ماشین خود نسخه پشتیبان تهیه کرده و سپس اقدام به انجام تمرینات نمایید.

- نسخه فعلی هسته سیستم خود را از طریق خط فرمان و فایل‌های سیستمی مربوطه به دست آورید. بررسی کنید که چه هسته‌هایی روی سیستم شما نصب شده و کدام یک در حال اجراست. فایل پیکربندی هسته فعلی که در مسیر /boot/config-$(uname -r) قرار دارد را یافته و بررسی کنید که آیا گزینه‌های ماژولار مانند CONFIG_MODULES در آن فعال هستند یا نه.
- با استفاده از ابزارهای مدیریت بسته در توزیع خود، آخرین نسخه هسته موجود در مخازن را نصب کرده و سیستم را راه‌اندازی مجدد نمایید. در منوی GRUB وجود هسته جدید و قدیمی را مشاهده کرده و پس از ورود به سیستم، با استفاده از uname -r صحت نصب را بررسی نمایید.

گزارش کار

- جدیدترین نسخه از کدمنبع هسته لینوکس را به‌صورت دستی کامپایل کرده و بر روی توزیع خود نصب نمایید. در طی مراحل نصب، با استفاده از make menuconfig یک تغییر ساده در پیکربندی پیش‌فرض ایجاد کنید. درباره تغییری که ایجاد کرده‌اید و تاثیر آن بر سیستم توضیح دهید.
- فرض کنید پس از کامپایل و نصب یک هسته جدید، سیستم با مشکل بوت مواجه شده و به‌درستی بالا نمی‌آید. با استفاده از منوی GRUB، به هسته قبلی بازگشته و با بررسی لاگ‌های سیستم در dmesg و /var/log/syslog و فایل پیکربندی هسته، خطاهای فرضی را شناسایی کنید. بررسی کنید که علل رایج در بروز مشکلات بوت، چه مواردی می‌باشد؟

منابع و مراجع:

- Kroah-Hartman G. *Linux Kernel in a Nutshell*, O'Reilly Media, 2006.
- Love R. *Linux Kernel Development*, Indianapolis, 2005.
- https://www.kernel.org, visited 2026 Sep.
- https://docs.kernel.org/6.8/admin-guide/quickly-build-trimmed-linux.html, visited 2024 Jul.

آزمایش ۱۲ : ماژول‌های هسته

اهداف:

- ایجاد، کامپایل و مدیریت ماژول‌های هسته در لینوکس

ابزار و مفاهیم مورد نیاز:

- سیستم‌عامل لینوکس نصب شده به‌صورت مجازی یا محلی
- ابزارهای خط فرمانی کامپایل
- کتابخانه‌های ضروری

محتوا:

- معرفی ماژول‌های هسته
- مزایای ماژول‌های شخصی بر ماژول‌های build-in
- ایجاد یک ماژول ساده
  - توابع آغاز و پایان
  - ایجاد makefile و کامپایل
  - بارگذاری و حذف ماژول از هسته
  - بررسی وضعیت بارگذاری ماژول و نتیجه اجرای آن

۱۲-۱- ماژول هسته چیست؟

ماژول‌ها، تکه‌کدهایی هستند که در حین اجرای لینوکس می‌توانند وارد هسته شده و یا از آن خارج گردند. این تکه‌کدها، عملکرد هسته را بدون نیاز به راه‌اندازی دوباره کامپیوتر توسعه می‌دهند. اگر ماژول‌ها وجود نداشتند، برای هر قابلیتی که می‌خواستیم به هسته اضافه کنیم یا از آن کم کنیم، می‌بایست کل هسته را یک بار کامپایل کرده و برای استفاده از ان قابلیت یا حذف آن، یک بار سیستم را از نو راه‌اندازی می‌کردیم.

به‌عنوان مثال یک نوع از ماژول‌ها device drivers هستند که به هسته، امکان استفاده از قابلیت سخت‌افزارها را می‌دهند. با شناسایی اتصال یک چاپگر یا فلش درایو به سیستم، هسته بلافاصله ماژول مربوطه را بارگذاری کرده و آن سخت‌افزار، بدون نیاز به راه‌اندازی مجدد سیستم، آماده استفاده خواهد بود. درصورتی که درایور از قبل نصب نباشد نیز، پس از نصب اولیه و باز هم بدون نیاز به راه‌اندازی مجدد، دستگاه قابل استفاده می‌شود.

۱۲-۲- ماژول‌های سیستمی

با اجرای دستور lsmod می‌توان ماژول‌هایی که هم اکنون در هسته حضور دارند و اطلاعات آن‌ها را مشاهده نمود. این دستور، اطلاعات خود را از فایل /proc/modules دریافت می‌کند.

هنگامی که هسته، به قابلیت یا عملکردی نیاز داشته باشد که هم اکنون درون هسته قرار ندارد، با کمک یکی از مکانیزم‌های خود به نام kmod، دستور modprobe را در فضای کاربر اجرا می‌کند تا ماژولی که عملکرد مدنظر را دارد، وارد هسته گردد. هنگامی که modprobe اجرا شود، یک رشته کاراکتر به یکی از دو صورت زیر دریافت می‌کند:

- نام ماژول: مثلا softdog یا ppp
- یک مشخصه کلی: مانند char-major-10-30

اگر حالت اول به modprobe داده شود، این دستور به‌دنبال فایلی به نام softdog.ko یا ppp.ko می‌گردد. ولی اگر حالت دوم داده شود، این دستور ابتدا به‌دنبال رشته ذکر شده در فایل‌های پیکربندی موجود در مسیر

/etc/modprobe.d می‌گردد و اگر توانست نام مستعاری[1] مانند softdog برای مشخصه char-major-10-30 پیدا کند، متوجه خواهد شد که این مشخصه به ماژول softdog اشاره دارد و بنابراین، نام فایل اجرایی مربوطه‌اش softdog.ko می‌باشد.

در مرحله بعد، modprobe فایل /lib/modules/version/modules.dep.bin را باز می‌کند. این فایل به وسیله دستور depmod -a ایجاد می‌شود و حاوی وابستگی بین ماژول‌هاست. سپس در آن فایل به‌دنبال ماژول‌هایی می‌گردد که باید قبل از ماژول موردنظر به هسته اضافه شوند. این ماژول‌ها پیش‌نیازهای[2] ماژول موردنظر می‌باشند.

به‌عنوان مثال اگر به‌دنبال ماژول msdos.ko در این فایل بگردید، خواهید دید که به ماژول دیگری به نام fat.ko وابسته است؛ یعنی برای اینکه msdos.ko وارد هسته شود حتما باید قبل از آن، fat.ko وارد شده باشد. این مساله برای fat.ko نیز تکرار می‌شود تا به مرحله‌ای برسیم که دیگر وابستگی‌ای وجود نداشته باشد. در نهایت، modprobe دستور insmod را به‌کار می‌برد تا ابتدا وابستگی‌ها و سپس ماژول موردنظر را به هسته وارد نماید.

بنابراین modprobe وظیفه پیدا کردن ماژول، تعیین وابستگی‌های آن و وارد کردن آن به هسته به وسیله صدا کردن insmod را دارد، در حالی که insmod فقط وظیفه وارد کردن ماژول‌ها به هسته را دارد.

اگر بخواهیم به‌صورت دستی msdos.ko را وارد هسته کنیم، با دسترسی روت یا با یک کاربر sudoer، به‌صورت زیر ابتدا وابستگی‌ها و سپس ماژول موردنظر را در هسته بارگذاری می‌نماییم.

```
insmod     /lib/modules/2.6.11/kernel/fs/fat/fat.ko
insmod     /lib/modules/2.6.11/kernel/fs/fat/msdos.ko
```

معادل دو دستور بالا با modprobe، به‌صورت زیر خواهد بود.

```
modprobe msdos
```

---

[1] Alias

[2] Dependency

توجه کنید که insmod مسیر کامل تا فایل ماژول را می‌خواهد، در حالی که modprobe فقط نام ماژول را می‌گیرد.

نکات:

۱) **نسخه‌بندی ماژول**: هسته به‌طور پیش‌فرض ماژول‌هایی را که برای نسخه‌ی دیگری کامپایل شده باشند، بارگذاری نمی‌کند؛ مگر اینکه مکانیزم CONFIG_MODVERSIONS فعال شود. اما استفاده از این قابلیت توصیه نمی‌شود، زیرا می‌تواند منجر به ناپایداری سیستم گردد.

۲) **نمایش اطلاعات توسط ماژول**: ماژول‌ها نمی‌توانند چیزی به غیر از پیام‌های خطا و هشدار را بر روی صفحه، نمایش دهند. بلکه برای نشان دادن اطلاعات خود، آن‌ها را در فایل‌های لاگ می‌نویسند.

۳) **مشاهده‌ی اطلاعات ماژول**: برای مشاهده‌ی اطلاعات یک ماژول مانند نام، توصیف، وابستگی‌ها و پارامترها، از دستور modinfo استفاده می‌شود. برای ماژول‌های سیستمی تنها ذکر نام ماژول پس از دستور کافی است. اما برای ماژول‌های کاربر، باید نام و مسیر کامل ماژول با پسوند ko. در اختیار دستور قرار داده شود.

۴) **مشکلات احتمالی**: برای جلوگیری از مشکلات ناشی از فایل‌های سرآیند[3] ناقص برای هسته، حتما پیش از شروع ماژول نویسی، بسته linux-headers-$(uname -r) را نصب کنید.

۱۲-۳- ماژول‌های کاربر

در بخش قبل، با ماژول‌های سیستمی آشنا شدیم؛ ماژول‌هایی که از ابتدا همراه با هسته عرضه می‌شوند و برای مدیریت سخت‌افزار و قابلیت‌های اساسی سیستم طراحی شده‌اند. همان‌طور که از دستورکارهای پیشین به یاد دارید؛ به دلیل انتشار آزاد، امکان توسعه‌ی هسته لینوکس توسط کاربران و جامعه‌ی مصرف‌کننده وجود دارد. بنابراین، یک برنامه‌نویس

---

[3] Header

می‌تواند با نوشتن ماژول‌های سفارشی و شخصی خود، بدون اینکه تغییری در اساس هسته ایجاد کند، قابلیت جدیدی را به آن بیفزاید. در ادامه، با ساختار یک ماژول ساده، روش کامپایل، بارگذاری و حذف آن از هسته آشنا خواهیم شد.

۱۲-۳-۲- ایجاد ماژول شخصی

با نوشتن یک ماژول شخصی ساده شروع می‌کنیم. این ماژول یک خط پیام را در فایل لاگ هسته چاپ خواهد کرد. اما ابتدا باید ابزارهای مورد نیاز برای ایجاد، کامپایل و بارگذاری ماژول‌ها را با دو دستور زیر نصب کنیم. ممکن است برخی از این ابزارها را در دستورکارهای قبلی نصب کرده باشید و یا به‌صورت پیش‌فرض روی توزیع لینوکسی خود داشته باشید.

```
sudo  apt-get  update
sudo  apt-get  install gcc build-essential linux-headers-$(uname -r)
```

پس از کسب اطمینان از نصب ابزارها، یک فایل متنی ساده به نام hello.c ایجاد کرده و کد C زیر را در آن بنویسید.

```
// hello.c
# include  < linux/ module.h >
# include  < linux/ kernel.h >
# include  < linux/ init.h >

static  int  __init  hello_init( void ) {
printk ( KERN_INFO "Hello, world!\n" );
return 0; }

static  void  __exit  bye_exit( void ) {
printk ( KERN_INFO "Goodbye, world!\n" ); }

module_init ( hello_init );
module_exit ( bye_exit );

MODULE_LICENSE( "GPL" );
MODULE_DESCRIPTION( "Simple Hello World Module" );
MODULE_AUTHOR( "OSLaboratory of Urmia University" );
```

هر ماژول هسته، حداقل باید ۲ تابع داشته باشد. یک تابع شروع، که هنگام بارگذاری[۴] ماژول در هسته صدا زده می‌شود و دیگری تابع پایان که هنگام حذف[۵] ماژول از هسته، صدا زده می‌شود. پس از تعریف این دو تابع، لازم است در انتهای کد و با استفاده از ماکروهای مقداردهی اولیه، به نام‌های module_init() و module_exit()، مشخص کنیم که کدام یک از این دو تابع، برای شروع و کدام یک برای پایان در نظر گرفته شده است.

در مثال بالا، تابع hello_init، تابع شروع است و بنابراین به‌صورت module_init (hello_init) معرفی شده و تابع پایان هم به همین ترتیب module_exit (bye_exit) در انتهای کد اضافه شده است. توابع شروع و پایان، از نسخه ۲.۳.۱۳ هسته به بعد می‌توانند هر نام دلخواهی داشته باشند. اما تا پیش از آن، با نام init_module() و cleanup_module() شناخته می‌شدند.

سه فایل سرآیند در این کد ضمیمه شده‌اند. سرآیند linux/module.h که برای هر ماژولی مورد نیاز است و تعریف خیلی از توابع را در خود دارد، از جمله اطلاعات مجوز و نام نویسنده که در سه سطر انتهایی مثال بالا می‌بینید.

سرآیند linux/kernel.h حاوی تعدادی ماکرو و تابع می‌باشد؛ مانند printk().سرآیند linux/init.h، دو کاربرد اصلی در ماژول‌نویسی هسته لینوکس دارد. اول اینکه شامل تعریف ماکروهای مقداردهی اولیه است؛ که بالاتر به آن‌ها اشاره شد و دوم ماکروهای مدیریت حافظه را در خود دارد؛ مانند __init و __exit یا __initdata و __exitdata

همان‌طور که در مثال بالا می‌بینید، قبل از تابع شروع، عبارت __init آمده است که یعنی این تابع فقط در زمان راه‌اندازی[۶] ماژول استفاده می‌شود و سپس حافظه آزاد می‌گردد. ماکروی __exit نیز برای توابعی که فقط هنگام خروج ماژول از هسته اجرا می‌شوند، کاربرد دارد. توجه داشته باشید که این ماکرو برای ماژول‌هایی که به‌صورت مستقیم در هسته کامپایل شده‌اند، قابل استفاده نیست زیرا چنین ماژول‌هایی قابل بارگذاری یا حذف پویا نیستند (به عبارتی insmod یا rmmod روی آن‌ها کار نمی‌کند)، بلکه تا هنگام خاموشی سیستم در حافظه‌ی هسته باقی و فعال می‌مانند.

---

[4] Load
[5] Unload
[6] Initialization

همچنین ملاحظه می‌کنید که هر دو تابع __init و __exit بر اساس استانداردهای برنامه‌نویسی هسته به‌صورت ایستا تعریف شده‌اند تا فقط درون همین فایل C قابل مشاهده باشند و در جدول سمبل‌های سراسری هسته ظاهر نشوند. با این کار، از تداخل نام‌گذاری[7] جلوگیری به عمل می‌آید. به‌علاوه، اگر این دو تابع به‌صورت ایستا تعریف نشوند، ممکن است سایر ماژول‌ها یا کدهای هسته به طور تصادفی آن‌ها را صدا بزنند.

تابع printk() که در کد مورد استفاده قرا گرفته است، به‌صورت پیش‌فرض چیزی در صفحه‌نمایش چاپ نمی‌کند و برای تعامل مستقیم با کاربر نیست. بلکه برای مکانیزم لاگ هسته به‌کار می‌رود. هر printk() با یک اولویت می‌آید که در مثال بالا، ماکروی KERN_INFO برای این‌منظور به‌کار رفته است و نشان می‌دهد پیام از نوع informational است. به تعداد ۸ اولویت (از ۰ تا ۷) وجود دارند که به‌صورت ماکرو در فایل include/linux/klog.h تعریف شده‌اند. اگر کاربر این اولویت را تعیین نکند به طور پیش‌فرض DEFAULT_MESSAGE_LOGLEVEL به آن تخصیص می‌یابد.

برای اینکه پیام printk() بر روی کنسول ظاهر شود، باید اولویت console_loglevel بالاتر از اولویت پیام باشد. اولویت console_loglevel را متغیری به همین نام تعیین می‌کند. برای تنظیم مقدار این متغیر، می‌توان از طریق دستور زیر اقدام نمود.

```
sudo    dmesg    -n    اولویت مورد نظر
```

نکته: این تغییرات به‌صورت موقت در سیستم اعمال خواهند شد و با راه‌اندازی مجدد از بین می‌روند. برای ثبت دائمی، باید فایل etc/sysctl.conf/ را ویرایش کنید.

۱۲-۳-۲- کامپایل

برای اینکه یک ماژول هسته را به درستی کامپایل گردد، نیاز به تنظیمات بسیار زیادی دارد که با پیچیده‌تر شدن ماژول، این تنظیمات نیز پیچیده‌تر می‌شوند. اما مکانیزمی به نام kbuild وجود دارد که تمام این تنظیمات را انجام

---

[7] Namespace Pollution

داده و امکان کامپایل ماژول را بدون نیاز به تنظیمات دستی فراهم می‌آورد. برای اگاهی بیشتر درباره این مکانیزم، به مستندات سایت رسمی هسته لینوکس مراجعه کنید.

برای استفاده از kbuild، بایستی یک Makefile مطابق با استاندارد آن بنویسیم. بدین‌منظور، در همان دایرکتوری که کد ماژول در آن قرار دارد، یک فایل متنی ساده به نام Makefile ایجاد کرده و دستورات زیر را در آن بنویسید.

```
obj-m  +=  hello.o
all :
     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

نکته ۱: نام فایل باید دقیقا معادل عبارت Makefile و با رعایت حروف بزرگ و کوچک باشد.

نکته ۲: فایل را در همان دایرکتوی حاوی کدمنبع، یعنی دایرکتوری‌ای که فایل hello.c در آن قرار دارد، ایجاد کنید.

نکته ۳: درون فایل به‌منظور ایجاد تورفتگی[8] از کلید Tab استفاده نمایید و به هیچ وجه کلید Space را به‌کار نبرید، چرا که ممکن است در هنگام کامپایل، موجب بروز خطا گردد. توصیه می‌شود از ویرایشگرهای خط فرمانی برای ایجاد این فایل استفاده نمایید، زیرا ویرایشگرهای گرافیکی گاهی به‌صورت خودکار تورفتگی‌ها را به کاراکترهای فاصله تبدیل می‌نمایند.

آنچه که به‌عنوان محتوای Makefile مشاهده کردید، از یک الگوی ثابت و استاندارد به شرح زیر پیروی می‌کند:

در سطر اول، obj-m که یک متغیر از پیش تعریف‌شده در سیستم «ساخت[9] هسته» است، به سیستم اعلام می‌کند که فایل hello.o به‌عنوان یک ماژول هسته، ساخته خواهد شد. در زیربخش all، ابتدا با استفاده از سوئیچ C- به دایرکتوری ساخت هسته جاری می‌رویم. نسخه هسته جاری، با استفاده از دستور uname -r استخراج شده است. متغیر M نیز مسیر دایرکتوری فعلی را دربردارد و از هسته می‌خواهد تا ماژول مدنظر را در این دایرکتوری بسازد. عبارت

---

[8] Indention
[9] Build

modules در انتهای این سطر بیانگر این است که هدف این سطر، ساخت ماژول می‌باشد نه پاک کردن آن. در بخش clean نیز به همان ترتیب بالا، عملیات پاک‌سازی فایل‌های موقت و خروجی‌های حاصل از ساخت، انجام می‌پذیرد. پس از ایجاد Makefile، وارد دایرکتوری کدمنبع شده و دستور make را اجرا کنید تا ماژول شما کامپایل شود.

نکته: اگر قصد کامپایل فایل بدون استفاده از Makefile را دارید، کافی است وارد دایرکتوری کدمنبع شده و دستور زیر را در ترمینال اجرا نمایید.

```
make    -C    /lib/modules/$(uname -r)/build     M=$(pwd)     modules
```

پس از کامپایل، چندین فایل ایجاد می‌شود. از این میان، hello.ko فایل اجرایی موردنظر ماست. از هسته ۲.۶ به بعد، از پسوند ko برای ماژول‌های هسته استفاده شده است تا از پسوند o که مخصوص فایل‌های object است، به راحتی قابل تمیز باشند.

برای به دست آوردن اطلاعاتی از ماژول خود، در حالیکه درون دایرکتوری حاوی ماژول قرار دارید دستور زیر را اجرا نمایید.

```
modinfo    hello.ko
```

۱۲-۳-۳- بارگذاری، بررسی اجرا و خروج

برای بارگذاری ماژول در هسته‌ی یک سیستم در حال اجرا، از دستور زیر استفاده می‌شود.

```
sudo    insmod    hello.ko
```

نکته: گاهی لازم است از hello.ko./ به جای hello.ko استفاده کنید. تا سیستم به راحتی ماژول را پیدا کند.

پس از بارگذاری ماژول، با استفاده از دستور زیر، لاگ هسته را مشاهده کنید. خواهید دید که پیام Hello, world! در انتهای آن چاپ شده است که یعنی ماژول موردنظر با موفقیت در هسته بارگذاری شده است.

```
dmesg | tail
```

دستور dmesg لاگ هسته را در ترمینال نمایش می‌دهد. برای سادگی، با کمک هدایتگر پایپ و دستور tail، تنها ده خط انتهایی لاگ را نمایش می‌دهیم. با استفاده از دستور lsmod نیز می‌توانید ماژول بارگذاری شده را در بین اسامی لیست ماژول‌های فعال ببینید.

برای خارج کردن ماژول از هسته از دستور rmmod به‌صورت زیر استفاده کنید.

```
sudo    rmmod    hello
```

دوباره اگر دستور dmesg | tail را اجرا نمایید، خواهید دید که پیام Goodbye, world! در انتهای فایل چاپ شده است که به معنی خروج موفقیت‌آمیز ماژول hello از هسته می‌باشد.

برای پاک‌سازی فایل‌های حاصل از دستور make، به‌صورت زیر اقدام می‌کنیم. انجام این محله اختیاری می‌باشد.

```
make    clean
```

۱۲-۳-۴- دریافت پارامتر

در این مثال، چگونگی تعریف و دریافت پارامتر از کاربر را در زمان بارگذاری ماژول هسته، بررسی خواهیم کرد. در ادامه، کد را به‌صورت بخش به بخش و همراه با توضیحات مربوطه ملاحظه می‌کنید. یک فایل متنی ساده با نام param_module.c ایجاد کرده و تمام بخش‌های کد را پشت سر هم در آن قرار دهید.

در ابتدای کد، یک سرآیند جدید به نام linux/moduleparam.h اضافه شده است که برای تعریف پارامتر قابل تنظیم توسط کاربر، به آن نیاز داریم.

```
// param_module.c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/moduleparam.h>
```

از تابع module_param برای ثبت متغیر param به‌عنوان یک پارامتر نوع صحیح استفاده شده است. این تابع، سه آرگومان دریافت می‌کند: نام متغیر(param)، نوع داده(int)، سطح دسترسی (۰ یعنی فقط از طریق هسته قابل تنظیم بوده و از فضای کاربر قابل تغییر نیست). در تابع شروع، مقدار این پارامتر در لاگ هسته چاپ خواهد شد تا بتوان بررسی کرد که مقدار ورودی درست دریافت شده است یا نه.

```
static int param = 0;
module_param(param, int, 0);
```

به کمک MODULE_PARM_DESC نیز یک توضیح کوتاه برای پارامتر تنظیم می‌کنیم.

```
MODULE_PARM_DESC(param, "An integer parameter");
```

این توضیح در مسیر زیر قابل مشاهده خواهد بود.

```
/sys/module/<module_name>/parameters/
```

سپس، «تابع شروع» را تعریف می‌کنیم. این تابع هنگام بارگذاری ماژول، اجرا می‌شود که در این مثال، مقدار پارامتر دریافتی را در لاگ هسته چاپ خواهد کرد.

```
static int __init param_module_init(void) {
printk(KERN_INFO "param_module: Module loaded with parameter %d\n",
param);
return 0;   }
```

پس از آن، به تعریف «تابع پایان» می‌پردازیم. این تابع هنگام حذف ماژول از حافظه، اجرا شده و پیامی را به لاگ هسته ارسال خواهد کرد.

```
static void __exit param_module_exit(void) {
printk(KERN_INFO "param_module: Module unloaded\n");    }
```

سپس با استفاده ازخطوط زیر، مشخص می‌کنیم که کدام یک از توابع تعریف شده در بالا، «تابع شروع» و کدام یک، «تابع پایان» می‌باشد.

```
module_init(param_module_init);
module_exit(param_module_exit);
```

و در نهایت اطلاعات فراداده‌ای ماژول را به ترتیب زیر ثبت می‌نماییم.

```
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple Parameter Handling Module");
MODULE_AUTHOR("OSLaboratory of Urmia University");
```

سطر اول، به هسته اعلام می‌دارد که این ماژول تحت مجوز GPL منتشر شده است. سطر دوم، یک توضیح کلی درباره کارکرد ماژول ارائه می‌دهد که در مستندات ماژول قابل نمایش خواهد بود و سطر آخر، نام نویسنده یا گروه توسعه‌دهنده ماژول را ثبت می‌نماید.

برای کامپایل این ماژول نیز دقیقا مانند مثال قبل عمل می‌کنیم. فقط کافی است نام ماژول جدید، جایگزین شود. اما هنگام اجرا، یک تفاوت کوچک وجود دارد. به این صورت که هنگام وارد کردن ماژول به هسته، باید مقدار پارامتر ورودی را نیز تعیین نماییم.

```
sudo   insmod   param_module.ko   param=42
```

تمرین

ماژولی بنویسید که یک پارامتر ورودی از نوع صحیح مانند count را از کاربر دریافت کرده و هنگام بارگذاری، به تعداد count بار، یک پیام خوش‌آمدگویی را به‌صورت زیر در لاگ ثبت نماید که در آن n شماره تکرار است. هنگام خروج از هسته نیز یک پیام خداحافظی با همین تعداد و قالب، در لاگ چاپ گردد.

```
n) Welcome to Kernel Modules
```

برای تعریف پارامتر، از module_param استفاده نمایید.

گزارش کار

- ماژولی بنویسید که یک دستگاه کاراکتری[10] به نام خودتان در /dev/yourname_char_dev ایجاد کند. توجه داشته باشید که برای جلوگیری از خطای نام‌گذاری، لازم است نام خود را با حروف کوچک و بدون فاصله وارد نمایید.
  - این دستگاه باید طوری طراحی شود که هنگام نوشتن در آن، پیام را در لاگ کرنل ثبت کند و هنگام خواندن، آخرین پیام در آن لحظه را برگرداند. به عبارت دیگر، تازمانی‌که پیام جدیدی نوشته نشود، همان پیام برگردانده شود. طول پیام را حداکثر ۱۰۰ کاراکتر در نظر بگیرید.
  - دستگاه کاراکتری، نوعی ماژول هسته در لینوکس است که یک رابط برای برنامه‌های فضای کاربر فراهم می‌کند تا این برنامه‌ها بتوانند بر اساس انتقال کاراکتر به کاراکتر، با دستگاه‌های سخت‌افزاری یا سرویس‌های هسته تعامل داشته باشند.
  - دسترسی به این دستگاه‌ها از طریق فایل‌های ویژه در دایرکتوری /dev انجام می‌پذیرد و معمولا برای دستگاه‌هایی که داده‌ها را به‌صورت «هر بار یک کاراکتر/بایت» منتقل می‌کنند - مانند صفحه‌کلید، ماوس

[10] Character Device Module

و پورت‌های سریال - استفاده می‌شود. این دستگاه‌ها با Block Devices که داده را در قالب بلوک‌های بزرگ منتقل می‌کنند متفاوتند.

- ماژولی بنویسید که هر ۵۰۰ میلی‌ثانیه یک‌بار، وضعیت یک LED از صفحه‌کلید مانند Num Lock را تغییر دهد. به عبارت دیگر باعث خاموش و روشن شدن متناوب آن شده و هنگام خروج از هسته، LED را به حالت اولیه بازگرداند.

منابع و مراجع:

- Salzman PJ. *The Linux Kernel Module Programming Guide*. CreateSpace, 2022 .
- Madieu J. *Linux Device Driver Development*, Chapter 3. Packt Publishing Ltd, 2017.
- https://www.kernel.org/doc/Documentation/kbuild/modules.txt, visited 2026 Jan.
- https://docs.kernel.org/kbuild/makefiles.html, visited 2026 Jan

آزمایش ۱۳ : مجازی‌سازی با KVM

اهداف:

- آشنایی با مفاهیم مجازی‌سازی
- آشنایی با مجازی‌سازی در لینوکس

ابزار و مفاهیم مورد نیاز:

- یک توزیع لینوکس مبتنی بر اوبونتو
- پردازنده با پشتیبانی از مجازی‌سازی سخت‌افزاری Intel VT-x یا AMD-V
- دستورات مدیریت کابران
- دستورات مدیریت بسته

محتوا:

- نصب و تنظیمات KVM
- ایجاد و مدیریت ماشین‌های مجازی با KVM

۱۳-۱- مجازی‌سازی

۱۳-۱-۱- ضرورت مجازی‌سازی

با ظهور محاسبات سمت سرور به‌عنوان یک سرویس، ارائه تضمینی منابع و ایزوله‌سازی در محیط‌های چند مستاجری[1] از اهمیت بالایی برخوردار شدند و این ضرورت ایجاد شد که این زیرساخت‌ها، اهدافی همچون جداسازی[2] برنامه‌ها و کارایی منابع را برآورده سازند. برای دستیابی به این اهداف و با توجه به مسائل اقتصادی زیرساخت، باید اجازه داده می‌شد که سرورها بین چند کاربر به اشتراک گذاشته شوند. در عین حال بایستی جداسازی عملیاتی برنامه‌ها نیز تضمین می‌شد. مجازی‌سازی رایج‌ترین راه حل برای اطمینان از این اهداف است. ادغام[3] با استفاده از مجازی‌سازی منجر به جداسازی برنامه‌ها، استفاده بهتر از منابع و کاهش هزینه‌های عملیاتی می‌شود.

۱۳-۱-۲- فن آوری‌های مجازی‌سازی

فناوری‌های مختلف مجازی‌سازی در جدول ۱۳-۱ خلاصه شده است. دو نوع اصلی مجازی‌سازی وجود دارد: نوع یک[4] و نوع دو[5]. با این حال، برای اینکه بتوانیم تکنیک‌های مجازی‌سازی را بدون هم‌پوشانی در ویژگی‌های آن‌ها طبقه‌بندی کنیم، نوع یک را به دو کلاس مختلف تقسیم می‌کنیم. به‌علاوه، طبقه‌بندی کلاسیک، شامل فناوری جدید مجازی‌سازی از نوع کانتینر نمی‌شود.

Para-Virtualization (نوع یک تغییر داده شده) نوعی مجازی‌سازی است که در آن سیستم‌عامل مهمان (سیستم‌عاملی که مجازی شده است) می‌داند که مهمان است و بر این اساس درایورهایی دارد که به جای صدور دستورات سخت‌افزاری، دستورات را به مقصد مانیتور ماشین‌مجازی صادر می‌کنند. این کار شامل مدیریت حافظه و رشته‌ها نیز می‌شود که معمولا به دستورالعمل‌های ممتاز در پردازنده نیاز دارد.

---

[1] Multi-Tenant
[2] Isolation
[3] Consolidation
[4] Type1
[5] Type2

Type (Classic Definition)	Layer	Virtual Machine Monitor	Virtualization Technology	Example
Type 1	VMM	ISA (HW)	Native Virtualization	XenServer
Type 1 – Modified	ISA (HW) and Hypercalls	Hosted	Para-Virtualization	Hyper-V, VMwre ESX(i)
Type 2	ISA (HW)	Hosted	Full-Virtualization	VMware Workstation, Virtual Box
Container Virtualization	ABI (OS)	Kernel	Container-Virtualization	LXC, Docker

جدول ۱۳-۱-فناوری‌های مجازی‌سازی

اما در مجازی‌سازی نوع دو، سیستم‌عامل مهمان از قرارگیری خود در محیط مجازی بی‌اطلاع است. بنابراین، سخت‌افزار توسط سیستم‌عامل میزبان مجازی‌سازی می‌شود تا مهمان بتواند فرمان‌هایی را به آنچه که فکر می‌کند سخت‌افزار واقعی است صادر کند، اما در واقع فقط با دستگاه‌های سخت‌افزاری شبیه‌سازی شده توسط میزبان، سروکار دارد.

مجازی‌سازی بومی (نوع یک) نوعی مجازی‌سازی کامل است که در آن، ریزپردازنده دستورالعمل‌های ویژه‌ای برای کمک به مجازی‌سازی سخت‌افزار دارد. این دستورالعمل‌ها ممکن است اجازه دهند یک زمینه مجازی تنظیم شود تا مهمان بتواند بدون تغییر میزبان، دستورالعمل‌های ممتاز را مستقیما روی پردازنده اجرا کند. چنین مجموعه‌ای از ویژگی‌ها، اغلب ناظر ماشین‌مجازی[6] نامیده می‌شوند. حتی اگر دستورالعمل‌های مذکور وجود نداشته باشند، مجازی-سازی کامل همچنان امکان‌پذیر است. با این حال، این کار باید از طریق تکنیک‌های نرم‌افزاری مانند کامپایل مجدد پویا انجام شود که در آن میزبان دستورالعمل‌های ممتاز جاری در مهمان را دوباره کامپایل می‌کند تا بتوانند به‌عنوان دستور غیرممتاز روی میزبان اجرا شوند.

---

[6] Virtual Machine Monitor/ Manager (VMM)

راه‌کارهای مجازی‌سازی مبتنی بر ابر، عمدتا به مجازی‌سازی نوع یک که مبتنی بر هایپروایزر[7] می‌باشد، متکی هستند؛ زیرا می‌توانند از انواع مختلف سیستم‌عامل مانند لینوکس، ویندوز و غیره پشتیبانی کنند. به‌علاوه، با گسترش مجازی‌سازی سخت‌افزار در اکثر معماری‌های مدرن، این امر به سیستم‌های عامل مهمان اجازه می‌دهد تا بدون تغییر اجرا شوند. با این حال، ممکن است مجازی‌سازی مبتنی بر هایپروایزر از کاهش کارایی رنج ببرد، زیرا در معماری مجموعه دستورالعمل[8]، باید متحمل سربار مدیریت ماشین‌مجازی توسط هایپروایزر شود.

به این ترتیب، مجازی‌سازی میزبان نوع دو نیز در حال تبدیل شدن به یک جایگزین گسترده است زیرا هر مهمان می‌تواند هسته خود را داشته باشد و میزبان شامل یک هسته اصلاح‌شده با افزونه‌هایی[9] برای مدیریت و اجرای چندین ماشین‌مجازی است.

از طرف دیگر، ازآنجایی‌که مجازی‌سازی کانتینر، لایه اضافی برای مسیریابی ندارد و هسته یکسانی را به اشتراک می‌گذارد، می‌تواند برنامه‌ها را با سرعتی نزدیک به حالت بومی اجرا کند. این نوع مجازی‌سازی، لایه Application Binary Interface (ABI) را مجازی می‌کند. با این وجود، ازآنجایی‌که هسته مشترک است، فقط می‌تواند سیستم‌های عامل مهمانی را اجرا کند که از هسته میزبان پشتیبانی می‌کنند.

کانتینر لینوکس[10](LXC)، یک نمونه نوعی از مجازی‌سازی کانتینر است. LXC یک روش مجازی‌سازی در سطح سیستم‌عامل برای اجرای چندین سیستم لینوکس مجزا (به روش کانتینر) روی یک میزبان کنترلی با استفاده از یک هسته لینوکس است. Docker نیز می‌تواند از LXC به‌عنوان یکی از درایورهای اجرایی خود استفاده کند که مدیریت image و ارائه خدمات استقرار را ممکن می‌سازد. کانتینرهای Docker تکه‌ای از یک نرم‌افزار را در یک فایل‌سیستم کامل می‌پیچند که شامل همه چیزهایی است که برای اجرا نیاز دارد: کد، زمان اجرا[11]، ابزارهای سیستم، کتابخانه‌های

---

[7] Hypervisor
[8] Instruction Set Architecture (ISA)
[9] Extensions
[10] Linux Container
[11] Run Time

سیستم، به عبارتی تمام چیزهایی که می‌توانید روی یک سرور نصب کنید. این کار تضمین می‌کند که بدون در نظر گرفتن محیط بومی که اجرا در آن صورت می‌پذیرد، همیشه اجرای یکسانی را ارائه خواهد شد.

تراکم مجازی‌سازی به تعداد ماشین‌های مجازی اشاره دارد که یک میزبان فیزیکی می‌تواند در خود داشته باشد به‌طوری‌که برای هر ماشین‌مجازی، منابع محاسباتی کافی برای عملکرد خوب فراهم کند. این به عوامل متعددی همچون سخت‌افزار سرور، نرم‌افزار مجازی‌سازی، نوع سرویس و تنوع حجم کاری بستگی دارد. این عوامل مختلف، دستیابی به یک عدد مطلق برای تراکم ماشین‌مجازی را در تمام سناریوها دشوار می‌کند. افزایش تراکم ماشین‌مجازی هزینه‌های تحمیل شده را کاهش می‌دهد، اما در عین حال، به دلیل مشاجره بر سر منابع مشترک، چالش‌های بیشتری را در تضمین کارایی ایجاد می‌کند. معمولا گلوگاه در زیرسیستم حافظه و مقدار حافظه دردسترس، آشکار می‌شود. ماشین‌های commodity در چنین منابعی کمیاب هستند و به این ترتیب، سیستم برای تضمین بهترین تلاش[12]، از تدارکی محافظه‌کارانه بهره‌مند خواهد شد.

۱۳-۱-۳- مجازی‌سازی در زمینه P2P

اگرچه تکنیک‌های مجازی‌سازی مختلفی وجود دارد، مجازی‌سازی در زمینه همتابه‌همتا[13] (P2P) چالش‌های خاصی دارد که باید مورد توجه قرار گیرند. P2P از ماشین‌های commodity تشکیل شده است که در درجه سرور نیستند و در نتیجه قابلیت محاسباتی بالایی ندارند. در مراکز داده، گرچه فناوری مجازی‌سازی متداول، فناوری مبتنی بر هایپروایزر است؛ ولی مجازی‌سازی میزبانی شده نیز رواج یافته است. سرورهای P2P از نظر منابع دردسترس، محدود شده‌اند. بنابراین حصول اطمینان از اینکه مجازی‌سازی هزینه بالایی را بر روی این ماشین‌های با منابع محدود تحمیل نمی‌کند، ضروری است.

تکنیک‌های مجازی‌سازی مبتنی بر Hypervisor مانند Xen به دلیل وجود یک مرحله مسیریابی اضافی، سربار بالایی را تحمیل می‌کنند، زیرا اجرای هر نوع سیستم‌عاملی را پشتیبانی می‌نمایند. بنابراین، مجازی‌سازی کانتینر به دلیل

---

[12] Best-effort

[13] Peer to Peer

سرعت نزدیک به اجرای بومی که سربار کمی به همراه دارد، برای چنین محیط‌هایی با منابع محدود مناسب‌تر است. مجازی‌سازی میزبانی‌شده نیز یک جایگزین جذاب برای محیط‌های با منابع محدود است زیرا حداقل هزینه‌های سربار را تحمیل می‌کند. با این حال، هر مهمان می‌تواند هسته خود را داشته باشد که بر کارایی ایجاد یک VM جدید تاثیر دارد. LXC از این‌منظر سربار کمتری دارد. باوجود مزایا و معایب نسبی، به نظر می‌رسد LXC انتخاب معقولی از تکنیک مجازی‌سازی برای استفاده در یک محیط P2P باشد.

۱۳-۲- مجازی‌سازی با KVM در لینوکس

یکی از پرکاربردترین راه‌حل‌های مجازی‌سازی در لینوکس، ماشین‌مجازی مبتنی بر هسته[14](KVM) است که یک ماژول هسته بوده و با استفاده از قابلیت‌های مجازی‌سازی سخت‌افزاری پردازنده (Intel VT-x و AMD-V)، اجرای ماشین‌های مجازی را با کارایی بالایی فراهم می‌سازد. این ماژول تقریبا در تمام توزیع‌های لینوکسی قابل استفاده است. اما از آنجاییکه تمرکز آزمایش‌های این کتاب بر توزیع‌های مبتنی بر دبیان می‌باشد، در این آزمایش نیز مراحل نصب و راه‌اندازی بر روی این سیستم‌ها شرح داده شده است.

۱۳-۲-۱- نصب و راه‌اندازی kvm

مرحله ۱: به‌روزرسانی سیستم. لیست و بسته‌های سیستم را به‌روزرسانی کنید تا مطمئن شوید آخرین نسخه نرم‌افزارها را دارید.

```
sudo   apt   update
sudo   apt   upgrade
```

[14] Kernel-based Virtual Machine

مرحله ۲: بررسی قابلیت مجازی‌سازی پردازنده. برای کسب اطمینان از پشتیبانی پردازنده از فناوری مجازی‌سازی، دستور زیر را اجرا نمایید.

```
egrep   -c   '(vmx|svm)'   /proc/cpuinfo
```

اگر خروجی بزرگتر از ۰ باشد، مجازی‌سازی پشتیبانی می‌شود. خروجی ۰ به معنای عدم پشتیبانی است. همان‌طور که در تصویر ۱۳-۱ ملاحظه می‌نمایید، خروجی دستور بر روی سیستم مورد استفاده در این آزمایش برابر با عدد ۸ است.

```
soltani@LIFEBOOK:~$ egrep -c '(vmx|svm)'  /proc/cpuinfo
8
```

تصویر ۱۳-۱- بررسی قابلیت مجازی سازی پردازنده

نکته: برای استفاده از حالت مجازی‌سازی تودرتو[15]، یعنی نصب KVM روی لینوکسی که خود به‌صورت مجازی نصب شده است (مثلا روی VMWare Workstation) نیاز به انجام دو نوع تنظیمات دارید. اول اینکه سیستم عامل میزبان شما از قابلیت مجازی‌سازی پشتیبانی کند. که این مورد را می‌توانید در تنظیمات BIOS/UEFI مشاهده و تنظیم نمایید. دوم اینکه خود VMWare Workstation نیز این حالت را پشتیبانی کند. برای فعال‌سازی این قابلیت، ابتدا ماشین‌مجازی خود را خاموش کرده و سپس از منوی

```
VM > Settings > Hardware > Processors
```

دو گزینه زیر را تیک بزنید.

```
Virtualize Intel VT-x/EPT or AMD-V/RVI
Virtualize CPU performance counters
```

---

[15] Nested Virtualization

گاهی با وجود این تنظیمات باز هم مجازی‌سازی تودرتو پشتیبانی نمی‌شود. اگرچه در صورت پشتیبانی نیز، عملکرد بسیار کندتری از حد معمول ارائه خواهد کرد. به همین دلیل، در انجام این آزمایش از یک لینوکس محلی استفاده شده است.

مرحله ۳: نصب بسته‌های مورد نیاز. در خط فرمان دستور زیر را برای نصب بسته‌های مورد نیاز اجرا کنید.

```
sudo apt -y install bridge-utils cpu-checker libvirt-clients libvirt-daemon qemu-kvm virtinst
```

مرحله ۴: **بررسی امکان مجازی‌سازی با KVM پس از نصب**. برای اطمینان از امکان استفاده از KVM بر روی سیستم خود، دستور زیر را اجرا کنید.

```
kvm-ok
```

خروجی این دستور بسیار ساده و واضح است و نشان می‌دهد که آیا KVM قابل استفاده است یا خیر.

```
INFO: /dev/kvm exists
KVM acceleration can be used
```

مرحله ۵: افزودن کاربر جاری به گروه‌ها. کاربر خود را به گروه‌های libvirt-qemu و kvm اضافه کنید تا امکان مدیریت ماشین‌های مجازی KVM را بدون سوئیچ به کاربر ریشه، داشته باشد.

```
sudo usermod -aG libvirt-qemu $USER
sudo usermod -aG kvm $USER
```

برای اعمال تغییرات، یک‌بار کاربر را خارج[16] کرده و دوباره وارد شوید. البته اگر در مراحل آتی، سیستم را راه‌اندازی مجدد خواهید کرد، نیازی به این کار نیست.

---

[16] Log Out

مرحله ۶: نصب Virt-Manager. اگر می‌خواهید برای ایجاد و مدیریت ماشین‌های مجازی، یک رابط کاربری گرافیکی در اختیار داشته باشید، Virt-Manager را نصب کنید.

```
sudo   apt   install   virt-manager
```

نکته: درصورتی‌که سیستم‌عامل میزبان یا مهمان شما صرفا مبتنی بر رابط خط فرمانی است، از این مرحله رد شوید.

مرحله ۷: راه‌اندازی مجدد سیستم. برای اعمال تغییرات، سیستم خود را مجددا راه‌اندازی نمایید.

```
sudo   reboot
```

مرحله ۸: فعال سازی سرویس libvirtd. ابتدا چک کنید که سرویس موردنظر فعال شده باشد.

```
sudo   systemctl   status   libvirtd
```

درصورتی‌که به هر دلیلی سرویس فعال نشده بود، با دستورات زیر به‌صورت دستی این کار را انجام دهید.

```
sudo   systemctl   start    libvirtd
sudo   systemctl   enable   libvirtd
```

نکته: در برخی نسخه‌ها نام این سرویس، libvirt می‌باشد.

مرحله ۹: بررسی وضعیت KVM. بررسی کنید که ماژول‌های KVM بارگذاری شده باشند.

```
lsmod | grep kvm
```

۱۳-۲-۲- نصب سیستم‌عامل مجازی در KVM

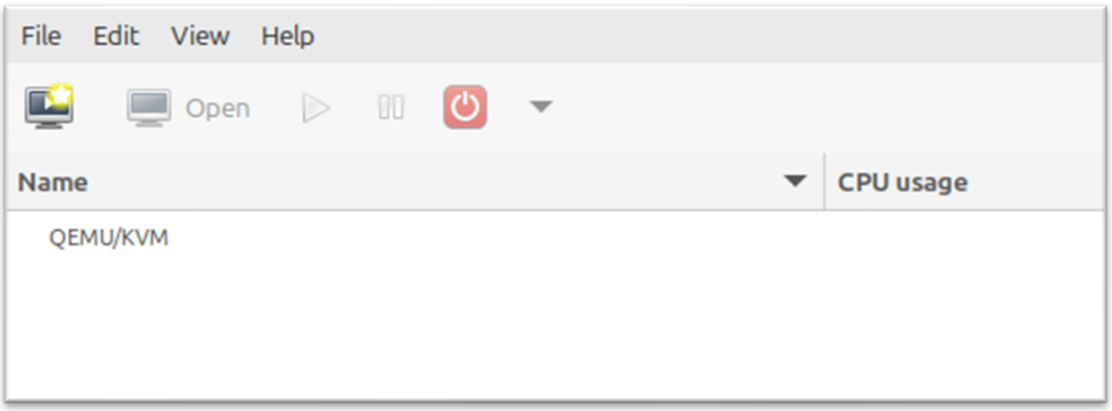
برای ایجاد یک ماشین‌مجازی جدید و نصب یک سیستم‌عامل مهمان در آن، به دو روش می‌توانید اقدام نمایید. درصورتی‌که در مراحل قبل Virt-Manager را نصب کرده‌اید، روش اول را دنبال کنید. درغیراین‌صورت، به روش مبتنی بر خط فرمان که در ادامه آمده است مراجعه نمایید.

روش اول: نصب از طریق Virt-Manager

با اجرای دستور زیر، رابط گرافیکی KVM با نام Virtual Machine Manager باز می‌شود. از این برنامه می‌توانید به‌منظور نصب و مدیریت ماشین‌های مجازی استفاده نمایید.

```
virt-manager
```

محیط برنامه در تصویر ۱۳-۲ نمایش داده شده است.



تصویر۱۳-۲- واسط گرافیکی KVM

حال در این پنجره، از منوی «فایل» گزینه «ایجاد ماشین‌مجازی جدید» را انتخاب کرده و سپس با انتخاب گزینه‌هایی که در ادامه توضیح داده شده، مراحل ساخت را به پایان برسانید. قبل از شروع مراحل نصب، فایل ISO نصب را -که فرض کردیم در دایرکتوری خانگی کاربر قرار دارد - به مسیر زیر منتقل کنید.

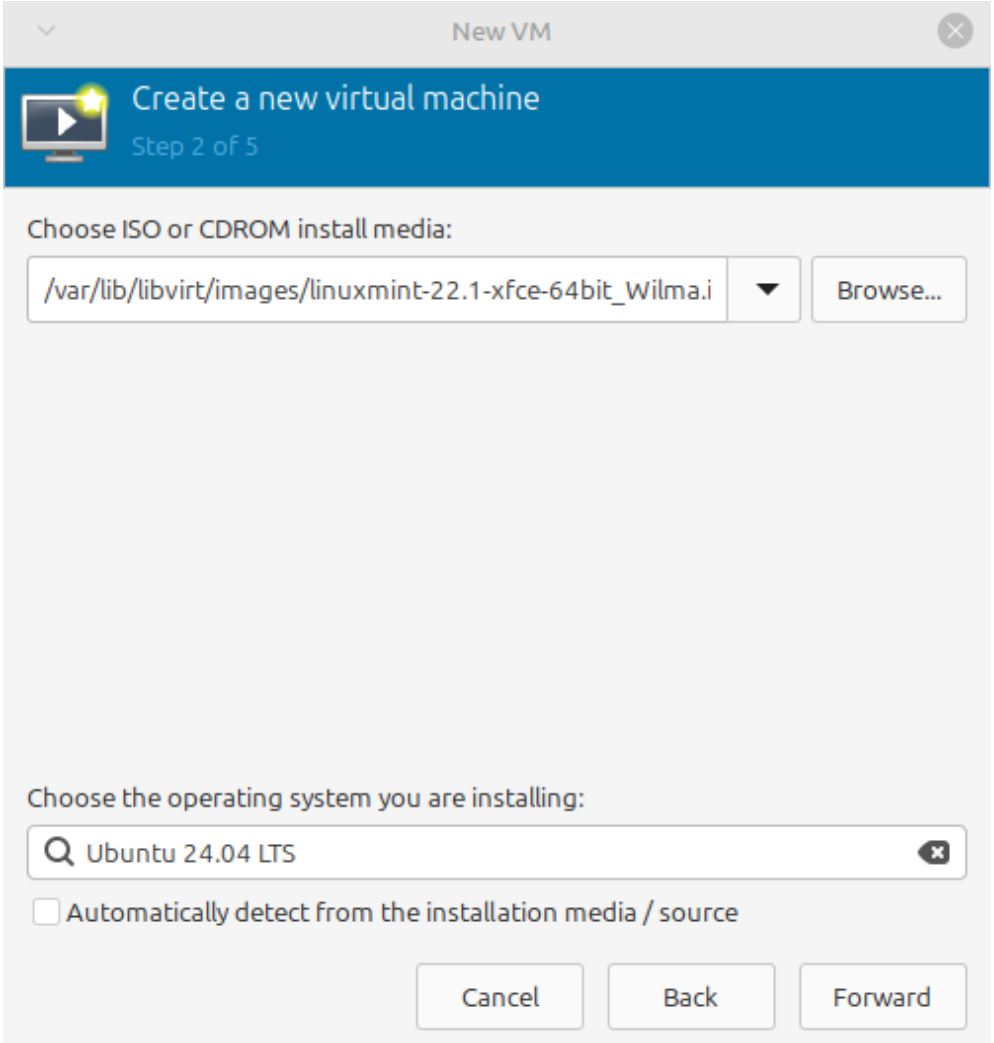
```
sudo mv /home/soltani/linuxmint.iso  /var/lib/libvirt/images
```

در مرحله اول، باید نحوه نصب را مشخص کنیم. با توجه به اینکه قصد نصب محلی را داریم، گزینه زیر را انتخاب می‌کنیم.

```
Local install media (ISO Image or CDROM)
```

نکته: در صورتی که قصد نصب از مخازن را دارید، باید در این مرحله، گزینه Network Install را انتخاب کرده و در مرحله بعدی نیز لینک مربوطه را وارد نمایید.

حال به پنجره نصب بازگشته و مانند تصویر ۱۳- ۳ محل قرارگیری فایل ISO و همچنین نوع سیستم‌عامل را مشخص می‌نماییم.



تصویر ۱۳- ۳- مراحل نصب سیستم‌عامل مجازی با استفاده از واسط گرافیکی

در طی مراحل بعدی، باید میزان حافظه، تعداد پردازنده، اندازه دیسک، نام و نوع اتصال به شبکه را برای ماشین‌مجازی خود تعیین کنیم. معمولا مقادیر پیش‌فرض، انتخاب‌های مناسبی هستند و می‌توانید همین مقادیر را بدون تغییر باقی گذاشته و به مرحله بعدی بروید. در نهایت، با زدن کلید «پایان» فرآیند نصب سیستم عامل آغاز خواهد شد.

روش دوم: نصب از طریق خط فرمان

برای نصب یک سیستم عامل مجازی به‌صورت خط فرمانی، از دستور virt-install استفاده می‌کنیم و در ادامه ویژگی‌های ماشین‌مجازی را با سوئیچ‌های مناسب مشخص می‌نمایم.

```
sudo virt-install \
  --name myvm \
  --memory 2048 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/myvm.qcow2,size=20 \
  --os-variant ubuntu22.04 \
  --cdrom /var/lib/libvirt/images/linuxmint.iso \
  --graphics vnc \
  --network network=default
```

نکته ۱: علامت ممیز معکوس که در انتهای هر سطر استفاده شده به این معنی است که دستور - به دلیل طولانی بودن - در خط بعدی ادامه پیدا کرده است.

نکته ۲: در بخش‌هایی از دستور که علامت ویرگول قرار دارد، به هیچ وجه از کاراکتر فاصله استفاده ننمایید.

همان‌طور که در تصویر ۱۳-۴ مشاهده می‌نمایید پس از اجرای دستور بالا، عملیات ایجاد ماشین‌مجازی آغاز می‌شود. چون در اینجا از یک توزیع لینوکس دسکتاپ استفاده کرده‌ایم که دارای واسط گرافیکی می‌باشد، پس از مراحلی که در تصویر می‌بینید، یک پنجره جدید باز شده و نصاب گرافیکیِ سیستم‌عامل مهمان را برای ادامه مراحل نصب، نمایش خواهد داد.

```
soltani@LIFEBOOK:~$ sudo virt-install --name myvm --memory 2048 --vcpus 2 --disk path=/var/lib/libvirt/
images/myvm.qcow2,size=20 --os-variant ubuntu22.04 --cdrom /var/lib/libvirt/images/linuxmint-22.1-xfce-
64bit_Wilma.iso --graphics vnc --network network=default

Starting install...
Allocating 'myvm.qcow2'
                                    | 2.9 MB  00:00:03 ...
Creating domain...
                                    |    0 B  00:00:00
Running graphical console command: virt-viewer --connect qemu:///system --wait myvm
```

تصویر ۱۳-۴- ایجاد ماشین‌مجازی از خط فرمان

۱۳-۲-۳- دسترسی به سیستم‌عامل مجازی

پس از اتمام مراحل نصب، برای دسترسی به ماشین‌های مجازی و مدیریت آن‌ها، به دو روش می‌توان اقدام کرد.

روش اول: با استفاده از واسط گرافیکی. ابتدا برای دسترسی به برنامه «مدیر ماشین‌مجازی» دستور virt-manager را در خط فرمان اجرا کنید و یا از پنجره استارت، Virtual Machine Manager را پیدا کرده و برنامه را باز نمایید. همان رابط گرافیکی که در تصویر ۱۳-۲ نمایش داده شده بود، باز خواهد شد. اما این بار در صفحه اصلی برنامه، لیست ماشین‌های مجازی موجود نمایش داده خواهد شد. کافی است ماشین موردنظر خود را انتخاب کرده و کلید Open را بزنید.

روش دوم: با استفاده از خط فرمان. با اجرای دو دستور زیر، ابتدا ماشین‌مجازی روشن شده و سپس به اجرا درمی‌آید.

```
sudo   virsh   start   myvm
virt-viewer   --connect   qemu:///system   myvm
```

۱۳-۲-۴- مدیریت ماشین‌های مجازی

مدیریت ماشین‌های مجازی با استفاده از رابط گرافیکی virt-manager به سادگی قابل انجام می‌باشد. در حالت خط فرمانی نیز با استفاده از دستورات virsh، امکان ایجاد تغییرات بر روی ماشین‌های مجازی فراهم شده است. دستورات زیر برخی از این موارد را نشان می‌دهند.

نمایش لیست تمام ماشین‌های مجازی در حال اجرا

```
virsh   list
```

نکته: با اضافه کردن سوئیچ all-- ماشین‌های خاموش نیز لیست می‌شوند.

نمایش اطلاعات یک ماشین‌مجازی

```
virsh   dominfo   myvm
```

شروع یک ماشین‌مجازی

```
virsh   start   myvm
```

اتصال به کنسول یک ماشین‌مجازی

```
virsh   console   myvm
```

نکته ۱: در مواردی که سیستم عامل مجازی، بدون حالت گرافیکی نصب شده است و فقط مبتنی بر خط فرمان می‌باشد، از طریق کنسول می‌توانیم به آن دسترسی پیدا نماییم.

نکته ۲: برای خروج از کنسول سریال، کلید ترکیبی [+ Ctrlرا بزنید.

خاموش کردن یک ماشین‌مجازی

```
virsh   shutdown   myvm
```

حذف کامل یک ماشین‌مجازی

```
virsh   undefine   myvm
sudo   virsh   vol-delete   --pool   default   myvm.qcow2
```

دستور اول، تعریف ماشین‌مجازی را حذف می‌کند و دستور دوم، فایل دیسک مجازی را که در استخر[17] ذخیره‌سازی پیش‌فرض قرار گرفته، به طور کامل حذف می‌نماید.

نکته: به جای دو دستور بالا، می‌توانید فقط از دستور اول به همراه سوئیچ مناسب استفاده نمایید.

```
sudo virsh undefine --remove-all-storage myvm
```

ویرایش تنظیمات ماشین‌مجازی

```
sudo virsh edit myvm
```

با اجرای این دستور، فایل تنظیمات ماشین‌مجازی در ویرایشگر متنی باز می‌شود و به این ترتیب می‌توانید تغییرات لازم را در آن اعمال و ذخیره نمایید. البته دستورات مجزایی نیز برای تغییر دادن ویژگی‌های منفرد، وجود دارند که در ادامه به دو مورد از آن‌ها اشاره شده است.

تغییر حافظه اختصاص یافته به ماشین‌مجازی

```
virsh setmem myvm 1024
```

تغییر تعداد پردازنده‌های اختصاص یافته به ماشین‌مجازی

```
virsh setvcpus myvm 4
```

---

[17] Pool

تمرین

- یک توزیع دلخواه را به‌صورت مجازی در KVM نصب نمایید.
- با کمک دستور virsh، از ماشین‌مجازی خود (هم از فایل تنظیمات و هم از دیسک مربوطه)، نسخه پشتیبان تهیه کنید.
- ماشین‌مجازی را از آن فایل پشتیبان، بازیابی نمایید.

گزارش کار

- یک نمونه همسان[18] از ماشین‌مجازی خود ایجاد نمایید.
- نمونه ایجاد شده را راه‌اندازی کرده و بررسی نمایید که به درستی کار کند.
- برای جلوگیری از تداخل با ماشین‌مجازی اصلی، نام و تنظیمات شبکه را در نمونه جدید تغییر دهید.

[18] Clone

منابع و مراجع:

- Dakic V, Chirammal H. *Mastering KVM Virtualization*, Packt Publishing, 2020.
- https://www.linux-kvm.org/page/Networking, visited 2024 Aug.
- https://ubuntu.com/blog/kvm-hyphervisor, visited 2024 Aug.
- https://www.tecmint.com/install-qemu-kvm-ubuntu-create-virtual-machines,visited 2024 Aug.
- https://linuxconfig.org/setting-up-virtual-machines-with-qemu-kvm-and-virt-manager-on-debian-ubuntu, visited 2024 Aug.

آزمایش ۱۴ : مجازی‌سازی نوع یک با ESXi

اهداف:

- آشنایی با ابزارهای مجازی‌سازی نوع یک
- مدیریت ماشین‌های مجازی

ابزار و مفاهیم مورد نیاز:

- ESXi Server
- شبکه محلی
- یک مرورگر وب در سیستم کلاینت

محتوا:

- نصب و پیکربندی ESXi سمت سرور
  - رابط کاربری مبتنی بر وب
  - ایجاد و مدیریت ماشین‌مجازی
  - پیکربندی و رجیستر
  - نصب سیستم‌عامل
  - حذف ماشین‌های ایجاد شده
- اتصال به ماشین‌مجازی ایجاد شده از طریق کلاینت

۱۴-۱- معرفی

همانگونه که در آزمایش قبل اشاره شد، VMware ESXi یک هایپروایزر نوع یک است که مستقیما روی سخت‌افزار فیزیکی سرور نصب و اجرا می‌شود. هسته این هایپروایزر که VMkernel نام دارد، وظیفه مدیریت مستقیم منابع سخت‌افزاری مانند پردازنده، حافظه و تجهیزات جانبی را بر عهده دارد و امکان تخصیص آن‌ها را به ماشین‌های مجازی فراهم می‌کند. VMWare برای ایجاد و مدیریت یک مرکز داده مجازی بزرگ و یکپارچه، یک پلتفرم کامل مجازی‌ساز به نام vSphere نیز منتشر کرده است که ESXi و ابزارهای مدیریتی آن را در بر می‌گیرد. در این مجموعه، امکان مدیریت یکپارچه چندین سرور ESXi، تخصیص منابع به‌صورت خوشه‌ای[1] و فعال‌سازی ویژگی‌های پیشرفته‌ای مانند انتقال ماشین‌های مجازیِ در حال اجرا بین سرورها را فراهم می‌آورد.

۱۴-۲- نصب سرور

۱۴-۲-۱- پیش‌نیازهای سخت‌افزاری

پیش از دانلود و نصب ESXi، ابتدا مشخصات سرور خود را بررسی کرده و مطمئن شوید که حداقل ویژگی‌های موردنیاز نسخه مد نظر شما را برآورده می‌کند. در این آزمایش از نسخه ۸ استفاده شده است و بنابراین در ادامه الزامات سخت‌افزاری ESXi 8.0 را ملاحظه می‌نمایید.

- یک پردازنده ۶۴ بیتی x86 با حداقل دو هسته که از مجازی‌سازی سخت‌افزاری مانند Intel VT-x یا AMD RVI پشتیبانی می‌کند.
- حافظه‌ای با حداقل ۸ و یا ترجیحا ۱۲ گیگابایت فضا تا اجرای ماشین‌های مجازی به راحتی و با سرعت کافی انجام پذیرد.
- یک دیسک بوت با حداقل ۳۲ گیگابایت فضای ذخیره‌سازی پایدار مانند HDD یا SSD.
- یک کنترلر اترنت گیگابیتی برای اتصال به شبکه.

---

[1] Clustering

- امکان بوت شدن از طریق UEFI چون در نسخه‌های جدید، BIOS به‌صورت محدودی پشتیبانی می‌شود.

۱۴-۲-۲- مراحل نصب

آماده‌سازی رسانه نصب. ابتدا فایل ISO نصب VMware ESXi 8.0 را از وب‌سایت رسمی[۲] دانلود کنید . سپس آن را روی یک DVD رایت کنید یا همان‌طور که در آزمایش دوم آموختید، یک فلش قابل بوت از آن بسازید.

بوت شدن از رسانه نصب. رسانه نصبی که در مرحله قبل آماده کردید را به سرور متصل کنید و سرور را روشن یا راه‌اندازی مجدد نمایید. در هنگام بوت شدن، وارد تنظیمات UEFI شده و ترتیب بوت را طوری تغییر دهید که ابتدا رسانه نصب شما، مورد استفاده قرار بگیرد.

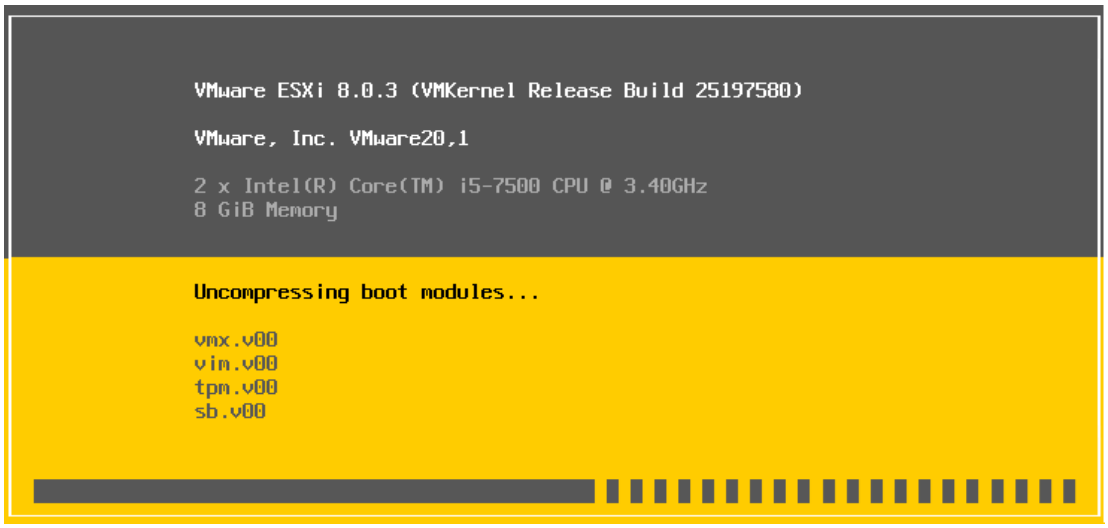
تنظیمات پیش از نصب. پس از بوت شدن از رسانه نصب، باید چندین مرحله را طی کرده و تنظیمات دلخواه خود را به انجام برسانید تا فرآیند نصب آغاز گردد. این مراحل به شرح زیر می‌باشند:

- ابتدا صفحه خوش‌آمدگویی ESXi مطابق با تصویر ۱۴-۱ نمایش داده می‌شود. برای شروع، کلید ENTER را فشار دهید.
- در صفحه بعدی، صفحه مجوز[۳] نمایش داده می‌شود. کلید F11 را به منزله پذیرش مجوز فشار دهید.
- برنامه نصاب، تمام دیسک‌های موجود را اسکن کرده و لیستی را به شما نشان می‌دهد. دیسک موردنظر خود را برای نصب ESXi انتخاب نمایید. توجه داشته باشید که تمام داده‌های موجود روی این دیسک، به طور دائمی پاک خواهند شد.

---

[2] https://support.broadcom.com/group/ecx/downloads

[3] End User License Agreement



تصویر ۱۴-۱- نصاب ESXi

- در صفحه بعد، لازم است که چیدمان صفحه‌کلید را انتخاب کنید.
- سپس یک رمزعبور قوی برای مدیر سیستم که root نام دارد، وارد نمایید.
- در نهایت، خلاصه‌ای از تنظیمات برای شما نمایش داده می‌شود. با فشردن کلید F11، فرآیند نصب آغاز خواهد شد.

نصب و راه‌اندازی مجدد. پس از به پایان رسیدن کپی فایل‌ها و عملیات نصب، مانند آنچه در تصویر ۱۴-۲ می‌بینید، پیامی مبنی بر موفقیت‌آمیز بودن نصب نمایش داده خواهد شد. رسانه نصب را از سرور خارج کرده و کلید ENTER را بزنید تا سرور راه‌اندازی مجدد شود.



تصویر ۱۴-۲- پایان نصب

ورود و پیکربندی اولیه. وقتی سیستم به طور کامل بوت شد، صفحه اصلی مدیریت ESXi نمایش داده می‌شود. کلید F2 را فشار داده و نام کاربری root و رمز خود را وارد کنید. پس از ورود، تنظیمات شبکه را برای مدیریت سرور به‌صورت زیر پیکربندی نمایید:

ابتدا در منوی اصلی، گزینه "Configure Management Network" را انتخاب کرده و سپس آدرس IP را به‌صورت ایستا یا از طریق سرور DHCP تنظیم نمایید.

در این آزمایش، آدرس آی‌پی شبکه محلی ما 192.168.154.0/24 می‌باشد و سرور با آدرس 192.168.154.4 در این شبکه قرار دارد.

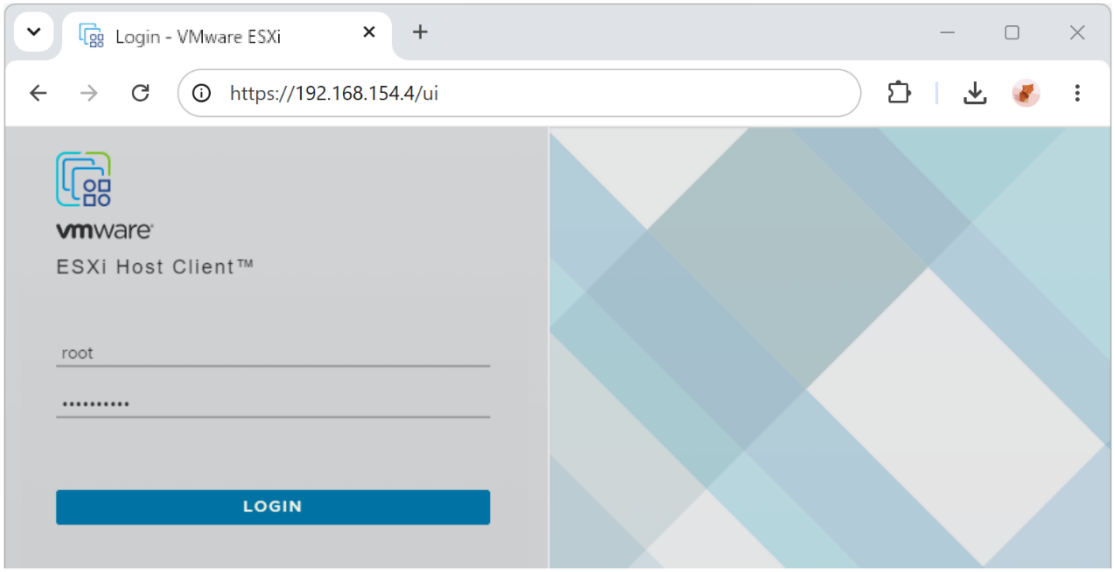
۱۴-۳- مدیریت ماشین‌های مجازی

پس از اتمام عملیات نصب، می‌توان تمام عملیات مدیریتی از جمله ایجاد، تغییر و حذف ماشین‌های مجازی را از طریق یک رابط کاربری تحت وب با نام VMware Host Client که به‌منظور مدیریت سرور ESXi طراحی شده است، به انجام رساند.

برای دسترسی به این رابط تحت وب، ابتدا در یک سیستم دیگر که در همان شبکه محلی قرار دارد، یک مرورگر وب را باز کرده و در نوار آدرس، آی‌پی سرور را به‌صورت زیر وارد کنید.

```
https://192.168.154.4/ui
```

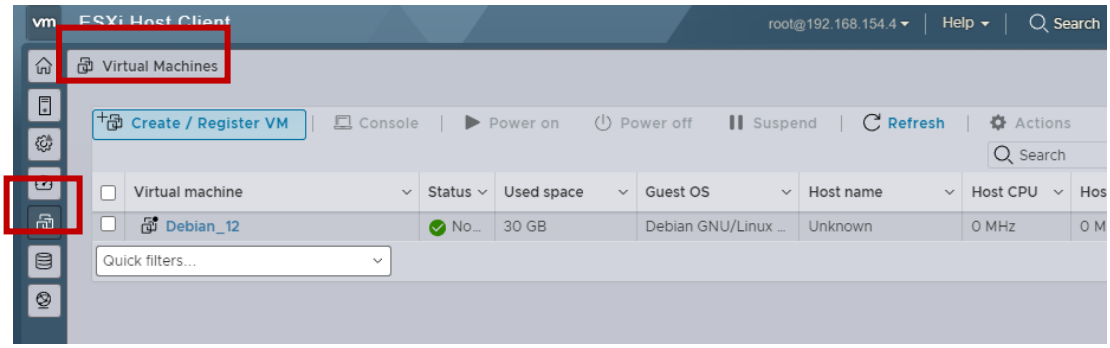
یک صفحه مانند تصویر ۱۴-۳ باز شده و نام کاربری و رمزعبور از شما درخواست می‌گردد. نام کاربری root و رمزعبوری که هنگام نصب انتخاب کرده بودید را وارد کنید. حال به تمام امکانات و تنظیمات سرور دسترسی دارید.



تصویر ۱۴-۳- صفحه ورود در رابط کاربری تحت وب

۱۴-۳-۱- ایجاد ماشین‌مجازی

پس از ورود به سرور، مانند آنچه در تصویر ۱۴-۴ مشاهده می‌نمایید، از منوی سمت چپ گزینه Virtual Machines را انتخاب کنید. حال در قسمت بالای صفحه روی دکمه Create / Register VM کلیک نمایید. یک پنجره جدید باز می‌شود که در پنج مرحله شما را برای ساخت ماشین‌مجازی جدید هدایت خواهد کرد.



تصویر ۱۴-۴- صفحه اصلی رابط کاربری

مرحله اول: گزینه Create a new virtual machine را انتخاب کرده و Next را بزنید.

مرحله دوم: برای ماشین‌مجازی خود یک نام انتخاب کرده، نوع و نسخه سیستم‌عامل مهمان را مشخص نمایید.

مرحله سوم: محل ذخیره‌سازی[۴] فایل‌های ماشین‌مجازی را از لیست انتخاب انتخاب کنید.

مرحله چهارم: در این مرحله، منابع سخت‌افزاری ماشین‌مجازی مانند تعداد پردازنده‌ها، مقدار حافظه اصلی و دیسک را تنظیم می‌شود. برای نصب سیستم‌عامل، در بخش CD/DVD Drive گزینه Datastore ISO file را انتخاب کرده و سپس در بخش CD/DVD Media با زدن دکمه Browse محل فایل ISO را مشخص کنید.

مرحله پنجم: خلاصه‌ای از تنظیمات انجام شده را نمایش می‌دهد. در صورت اطمینان از انتخاب‌های خود، کلید Finish را کلیک کنید تا ماشین‌مجازی ساخته شود.

توجه: در پایان مراحل بالا، فقط یک ماشین‌مجازی با مشخصات تعیین شده ایجاد می‌گردد ولی سیستم عامل نصب نخواهد شد.

---

[4] Datastore

۱۴-۳-۲- نصب سیستم‌عامل در ماشین‌مجازی

پس از اتمام مراحل بخش قبل، نام ماشین‌مجازی شما در بخش Virtual Macnines لیست شده است. درصورتی که قصد نصب سیستم‌عامل را دارید، ابتدا ماشین‌مجازی مد نظر را انتخاب کرده و سپس کلید Power on را از بالای پنجره کلیک نموده و سپس کلید Console را که در سمت چپ آن قرار دارد بفشارید تا یک لیست باز شود. حال گزینه Open Browser Console را انتخاب نمایید. یک پنجره جدید باز شده و مراحل نصب آغاز می‌گردد.

۱۴-۳-۳- مدیریت ماشین‌مجازی

برای ایجاد تغییر در تنظیمات یک ماشین‌مجازی دوباره به لیست Virtual Machines باز گردید. حال روی ماشین‌مجازی موردنظر راست‌کلیک کرده و گزینه Edit Settings را انتخاب نمایید. در پنجره باز شده، می‌توانید منابع سخت‌افزاری را تغییر داده و یا دستگاه‌ها را اضافه یا حذف کنید. فراموش نکنید که در انتها، تغییرات را ذخیره نمایید.

۱۴-۳-۴- حذف ماشین‌مجازی

ابتدا ماشین‌مجازی موردنظر را انتخاب کرده و با زدن دکمه Power off، آن را خاموش نمایید. در صورت روشن بودن ماشین‌مجازی، گزینه حذف غیرفعال خواهد بود. حال روی ماشین‌مجازی راست‌کلیک کرده و گزینه Delete را انتخاب کنید. در پنجره تایید نیز روی Delete کلیک نمایید تا ماشین‌مجازی و تمام فایل‌های مرتبط با آن به‌طور کامل از محل ذخیره داده‌ها حذف شوند.

توجه: حذف یک ماشین‌مجازی، به معنای حذف دائمی تمام فایل‌های آن ماشین از روی دیسک می‌باشد. بنابراین اگر می‌خواهید ماشین‌مجازی را بدون پاک کردن، فقط از لیست مدیریتی پنهان نمایید؛ باید از گزینه Unregister استفاده کنید. برای دسترسی به این گزینه، کافی است روی نام ماشین‌مجازی راست‌کلیک کنید.

۱۴-۳-۵- ایجاد ومدیریت کاربران

برای اینکه کاربران بتوانند به ماشین‌های مجازی خود در ESXi دسترسی داشته باشند، ابتدا باید یک حساب کاربری برای آنها ایجاد کرده و سپس سطح دسترسی[5] مناسب را به آن‌ها اختصاص دهیم. به طور پیش‌فرض کاربران جدید، هیچ دسترسی‌ای به ماشین‌های مجازی ندارند.

ایجاد یک کاربر جدید. در صفحه اصلی رابط تحت وب، روی گزینه Manage از منوی سمت چپ کلیک کنید. به زبانه Security & Users رفته و سپس روی Users کلیک نمایید. گزینه Add user را زده و اطلاعات کاربر، شامل یک نام کاربری یکتا و رمزعبور را وارد کنید.

نکته: ESXi هم در نام کاربری و هم در رمزعبور به حروف بزرگ و کوچک حساس است.

اختصاص دسترسی به کاربر. ابتدا از لیست Virtual Machines روی ماشین موردنظر راست‌کلیک کرده و گزینه Permissions را انتخاب کنید. در پنجره باز شده، روی Add user کلیک کرده و سپس در بخش Select a user، کاربر موردنظر را از لیست انتخاب نمایید. در نهایت، در بخش Select a role، یک نقش مناسب به کاربر اختصاص دهید. مثلا برای یک کاربر عادی که فقط ماشین‌مجازی خود را مدیریت می‌کند، توصیه می‌شود یک نقش سفارشی[6] با کمترین دسترسی‌های لازم ایجاد کنید.

بری ایجاد نقش سفارشی، پس از طی کردن مسیر زیر، یک نام دلخواه وارد کرده و سپس دسترسی‌های مورد نیاز را انتخاب کنید.

```
Manage > Security & Users > Roles > Add Role
```

---

[5] Role
[6] Custom Role

به‌عنوان مثال برای یک کاربرعادی، گزینه‌های Console interaction، Power On و Power Off از بخش زیر مکفی خواهد بود.

```
Virtual machine > Interaction
```

حال اطلاعات زیر را در اختیار کاربر مربوطه قرار دهید تا بتواند به ماشین‌مجازی خود متصل شده و از آن استفاده نماید:

- آدرس آی‌پی یا نام دامنه ثبت شده برای سرور
- نام کاربری و رمزعبور کاربر
- نام ماشین‌مجازی ایجاد شده برای کاربر

۱۴-۴- اتصال کاربران

به‌عنوان یک کاربر، از طریق مرورگر وب به آدرس سرور مراجعه کرده و با استفاده از نام کاربری و رمزعبور وارد شده و مراحل زیر را دنبال نمایید:

- از منوی سمت چپ، بخش Virtual Machines را انتخاب کنید.
- از لیست ماشین‌های مجازی، روی ماشین موردنظر خود کلیک نمایید.
- از نوار ابزار بالای پنجره، روی آیکون Console کلیک کرده و گزینه Open Browser Console را انتخاب نمایید.

نکته ۱: برای آزاد کردن ماوس، کلیدهای Ctrl + Alt را فشار دهید.

نکته ۲: بسته به میزان دسترسی‌هایی که به شما اعطا شده است می‌توانید از ماشین‌مجازی خود استفاده کرده و یا در آن تغییرات لازم مانند نصب سیستم‌عامل را انجام بدهید.

تمرین

با استفاده از یک سیستم کامپیوتری در آزمایشگاه سیستم‌عامل که به شبکه محلی آزمایشگاه متصل است، به سرور متصل شوید. برای اتصال، از اطلاعات حساب کاربری که برای شما ایجاد و در اختیارتان قرار داده شده استفاده نمایید.

- در صفحه اصلی، اطلاعات سرور شامل پردازنده، حافظه اصلی و فضای ذخیره‌سازی را مشاهده نمایید.
- از منوی Storage، روی Datastore سرور کلیک کرده و محتویات آن را با زدن Datastore browser مشاهده کنید. درصورتی‌که بر روی سرور Datastore وجود ندارد، خودتان یک مورد بسازید.
- در یکی از Datastoreها، پوشه‌ای به نام خودتان ایجاد کنید.
- فایل ISO یک توزیع لینوکسی مانند Ubuntu Server را در پوشه مذکور آپلود نمایید.
- با استفاده از بخش Create / Register VM، یک ماشین‌مجازی جدید ایجاد کرده و تنظیمات لازم را انجام دهید تا به فایل ISO که در بخش قبل آپلود کردید متصل بشود. سپس با باز کردن کنسول VM، سیستم‌عامل را روی ماشین‌مجازی ایجاد شده نصب نمایید.

گزارش کار

در ماشین‌مجازی که در تمرین بالا ایجاد کردید، تنظیمات آداپتور شبکه را روی Bridged قرار دهید تا VM مانند یک میزبان مستقل در شبکه ظاهر شود. همچنین یک آدرس IP ثابت و قابل دسترسی در شبکه محلی - مانند 192.168.154.100 - برای آن تنظیم نمایید.

- از یک سیستم دیگر در آزمایشگاه، دسترسی به آن آدرس را با استفاده از دستور ping بررسی کرده و با استفاده از یک مرورگر وب به ماشین‌مجازی متصل شوید. در نهایت ماشین‌مجازی را خاموش کرده و خارج شوید.
- دوباره به پشت سیستم قبلی با دسترسی مدیریتی بازگردید. از ماشین‌مجازی که ایجاد کرده بودید، یک snapshot تهیه نمایید. سپس، داخل VM یک فایل متنی ساده بسازید و در آن یک خط متن بنویسید. با استفاده از کنسول Host Client، ماشین‌مجازی را به‌صورت ناگهانی خاموش کنید. به عبارت دیگر، به جای

Shut Down Guest از Power Off استفاده نمایید. دوباره ماشین را روشن کرده و بررسی کنید چه پیامی هنگام بوت شدن لینوکس نمایش داده می‌شود و آیا فایل متنی، سالم باقی مانده است یا خیر.

- از ماشین‌مجازی که قبلا ایجاد کرده بودید، یک همسان[۷] با تنظیمات همسان‌سازی کامل، ایجاد کرده و سپس نام سیستم‌عامل را در آن تغییر داده و همچنین یک آدرس IP جدید و معتبر به آن اختصاص بدهید. با استفاده از Export/Import توضیح دهید که چگونه می‌توان این ماشین‌مجازی را به یک سرور فیزیکی دیگر منتقل کرد.

---

[7] Clone

منابع و مراجع:

- Marshall N. *Mastering VMware vSphere 6.7*. John Wiley & Sons, 2018.
- https://docs.vmware.com/en/VMware-vSphere/index.html, visited 2024 Aug.
- https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0/esx-installation-and-setup/installing-and-setting-up-esxi-install.html, visited 2026 Aug.

آزمایش ۱۵ : حافظه مجازی

اهداف:

- درک مفهوم حافظه مجازی
- آشنایی با ابزارهای نظارت و تخصیص حافظه به برنامه‌ها

ابزار و مفاهیم مورد نیاز:

- ابزارهای شبیه‌ساز تخصیص حافظه مانند stress
- دستورات نظارت بر حافظه مانند vmstat, free, lshw
- ابزارهای کامپایل خط فرمانی

محتوا:

- تکوین روش‌های مختلف مدیریت حافظه در سیستم‌عامل
- حافظه مجازی
  - صفحه‌بندی و پارتیشن‌بندی
  - جدول صفحه و ترجمه آدرس
  - TLB
  - تخصیص حافظه با کمک برنامه‌های مختلف
  - نظارت بر میزان مصرف حافظه

۱۵-۱- شیوه‌های مدیریت حافظه

برنامه‌ای که بر روی یک کامپیوتر نصب شده، در قسمتی از هارد دیسک قرار گرفته است. هنگامی که برنامه را اجرا می‌کنید، سیستم‌عامل آن برنامه را بر روی حافظه اصلی سیستم بارگذاری کرده و پس از ایجاد فضای حافظه و سایر موارد لازم، آن را برای اجرا به پردازنده اعزام می‌نماید. همان‌طور که می‌دانید این برنامه‌ی در حال اجرا، فرآیند نامیده می‌شود.

با توجه به محدودیت فضای حافظه اصلی، لازم است نحوه بارگذاری و ورود وخروج این فرآیندها در حافظه مدیریت شود. شیوه‌های مختلفی بدین‌منظور ایجاد و تکامل یافته‌اند که در ادامه برخی از مهمترین این روش‌ها را به ترتیب بررسی می‌کنیم.

۱۵-۱-۱- تک برنامگی

در هر لحظه علاوه بر سیستم عامل، تنها یک برنامه دیگر در حافظه قرار دارد. برنامه، اجرا شده و از حافظه خارج می‌گردد و سپس نوبت به بارگذاری برنامه بعدی می‌رسد و با همین منوال ادامه می‌یابد. در این روش اتلاف حافظه و پردازنده بالاست و بنابراین با کاهش بهره‌وری مواجهیم.

۱۵-۱-۲- قسمت‌بندی

در این روش، با استفاده از قسمت‌بندی[1]، امکان چندبرنامگی فراهم شد. یعنی حافظه اصلی به چندین قسمت[2] با اندازه‌های مختلف تقسیم شده و سیستم عامل و برنامه‌ها، هرکدام در یکی از این قسمت‌ها قرار می‌گیرند. به این ترتیب امکان استفاده همزمان از منابع فراهم شده و بهره‌وری افزایش می‌یابد. در این روش، هر یک از برنامه‌ها باید به طور کامل و فقط در یک قسمت از حافظه بارگذاری بشوند.

---

[1] Partitioning
[2] Partition

قسمت‌بندی حافظه، به دو روش ایستا و پویا انجام می‌پذیرد. در حالت ایستا، اندازه قسمت‌ها ثابت است و پس از روشن شدن سیستم، امکان تغییر ندارد. بنابراین اگر برنامه‌ای فقط نیمی از یک قسمت را پر کند، نیمه دیگر از آن قسمت بلااستفاده باقی خواهد ماند زیرا هیچ برنامه دیگری اجازه استفاده از آن قسمت را ندارد. و همچنین اگر اندازه‌ی برنامه‌ای از اندازه بزرگترین قسمت حافظه بزرگتر باشد، آن برنامه روی این سیستم امکان اجرا ندارد. مگر اینکه سیستم خاموش شده و مجددا قسمت‌بندی انجام بگیرد تا درصورت امکان، یک قسمت با اندازه موردنیاز ایجاد گردد.

در حالت پویا، اندازه قسمت‌ها از قبل تعیین نمی‌گردد. بلکه هنگام بارگذاری هر برنامه، یک قسمت با اندازه مناسب برای آن برنامه ایجاد می‌شود. در این روش نیز برای اجرای یک برنامه لازم است که کل آن برنامه در حافظه بارگذاری گردد. اما درصورت کمبود فضا در حافظه، امکان مبادله[3] فرآیندها با دیسک جانبی وجود دارد. مثلا اگر کاربر بخواهد برنامه‌ای را اجرا کند و در حال حاضر فضای خالی حافظه کمتر از فضای مورد نیاز آن برنامه باشد، آن‌گاه سیستم‌عامل یک یا چند فرآیند معلق[4] را به‌صورت موقت از حافظه اصلی خارج کرده و در حافظه جانبی قرار می‌دهد و سپس فرآیند جدید را در حافظه بارگذاری می‌نماید.

نکته: وجه مشترک روش‌های تک‌برنامگی و قسمت‌بندی ایستا و پویا، در این است که در هر سه روش، باید کل فرآیند به‌صورت یک‌جا و پشت سرهم در حافظه قرار بگیرد. این ضرورت باعث ایجاد حفره‌های خالی در حافظه و در نتیجه هدررفت حافظه می‌شود.

۱۵-۱-۳- صفحه‌بندی

در این روش فرآیندها به اندازه‌های ثابتی به اسم صفحه[5] یا بخش[6] و یا ترکیبی از این دو، تقسیم شده و به‌صورت تکه‌تکه درمی‌آیند. صفحات حاصل می‌توانند بر اساس ظرفیت خالی موجود، در نقاط مختلفی از حافظه قرار بگیرند. به

---

[3] Swap
[4] Suspended
[5] Page
[6] Segment

این ترتیب، بر خلاف روش‌های قبلی، در این روش دیگر الزامی برای بارگذاری یکپارچه‌ی فرآیند در حافظه وجود ندارد اما همچنان باید تمام بخش‌های آن فرآیند در حافظه بارگزاری گردد.

با توجه به اینکه یک فرآیند به تکه‌های مختلفی تقسیم و در حافظه اصلی پراکنده می‌شود، برای مراجعه به هر یک از این تکه‌ها نیاز به دانستن آدرس آن تکه در حافظه داریم. به همین دلیل، مفاهیم جدول صفحه و ترجمه آدرس بوجود آمدند که در بخش‌های بعدی به آن‌ها خواهیم پرداخت.

۱۵-۲- حافظه مجازی

آخرین تکنیک مدیریت حافظه که بدان خواهیم پرداخت حافظه مجازی می‌باشد که در سیستم‌های عامل امروزی مورد استفاده قرار گرفته و امکان استفاده بهینه از حافظه را فراهم آورده است. حافظه مجازی به‌عنوان یک لایه انتزاعی، بین حافظه فیزیکی و فرآیندهای در حال اجرا بر روی سیستم عمل کرده و به فرآیندها این تصور را می‌دهد که سیستم دارای حافظه‌ای پیوسته و نامحدود است - حتی اگر حافظه فیزیکی در واقعیت محدود باشد. جداسازی فرآیندها نیز از دیگر مزایای حافظه مجازی می‌باشد.

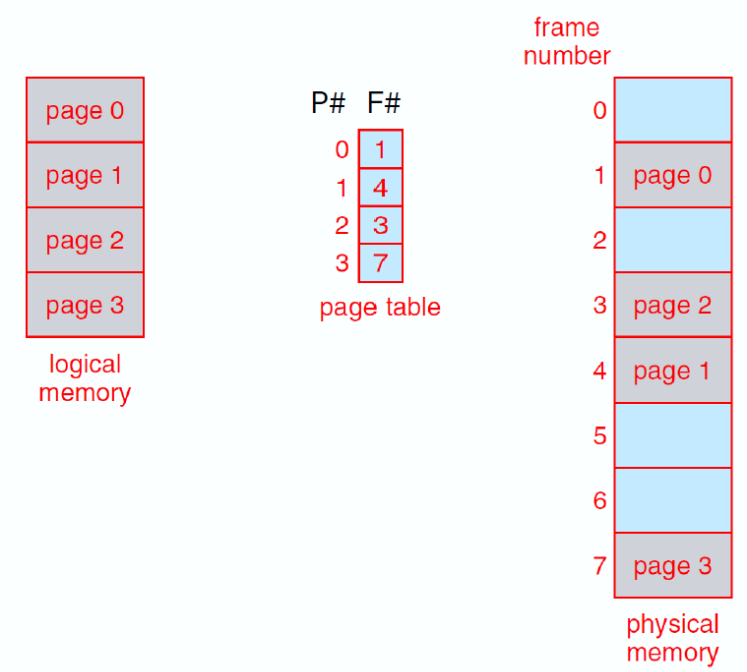
حافظه مجازی نه تنها از صفحه‌بندی استفاده می‌کند، بلکه فقط صفحات مورد نیاز فعلی فرآیند را در حافظه فیزیکی بارگذاری می‌نماید. بنابراین برخلاف روش‌های قبلی، دیگر نیازی به بارگذاری کامل فرآیند در حافظه نداریم.

۱۵-۲-۱- جدول صفحه

به هر فرآیند، فضای آدرس مجازی مخصوص به خود داده شده و هر صفحه از این حافظه مجازی، در یکی از قاب‌های[7] حافظه اصلی قرار می‌گیرند. این صفحات و قاب‌ها توسط سیستم‌عامل و با استفاده از جدول صفحه به یکدیگر نگاشت می‌شوند. در تصویر ۱۵-۱ می‌توانید یک جدول صفحه ساده را مشاهده کنید.

---

[7] Frame



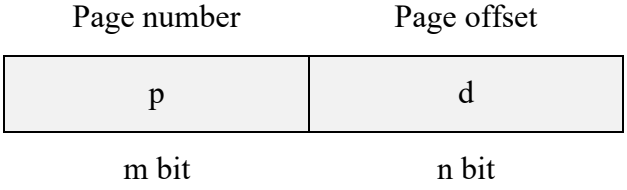
تصویر ۱۵-۱- صفحات حافظه مجازی، جدول صفحه و قاب‌های حافظه فیزیکی

هنگامی که یک فرآیند نیاز به دسترسی به داده‌هایش داشته باشد، سیستم عامل با استفاده از جدول صفحه، آدرس مجازی را به آدرس فیزیکی ترجمه می‌کند. اگر فرآیندی سعی کند به صفحه‌ای دسترسی یابد که در حال حاضر در حافظه فیزیکی نیست، نقص‌صفحه[۸] رخ می‌دهد و بنابراین سیستم عامل، صفحه مورد نیاز را از حافظه ثانویه به حافظه اصلی بارگذاری می‌نماید. همچنین هرگاه حافظه اصلی پر باشد، سیستم‌عامل باید تصمیم بگیرد که کدام صفحات را مبادله نماید تا فضا برای صفحات جدید باز شود. این مورد توسط الگوریتم‌های جایگزینی صفحه مدیریت می‌شود، مثلا الگوریتم اخیرا کمتر استفاده شده[۹](LRU) یا به ترتیب ورود[۱۰](FIFO)

[8] Page Fault
[9] Least Recently Used
[10] First In First Out

۱۵-۲-۲- ترجمه آدرس

آدرس مجازی به دو بخش شماره صفحه و افست[11] تقسیم می‌شود. شماره صفحه، برای ایندکس کردن در جدول صفحه استفاده می‌شود و نشان می‌دهد که داده موردنظر در کدام صفحه منطقی قرار گرفته است. افست نیز محل دقیق داده موردنظر را در داخل صفحه مذکور مشخص می‌نماید. برای ترجمه آدرس در صفحه بندی، کافی است شماره صفحه را با شماره قاب جایگزین کنیم.



تصویر ۱۵-۲- آدرس مجازی

۱۵-۲-۳- TLB

برای حل چالش سربار کارایی و کندی سرعت ترجمه آدرس، از TLB[12] استفاده می‌شود که نوعی رجیستر با قابلیت جستجوی موازی است. اطلاعات صفحاتی که اخیرا توسط یک فرآیند مورد استفاده قرار گرفته‌اند را در این حافظه نهان ذخیره می‌کنیم تا در مراجعات بعدی، سرعت دستیابی به آن‌ها افزایش یابد.

---

[11] Offset

[12] Translation Lookaside Buffer

۱۵-۳- نظارت بر تخصیص حافظه مجازی

یکی از روش‌های نظارت بر استفاده از حافظه، دستور vmstat است. این دستور، آمار دقیقی در مورد حافظه مجازی، فرآیندها و میزان استفاده از پردازنده ارائه می‌دهد. ابتدا بنویسید vmstat و سپس فاصله زمانی (بر حسب ثانیه) و تعداد به‌روزرسانی‌های موردنظرتان را اضافه کنید. به‌عنوان مثال، برای به‌روزرسانی هر ۲ ثانیه به مدت ۵ بار:

```
vmstat   2   5
```

در جدول ۱۵-۱ خروجی این دستور را مشاهده می‌نمایید. این اطلاعات درباره کل فرآیندها و سیستم هستند. در ادامه اطلاعات یک تک فرآیند را بررسی خواهیم کرد.

خروجی	توضیحات
Procs	تعداد فرآیندها
Memory	اطلاعاتی درباره میزان مصرف حافظه
Swap	میزان فضای مبادله مورد استفاده
IO	عملیات ورود و خروج
System	فرآیندهای سیستمی
CPU	آمار میزان استفاده از پردازنده

جدول ۱۵-۱- خروجی دستور vmstat

برنامه‌ای را که می‌خواهید نظارت کنید، اجرا نمایید. به‌عنوان مثال، یک برنامه ساده مانند stress[13] را برای شبیه‌سازی استفاده از حافظه راه‌اندازی کنید. درصورتی‌که این برنامه روی سیستم شما موجود نیست، ابتدا آن را با دستور زیر نصب نمایید.

```
sudo   apt-get   install   stress
```

[13] Simulate Memory Usage

سپس با دستور زیر ۲۵۶ مگابایت حافظه را به خود اختصاص بدهید.

```
stress   --vm  1   --vm-bytes  256M   --vm-hang  10
```

برای مشاهده حافظه تخصیص یافته، یک ترمینال دیگر باز کرده و دستور top را اجرا کنید. با این دستور قبلا در بخش خط فرمان آشنا شده‌اید. پس از اجرای این دستور، اطلاعاتی در ترمینال برایتان نمایش داده خواهد شد. قسمت بالایی، شامل اطلاعات کلی سیستم از جمله میزان مصرف حافظه می‌باشد. در قسمت پایینی، تمام فرآیندهای در حال اجرا و اطلاعات مرتبط با آن‌ها، بویژه میزان مصرف حافظه برای هر فرآیند فهرست شده است. برای مرتب‌سازی ستون‌ها بر اساس میزان مصرف حافظه، Shift + M را فشار دهید. احتمالا فرآیند مرتبط با stress را در خطوط ابتدایی مشاهده خواهید کرد.

۱۵-۳-۱- نقشه حافظه مجازی

در حالیکه برنامه stress هنوز در حال اجراست، دریک ترمینال دیگر، با دستور زیر (یا هر دستور مناسب دیگر) شناسه فرآیند را شناسایی کنید.

```
ps   -aux  |  grep  stress
```

با در دست داشتن شناسه یک فرآیند، می‌توانید نقشه حافظه مجازی مرتبط با آن را با خواندن فایل /proc/[pid]/maps مشاهده نمایید. به‌عنوان مثال، اگر PID = 4025 باشد، دستور زیر محتویات نقشه حافظه مربوط به این فرآیند را در ترمینال نمایش خواهد داد.

```
cat   /proc/4025/maps
```

یک نمونه maps در زیر آورده شده است.

```
594259cdc000-594259cdd000 r--p 0000000 08:03 656211 /usr/bin/stress
594259cdd000-594259ce0000 r-xp 0001000 08:03 656211 /usr/bin/stress
594259ce2000-594259ce3000 rw-p 0006000 08:03 656211 /usr/bin/stress
```

```
7930ee000000-7930fe001000 rw-p 0000000 00:00 0      [heap]
7930fe200000-7930fe228000 r--p 0000000 08:03 687954 /usr/lib/x86_64-
linux-gnu/libc.so.6
…
```

این نقشه نشان می‌دهد که چگونه فضای حافظه مجازی یک فرآیند، به بخش‌های مختلف مانند فایل اجرایی، کتابخانه‌های اشتراکی، heap، stack و غیره نگاشت شده است.

توضیح بخش‌های نقشه نگاشت حافظه

ستون اول، محدوده آدرس‌های مجازی که به این ناحیه از حافظه نگاشت شده‌اند را نشان می‌دهد. نقشه، هر ناحیه از حافظه را به‌صورت «آدرس شروع : آدرس پایان» مشخص می‌کند.

ستون دوم، مجوزهای حافظه را با علائم اختصاری r=read، w=write، x=execute، s=shared / p=private نمایش می‌دهد. کاراکتر p یعنی حافظه با سایر فرآیندها به اشتراک گذاشته نمی‌شود.

ستون سوم، افست فایل یا حافظه را نشان می‌دهد. عدد ۰ یعنی این نگاشت به فایل خاصی مرتبط نمی‌شود؛ مثلا ممکن است یک تخصیص حافظه ناشناس[۱۴] باشد.

ستون چهارم، شماره دستگاه مربوط به حافظه را با فرمت major:minor نشان داده و مشخص می‌کند که فایل به کدام دستگاه مرتبط می‌شود. عدد ۰ یعنی حافظه با فایل خاصی مرتبط نیست.

ستون پنجم، عدد iNode فایلی را نشان می‌دهد که به حافظه نگاشت شده است. iNode یک ساختار داده است که توسط فایل‌سیستم برای نمایش یک فایل یا دایرکتوری، مورد استفاده قرار می‌گیرد. درباره این ساختار، در آزمایش فایل‌سیستم به‌طور کامل توضیح داده شده است.

---

[14] Anonymous

ستون ششم، pathname فایل مرتبط با ناحیه حافظه را در صورت وجود، نشان می‌دهد. اگر نگاشت برای یک ناحیه ناشناس مثل Heap یا Stack صورت بگیرد، این فیلد خالی می‌ماند و یا عبارت [heap] یا [stack] را نشان خواهد داد.

۱۵-۴- برنامه‌ای برای تخصیص حافظه

در این بخش، یک برنامه ساده به نام «تخصیص حافظه» با زبان C می‌نویسیم تا یک بلوک بزرگ از حافظه را با استفاده از ()malloc تخصیص داده و به آن دسترسی پیدا کند. سپس برنامه به مدت ۳۰ ثانیه به حالت توقف درمی‌آید تا کاربر فرصت کافی برای مشاهده چگونگی تخصیص حافظه مجازی را با استفاده از ابزارهای مناسب، داشته باشد.

```
//Memory_Allocation.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    size_t size = 100 * 1024 * 1024; // 100 MB
    int* buffer = (int*)malloc(size);

    if (buffer == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;   }

    // برای اطمینان از تخصیص حافظه، به آن دسترسی پیدا کرده و اطلاعاتی را در آن می‌نویسیم
    size_t elements = size / sizeof(int);
    for (size_t i = 0; i < elements; ++i) {
        buffer[i] = i;   }

    printf("Memory allocated and used. Check process memory with
`ps` or `top`.\n");
    sleep(30); // Pause to allow observation

    free(buffer);
    return 0;   }
```

برنامه را کامپایل کرده و اجرا نمایید. سپس یک ترمینال دیگر باز کرده و با استفاده از دستور top وضعیت حافظه تخصیص یافته را بررسی کنید.

نکته: size_t یک نوع داده بدون علامت در زبان C است که برای نمایش اندازه‌ها و مقادیر حافظه استفاده می‌شود و بسته به پلتفرم معمولا معادل unsigned long یا unsigned long long است.

۱۵-۵- دستورات بررسی حافظه سیستم

۱۵-۵-۱- میزان حافظه فیزیکی

برای بررسی میزان حافظه فیزیکی[15] سیستم دستور زیر را اجرا کنید. این دستور میزان حافظه کل، حافظه استفاده شده و حافظه آزاد را در قالبی قابل خواندن برای انسان نمایش می‌دهد.

```
free   -h
```

دستور دیگر، lshw می‌باشد که اطلاعات دقیق سخت افزاری از جمله اندازه حافظه را ارائه می‌دهد.

```
sudo    lshw   -C    memory
```

پس از بررسی این موارد، میزان حافظه فیزیکی سیستم خود را به دست آورده و یادداشت نمایید. در بخش بعد، رفتار سیستم‌عامل را در شرایطی که حافظه درخواستی از حافظه فیزیکی سیستم فراتر رود، مشاهده خواهیم کرد. بدین منظور لازم است که از میزان حافظه واقعی سیستم که با دستورات بالا به‌دست آوردیم، اطلاع داشته باشیم و برای ایجاد چنین شرایطی از برنامه تخصیص حافظه که در بخش قبل نوشتیم، استفاده خواهیم کرد.

---

[15] RAM

۱۵-۵-۲- تخصیص حافظه بیش‌از حد

به برنامه تخصیص حافظه که در قسمت قبل نوشتیم رفته و میزان حافظه تخصیص یافته را به مقداری بسیار بیشتر از حافظه موجود در سیستم، تغییر دهید. مثلا برای سیستمی با ۲.۵ گیگابایت حافظه، مقدار را برابر با ۶ یا ۸ گیگابایت قرار دهید.

```
size_t size = 8 * 1024 * 1024 * 1024ULL;    // 8 GB
```

نکته: پسوند ULL در انتهای خط باعث می‌شود تمام محاسبه به unsigned long long ارتقا یابد و جلوی خطای احتمالی سرریز عددصحیح گرفته شود.

برنامه را دوباره کامپایل و اجرا نمایید. به احتمال زیاد با خطای عدم امکان تخصیص حافظه مواجه خواهید شد. علت این است که هرگاه میزان حافظه‌ی درخواستی، از مجموع حافظه‌ی فیزیکی به اضافه فضای مبادله بیشتر باشد، یکی از مکانیزم‌های هسته به نام OOM Killer[16] فعال شده و یکی از فرآیندها - که اغلب همان فرآیند متقاضی تخصیص حافظه زیاد است - را می‌کشد تا سیستم پایدار بماند.

نکته: در برخی موارد، ممکن است malloc به‌جای OOM Killer، خودش با شکست مواجه شده و پیام « Memory allocation failed» که در برنامه در نظر گرفته شده است، چاپ گردد. این امر به تنظیمات vm.overcommit_memory و میزان حافظه‌ی درخواستی بستگی دارد.

---

[16] Out-Of-Memory Killer

۱۵-۵-۳- تعهد بیش از حد و نقص صفحه

این بار، عدد مذکور را به مقداری که کمی بیشتر از حافظه واقعی سیستم باشد، مثلا ۳ یا ۴ گیگابایت، تغییر دهید. در این حالت، حافظه درخواستی به برنامه تخصیص داده خواهد شد - حتی با اینکه از حافظه فیزیکی سیستم بیشتر است. علت این است که هسته لینوکس با استفاده از زیرسیستم مدیریت حافظه[۱۷] خود، مکانیزم «تعهد بیش از حد[۱۸]» را اجرا می‌کند که به malloc اجازه می‌دهد حتی بیشتر از حافظه‌ی فیزیکی موجود، درخواست حافظه کند. در پاسخ به این درخواست، فضای آدرس مجازی، به میزان درخواست شده، رزرو می‌گردد؛ بدون اینکه هیچ حافظه‌ی فیزیکی‌ای تخصیص داده شود. تخصیص واقعی حافظه فیزیکی، بر اساس مکانیزم «تخصیص دیرهنگام[۱۹]» و تنها زمانی رخ می‌دهد که برنامه به آن حافظه دسترسی پیدا کند. که در این مثال، دسترسی به حافظه با نوشتن در آن، انجام می‌پذیرد.

یکی از دو دستور زیر را اجرا کرده و سپس در یک ترمینال دیگر، برنامه تخصیص حافظه را اجرا نمایید. مشاهده خواهید کرد که تعداد نقص صفحه در حین اجرای برنامه، به طرز چشمگیری افزایش خواهد یافت.

برای مشاهده‌ی نقص صفحه در کل سیستم:

```
// Page Faults of the System
watch -n 1 "cat /proc/vmstat | grep -E 'pgfault|pgmajfault'"
```

و یا برای مشاهده‌ی نقص صفحه برای یک فرآیند خاص:

```
//Page Fault Monitor
watch -n 1 "ps -eo pid,comm,min_flt,maj_flt | grep $(PID)"
```

در این دستور، شناسه برنامه خود را با $(PID) جایگزین نمایید.

---

[17] Memory Management Subsystem
[18] Overcommit
[19] Lazy Allocation

علت این است که وقتی برنامه به بخشی از کد می‌رسد که طی آن به صفحات حافظه دسترسی پیدا کرده و در آن‌ها می‌نویسد، سیستم‌عامل باید آن صفحات را در حافظه فیزیکی بارگذاری کند. هر بار که یک صفحه در حافظه فیزیکی نباشد، یک نقص صفحه رخ می‌دهد.

نکته: major fault زمانی اتفاق می‌افتد که صفحه موردنیاز در حافظه اصلی وجود ندارد و باید از حافظه جانبی یا فضای مبادله فراخوانی شود. اما در minor fault، صفحه مدنظر در حافظه اصلی قرار دارد اما به هر دلیلی، هنوز در جدول صفحه‌ی فرآیند، ثبت نشده است. مثلا ممکن است صفحه موردنظر، یک صفحه اشتراکی بین فرآیندها باشد.

۱۵-۵-۴- دستورات تکمیلی

برنامه را سه بار دیگر نیز اجرا کرده و در هر اجرا، یکی از دستورات زیر را در یک ترمینال دیگر اجرا نمایید. تغییرات نتایج دستورات را در حین اجرای برنامه و پس از اتمام آن، به‌شرح زیر مشاهده کنید.

مشاهده‌ی حافظه‌ی آزاد. به کاهش مقدار MemFree و افزایش مقدار SwapUsed در حین اجرای برنامه و بازگشت این مقادیر به حالت قبلی خود، پس از اتمام برنامه توجه نمایید.

```
// Memory Monitor
watch -n 1 "free -h"
```

مشاهده‌ی آمار حافظه‌ی مجازی. از آنجاییکه حافظه درخواستی بیش‌تر از حافظه فیزیکی است، سیستم‌عامل مجبور به استفاده از فضای مبادله خواهد شد؛ که این امر منجر به افزایش تعداد مبادلات می‌گردد و اثرش در تغییر مقادیر ستون‌های Swap In (si) و Swap Out (so) قابل مشاهده است.

```
// Swap Monitor
```

```
watch -n 1 "vmstat 1"
```

مشاهده‌ی جزئیات حافظه. با دستور زیر نیز می‌توانید تغییرات حافظه آزاد، فضای مبادله و جدول صفحه را در حین اجرای برنامه و پس از اتمام آن مشاهده نمایید.

```
//Memory Monitor
watch -n 1 "cat /proc/meminfo | grep -E
'MemFree|SwapFree|PageTables|SwapCached'"
```

تمرین

- برنامه‌ای بنویسید که اطلاعات حافظه مجازی فرآیند جاری خود را با استفاده از تابع ()getrusage استخراج کرده و سپس، حداکثر حافظه مصرفی (ru_maxrss)، تعداد نقص‌های صفحه از نوع مینور (ru_minflt) و تعداد نقص‌های صفحه از نوع ماژور (ru_majflt) را چاپ نماید. سپس برنامه را با تخصیص ۱ مگابایت حافظه توسط malloc اجرا کرده و تغییر در مقادیر مذکور را مشاهده و مقایسه نمایید.
- یک بار دیگر برنامه را با تخصیص۱۰۰ مگابایت حافظه اجرا کرده و اطلاعات را دوباره ثبت کنید. سپس بدون ایجاد دسترسی، حافظه را آزاد کرده و بلافاصله اطلاعات را ثبت کنید. تفاوت اعداد را بررسی کرده و توضیح دهید چرا malloc به تنهایی باعث افزایش ru_maxrss نمی‌شود.

گزارش کار

- برنامه‌ی تمرین قبل را طوری تغییر دهید که ثبت اطلاعات را در سه مرحله شامل «قبل از تخصیص حافظه»، «بعد از تخصیص حافظه با استفاده از malloc، ولی قبل از دسترسی به حافظه» و «بعد از نوشتن در تمام خانه‌های حافظه تخصیص داده شده»، انجام دهد. تفاوت اطلاعات ثبت شده در هر مرحله را با مرحله قبلی خود محاسبه کرده و توضیح دهید که چرا در مرحله ۲ هیچ نقص صفحه‌ای رخ نمی‌دهد اما در مرحله ۳ تعداد زیادی نقص صفحه ثبت می‌شود.
- در برنامه‌ی قبلی یک بخش جدید اضافه کنید تا علاوه بر تخصیص حافظه با malloc، یک بار هم با calloc به اندازه یک آرایه ۱۰۰ مگابایتی حافظه تخصیص دهد. در هر دو حالت، نقص صفحه‌های مینور و ماژور را قبل و بعد از تخصیص و همینطور بعد از دسترسی به حافظه ثبت کنید. تفاوت نتایج را بررسی کرده و توضیح دهید که چرا calloc ممکن است در برخی سیستم‌ها، نقص صفحه کمتری ایجاد کند.

منابع و مراجع:

- Silberschatz A. *Operating System Concepts*, Chapter 9. Wiley,10th Edition; 2018.
- ArpaciDusseau RH. *Operating Systems: Three Easy Pieces*. 2018.
- Haghighat A, *Operating Systems with Problem Solving approach*, Pars Resaneh, 2023.
- https://man7.org/linux/man-pages/man5/proc.5.html, visited 2026 Feb.

آزمایش ۱۶ : فایل‌سیستم

اهداف:

- آشنایی با مفاهیم اصلی فایل‌سیستم
- آشنایی با ابزارهای ایجاد و مدیریت فایل‌سیستم

ابزار و مفاهیم مورد نیاز:

- یک سیستم لینوکسی
- ابزارهای خط فرمانی مرتبط با دیسک
- یک پارتیشن خالی یا یک فایل loopback

محتوا:

- معرفی انواع فایل‌سیستم
- مشاهده و بررسی اطلاعات iNode
- ایجاد و مدیریت فایل‌سیستم‌های آزمایشی و واقعی
- بررسی سلامت فایل‌سیستم و اشکال‌زدایی

۱۶-۱- انواع فایل‌سیستم

در آزمایش ۲، درباره انواع فایل‌سیستم به اختصار توضیحاتی داده شد. در این بخش، سه نوع متداول و با اهمیت را با جزئیات بیشتری بررسی خواهیم کرد. یعنی فایل‌سیستم خانواده FAT، فایل‌سیستم ویندوزی NTFS و فایل‌سیستم لینوکسی Ext. در ادامه به توضیح هر یک می‌پردازیم.

۱۶-۱-۱- FAT

خانواده‌ی فایل‌سیستم‌های FAT[1] شامل نسخه‌های متعددی شامل FAT12، FAT16، FAT32 و exFAT می‌باشد. این فایل‌سیستم‌ها با ویژگی‌هایی همچون سادگی ساختار، سازگاری بالا و فراداده محدود شناخته می‌شوند که آن‌ها را برای حافظه‌های قابل حمل و محیط‌های چندسیستم‌عاملی ایده‌آل می‌سازد.

این فایل‌سیستم در دهه‌ی ۱۹۷۰ توسط مایکروسافت و برای فلاپی دیسک‌ها معرفی شد. با گذر زمان، نسخه‌های جدیدتری توسعه یافتند. سه نسخه اول از این فایل‌سیستم، یعنی نسخه‌های ۱۲، ۱۶ و ۳۲ به‌طور تقریبا متوالی عرضه شدند و هدفشان توسعه تدریجی برای پشتیبانی از حجم‌های بزرگ‌تر، حفظ سازگاری عقب‌رو و کاهش محدودیت‌های اندازه فایل و پارتیشن بود. مثلا حداکثر اندازه فایل در FAT16 برابر با ۲ گیگابایت بود که در FAT32 به ۴ گیگابایت افزایش یافت. اما نسخه exFAT که در سال ۲۰۰۶ توسط مایکروسافت معرفی شد، دارای یک طراحی ویژه برای حافظه‌های فلش مدرن بود. در این نسخه، محدودیت‌های آخرین نسخه قبلی، یعنی FAT32 رفع شده بودند. به عبارتی، نسخه جدید از فایل‌های بسیار بزرگ و همچنین پارتیشن‌هایی تا ۱۲۸ پتابایت پشتیبانی می‌کرد و در عین حال هنوز هم سبک‌تر از NTFS بود. به دلیل سادگی و سازگاری گسترده با سیستم‌عامل‌های مختلف و همچنین طراحی بهینه نسخه‌ی اخیر برای حافظه‌های فلش، این فایل‌سیستم تبدیل به استاندارد فعلی برای کارت‌های SDXC و حافظه‌های فلش شده و امروزه همچنان در حافظه‌های USB، کارت‌های حافظه، دوربین‌های دیجیتال و ... مورد استفاده قرار می‌گیرد.

---

[1] File Allocation Table

ساختار FAT از چندین بخش اصلی به شرح زیر تشکیل شده است. این بخش‌ها را در تصویر ۱۶-۱ نیز می‌توانید مشاهده نمایید.

- **بوت سکتور**[۲]، که حاوی اطلاعات پایه مانند اندازه سکتور، تعداد نسخه‌های FAT و اندازه پارتیشن است و در ابتدای دیسک قرار می‌گیرد. همان‌طور که درتصویر مشاهده می‌کنید، یک نسخه پشتیبان نیز از جدول FAT بر روی دیسک ایجاد شده است تا در صورت خرابی یکی، دیگری قابل استفاده باشد.
- **جدول تخصیص فایل**[۳]، که یک جدول و در واقع یک لیست پیوندی از خوشه‌های[۴] فایل است و در هر خوشه، شماره خوشه بعدی فایل را نیز نگهداری می‌کند. در تصویر، دو فایل با نام‌های tux.jpg و GPL.txt را مشاهده می‌نمایید. فایل اول از خوشه ۲ شروع شده و طول آن ۱۴۰۰ بایت می‌باشد. فایل دوم نیز از خوشه ۵ شروع می‌شود و طولی برابر با ۹۸۰ بایت دارد.

در لیست پیوندی که در تصویر نمایش داده شده است، خوشه ۲ که شروع فایل tux.jpg است، به خوشه ۳ اشاره می‌کند. به این معنی که خوشه بعدی این فایل، خوشه شماره ۳ می‌باشد. خوشه ۳ نیز به خوشه ۴ اشاره می‌کند. و در نهایت، خوشه ۴ حاوی مقدار 0xFFF است که به معنای پایان فایل[۵] است. فایل دوم نیز خوشه‌های ۵ و ۶ را اشغال کرده است. خوشه‌هایی هم که با مقدار 0x000 پر شده‌اند، خوشه‌های آزاد و بدون استفاده هستند.

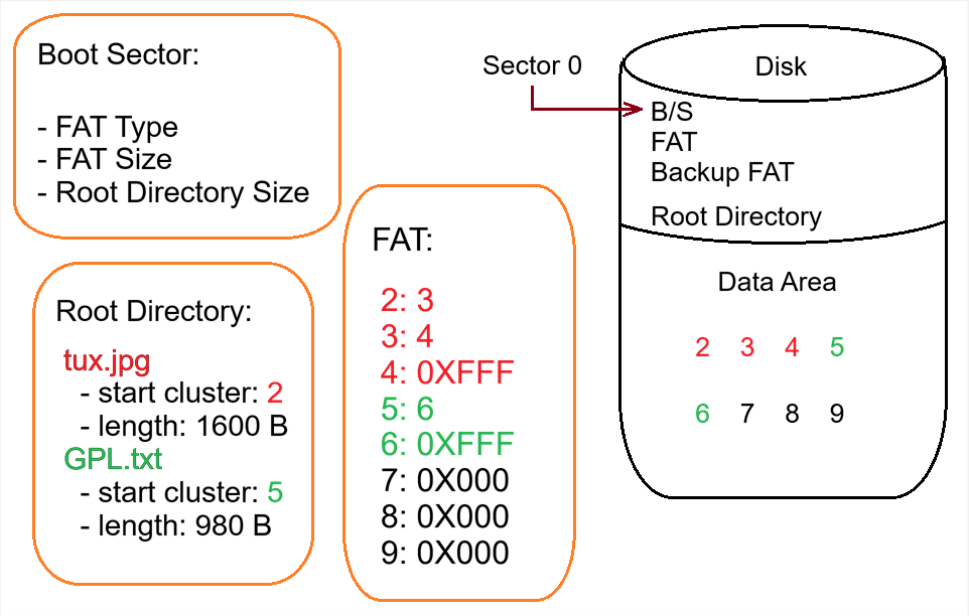
- **دایرکتوری ریشه**[۶]، که شامل لیستی از ورودی‌های دایرکتوری[۷] می‌باشد. از جمله این ورودی‌ها یکی نام فایل است که قبلا از فرمت ۸.۳ استفاده می‌کرد. به این ترتیب که نام فایل می‌توانست تا ۸ کاراکتر داشته باشد و سپس یک پسوند سه حرفی که با نقطه از آن جدا می‌شد. ورودی دیگر، ویژگی[۸] نام دارد. ویژگی‌ها، مواردی

---

[2] Boot Sector
[3] File Allocation Table
[4] cluster
[5] EoF
[6] Root Directory
[7] Directory Entry
[8] Attribute

همچون hidden، system و read-only بودن را برای فایل‌ها مشخص می‌کنند. همچنین یک ورودی دایرکتوری نیز وجود دارد که شماره خوشه شروع هر فایل را نگهداری می‌نماید.

- **ناحیه داده**[9]، که عبارت است از خوشه‌های واقعی که داده‌های فایل در آن ذخیره می‌شوند. در تصویر خوشه‌های ۲ تا ۹ را در این ناحیه مشاهده می‌کنید.



تصویر ۱۶-۱- ساختار فایل‌سیستم FAT

اگر بخواهیم مزایای این فایل‌سیستم را برشماریم، می‌توان اشاره کرد که فایل‌سیستم FAT بسیار ساده است؛ یعنی فاقد ساختارهای پیچیده‌ای مانند iNode بوده و پیاده‌سازی ساده و کم‌حجمی نیز دارد. این فایل‌سیستم، توانایی مدیریت فایل‌های تکه‌تکه[10] را دارا بوده و نیاز به مدیریت مجوزهای پیچیده ندارد زیرا دارای فراداده[11] محدودی

---

[9] Data Area
[10] Fragmented
[11] Metadata

می‌باشد. همچنین به‌دلیل پشتیبانی توسط ویندوز، لینوکس، مک و اغلب دستگاه‌های الکترونیکی، سازگاری گسترده‌ای از خود نشان داده و به فایل‌سیستمی تبدیل شده که با وجود معایبش، همچنان مورد استفاده قرار می‌گیرد.

از جمله معایب فایل‌سیستم FAT، تکه‌تکه‌شدگی بالا به دلیل استفاده از تخصیص پیوندی است. که این امر در فایل‌های بزرگ، منجر به کندی عملکرد می‌شود. زیرا سیستم‌عامل باید کل لیست پیوندی را برای دسترسی به آن فایل، دنبال کند. از طرف دیگر، عدم پشتیبانی از ژورنالینگ[۱۲] در این فایل‌سیستم، آن در برابر خاموشی ناگهانی آسیب‌پذیر می‌سازد. مفهوم ژورنالینگ، در بخش مربوط به فایل‌سیستم ext4 توضیح داده خواهد شد. به‌علاوه، عدم پشتیبانی از لیست‌های کنترل دسترسی[۱۳] (ACL) نیز منجر به ارائه امنیت پایین در این فایل‌سیستم می‌شود.

NTFS -۲-۱-۱۶

فایل‌سیستم NTFS[14] که در سال ۱۹۹۳ همراه با ویندوز NT معرفی شد، جایگزین پیشرفته‌ای برای خانواده قدیمی‌تر فایل‌سیستم‌های FAT به شمار می‌رود. این فایل‌سیستم تا به امروز به‌عنوان فایل‌سیستم اصلی در تمامی نسخه‌های مدرن ویندوز، از XP تا ویندوز ۱۱ و همچنین ویندوز سرور، مورد استفاده قرار گرفته است.

ساختار NTFS بر پایه‌ی چند بخش اصلی بنا شده است که یک نمای ساده‌شده از آن را در تصویر ۱۶-۲ مشاهده می‌نمایید. در ابتدای دیسک، بوت سکتور قرار دارد که اطلاعات بوت و پارامترهای اساسی فایل‌سیستم، مانند اندازه کلاسترها و تعداد سکتورها را در خود نگهداری می‌کند. در بطن این فایل‌سیستم، ساختاری جدولی به نام Master File Table(MFT) قرار گرفته است. این جدول، نقش فهرست اصلی تمامی محتویات دیسک را ایفا می‌کند، به‌طوری‌که به هر فایل یا دایرکتوری در دیسک، یک ردیف[۱۵] ویژه از این جدول اختصاص داده شده است. هر ردیف در MFT شامل مجموعه‌ای از اطلاعات و ویژگی‌ها درباره آن فایل یا دایرکتوری می‌باشد. این ویژگی‌ها، شامل اطلاعاتی همچون

---

[12] Journaling
[13] Access Control Lists
[14] New Technology File System
[15] Record

نام و اندازه فایل، مهرهای زمانی[۱۶]، مجوزهای دسترسی و داده/ اشاره‌گر به داده است. NTFS برای افزایش قابلیت اطمینان، یک نسخه‌ی پشتیبان از MFT نیز نگهداری می‌کند که MFT Mirror نام دارد و در صورت خرابی MFT اصلی، برای بازیابی اطلاعات مورد استفاده قرار می‌گیرد.

یکی از خصوصیات منحصربه‌فرد NTFS این است که از رویکرد مبتنی بر ویژگی، برای ذخیره‌سازی داده‌ها استفاده می‌کند. یعنی، حتی محتوای اصلی فایل نیز به‌عنوان یک ویژگی در نظر گرفته می‌شود که البته می‌تواند به دو صورت ذخیره شود: به‌صورت Resident که در خود رکورد MFT جای می‌گیرد و برای فایل‌های کوچک مناسب است، یا به‌صورت Non-Resident که در بلوک‌های داده‌ی خارجی ذخیره می‌شود. این بلوک‌های خارجی در بخشی از دیسک به نام ناحیه داده‌ها[۱۷] جای می‌گیرند. این بخش، بزرگ‌ترین قسمت دیسک را تشکیل می‌دهد و داده‌ها در آن در قالب کلاسترها - که واحدهای معمولا ۴ کیلوبایتی هستند - ذخیره می‌شوند.

فایل‌سیستم NTFS علاوه بر MFT و داده‌ها، مجموعه‌ای از فایل‌های سیستمی و فراداده را نیز در خود جای داده است که از آن‌ها برای مدیریت خود استفاده می‌کند. از جمله این فایلهای فراداده، یک فایل لاگ یا ژورنال برای ثبت تراکنش‌ها است که به‌منظور تضمین یک‌پارچگی فراداده‌ی فایل‌سیستم، در برابر قطعی ناگهانی برق یا ازکارافتادگی ناگهانی[۱۸] سیستم، مورد استفاده قرار می‌گیرد. همچنین یک فایل «نقشه بیتی[۱۹]» برای مدیریت فضای دیسک در نظر گرفته شده است که وضعیت بلوک‌های خالی و پر را رهگیری می‌کند.

Boot Sector	MFT (Master File Table)	MFT Mirror	Metadata	File Data

[16] Timestamp
[17] File Data
[18] Crash
[19] Bitmap

تصویر ۱۶-۲- ساختار فایل‌سیستم NTFS

از جمله قابلیت‌هایی که در این فایل‌سیستم پشتیبانی می‌شوند، می‌توان به موارد زیر اشاره کرد:

- قابلیت سهمیه‌بندی دیسک[۲۰] برای محدود کردن فضای مصرفی کاربران
- امکان ایجاد لینک‌های سخت و نمادین (هرچند پیاده‌سازی آن با سیستم‌عامل‌های یونیکسی متفاوت است)
- استفاده از لیست‌های کنترل دسترسی (ACL) که امکان تعریف مجوزهای پیچیده در سطح کاربران و گروه‌ها را فراهم می‌سازند.
- استفاده از جریان داده جایگزین[۲۱] (ADS) که به هر فایل اجازه می‌دهد تا جریان‌های داده اضافی و مخفی را در خود نگهداری نماید.
- پشتیبانی از فشرده‌سازی و رمزنگاری به‌صورت بومی
- پشتیبانی از فایل‌های بسیار بزرگ (فایل‌هایی تا حجم ۱۶ ترابایت و دیسک‌هایی تا ظرفیت ۲۵۶ ترابایت)
- بهبود کارایی سیستم با استفاده از قابلیت ذخیره‌سازی داده‌های کوچک در درون ردیف‌های MFT

با وجود قابلیت‌های یاد شده، NTFS دارای یک‌سری محدودیت‌ها و چالش‌ها نیز می‌باشد. از جمله آنکه در پلتفرم‌های غیرویندوزی مانند مک و همچنین در برخی توزیع‌های لینوکسی به‌صورت پیش‌فرض پشتیبانی نمی‌شود و نیاز به نصب درایورهای اختصاصی دارد. به دلیل ساختار پیچیده‌ای که دارد، بازیابی دستی داده‌ها در آن، در مقایسه با فایل‌سیستم‌های ساده‌ای همچون FAT دشوارتر است. در NTFS حجم نوشتن بالا است، که این امر به کاهش عمر حافظه‌های فلش می‌انجامد. بنابراین NTFS برای استفاده در این نوع از حافظه‌ها بهینه نیست.

---

[20] Disk Quota Management
[21] Alternative Data Stream

EXT -۳-۱-۱۶

خانواده فایل‌سیستم‌های گسترش‌یافته[۲۲] (ext) که در نسخه‌های ext، ext2، ext3 و ext4 عرضه شده‌اند، نقش مهمی در توسعه سیستم‌های عامل لینوکسی داشته‌اند. در سال ۱۹۹۲ ext به‌عنوان اولین فایل‌سیستم اختصاصی لینوکس معرفی شد. سپس در سال ۱۹۹۳، نسخه پایدارتر و قابل اطمینان‌تر ext2 ارائه گردید و به مدت یک دهه به‌عنوان فایل‌سیستم استاندارد و رایج در توزیع‌های لینوکسی مورد استفاده قرار گرفت.

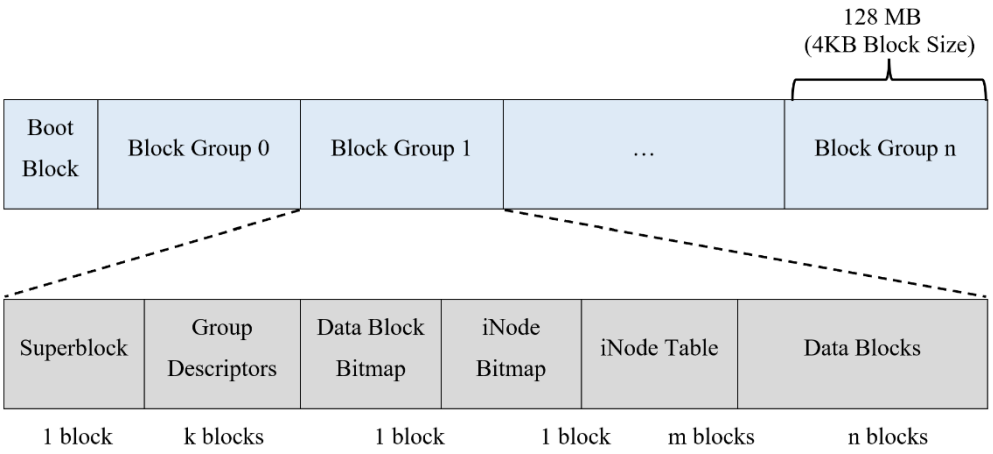
پس از آن در سال ۲۰۰۱، نسخه ext3 معرفی شد که قابلیت حیاتی ژورنالینگ را به معماری ext2 افزوده بود. این ویژگی امکان بازیابی سریع و مطمئن فایل‌سیستم را پس از قطع ناگهانی برق و یا ازکارافتادگی ناگهانی سیستم ممکن می‌ساخت. آخرین نسل خانواده در سال ۲۰۰۸ و تحت نام ext4 ارائه گردید که شامل بهبودهای چشمگیری می‌شد؛ از جمله پشتیبانی از extents، تخصیص تأخیری[۲۳] و کاهش قابل توجه محدودیت در اندازه فایل و پارتیشن. این نسخه امروزه همچنان در توزیع‌های لینوکسی مورد استفاده قرار می‌گیرد.

همان‌طور که در تصویر ۱۶-۳ نمایش داده شده است، ساختار فنی و معماری فایل‌سیستم ext4 بر پایه چندین مولفه کلیدی تعریف می‌شود. در بالای این تصویر، یک نوار افقی بلند مشاهده می‌کنید که نشان دهنده کل فضای دیسک است. بلوکی که در ابتدای دیسک در سمت چپ قرار گرفته، بلوک بوت[۲۴] نام دارد و اطلاعات بوت را در خود نگهداری می‌کند. پس از آن، فضای ذخیره‌سازی دیسک به واحدهای کوچک‌تری به نام گروه‌های بلوکی[۲۵] تقسیم می‌شود. در تصویر، گروه‌های بلوکی متوالی با شماره‌های ۰ تا n را مشاهده می‌کنید که هر کدام ساختاری مشابه دارند.

---

[22] Extended File System
[23] Delayed Allocation
[24] Boot Block
[25] Block Groups

نکته: تمام گروه‌های بلوکی دارای اندازه یکسانی بوده و به‌صورت متوالی روی دیسک ذخیره می‌شوند. این ویژگی باعث می‌شود که مدیریت فضای دیسک، ساده‌تر شود و دسترسی به بلوک‌ها سریع‌تر انجام گیرد.



تصویر ۱۶-۳- ساختار فایل‌سیستم EXT4

ساختار هر گروه بلوکی شامل شش جز اصلی می‌باشد: سوپربلاک، توصیف‌گرهای گروه، نقشه بیتی بلوک‌های داده، نقشه بیتی iNodeها، جدول iNode و در نهایت بلوک‌های داده. این اجزا در پایین تصویر نمایش داده شده‌اند. در ادامه به توضیح هر یک می‌پردازیم.

سوپربلاک[۲۶]: مانند هسته مرکزی فایل‌سیستم بوده و اطلاعات کلی و حیاتی درباره فایل‌سیستم از جمله اندازه کل فایل‌سیستم، تعداد کل بلوک‌ها و iNodeها، اندازه هر بلوک، وضعیت سلامتی فایل‌سیستم و زمان آخرین mount و آخرین نوشتن را در خود نگهداری می‌کند. سوپربلاک در همه گروه‌های بلوکی تکرار می‌شود تا در صورت خرابی یکی، نسخه‌های پشتیبان در دسترس باشند.

---

[26] Superblock

توصیف‌گرهای گروه[۲۷]: یک آرایه از ساختارها هستند که اطلاعات مربوط به هر گروه بلوکی را در خود نگهداری می‌کنند. برای هر گروه بلوکی، یک توصیف‌گر وجود دارد که شامل: شماره بلوک شروع نقشه بیتی بلوک‌ها، شماره بلوک شروع نقشه بیتی iNodeها، شماره بلوک شروع جدول iNode، تعداد بلوک‌ها و iNodeهای آزاد در آن گروه و تعداد دایرکتوری‌های موجود در آن گروه می‌باشد. تعداد بلوک‌های اشغال‌شده توسط توصیف‌گرها به اندازه گروه بلوکی بستگی دارد. در تصویر، این تعداد برابر با k در نظر گرفته شده است.

نقشه بیتی بلوک‌های داده[۲۸]: فقط یک بلوک را اشغال می‌کند و شامل یک نقشه بیتی است که وضعیت هر بلوک داده در آن گروه بلوکی را مشخص می‌کند. به این صورت که بیت 0 به منزله بلوک آزاد و بیت 1 به معنای بلوک اشغال شده، می‌باشد. این نقشه، به سیستم‌عامل کمک می‌کند تا بلوک‌های آزاد را برای تخصیص به فایل‌های جدید پیدا کند.

نقشه بیتی iNode: فقط یک بلوک را اشغال می‌کند و شامل یک نقشه بیتی است که وضعیت هر iNode را در آن گروه بلوکی مشخص می‌کند. بیت 0 یعنی iNode آزاد است و بیت 1 یعنی iNode اشغال شده است. این نقشه نیز به سیستم‌عامل کمک می‌کند تا iNodeهای آزاد را برای فایل‌های جدید پیدا کند.

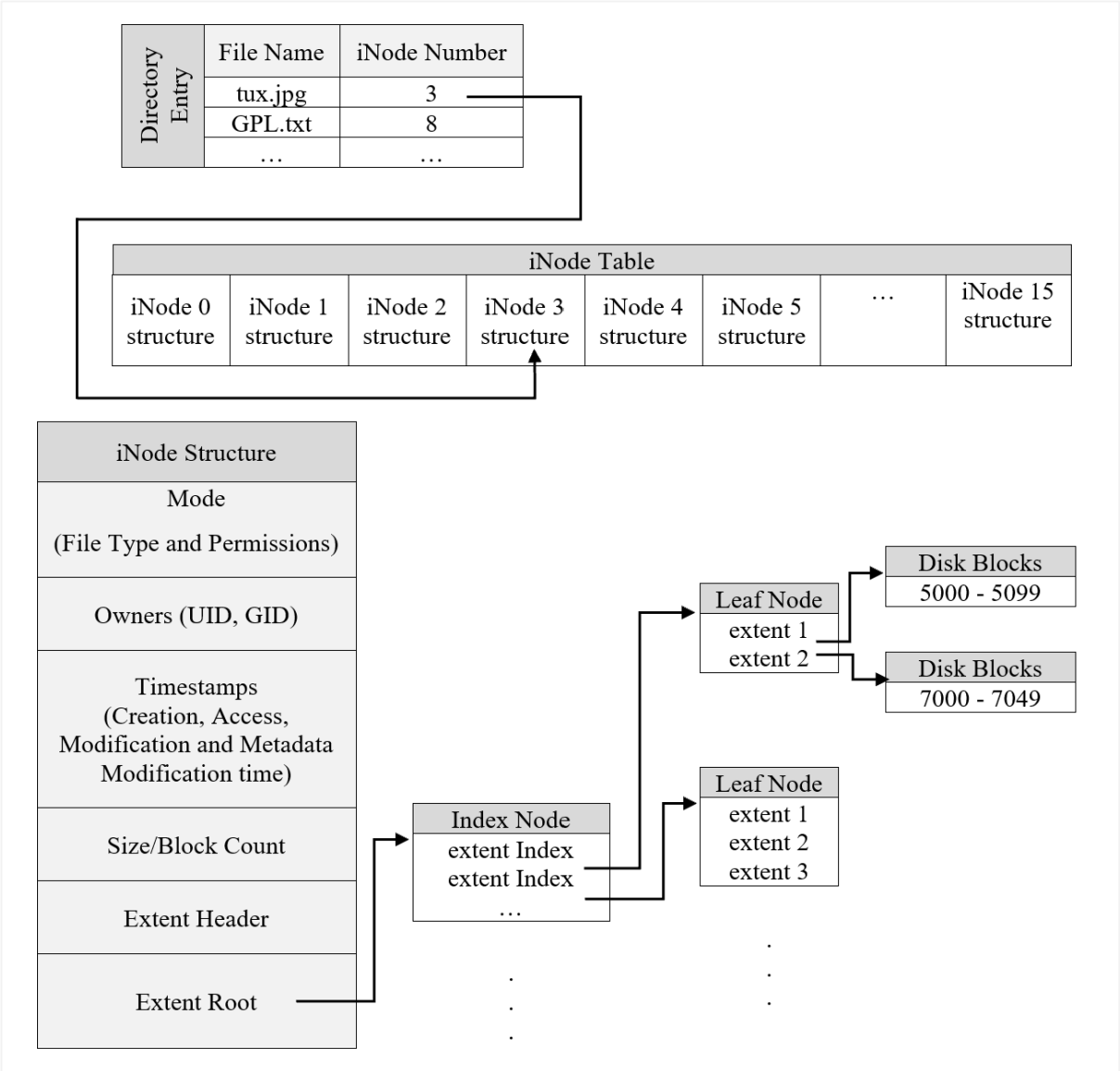
جدول iNode: یک آرایه از ساختارهای iNode است. تعداد بلوک‌های اشغال‌شده توسط جدول iNode به تعداد iNodeهای آن گروه بستگی دارد. در تصویر قبلی، این تعداد برابر با m در نظر گرفته شده است. نمای کلی خود این جدول را نیز در تصویر بعدی می‌توانید ببینید.

iNode یکی از مهم‌ترین مفاهیم در فایل‌سیستم‌های مبتنی بر یونیکس از جمله خانواده ext می‌باشد. هر فایل یا دایرکتوری در فایل‌سیستم، یک شماره iNode یکتا دارد. این شماره همراه با نام آن فایل یا دایرکتوری، به‌صورت یک ورودی دایرکتوری در دایرکتوری والد که حاوی آن فایل یا دایرکتوری است، نگهداری می‌شود. به بیان دیگر، هر ورودی

---

[27] Group Descriptors
[28] Data Blocks Bitmap

دایرکتوری، نام فایل را به شماره iNode متناظرش در جدولiNode ها می‌نگارد. به‌عنوان مثال، همان‌طور که در تصویر ۴-۱۶ مشاهده می‌کنید فایل tux.jpg در ورودی دایرکتوری، به iNode شماره 3 نگاشته شده است. و ارتباط بین این فایل با ساختار iNode مربوطه، با استفاده از یک فلش نمایش داده شده است.



تصویر ۱۶-۴- ساختار iNode

یک ساختار iNode شامل فراداده‌های مربوط به یک فایل یا دایرکتوری می‌باشد. این فراداده‌ها عبارتند از اطلاعاتی مانند نوع فایل، مالک فایل، مجوزهای دسترسی، اندازه فایل، مهرهای زمانی، تعداد لینک‌ها، و اشاره‌گر به محدوده‌ی[۲۹] بلوک‌های داده.

در ext4، به‌جای استفاده از اشاره‌گرهای مستقیم به بلوک‌های داده، از یک درخت Extent در ساختار iNode استفاده می‌شود. سرآیند این درخت، شامل یک سری اطلاعات کنترلی درباره درخت می‌باشد. ریشه درخت، به گره‌های داخلی و این گره‌ها نیز به برگ‌های درخت اشاره می‌کنند. هر برگ از درخت، اطلاعات مربوط به یک محدوده‌ی پیوسته از بلوک‌ها که Extent نام دارد را نگهداری می‌کند. این اطلاعات عبارتند از: شماره اولین بلوک منطقی محدوده، شماره اولین بلوک فیزیکی محدوده، و تعداد بلوک‌های پیوسته تشکیل دهنده آن محدوده. این مکانیزم باعث کاهش حجم فراداده، افزایش کارایی دسترسی و کاهش تکه‌تکه‌شدگی فایل‌ها می‌گردد.

نکته ۱: یک فایل بسته اندازه خود می‌تواند در چند extent ذخیره شده باشد.

نکته ۲: اندازه پیش‌فرض هر iNode برابر با ۲۵۶ بایت است. بنابراین در هر بلوک ۴ کیلوبایتی، ۱۶ عدد iNode قرار می‌گیرد.

```
( 4 × 1024 ) ÷ 256 = 16
```

بلوک‌های داده[۳۰]: بخش انتهایی در هر گروه بلوکی، بلوک‌هایی هستند که محتوای واقعی فایل‌ها در آن‌ها ذخیره می‌شود. این بخش بزرگ‌ترین قسمت هر گروه بلوکی را تشکیل می‌دهد. در تصویر n بلوک داده در نظر گرفته شده است. برای محاسبه ظرفیت گروه بلوکی، یک فرمول به شکل زیر وجود دارد.

```
One Block Size * 8 = k + m + n + 3
```

---

[29] Extents
[30] Data Blocks

سمت چپ فرمول نشان‌دهنده حداکثر تعداد بیت‌هایی است که یک نقشه بیتی می‌تواند مدیریت کند. هر نقشه بیتی فقط یک بلوک را اشغال می‌کند. بنابراین اگر اندازه یک بلوک را ۴ کیلوبایت در نظر بگیریم، آن‌گاه آن بلوک ۳۲۷۶۸ عدد بیت خواهد داشت.

```
4 × 1024 × 8 = 32768
```

ازآنجایی‌که نقشه بیتی برای نمایش وضعیت هر بلوک فقط به یک بیت نیاز دارد، پس با توجه به محاسبات بالا، هر نقشه بیتی حداکثر می‌تواند اطلاعات مربوط به وضعیت ۳۲۷۶۸ بلوک را در خود نگهداری نماید. بنابراین تعداد کل بلوک‌های یک گروه نباید از ۳۲۷۶۸ بیشتر باشد، وگرنه نقشه بیتی نمی‌تواند همه آن‌ها را پوشش دهد. در نهایت، با توجه به اندازه ۴ کیلوبایتی هربلوک، این تعداد بلوک معادل ۱۲۸ مگابایت فضای دیسک خواهد بود.

```
32768 × 4 (KB) = 32768 × 4096 (B) = 128 (MB)
```

در سمت راست فرمول، از k برای نمایش تعداد بلوک‌های توصیف‌گر، از m برای تعداد بلوک‌های جدول iNode و از n برای نمایش تعداد بلوک‌های داده استفاده شده است. عدد ۳ نیز بیانگر مجموع تعداد بلوک‌های ثابت شامل سوپربلاک، نقشه بیتی بلوک‌های داده و نقشه بیتی iNode، می‌باشد

نکته: فرمول محاسبه ظرفیت گروه بلوکی، یک محدودیت طراحی در ext4 را نیز آشکار می‌کند. در واقع این فرمول نشان می‌دهد که مجموع بلوک‌های یک گروه (سمت راست فرمول) نباید از ظرفیت نقشه بیتی (سمت چپ فرمول) بیشتر شود.

مزایای فایل‌سیستم ext4

کارایی بالا: به دلیل قرارگیری سوپربلاک و نقشه‌های بیتی در ابتدای هر گروه بلوکی، دسترسی به آن‌ها سریع‌تر انجام شده و نیازی به جستجوی کل دیسک نخواهد بود.

قابلیت اطمینان: به دلیل تکرار سوپربلاک در همه گروه‌ها، سیستم می‌تواند در صورت خرابی، از نسخه‌های پشتیبان استفاده نماید.

کاهش تکه‌تکه‌شدگی: به دلیل قرارگیری iNodeها و بلوک‌های داده در یک گروه بلوکی یکسان، تکه‌تکه‌شدگی کاهش می‌یابد و دسترسی به فایل‌ها سریع‌تر است.

مقیاس‌پذیری: این ساختار برای دیسک‌های بزرگ - حتی تا چند ترابایت - به خوبی مقیاس‌پذیر است.

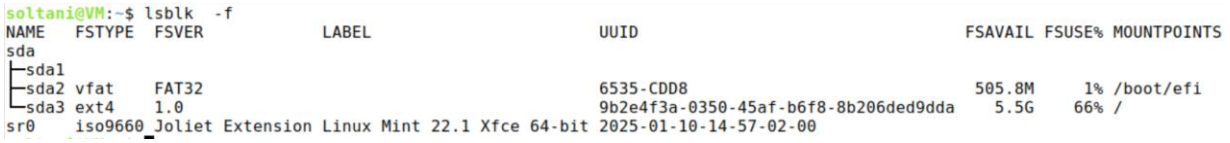
ژورنالینگ: از آنجاییکه اگر هنگام نوشتن روی دیسک، برق قطع شود داده‌ها دچار خرابی می‌شوند، راه‌حل ژورنالینگ معرفی شده است. این راه‌حل به این شکل است که فایل‌سیستم، ابتدا یک یادداشت در ژورنال خود می‌نویسد مبنی بر اینکه می‌خواهد فایل X را تغییر بدهد. سپس تغییر موردنظر را به انجام می‌رساند، یعنی فایل را واقعا تغییر می‌دهد. پس از پایان کار، یک تیک تایید در مقابل یادداشت مذکور می‌زند که به منزله انجام کار است و سپس رکورد مربوطه را از ژورنال خود پاک می‌کند. بنابراین اگر در میانه راه برق قطع شود، پس از راه‌اندازی مجدد، فایل‌سیستم ابتدا به یادداشت خود در ژورنال نگاه کرده و بر اساس آن کارهای ناتمام را کامل می‌نماید. بدین ترتیب دیگر داده‌ها خراب نمی‌شوند.

۱۶-۲- ایجاد فایل‌سیستم

حال که با انواع فایل‌سیستم و ساختار هر یک آشنا شدیم، در این بخش مراحل ایجاد یک فایل‌سیستم را دنبال خواهیم کرد. بدین منظور، از شناسایی دیسک‌ها و پارتیشن‌های موجود در سیستم شروع کرده و سپس فایل‌سیستم‌های جدیدی را ایجاد خواهیم نمود.

۱۶-۲-۱- مشاهده اطلاعات پارتیشن‌ها

با استفاده از دستور lsblk -f می‌توانید مشاهده کنید چند پارتیشن دارید و هر پارتیشن چه نوع فایل‌سیستمی دارد.



تصویر ۱۶-۴- مشاهده پارتیشن‌ها و فایل‌سیستم‌های موجود

همان‌طور که در تصویر ۱۶-۴ ملاحظه می‌نمایید، در این سیستم یک هارد دیسک (sda) وجود دارد که به ۳ پارتیشن تقسیم شده است و همچنین یک حافظه موقت که با نام sr0 نمایش داده می‌شود.

۱۶-۲-۲- اضافه کردن دیسک جدید

برای اضافه کردن یک دیسک جدید به سیستم، ابتدا ماشین‌مجازی را از منوی زیر به‌طور کامل خاموش کرده

```
VM > Power > Shut Down Guest
```

و سپس یک دیسک جدید تعریف می‌کنیم.

```
VM > Settings > Hard Disk > Add
```

در هر یک از پنجره‌های نمایش داده شده، گزینه «بعدی» را انتخاب کرده و صرفا در بخش «تعیین اندازه دیسک» سایز را به ۵ گیگابایت کاهش دهید. در آخرین پنجره گزینه finish را کلیک کنید. اکنون یک هارد دیسک جدید به سیستم مجازی شما اضافه شده است.

ماشین‌مجازی را روشن کرده و با دستور fdisk -l یا lsblk اضافه شدن دیسک خام را بررسی می‌نماییم. از بخش ۲-۶ به یاد دارید که دیسک جدید، با نام sdb در سیستم شناخته خواهد شد. حال برای ایجاد یک پارتیشن جدید بر روی این دیسک، همان‌طور که در تصویر ۱۶-۵ ملاحظه می‌نمایید، مطابق با مثال ۳ از بخش ۳-۱-۵ عمل می‌کنیم.

نکته: اگر در هنگام اجرای این دستور، برای مواردی که گزینه پیش‌فرض دارند کلید ENTER را بزنید، همان مقدار پیش‌فرض انتخاب خواهد شد. مثلا در تصویر زیر، در انتخاب آخرین سکتور به همین روش عمل شده است.

```
soltani@VM:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.39.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0xf60a17ac.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-10485759, default 2048): 2048
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-10485759, default 10485759):

Created a new partition 1 of type 'Linux' and of size 5 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

تصویر ۱۶-۵- ایجاد یک پارتیشن جدید

۱۶-۲-۳- ایجاد فایل‌سیستم آزمایشی

در این قسمت می‌خواهیم همان عملیات بخش قبل را روی یک فایل‌سیستم آزمایشی انجام دهیم. ازآنجاییکه انجام این عملیات روی یک فایل‌سیستم واقعی ممکن است به‌اشتباه به داده‌های موجود آسیب برساند، معمولا برای مقاصد آموزشی از یک loopback device استفاده می‌شود که در واقع، یک فایل معمولی است که مثل یک دیسک واقعی رفتار می‌کند و سیستم‌عامل هم آن را مثل یک دیسک می‌بیند. روی این دیسک فرضی، امکان پارتیشن‌بندی و ایجاد فایل‌سیستم را خواهیم داشت و به این ترتیب می‌توانیم بدون نیاز به دیسک واقعی، انواع فایل‌سیستم را تست نماییم. بدین‌منظور از دستور dd به شکل زیر استفاده می‌کنیم.

```
ایجاد یک فایل با اندازه ۱۰۰ مگابایت #
dd  if=/dev/zero  of=disk.img  bs=1M  count=100

#format it as ext4
mkfs.ext4   disk.img

# mount as loop
sudo  mkdir  -p  /mnt/testloop
sudo  mount  -o  loop  disk.img  /mnt/testloop
```

همان‌طور که از بخش دستورات خط فرمان به یاد دارید، دستور dd یک فایل خالی به نام disk.img و با اندازه ۱۰۰ مگابایت در مسیر جاری ایجاد می‌کند. فایل ایجاد شده، داده‌های واقعی دستگاه لوپ‌بک را درون خود ذخیره خواهد کرد. دستور mkfs[۳۱] ، برای ایجاد یک فایل‌سیستم جدید روی یک پارتیشن یا یک فایل خام استفاده می‌شود. که در اینجا، فایل‌سیستم جدید با فرمت ext4 ایجاد شده است. در دستور ایجاد دایرکتوری، با استفاده از سوئیچ p- از سیستم می‌خواهیم که اگر دایرکتوری‌های والد وجود نداشتد، آن‌ها را نیز ایجاد کند. دستور آخر، فایل disk.img را به‌عنوان یک دستگاه Loopback تعریف کرده و سپس از طریق /mnt/testloop آن را به درخت دایرکتوری سیستم، متصل می‌نماید. به این ترتیب، فایل‌سیستم جدید از طریق این دایرکتوری قابل دسترسی خواهد شد و می‌توانیم از آن مانند یک پارتیشن معمولی استفاده کرده و فایل‌ها را در آن ذخیره یا بازیابی نماییم.

---

[31] Make File System

```
soltani@VM:~$ dd if=/dev/zero of=disk.img bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.183131 s, 573 MB/s
soltani@VM:~$ mkfs.ext4 disk.img
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 25600 4k blocks and 25600 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done

soltani@VM:~$ sudo mkdir -p /mnt/testloop
[sudo] password for soltani:
soltani@VM:~$ sudo mount -o loop disk.img /mnt/testloop/
```

تصویر ۱۶-۶- ایجاد یک دستگاه لوپ‌بک

در ادامه و قبل از انجام هر کار دیگر، ابتدا باید مالک دایرکتوری را به کاربر جاری تغییر دهیم تا از خطاهای مجوز در ادامه جلوگیری شود.

```
sudo  chown  soltani:soltani  /mnt/testloop
```

حال برای ایجاد یک فایل متنی ساده در دیسک لوپ‌بک، از دستور echo کمک می‌گیریم.

```
echo "Operating Systems Laboratory Test"> /mnt/testloop/testfile.txt
```

اطلاعات فایل: برای دیدن iNode و بلوک‌های متعلق به فایل ایجاد شده، دستورات زیر را اجرا کنید.

```
# ورود به دایرکتوری mount
cd  /mnt/testloop

# نمایش اطلاعات و شماره iNode
ls  -li  testfile.txt
stat  testfile.txt
```

همان‌طور که در تصویر زیر ملاحظه می‌کنید، شماره iNode تخصیص یافته به این فایل، ۱۲ می‌باشد که در هر دو دستور نمایش داده شده است. توجه داشته باشید که ممکن است این عدد در سیستم‌های مختلف متفاوت باشد.

```
soltani@VM:/mnt/testloop$ ls -il testfile.txt
12 -rw-rw-r-- 1 soltani soltani 34 Jul  2 19:14 testfile.txt
soltani@VM:/mnt/testloop$ stat testfile.txt
  File: testfile.txt
  Size: 34              Blocks: 8          IO Block: 4096   regular file
Device: 7,0     Inode: 12          Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/ soltani)   Gid: ( 1000/ soltani)
Access: 2026-07-02 19:14:28.724971399 +0330
Modify: 2026-07-02 19:14:28.724971399 +0330
Change: 2026-07-02 19:14:28.724971399 +0330
 Birth: 2026-07-02 19:14:28.724971399 +0330
```

تصویر ۱۶-۷- مشاهده اطلاعات و بلوک‌های فایل بر روی دیسک

نگاشت بلوک‌ها: با دستور زیر می‌توانید شماره بلوک‌های تخصیص‌داده‌شده به فایل را پیدا کنید. به عبارت دیگر، این دستور بلوک‌های منطقی را به بلوک‌های فیزیکی می‌نگارد[۳۲].

```
filefrag   -v   testfile.txt
```

نکته: Ef53 همان کد هگز برای ext4 است.

[32] Maps

```
soltani@VM:/mnt/testloop$ filefrag -v testfile.txt
Filesystem type is: ef53
File size of testfile.txt is 34 (1 block of 4096 bytes)
 ext:     logical_offset:        physical_offset: length:   expected: flags:
   0:        0..       0:      24576..     24576:      1:             last,eof
testfile.txt: 1 extent found
```

تصویر ۱۶-۸- مشاهده شماره بلوک‌های تخصیص‌داده‌شده به فایل بر روی دیسک

خواندن بلوک خام : با کمک دو دستور hexdump و dd می‌توانیم محتوای یک بلوک را به‌صورت مستقیم از دیسک بخوانیم. این روش را می‌توان برای بررسی مستقیم محتوای دیسک در سطح بایت، بررسی ساختار داده‌های خام و یا بازیابی اطلاعات به‌صورت دستی مورد استفاده قرار داد. در مثال قبل با استفاده از filefrag شماره اولین بلوک فیزیکی فایل را برابر با ۲۴۵۷۶ به دست آوردیم. در این مثال نیز محتوای همان بلوک را نمایش خواهیم داد.

```
sudo  dd  if=disk.img  bs=4096  skip=24576 count=1 | hexdump -C
```

دستور dd در این مثال صرفا کار خواندن داده از فایل disk.img را انجام می‌دهد. سوئیچ skip به این دلیل استفاده شده است تا ۲۴۵۷۶ بلوک از ابتدای فایل -که اندازه هر کدام برابر با ۴۰۹۶ بایت است - نادیده گرفته شده و عملیات خواندن از بلوک بعدی شروع گردد. توجه داشته باشید که چون شماره‌گذاری بلوک‌ها از ۰ شروع می‌شود، دقیقا بلوک شماره ۲۴۵۷۶ خوانده خواهد شد.

hexdump ابزاری برای نمایش محتوای فایل‌ها با فرمت هگزادسیمال می‌باشد. سوئیچ C- نیز نمایش را بصورت دو ستونی انجام می‌دهد به‌طوری‌که در سمت راست، نمایش کاراکتری[۳۳] داده‌ها را مشاهده می‌کنید و در سمت چپ، نمایش هگزادسیمال هر بایت را می‌بینید. در نمایش کاراکتری، اگر داده موجود یا قابل چاپ نباشد، یک نقطه به‌جایش نشان داده خواهد شد. همان‌طور که در تصویر ۱۶-۹ مشاهده می‌نمایید، خروجی مثال بالا همان متنی است که هنگام ایجاد فایل متنی در آن نوشته بودیم.

---

[33] ASCII

```
soltani@VM:~$ sudo dd if=/home/soltani/disk.img bs=4096 skip=24576 count=1| hexdump -C
[sudo] password for soltani:
00000000  4f 70 65 72 61 74 69 6e  67 20 53 79 73 74 65 6d  |Operating System|
00000010  73 20 4c 61 62 6f 72 61  74 6f 72 79 20 54 65 73  |s Laboratory Tes|
00000020  74 0a 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |t...............|
00000030  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 4.3365e-05 s, 94.5 MB/s
00001000
```

تصویر ۱۶- ۹- نمایش اطلاعات خام یک بلوک

۱۶-۲-۴- تنظیمات فایل‌سیستم آزمایشی

همان‌طور که در تصویر ۱۶-۶ مشاهده کردید، فایل لوپ‌بک ایجادشده را از طریق یک دایرکتوری به سیستم سوار و متصل نمودیم. پس از انجام این کار، سیستم با فایل مذکور به‌عنوان یک دستگاه برخورد خواهد کرد. این دستگاه با عبارت loop و یک عدد در انتهای آن به‌صورت /dev/loopX نام‌گذاری می‌شود.

برای تنظیم و مدیریت دستگاه‌های لوپ‌بک در لینوکس، یک ابزار خط فرمانی به نام losetup[34] وجود دارد که امکاناتی را برای کار با دستگاه‌های لوپ برای ما فراهم می‌نماید. به‌عنوان مثال، برای مشاهده نام و شماره‌ی تخصیص یافته به دستگاه‌های لوپ فعال در سیستم از دستور زیر استفاده کنید.

```
losetup   -a
```

که در مورد مثال ما، عبارت زیر نمایش داده خواهد شد. که یعنی disk.img به‌عنوان یک دستگاه loop و تحت عنوان /dev/loop0 در سیستم تعریف شده و قابل شناسایی و دسترسی است.

```
/dev/loop0: []: (/home/soltani/disk.img)
```

---

[34] Loop Setup

در صورتی که نیاز به اطلاعات کامل‌تری دارید از سوئیچ l- استفاده کنید. در این حالت جزئیات کامل همه دستگاه‌های لوپ و فایل پشتیبان هر یک را مشاهده خواهید کرد.

```
losetup   -l
```

همان‌طور که در تصویر ۱۶-۱۰ مشاهده می‌نمایید، در خروجی این دستور، اطلاعات یک دستگاه لوپ‌بک نمایش داده شده است.

```
soltani@VM:~$ losetup -l
NAME       SIZELIMIT OFFSET AUTOCLEAR RO BACK-FILE               DIO LOG-SEC
/dev/loop0         0      0         0  0 /home/soltani/disk.img   0     512
```

تصویر ۱۶-۱۰- لیست دستگاه‌های لوپ‌بک

معنی ستون‌های مختلف خروجی این دستور به شرح زیر می‌باشد:

- ستون اول: نام دستگاه را نمایش می‌دهد.
- ستون دوم: محدودیت اندازه را مشخص می‌کند. عدد صفر یعنی هیچ محدودیت اندازه‌ای اعمال نشده است. اما اعداد غیر صفر، به این معنی هستند که دستگاه لوپ فقط می‌تواند به اندازه مشخصی از فایل دسترسی داشته باشد.
- ستون سوم: افست دسترسی به فایل را برای دستگاه معین می‌کند که صفر به منزله شروع از ابتدای فایل است و هر عدد غیرصفر یعنی دستگاه فقط از آن بایت به بعد را می‌تواند از فایل بخواند.
- ستون چهارم: اگر auto clear مقدار صفر داشته باشد، یعنی دستگاه پس از unmount شدن به‌صورت خودکار حذف نخواهد شد و باید به‌صورت دستی و با دستور losetup -d آن را از سیستم جدا نمایید. اگر ستون ro برابر با یک باشد، یعنی دستگاه به‌صورت فقط خواندنی تعریف شده است و قابلیت نوشتن ندارد.

- ستون پنجم: Back-file مسیر کامل فایلی که به‌عنوان دستگاه لوپ‌بک به سیستم متصل شده است را مشخص می‌کند.
- ستون ششم: اگر DIO[35] مقدار صفر داشته باشد، به این معنی است که برای عملیات ورودی/خروجی از کش سیستم استفاده می‌شود و دسترسی به‌صورت مستقیم به فایل انجام نمی‌گیرد. به این ترتیب سرعت خواندن و نوشتن‌های مکرر، افزایش می‌یابد.
- ستون هفتم: این ستون اندازه بخش منطقی[۳۶] دستگاه را بر حسب بایت نشان می‌دهد که اغلب برابر با ۵۱۲ بایت می‌باشد.

برای اینکه دستگاه لوپ به‌صورت فقط خواندنی[۳۷] در سیستم تعریف بشود، سوئیچ r- را اضافه کنید.

```
losetup   -r   /dev/loop0   disk.img
```

توجه داشته باشید که losetup دستگاه لوپ‌بک را از روی یک فایل معمولی و به‌عنوان یک دیسک خام ساخته و آن را به سیستم معرفی می‌کند، اما فایل‌سیستم داخل آن را متصل نمی‌نماید و بدین‌منظور باید از دستور mount استفاده کنید.

سوئیچ d- ارتباط دستگاه لوپ را با فایل پشتیبان قطع کرده و این دو را از هم جدا می‌نماید.

```
losetup   -d   /dev/loop0
```

---

[35] Direct I/O
[36] Logical Sector Size
[37] Read-Only

توجه داشته باشید که این کار، فقط ارتباط فایل و دستگاه را قطع می‌کند ولی فایل‌سیستم را unmount نخواهد کرد. بدین‌منظور باید از دستور umount استفاده نمایید.

۱۶-۳- بررسی سلامت فایل‌سیستم

دستور fsck[38] یک ابزار خط فرمانی برای بررسی سلامت فایل‌سیستم و تعمیر آن می‌باشد. این دستور به‌صورت خودکار در هنگام بالا آمدن سیستم و یا پیش از متصل شدن پارتیشن‌ها اجرا می‌شود تا خطاهای ساختاری مانند سوپربلاک خراب، inodeهای آسیب‌دیده یا بلوک‌های گم‌شده را شناسایی و در صورت امکان این اشکالات را برطرف نماید. درصورتی‌که بخواهیم به‌صورت دستی این عملیات را برای یک فایل‌سیستم انجام بدهیم، به‌صورت زیر عمل می‌کنیم.

دستور زیر سلامت یک فایل‌سیستم را بررسی می‌کند ولی هیچ تعمیری اعمال نمی‌نماید.

```
sudo fsck -n /dev/loop0
```

سوئیچ y- باعث می‌شود که سلامت فایل‌سیستم بررسی شده و سپس تعمیرات لازم به‌صورت خودکار اعمال گردد. برای نمایش جزئیات بیشتر، می‌توانید سوئیچ V- را نیز اضافه کنید.

```
sudo fsck -y /dev/loop0
```

توجه داشته باشید که همیشه قبل از شروع اسکن، پارتیشن را unmount کرده باشید تا از بروز آسیب بیشتر، جلوگیری به عمل آید. به‌علاوه اگر قصد بررسی دیسک ریشه سیستم را دارید باید از Live USB استفاده کنید، زیرا امکان بررسی دیسک در حالت اجرا وجود ندارد.

```
sudo umount /mnt/testloop
```

---

[38] File System Consistency Check

در تصویر زیر، دستگاه لوپ مورد بررسی قرار گرفته است. مشاهده می‌کنید که از ۲۵۶۰۰ ظرفیت iNode موجود در فایل‌سیستم، ۱۲ تا مورد استفاده قرار گرفته و همچنین از مجموع ۲۵۶۰۰ بلوک موجود، ۲۶۴۷ مورد اشغال شده‌اند. برخی از این بلوک‌ها توسط فراداده‌های خود فایل‌سیستم مانند سوپر بلوک، جدول iNode و اطلاعات ژورنال پر شده‌اند. به‌علاوه، حتی اگر فایلی را پاک کرده باشید، ممکن است هنوز بلوک‌های آن به طور کامل آزاد نشده باشند.

```
soltani@VM:~$ sudo umount /mnt/testloop
soltani@VM:~$ sudo fsck -V /dev/loop0
fsck from util-linux 2.39.3
[/usr/sbin/fsck.ext4 (1) -- /dev/loop0] fsck.ext4 /dev/loop0
e2fsck 1.47.0 (5-Feb-2023)
/dev/loop0: clean, 12/25600 files, 2647/25600 blocks
```

تصویر ۱۶-۱۱- قطع اتصال و بررسی دستگاه لوپ

۱۶-۴- اشکال‌زدایی فایل‌سیستم

دستور debugfs، برای تشخیص و تعمیر مشکلات فایل‌سیستم، دسترسی مستقیم به ساختارهای داخلی فایل‌سیستم، و بازیابی فایل‌های حذف شده، مورد استفاده قرار می‌گیرد. پیش از شروع استفاده از این دستور، بهتر است مطمئن شوید که در مسیر دایرکتوری‌ای که فایل‌سیستم بر آن سوار شده است، قرار دارید. به این ترتیب، برای دسترسی به فایل‌ها دچار خطا یا اشتباه نخواهید شد.

```
# ورود به دایرکتوری mount
cd  /mnt/testloop

sudo  debugfs  /home/soltani/disk.img
```

و سپس در محیط debugfs می‌توانید از دستورات داخلی آن استفاده نمایید.

برای دیدن لیست دستورات داخلی، علامت سوال را وارد نموده و کلید ENTER را بزنید.

```
debugfs: ?
```

دستور داخلی stat : برای نمایش اطلاعات دقیق و جزئی فایل به‌کار می‌رود.

```
debugfs:  stat  testfile.txt
```

```
Inode: 12   Type: regular    Mode:  0664   Flags: 0x80000
Generation: 980657374    Version: 0x00000000:00000001
User:  1000   Group:  1000   Project:     0   Size: 34
File ACL: 0
Links: 1   Blockcount: 8
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6a46875c:acd8be1c -- Thu Jul  2 19:14:28 2026
 atime: 0x6a46875c:acd8be1c -- Thu Jul  2 19:14:28 2026
 mtime: 0x6a46875c:acd8be1c -- Thu Jul  2 19:14:28 2026
crtime: 0x6a46875c:acd8be1c -- Thu Jul  2 19:14:28 2026
Size of extra inode fields: 32
Inode checksum: 0x10487288
EXTENTS:
(0):24576
(END)
```

تصویر ۱۶-۱۲- نمایش اطلاعات فایل در debugfs

همان‌طور که در تصویر ۱۶-۱۲ مشاهده می‌کنید خروجی دستور حاوی اطلاعات مبسوطی می‌باشد که در ادامه به توضیح هر یک از بخش‌های آن می‌پردازیم.

خروجی	توضیحات
Inode: 12	شماره شناسه داخلی فایل
Type: regular file	نوع: فایل معمولی
Mode: 0644	مجوزهای دسترسی به‌صورت rw-r--r--
Flags: 0x80000	فلگ‌ها
Generation: 980657374 Version 0x00000000:00000001	شماره نسل و نسخه
User: 1000	شناسه کابر مالک: 1000
Group: 1000	شناسه گروه: 1000
Size: 34	حجم: ۳۴ کیلوبایت
File ACL: 0	لیست کنترل دسترسی
Links: 1	تعداد لینک‌های سخت
Blockcount: 8	تعداد بلوک‌های اختصاص داده شده. اندازه پیش‌فرض هر بلوک در دستور stat برابر با ۵۱۲ بایت در نظر گرفته می‌شود.
Fragment	اطلاعات فرگمنت شامل آدرس، شماره و اندازه
ctime: 0x6a46875c:acd8be1c-- Thu Jul 2 19:14:28 2026	زمان تغییر فراداده‌ها (نمایش به‌صورت هگزادسیمال و خوانا)
atime: 0x6a46875c:acd8be1c-- Thu Jul 2 19:14:28 2026	زمان آخرین دسترسی
mtime: 0x6a46875c:acd8be1c-- Thu Jul 2 19:14:28 2026	زمان آخرین تغییر محتوا
crtime: 0x6a46875c:acd8be1c-- Thu Jul 2 19:14:28 2026	زمان ایجاد فایل (مخصوص ext4)
Size of extra iNode fields: 32	اندازه فیلدهای اضافی در iNode بر حسب بایت. (ext4 از فیلدهای بیشتری نسبت به نسخه‌های قبلی پشتیبانی می‌کند)
Inode checksum: 0x10487288	نمایش چک‌سام برای اطمینان از صحت داده‌های iNode و تشخیص خرابی
EXTENTS:	فایل از مکانیزم Extent برای نگاشت بلوک‌ها استفاده می‌کند
(0): 24576	شماره اولین بلوک منطقی : شماره بلوک فیزیکی روی دیسک

جدول ۱۶-۱- شرح خروجی دستور stat

دستور داخلی blocks : حاصل این دستور به‌صورت یک یا چند عدد صحیح نمایش داده می‌شود که شماره بلوک‌های فیزیکی تخصیص داده شده به فایل را مشخص می‌نماید.

```
debugfs:  blocks  testfile.txt
```

مثلا اگر خروجی دستور به‌صورت زیر باشد، یعنی این فایل در بلوک فیزیکی شماره ۲۴۵۷۶ تا ۲۴۵۷۹ ذخیره شده است که مجموعا ۴ بلوک می‌شود.

```
24576  24577  24578  24579
```

در مثال بالا، بلوک‌ها به‌صورت پیوسته هستند. اما ممکن است در یک فایل‌سیستم این‌گونه نباشد و بلوک‌های فایل به‌صورت ناپیوسته در سطح دیسک، پراکنده شده باشند. در صورت پیوستگی، یعنی فایل در یک بخش[۳۹] یکپارچه ذخیره شده است و بنابراین سرعت دسترسی به فایل افزایش می‌یابد. زیرا با خواندن چند بلوک پیوسته در یک عملیات، تعداد درخواست‌های ورودی/خروجی به دیسک کاهش یافته و از طرفی نیز سیستم‌عامل برای دسترسی به بلوک‌های پراکنده نیاز به جستجوی[۴۰] اضافی ندارد. به این ترتیب سرعت دسترسی به فایل، کارآمد می‌شود.

دستور داخلی imap : برای پیدا کردن مکان فایل روی دیسک، استفاده می‌شود.

```
debugfs: imap  testfile.txt
```

همان‌طور که در ابتدای این فصل اشاره کردیم در فایل‌سیستم ext4، دیسک به گروه‌های بلوکی تقسیم می‌شود که هر گروه، شامل فراداده‌های مخصوص خود - مانند جدول و نگاشت بیتی iNodeها، نگاشت بیتی بلوک‌ها - و بلوک‌های داده واقعی می‌باشد.

---

[39] Extent
[40] Seek

```
debugfs:  imap testfile.txt
Inode 12 is part of block group 0
        located at block 46, offset 0x0b00
```

تصویر ۱۶-۱۳- دستور داخلی imap

در خروجی دستور imap در تصویر ۱۶-۱۳، گروه بلوکی با شماره صفر مشخص شده است که اولین گروه بلوکی در فایل‌سیستم می‌باشد. و این یعنی، داده‌های فایل مدنظر ما، در ابتدای دیسک قرار دارند. سطر دوم خروجی نیز می‌گوید که اطلاعات مربوط به iNodeهایی که در این گروه بلوکی قرار دارند، بر روی دیسک و در بلوک فیزیکی شماره ۶۴ - و احتمالا بلوک‌های بعد از آن - ذخیره شده‌اند. بخش افست[41] نیز موقعیت دقیق اطلاعات را در بلوک مشخص می‌نماید. در مثال بالا افست در مبنای شانزده برابر با 0X0b00 است که در مبنای ده می‌شود ۲۸۱۶. بنابراین باید به تعداد ۲۸۱۶ بایت از ابتدای بلوک ۶۴ به جلو حرکت کنید تا به iNode موردنظر برسید. آدرس بایتی دقیق بر روی دیسک، به شکل زیر محاسبه می‌شود.

آدرس شروع iNode = افست + (شماره بلوک × اندازه بلوک) = ۲۸۱۶ + (۶۴ × ۴۰۹۶) = ۲۶۴۹۶۰

بنابراین iNode موردنظر دقیقا از بایت شماره ۲۶۴۹۶۰ روی دیسک شروع می‌شود.

نکته: در اکثر سیستم‌های مدرن، اندازه بلوک فیزیکی در حالت پیش‌فرض برابر با ۴۰۹۶ بایت (۴ کیلوبایت) است. مگر اینکه در هنگام ساخت فایل‌سیستم، با استفاده از سوئیچ b- مقدار متفاوتی برای اندازه بلوک تعیین گردد.

دستور داخلی filefrag : این دستور تعداد بخش‌های پیوسته فایل را نشان می‌دهد.

```
debugfs: filefrag   testfile.txt
```

---

[41] Offset

خروجی این دستور برای مثال ما به‌صورت زیر می‌باشد.

```
/home/soltani/testfile.txt:  1  extent  found
```

همان‌طور که ملاحظه می‌کنید، فقط یک extent پیدا شده است که یعنی کل فایل، در یک بخش پیوسته ذخیره شده است. در حالت کلی، هرچه تعداد extentها بیشتر باشد، یعنی فایل بیشتر تکه‌تکه شده است.

بازیابی: برای بازیابی فایل‌هایی که به اشتباه از فایل‌سیستم پاک شده‌اند نیز می‌توانیم از دستورات داخلی debugfs کمک بگیریم. به این ترتیب که ابتدا با دستور داخلی lsdel لیست فایل‌های پاک شده از فایل‌سیستم و شماره iNode متناظرشان را پیدا کرده و سپس با دستور stat مطمئن می‌شویم که بلوک‌های فایل هنوز بر روی دیسک وجود دارند و رونویسی نشده‌اند. در نهایت با استفاده از دستور dump اطلاعات فایل را به یک فایل دیگر کپی می‌نماییم.

```
# لیست کردن iNodes که اخیرا پاک شده اند
debugfs: lsdel

# بررسی اندازه و بلوک‌های iNode پاک شده
debugfs: stat <iNode_Number>

# نوشتن بلوک‌های باقیمانده از iNode در یک فایل جدید
debugfs: dump  <iNode_Number>  /mnt/recovered_file.txt
```

توجه داشته باشید که عملیات بازیابی در سیستم‌های مجازی به علت دسترسی غیرمستقیم به دیسک، امکان بسیار کمی دارد. به‌علاوه، به‌دلیل ژورنالینگ و آزادسازی سریع بلوک‌های داده، این عملیات در فایل‌سیستم ext4 شانس بسیار کمتری نسبت به ext2 و ext3 دارد.

خروج: برای خروج از محیط debugfs، دستور quit یا q را نوشته و کلید ENTER را بزنید.

```
debugfs: q
```

تمرین

- یک فایل‌سیستم آزمایشی با فرمت ext4 ایجاد کرده و به سیستم متصل نمایید. نامی که دستگاه لوپ تحت آن به سیستم متصل شده است را به دست آورید. سپس با استفاده از خط فرمان یک فایل متنی را به درون آن کپی کرده و مشخص نمایید که این فایل دقیقا روی کدام بلوک‌های فیزیکی بر روی دیسک نوشته شده است. در نهایت، با استفاده از دستور مقتضی از سلامت فایل‌سیستم خود اطمینان حاصل نمایید.
- یک فایل متنی ساده به نام خودتان در سیستم ایجاد کرده و چند خط متن در آن بنویسید. با استفاده از دستور ls همراه با سوئیچ‌های مقتضی، شماره iNode فایل را پیدا کرده و سپس با debugfs به فایل‌سیستم متصل شده و دستور stat را به شکل صحیح اجرا نمایید. نوع و اندازه فایل، مالک آن، تعداد بلوک‌های اختصاص‌یافته و شماره بلوک‌های داده یا Extents را استخراج کرده و در پاسخ تمرین یادداشت نمایید.

گزارش کار

- یک فایل loopback با فایل‌سیستم FAT32 و یک فایل loopback با فایل‌سیستم ext4 ایجاد کنید. هر دو را mount کرده و یک فایل متنی یکسان در آن‌ها قرار دهید. با کمک ابزارهای مناسب، مانند debugfs برای ext4 و fsck.vfat یا dosfsck برای FAT، ساختار تخصیص بلوک‌ها را بررسی کرده و گزارش نمایید که هر فایل‌سیستم چگونه بلوک‌های فایل را نگهداری می‌کند. کدام روش کارآمدتر است و چرا؟
- یک فایل loopback با فایل‌سیستم ext4 ایجاد کرده و یک فایل متنی با محتوای مشخص در آن قرار دهید. سپس فایل را حذف کرده و فایل‌سیستم را unmount کنید. حال با استفاده از dump یا یک ابزار کارآمد خط فرمانی دیگر مانند extundelete، فایل پاک شده را بازیابی نمایید. و در نهایت فایل بازیابی‌شده را با فایل اصلی مقایسه کنید.

منابع و مراجع:

- Bovet DP, Cesati M. *Understanding the Linux Kernel*, O'Reilly, 3rd Edition; 2006.
- Silberschatz A. *Operating System Concepts*, Wiley,10th Edition; 2018.
- Tanenbaum AS. *Modern Operating Systems*, Pearson Education; 2015.
- Pate SD. *UNIX Filesystems: Evolution, Design, and Implementation*, John Wiley & Sons; 2003.

پیوست ۱ : راهنمای گزارش کارنویسی

برای نوشتن یک گزارش کار کامل، به نکات و موارد زیر توجه نمایید:

- درج مشخصات کامل دانشجو، کلاس، شماره تمرین و تاریخ تحویل در ابتدای گزارش
- مقدمه‌ای شامل اطلاعات پیش‌زمینه و معرفی موضوع آزمایش

به‌عنوان مثال در گزارش کار مرتبط با موضوع اسکریپت‌نویسی، در یک الی دو پاراگراف به طور خلاصه درباره خودکارسازی وظایف توضیح داده و بگویید مثلا خودکارسازی چه هدف یا اهمیتی برای مدیران سیستم دارد؟ خودکارسازی یک وظیفه روتین چگونه انجام می‌گیرد؟ چه روش‌های دیگری بدین‌منظور وجود دارد؟ همچنین درباره کاربرد اسکریپتی که طراحی کرده‌اید در دنیای واقعی توضیح دهید.

- روش، ابزار و نرم‌افزار مورد استفاده

تمامی موارد جدیدی که در آزمایش مورد استفاده قرار گرفته‌اند، از جمله نام و نسخه سیستم‌عامل، نوع پوسته، نرم‌افزارها، ابزارها، دستورات خط فرمانی و کتابخانه‌های مورد نیاز را لیست کرده و به‌صورت مختصر هر کدام را معرفی نمایید.

- توضیح مراحل آزمایش و نحوه پیاده‌سازی به‌صورت مشروح و با جزئیات کافی، به‌طوری‌که کار شما قابل تکرار[1] توسط دیگران باشد. این موضوع، از جمله شرایطی است که یک آزمایش علمی باید دارا باشد.

به‌عنوان مثال؛ تمام مراحل ایجاد و اجرای اسکریپت پوسته، تنظیمات خودکارسازی و تست آن برای اطمینان از اجرای صحیح را با جزئیات توضیح داده و از مراحل مختلف اسکرین‌شات اضافه نمایید.

---

[1] Replicable

- نتایج

به‌عنوان مثال؛ تصاویری از نتایج حاصل از اجرای دستی و خودکار اسکریپت، پیام‌های چاپ شده در ترمینال یا لاگ، تاثیر اجرای اسکریپت بر روی سیستم و هر نتیجه ماخوذ دیگر را در فایل گزارش خود درج نمایید.

- بحث و نتیجه گیری

نتایج به دست آمده را توضیح داده وتفسیر نمایید. به‌عنوان مثال؛ آیا خودکارسازی با موفقیت عمل کرد؟ در اثر اجرای اسکریپت چه اتفاق یا تغییری در سیستم حاصل شد و آیا این نتیجه با انتظار شما همخوانی داشت یا نه؟ همچنین هر گونه خطا یا استثنا که در هنگام تنظیم یا مراحل اجرا رخ داده و نحوه حل آن، نکات خاصی که باید در طی آزمایش رعایت شوند و هر موردی از این دست را نیز ذکر کنید. به‌علاوه، به سوالاتی که در انتهای دستورکار تعریف شده (درصورت وجود) به‌صورت دست‌نویس پاسخ داده و تصویر آن را به گزارش خود ضمیمه نمایید.

- منابع

تمام منابعی که در پیاده‌سازی، اجرا و رفع اشکال تمرین و نوشتن گزارش خود از آن‌ها استفاده کرده‌اید را لیست کنید. در تمامی گزارش کارها، از یک سبک ارجاع‌دهی ثابت استفاده نمایید.

- ضمائم:

سایر مواردی که در متن اصلی ذکر نشده اما ضروری هستند را (در صورت وجود) به گزارش خود ضمیمه نمایید. به‌عنوان مثال؛

  - کد اصلی اسکریپت (در صورت عدم درج کامل در متن گزارش)
  - افزودن و انجام بخش‌های اختیاری یا اضافی برای گزارش
  - ضرورت ارسال فایل‌هایی با سایر فرمت‌ها (مثلا ویدئویی) برای گزارش دقیق نتایج

نکات مهم:

- گزارش را با ظاهری تمیز، مرتب و با چیدمان صحیح تهیه نمایید. توضیحات را شفاف و با دقت کافی بنویسید.
- نکات و علائم نگارشی و ویرایشی را (مشابه با اصول مقاله‌نویسی) در نوشتن متن گزارش رعایت کنید. یک قالب[۲] و سبک[۳] ثابت را در سراسر گزارش و ترجیحا در تمامی گزارش کارهای خود رعایت نمایید.
- گزارش خود را برای وجود خطاهای احتمالی، غلط املایی و همچنین وضوح کافی، مجددا بررسی کرده و بازخوانی[۴] نمایید.
- کدها، ضمایم و بویژه تصاویر اسکرین‌شات را به‌صورتی تهیه نمایید که در آن‌ها **اصالت اثر** و تعلق کار به خودتان، به نحوی مشخص باشد.

فایل pdf نهایی را که به نام یا شماره دانشجویی خودتان نام‌گذاری شده، در بخش تمرین مربوطه در سامانه آموزش آنلاین دانشگاه و یا در سایت‌های جایگزین که در کلاس معرفی می‌شود، آپلود نمایید.

معیارهای ارزیابی گزارش کارها

گزارش کارهای ارسالی باید تمام موارد و نکات ذکر شده در این راهنما را به‌طور کامل رعایت نمایند. از جمله:

- کامل و دقیق بودن گزارش کار
- در برداشتن جزئیات کافی
- تصاویر واضح و گویا از مراحل انجام کار و نتایج حاصله
- مشخص بودن اصالت اثر بویژه در تصاویر
- صحیح بودن کد (در صورت وجود) و کامپایل و اجرای موفقیت‌آمیز آن

---

[2] Format
[3] Style
[4] Proofreading

- حصول نتایج صحیح پس از اجرا
- درج تصاویر به‌صورت مرتب با چیدمان یکسان
- رعایت نکات نگارشی و ویرایش در نوشتار متن
- ارسال به موقع
  - تاخیر در تحویل گزارش کار، منجر به کاهش بارم به نصف خواهد شد.
  - درصورتی‌که گزارش کار خود را با تاخیر آماده کردید، تا انتهای ترم فرصت دارید فایل خود را از بخش تمرین‌ها، در قسمت «ارسال با تاخیر» ذیل سوال مربوطه آپلود نمایید.

پیوست ۲ : راهنمای سریع دستورات لینوکس

عملیات مرتبط با فایل و دایرکتوری		
دستور	**توضیح**	**مثال**
ls	نمایش محتوای دایرکتوری	ls -la
cd	تغییر دایرکتوری جاری	cd /home/user
pwd	نمایش آدرس دایرکتوری فعلی	pwd
mkdir	ساخت دایرکتوری جدید	mkdir new_dir
rmdir	حذف دایرکتوری خالی	rmdir empty_dir
rm	حذف فایل یا دایرکتوری	rm -rf dir
cp	کپی فایل یا دایرکتوری	cp file1 file2
mv	جابه‌جایی یا تغییر نام فایل	mv old.txt new.txt
touch	ساخت فایل جدید	touch new_file.txt

مشاهده و ویرایش فایل		
دستور	**توضیح**	**مثال**
cat	نمایش محتوای فایل	cat file.txt
less	مشاهده صفحه به صفحه فایل	less large_file.log
more	مشاهده صفحه به صفحه فایل	more file.txt
head	نمایش خطوط اول فایل	head -n 10 file.txt
tail	نمایش خطوط آخر فایل	tail -f logfile.log
nano	ویرایشگر متن ساده	nano file.txt
vim	ویرایشگر متن پیشرفته	vim file.txt

جستجوی فایل		
دستور	**توضیح**	**مثال**
find	جستجوی فایل یا دایرکتوری	"find /home -name "*.txt
grep	جستجوی متن در فایل‌ها	grep "error" logfile.log
locate	جستجوی سریع فایل	locate filename
which	پیدا کردن مسیر دستور	which python

whereis ls	پیدا کردن باینری یا منبع یا راهنما	**whereis**

مانیتورینگ سیستم		
مثال	**توضیح**	**دستور**
Top	نمایش فرآیندهای لینوکس	**top**
Htop	نمایش تعاملی فرآیندها	**htop**
ps -aux	گزارش وضعیت فرآیندها	**ps**
free -h	نمایش مصرف حافظه	**free**
df -h	نمایش فضای دیسک	**df**
du -sh *	نمایش فضای دایرکتوری‌ها	**du**
Uptime	زمان کارکرد و بار سیستم	**uptime**

مدیریت سیستم		
مثال	**توضیح**	**دستور**
sudo apt update	اجرای دستور با دسترسی کاربر ریشه	**sudo**
su – username	تغییر کاربر	**su**
chmod 755 script.sh	تغییر مجوزهای فایل	**chmod**
chown user:group file	تغییر مالک فایل	**chown**
Passwd	تغییر رمزعبور	**passwd**
uname -a	اطلاعات سیستم	**uname**
Date	نمایش یا تنظیم زمان	**date**

شبکه		
مثال	**توضیح**	**دستور**
ping google.com	تست اتصال شبکه	**ping**
ifconfig	پیکربندی رابط شبکه	**ifconfig**
ip addr show	پیکربندی پیشرفته شبکه	**ip**
wget http://example.com/file	دانلود از اینترنت	**wget**
Curl -O http://example.com/file	انتقال داده از URL	**curl**
ssh user@hostname	اتصال امن به سیستم دیگر	**ssh**
scp file.txt user@host:/path	کپی امن فایل	**scp**

مدیریت بسته در توزیع‌های مبتنی بر دبیان		
دستور	**توضیح**	**مثال**
apt update	بروزرسانی لیست بسته‌ها	sudo apt update
apt upgrade	ارتقاء بسته‌ها	sudo apt upgrade
apt install	نصب بسته	sudo apt install nginx
apt remove	حذف بسته	sudo apt remove package
apt search	جستجوی بسته	apt search "web server"
dpkg	مدیریت بسته دبیان	dpkg -i package.deb

فشرده‌سازی و آرشیو		
دستور	**توضیح**	**مثال**
tar	ابزار آرشیو	tar -czvf archive.tar.gz dir
gzip	فشرده‌سازی فایل	gzip file.txt
gunzip	باز کردن فایل فشرده	gunzip file.txt.gz
zip	بسته‌بندی و فشرده‌سازی	zip archive.zip file1 file2
unzip	استخراج فایل فشرده	unzip archive.zip

مدیریت کاربران و گروه‌ها		
دستور	**توضیح**	**مثال**
useradd	افزودن کاربر جدید	sudo useradd newuser
userdel	حذف کاربر	sudo userdel username
usermod	تغییر اطلاعات کاربر	sudo usermod -aG group user
groupadd	افزودن گروه جدید	sudo groupadd newgroup
groups	نمایش گروه‌های کاربر	groups username

پردازش متن		
دستور	**توضیح**	**مثال**
awk	پردازش و اسکن الگو	awk '{print $1}' file.txt
sed	ویرایشگر استریم	sed 's/old/new/g' file.txt

sort file.txt	مرتب‌سازی خطوط متن	**sort**
uniq file.txt	حذف خطوط تکراری	**uniq**
wc -l file.txt	شمارش کلمات و خطوط	**wc**

مدیریت فرآیندها		
مثال	**توضیح**	**دستور**
kill -9 PID	پایان دادن به فرآیند	**kill**
killall process_name	پایان دادن به فرآیند با نام	**killall**
bg %1	فرستادن فرآیند به پس‌زمینه	**bg**
fg %1	آوردن فرآیند به پیش‌زمینه	**fg**
Jobs	لیست کارهای پس‌زمینه	**jobs**
& nohup command	اجرای مستقل فرآیند	**nohup**

هدایت ورودی یا خروجی		
مثال	**توضیح**	**دستور**
ls > files.txt	هدایت خروجی به فایل	>
echo "text" >> file.txt	افزودن به انتهای فایل	>>
sort < file.txt	خواندن از فایل به‌عنوان ورودی	<
ls | grep ".txt"	انتقال خروجی به دستور دیگر	|
command 2> error.log	هدایت خطاها	**2>**

راهنما و مستندات		
مثال	**توضیح**	**دستور**
man ls	صفحات راهنما	**man**
info info coreutils	مستندات	**info**
whatis ls	توضیح یک خطی	**whatis**
shell help cd	راهنمای داخلی	**help**
ls –help	راهنمای دستور	**--help**

ابزارهای دیسک		
دستور	**توضیح**	**مثال**
fdisk	مدیریت پارتیشن‌ها	sudo fdisk -l
mount	mountکردن فایل‌سیستم	sudo mount /dev/sda1 /mnt
umount	umountکردن	sudo umount /mnt
fsck	بررسی فایل‌سیستم	sudo fsck /dev/sda1
lsblk	نمایش دستگاه‌های بلوک	lsblk

کلیدهای میان‌بر	
دستور	**توضیح**
Ctrl + C	لغو دستور جاری
Ctrl + Z	مکث فرآیند جاری
Ctrl + D	خروج از ترمینال
Ctrl + L	پاک کردن صفحه
Ctrl + R	جستجو در تاریخچه
Tab	تکمیل خودکار
Up and Down Arrow Keys (↑ or ↓)	پیمایش تاریخچه دستورات

کتاب‌شناسی[1]

به جهت سهولت دسترسی، منابع به‌صورت موضوعی مرتب شده‌اند. مواردی که با علامت ستاره مشخص شده‌اند، منابعی هستند که به صورت مستقیم در متن کتاب مورد استفاده قرار گرفته و در پایان هر فصل ارجاع داده شده‌اند. سایر منابع، برای مطالعه‌ی بیشتر پیشنهاد می‌شوند.

کتب و مراجع پایه در مباحث سیستم‌عامل:

- *Silberschatz A. *Operating System Concepts*, Wiley, 10th Edition, 2018.
- *Tanenbaum A.S. *Modern Operating Systems*, Pearson Education, 4th Edition, 2015.
- *Arpaci-Dusseau R.H., Arpaci-Dusseau A.C. *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018.
- *Tanenbaum A.S., Woodhull A.S. *Operating Systems: Design and Implementation*, Prentice Hall, 3rd Edition, 2006.
- *Haghighat A. *Operating Systems with Problem Solving Approaches*, Pars Resaneh, 2023.

لینوکس و دستورات خط فرمان:

- *Fox R. *Linux with Operating System Concepts*, Chapman and Hall/CRC, 2nd Edition, 2021.
- *Negus C. *Linux Bible*, Wiley, 2010.
- *Garrels M. *Introduction to Linux: A Hands-on Guide*, Fultus Corporation, 2010.
- *Dalheimer M.K., Welsh M. *Running Linux*, O'Reilly Media, 4th Edition, 2002.
- *Purcell J., Hat R. *Linux Complete Command Reference*, Sams Publishing, 1997.

اسکریپت‌نویسی پوسته:

- *Matthew N., Stones R. *Beginning Linux Programming*, Chapter 2: Shell Programming, Wiley, 4th Edition, 2008.

---

[1] Bibliography

- *WikiBooks. *Bash Shell Scripting*, WikiBooks, last revised November 2024.
- *Blum R., Bresnahan C. *Linux Command Line and Shell Scripting Bible*, Wiley, 4th Edition, 2021.
- *Cooper M. *Advanced Bash-Scripting Guide*, The Linux Documentation Project, 2017.

برنامه‌نویسی سیستمی:

- *Love R. *Linux System Programming: Talking Directly to the Kernel and C Library*, O'Reilly Media, 2nd Edition, 2013.
- Kerrisk M. *The Linux Programming Interface*, No Starch Press, 2010.
- Stevens W.R., Rago S.A. *Advanced Programming in the UNIX Environment*, Addison-Wesley, 3rd Edition, 2013.
- *Bovet D.P., Cesati M. *Understanding the Linux Kernel*, O'Reilly Media, 3rd Edition, 2006.

هسته و ماژول‌ها:

- *Salzman P.J., Burian M. *The Linux Kernel Module Programming Guide*, CreateSpace, 2022.
- *Kroah-Hartman G. *Linux Kernel in a Nutshell*, O'Reilly Media, 2006.
- *Billimoria K.N. *Linux Kernel Programming Part 2: Char Device Drivers and Kernel Synchronization*, Packt Publishing, 2021.
- *Madieu J. *Linux Device Driver Development*, Packt Publishing, 2nd Edition, 2022.
- *Love R. *Linux Kernel Development*, Addison-Wesley, 3rd Edition, 2010.

تحلیل کارایی:

- Gregg B. *Systems Performance: Enterprise and the Cloud*, Pearson, 2nd Edition, 2020.
- *Gregg B. *BPF Performance Tools*, Addison-Wesley Professional, 2019.

مجازی‌سازی:

- *Dakic V., Chirammal H., Mukhedkar P., Vettathu A. *Mastering KVM Virtualization*, Packt Publishing, 2nd Edition, 2020.

- *Marshall N. *Mastering VMware vSphere 6.7*, Wiley, 2018.

حافظه مجازی:

- Bhattacharjee A., Lustig D., Khandelwal A. *Architectural and Operating System Support for Virtual Memory*, Springer, 2nd Edition, 2026.
- Gorman M. *Understanding the Linux Virtual Memory Manager*, Prentice Hall, 2004.

فایل‌سیستم:

- *Pate S.D. *UNIX Filesystems: Evolution, Design, and Implementation*, Wiley, 2003.
- Carrier B. *File System Forensic Analysis*, Addison-Wesley, 2005.

مستندات رسمی و منابع آنلاین

آموزش‌های کاربردی لینوکس:

- https://www.thegeekstuff.com
- https://linuxconfig.org

مرجع دستورات لینوکس:

- https://explainshell.com
- https://linuxcommandlibrary.com

مجموعه راهنماها و کتب آزاد برای لینوکس(TLDP) :

- https://tldp.org

مستندات رسمی اوبونتو شامل راهنمای نصب، مدیریت سیستم و سرور:

- https://ubuntu.com
- https://ubuntu.com/server/docs

مستندات رسمی بوت‌لودر GRUB:

- https://www.gnu.org/software/grub/manual/grub

صفحات راهنمای رسمی توابع سیستمی لینوکس:

- https://man7.org/linux/man-pages

مستندات رسمی هسته لینوکس، شامل مستندات APIها، زیرسیستم‌ها و راهنماهای توسعه:

- https://www.kernel.org/doc/html/latest

استاندارد رسمی POSIX شامل توابع سیستمی و سمافورها:

- https://pubs.opengroup.org/onlinepubs/9699959399

تحلیل کارایی و ابزار perf:

- https://www.brendangregg.com/linuxperf.html

مستندات رسمی KVM:

- https://www.kernel.org/doc/html/latest/virt/kvm/index.html
- https://www.linux-kvm.org

مستندات رسمی VMware ESXi:

- https://docs.vmware.com/en/VMware-vSphere/index.html
- https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0.html

فهرست موضوعی[1]

A

ACL, ۳۴۹, ۳۵۱

C

CLI, ۶۹

D

dd, ۹۲, ۳۶۱

debugfs, ۳۶۹

E

exec, ۱۸۳

ext4, ۲۹, ۵۸, ۳۵۲

F

FAT, ۵۹, ۳۴۶

fdisk, ۸۸

fork, ۱۸۲

G

GPL, ۱۲, ۱۳, ۲۳۳, ۲۹۰

GRUB, ۳۳, ۶۲

I

I/O, ۱۵۳, ۲۳۳

iNode, ۱۸۸, ۳۵۴

K

KVM, ۳۰۰

L

lsblk, ۳۵۸

N

NTFS, ۵۹, ۳۴۹

P

perf, ۲۳۴, ۲۴۵

POSIX, ۲۲۲, ۲۲۸

proc, ۵۹, ۱۲۸, ۲۸۰

pthread, ۲۰۳

R

root, ۲۷, ۵۰, ۵۳

S

sem_post, ۲۱۹, ۲۲۲

sem_wait, ۲۱۹, ۲۲۲

Slackware, ۱۴

sudoer, ۵۱, ۷۷

T

TLB, ۲۴۹

---

[1] Index

W

WSL, ۳۹

ا

اجازه‌نامه عمومی همگانی, ۱۲
ارتباط بین فرآیندی, ۱۵۲, ۱۸٤
اسکریپت‌نویسی, ۱۱۰, ۱۳۲
اوبونتو, ۱٦, ۲۱, ٤٥, ۲۷۲
اولویت, ۹۸, ۲۸٥

ب

بافر ایستا, ۲۱۰
بافر پویا, ۲۱۲
بوت, ۱٤, ۳۳, ٦۱, ۳٤۹, ۳٥۲
بوت سکتور, ۳٤۷
بوت‌لودر, ۲۹, ٥٤, ٦۲, ۲۷۰

پ

پارتیشن, ۱۹, ۲۸, ٥٦, ۸۸, ۳٤٦, ۳٥۸, ۳٦۸
پایانه, ٤٦, ٤۸
پردازنده, ۹٥, ۹۹, ۱٥۱, ۲۳۸, ۲٥۹, ۲۹۷, ۳۱٤
پوسته, ٤۹, ۸۷, ۱۰٤, ۱۱۰, ۱۱۹, ۱۳۲, ۱٤۱, ۱۸۰

ت

تابع رشته, ۲۰٤, ۲۰۹
تخصیص دیرهنگام, ۳۳۹
تعهد بیش از حد, ۳۳۹
تعویض متن, ۲۰۱, ۲۱۱, ۲٤۷
توزیع, ۱٥, ۱۸, ٤٥
تولیدکننده, ۱٦۸, ۱۸٦, ۲۲٤

ج

جدول صفحه, ۳۳۰, ۳٤۱

ح

حافظه مجازی, ۹۸, ۳۳۰, ۳۳۳, ۳۳٦

خ

خط فرمان, ٤٥, ۳۰۳, ۳٦۱

د

دایرکتوری, ٥۱, ٦۹, ۸۳, ۹۰, ۳۳٥, ۳٤۹, ۳٥٤
دایرکتوری خانگی, ۳۰, ٥۳
دبیان, ۱٦, ٤٥, ۷٦
دیسک, ۲۷, ٥۲, ٥٦, ۸۸, ۹۲, ۳۲۹, ۳٤۷, ۳٥۹, ۳٦٤

ر

ردهت, ۱٦
رشته, ۱۸۷, ۲۰۱, ۲۰۳, ۲۱۳, ۲۱۹, ۲۲۳, ۲٦۰
ریچارد استالمن, ۱۱

ز

زمان‌بند, ٦۲
زمان‌بندی, ۱٥۱, ۲۰٥, ۲٦۰

س

سطح اجرایی, ٥۹, ٦۳
سمافور, ۱٦۸, ۲۱۹
سمافور باینری, ۲۲۲
سمافور شمارشی, ۲۲۳
سیستم‌عامل, ۸, ۱۱, ۱۲, ٤۹, ۲۰٦, ٦۹, ۲٥۹, ۲۹٦, ۳۱۹, ۳۲۹, ۳٤۱, ۳٥٤, ۳٦۰
سیگنال, ۹٦, ۱٥۱, ۱٦۸, ۱۷۳, ۱۸۹

ش

شناسه فرآیند, ۹۶, ۱۴۳, ۱۶۳, ۱۸۰

ص

صفحه‌بندی, ۳۲۹

ف

فایل‌سیستم, ۵۲, ۵۳, ۵۵, ۵۸, ۶۲, ۸۰, ۹۰, ۹۲, ۹۹, ۲۹۸, ۳۳۵, ۳۴۶, ۳۵۲, ۳۶۰, ۳۶۸, ۳۶۹
فدورا, ۱۶
فراخوان سیستمی, ۱۵۲, ۱۵۳, ۱۶۱, ۱۶۳, ۱۸۱, ۲۳۵, ۲۵۲
فراداده, ۸۵, ۲۹۰, ۳۴۶, ۳۶۹, ۳۷۲
فرآیند, ۴۶, ۶۳, ۹۶, ۱۵۰, ۱۸۰, ۲۰۱, ۲۱۹, ۲۳۳, ۲۵۲, ۲۶۰, ۳۲۸
فرآیند فرزند, ۱۸۰, ۱۸۲, ۱۸۵
فرآیند والد, ۱۸۰, ۱۸۲, ۱۸۵
فرمت, ۲۹, ۵۷, ۳۴۷
فضای آدرس, ۱۸۰, ۱۸۵, ۲۰۶, ۳۳۹
فضای کاربر, ۵۵, ۶۳, ۲۴۲, ۲۸۰
فضای مبادله, ۳۰, ۵۸, ۹۸, ۳۳۸, ۳۴۱
فضای هسته, ۵۹, ۲۴۲, ۲۶۳
فلش قابل بوت, ۲۲, ۳۱۵

ک

کاربر ریشه, ۵۰, ۷۶, ۷۷
کامپایلر, ۲۰۲, ۲۴۵, ۲۶۷
کدمنبع, ۱۱, ۸۳, ۱۵۷, ۲۶۵

گ

گنو, ۱۱, ۱۳, ۴۳, ۱۱۹, ۲۵۹

ل

لاگ هسته, ۱۶۱, ۲۸۸
لینوس تورووالدز, ۱۳
لینوکس, ۱۱, ۱۳, ۱۸, ۲۱, ۴۳, ۶۹, ۱۱۰, ۱۵۶, ۲۳۳, ۲۵۹, ۲۸۰, ۲۹۸, ۳۵۲

م

ماژول, ۲۶۳, ۲۸۰, ۲۸۳, ۲۸۷
ماژول هسته, ۱۵۸, ۳۰۰
ماشین‌مجازی, ۲۷, ۳۴, ۲۴۰, ۲۴۶, ۳۰۳, ۳۱۸, ۳۲۰, ۳۵۹
مجازی‌سازی, ۲۹۶, ۳۱۴
مجوز, ۵۱, ۷۲, ۳۱۵, ۳۳۵, ۳۵۱
مخزن نرم‌افزاری, ۴۳, ۱۰۳
مد کاربر, ۱۵۱
مد هسته, ۱۵۸, ۲۶۰
مدیر بسته, ۱۰۴
مدیر ماشین‌مجازی, ۳۴, ۳۰۷
مدیرحافظه, ۶۲
مدیریت استثنا, ۱۵۲
مدیریت حافظه, ۲۹۷, ۳۲۸, ۳۳۹
مصرف‌کننده, ۱۶۸, ۱۸۶, ۲۲۴
منوی بوت, ۲۲, ۲۷۴

ن

نرم‌افزار آزاد, ۱۱, ۱۲
نسخه پشتیبان, ۳۸, ۳۴۷, ۳۵۰, ۳۵۳
نقص صفحه, ۲۴۹, ۳۳۹
نقطه اتصال, ۲۹, ۵۴, ۹۱

ه

هایپروایزر, ۲۹۸, ۲۹۹, ۳۱۴
هدایتگر, ۱۱۴, ۲۸۸

هسته, ۱۲, ۱۳, ۶۲, ۸۹, ۹۸, ۱۱۹, ۲۳۳, ۲۴۷, ۲۵۲, ۲۵۹, ۲۶۵, ۲۷۰, ۲۸۰, ۳۳۸, ۳۳۹
هسته ماژولار, ۲۶۳, ۲۶۴
هسته یکپارچه, ۲۶۰, ۲۶۴
همگام‌سازی, ۱۱۱, ۱۶۸, ۱۸۴, ۲۰۱, ۲۰۶, ۲۱۲, ۲۱۹, ۲۲۳

و

وقفه, ۱۵۰, ۱۵۸, ۲۴۷, ۲۵۲